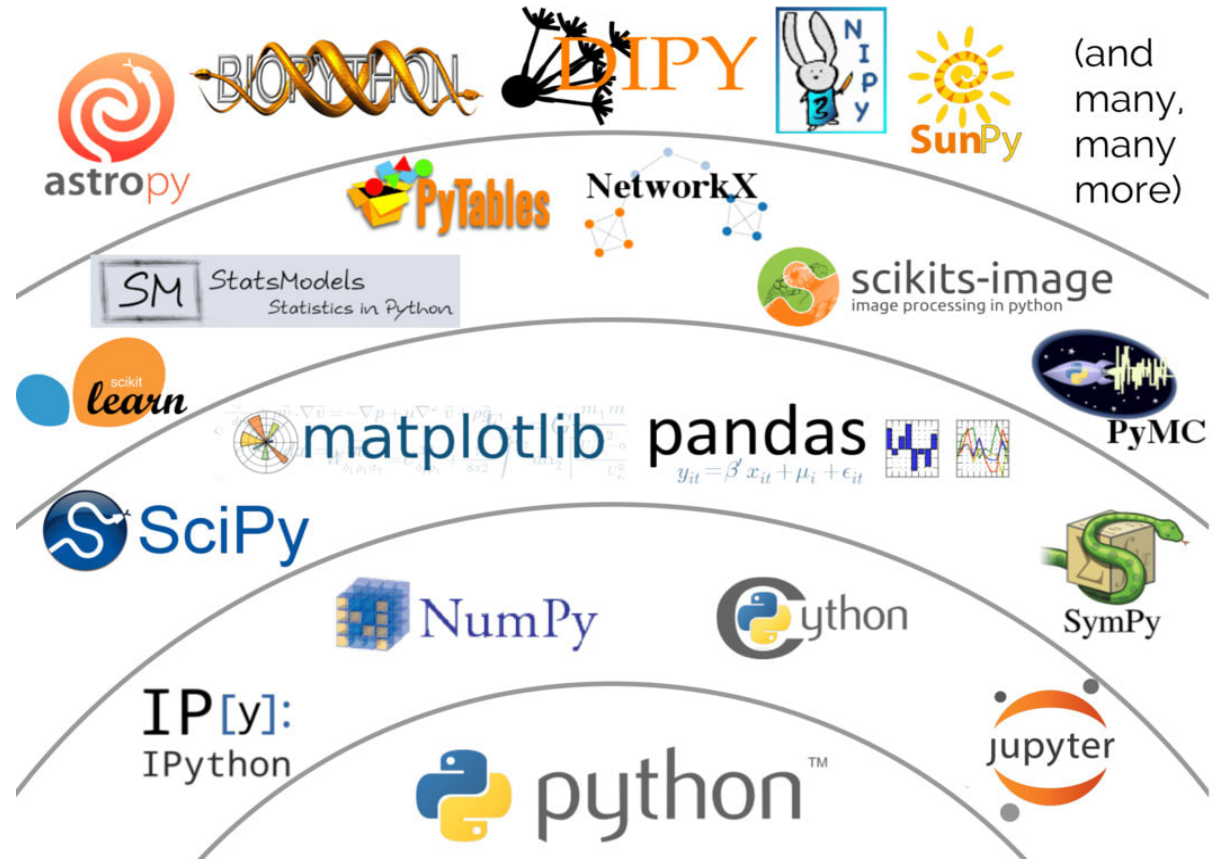


Korea University HRD
Machine Learning

KarL

• 파이썬 주요 라이브러리

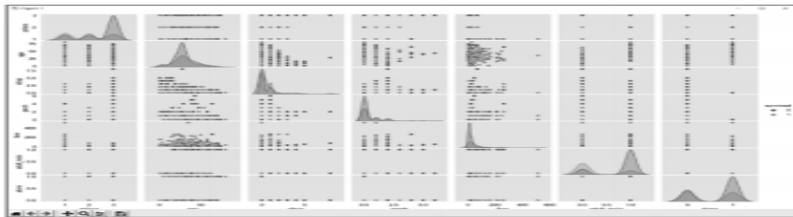
- ◆ 판다스
- ◆ 맷플로립
- ◆ 시분
- ◆ 넘파이
- ◆ 사이파이
- ◆ 심파이
- ◆ 머신러닝, 딥러닝 패키지
 - 사이킷런
 - 텐서플로
 - 케라스
 - 파이토치



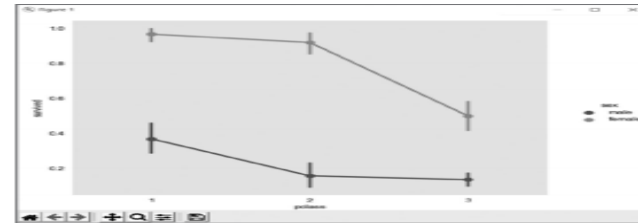
분석 예제

타이타닉호 생존을 분석하기	
목표	타이타닉호 승객 변수를 분석하여 생존율과의 상관관계를 찾는다.
핵심 개념	상관 분석, 상관 계수, 피어슨 상관 계수, 히트맵
데이터 수집	타이타닉 데이터: seaborn 내장 데이터셋
데이터 준비	결측치 치환: 중앙값 치환, 최빈값 치환
데이터 탐색	1. 정보 확인: <code>info()</code> 2. 차트를 통한 데이터 탐색: <code>pie()</code> , <code>countplot()</code>
데이터 모델링	1. 모든 변수 간 상관 계수 구하기 2. 지정한 두 변수 간 상관관계 구하기
결과 시각화	

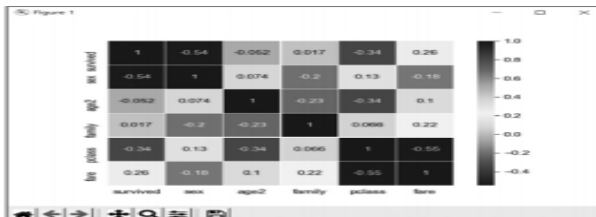
1. 산점도를 이용한 시각화



2. 특정 변수 간 상관관계 시각화



3. 히트맵을 이용한 시각화



타이타닉 자료로 분석하기

- `import seaborn as sns # 타이타닉 자료를 라이브러리에서 가져오기`
- `import pandas as pd`
- `titanic = sns.load_dataset("titanic")`
- `# 엑셀자료 형태로 저장하기`
- `titanic.to_csv(path_or_buf='titanic.csv', index=False)`
- `print(titanic.isnull().sum()) # missing value 확인하기`

타이타닉 자료(엑셀)로 분석하기

파일 편집 보기 입력 서식 수식 데이터 도구

오래 두기 복사하기 붙여넣기 셀 내용 맞추기 복사 셀 서식

일반 % , .00 00 줄 병합 테두리 셀 편집 숨기기 채우기 자르기 자동 합계 자동 필터 도형 개체 선택 찾기 글자 바꾸기

11.0 pt 가 가 가 가 가 가

fx A1 survived

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
1	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone											
2	0	3	male	22	1	0	7.25	S	Third	man	TRUE		Southampton	no	FALSE											
3	1	1	female	38	1	0	71.2833	C	First	woman	FALSE	C	Cherbourg	yes	FALSE											
4	1	3	female	26	0	0	7.925	S	Third	woman	FALSE		Southampton	yes	TRUE											
5	1	1	female	35	1	0	53.1	S	First	woman	FALSE	C	Southampton	yes	FALSE											
6	0	3	male	35	0	0	8.05	S	Third	man	TRUE		Southampton	no	TRUE											
7	0	3	male		0	0	8.4583	Q	Third	man	TRUE		Queenstown	no	TRUE											
8	0	1	male	54	0	0	51.8625	S	First	man	TRUE	E	Southampton	no	TRUE											
9	0	3	male	2	3	1	21.075	S	Third	child	FALSE		Southampton	no	FALSE											
10	1	3	female	27	0	2	11.1333	S	Third	woman	FALSE		Southampton	yes	FALSE											
11	1	2	female	14	1	0	30.0708	C	Second	child	FALSE		Cherbourg	yes	FALSE											
12	1	3	female	4	1	1	16.7	S	Third	child	FALSE	G	Southampton	yes	FALSE											
13	1	1	female	58	0	0	26.55	S	First	woman	FALSE	C	Southampton	yes	TRUE											
14	0	3	male	20	0	0	8.05	S	Third	man	TRUE		Southampton	no	TRUE											
15	0	3	male	39	1	5	31.275	S	Third	man	TRUE		Southampton	no	FALSE											
16	0	3	female	14	0	0	7.8542	S	Third	child	FALSE		Southampton	no	TRUE											
17	1	2	female	55	0	0	16	S	Second	woman	FALSE		Southampton	yes	TRUE											
18	0	3	male	2	4	1	29.125	Q	Third	child	FALSE		Queenstown	no	FALSE											
19	1	2	male		0	0	13	S	Second	man	TRUE		Southampton	yes	TRUE											
20	0	3	female	21	1	0	18	S	Third	woman	FALSE		Southampton	no	FALSE											
21	1	3	female		0	0	7.225	C	Third	woman	FALSE		Cherbourg	yes	TRUE											
22	0	2	male	35	0	0	26	S	Second	man	TRUE		Southampton	no	TRUE											
23	1	2	male	34	0	0	13	S	Second	man	TRUE	D	Southampton	yes	TRUE											
24	1	3	female	15	0	0	8.0292	Q	Third	child	FALSE		Queenstown	yes	TRUE											
25	1	1	male	28	0	0	35.5	S	First	man	TRUE	A	Southampton	yes	TRUE											
26	0	3	female	8	3	1	21.075	S	Third	child	FALSE		Southampton	no	FALSE											
27	1	3	female	38	1	5	31.3875	S	Third	woman	FALSE		Southampton	yes	FALSE											
28	0	3	male		0	0	7.225	C	Third	man	TRUE		Cherbourg	no	TRUE											
29	0	1	male	19	3	2	263	S	First	man	TRUE	C	Southampton	no	FALSE											
30	1	3	female		0	0	7.8792	Q	Third	woman	FALSE		Queenstown	yes	TRUE											
31	0	3	male		0	0	7.8958	S	Third	man	TRUE		Southampton	no	TRUE											
32	0	1	male	40	0	0	27.7208	C	First	man	TRUE		Cherbourg	no	TRUE											
33	1	1	female		1	0	146.5208	C	First	woman	FALSE	B	Cherbourg	yes	FALSE											
34	1	3	female		0	0	7.75	O	Third	woman	FALSE		Queenstown	yes	TRUE											

타이타닉

missing value (isnull() 사용하기)

missing value 결괏값 확인하기

```
survived.....0
pclass.....0
sex.....0
age.....177
sibsp.....0
parch.....0
fare.....0
embarked.....2
class.....0
who.....0
adult_male.....0
deck.....688
embark_town.....2
alive.....0
alone.....0
dtype: int64
```

결측값(missing value) 치환하기

- null 값에 의해서 차원이 안 맞아서 정확한 계산이 불가능하여 적절한 값으로 치환해야 한다.
- age → 중앙값(`median( )`)
- embark, embark_town → 최빈값(`fillna()` : 제일 많이 나온 값)
- deck(category) → 최빈 `category(fillna())`

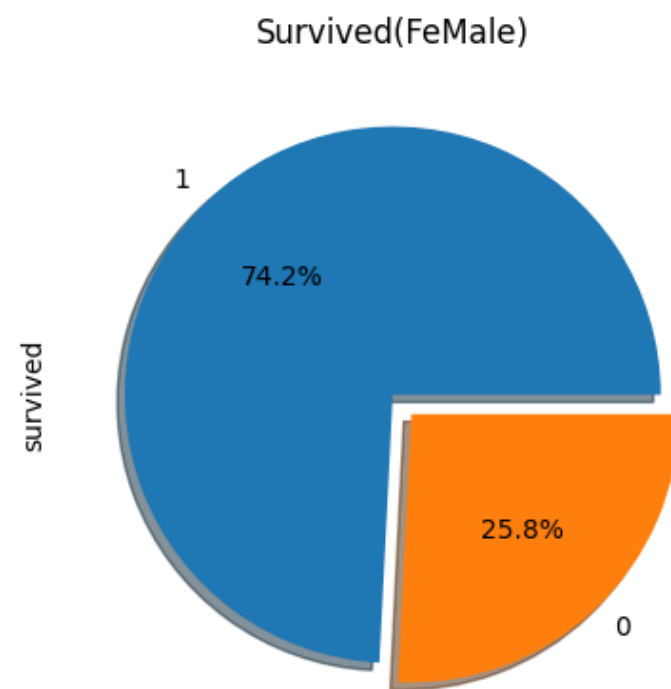
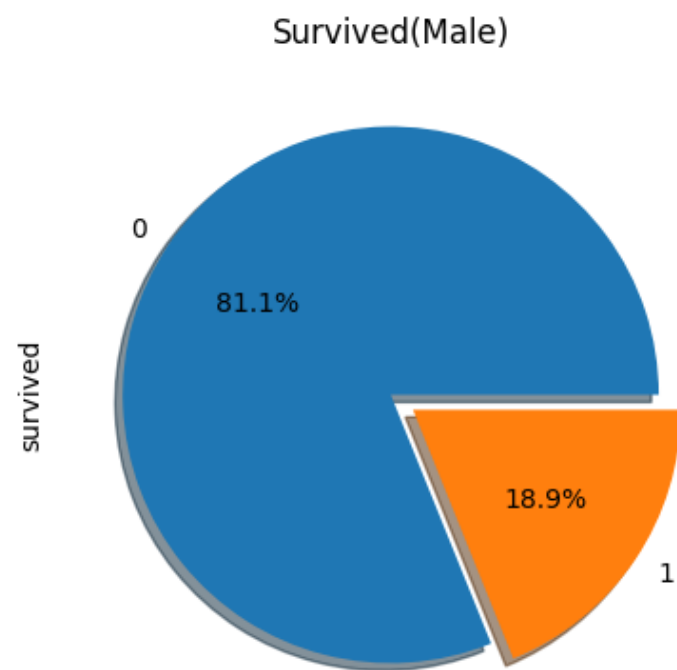
데이터 정리(결측값 치환하기)

- # age 에 있는 missing value 를 median() 중앙값으로 치환
- `titanic['age'] = titanic['age'].fillna(value=titanic['age'].median())`
- `print(f'titanic["age"] : \r\n{titanic["age"]}')'`
- # 최빈값(가장 많이 나온값)을 찾기 위해서 `.value_counts()`
- `titanic['embarked'].value_counts()`
- # 최빈값 S : 644 이것을 이용함
- `titanic['embarked'] = titanic['embarked'].fillna(value='S')`
- `print(f'titanic["embarked"] : \r\n{titanic["embarked"]}')'`
- `titanic['embark_town'].value_counts()`
- # Southampton 644 (최빈값)을 이용해서 "embark_town" 을 644로 치환한다.
- `titanic['embark_town'] = titanic['embark_town'].fillna(value='Southampton')`
- `print(f"titanic['embark_town'] : \r\n{titanic['embark_town']}")`
- `titanic['deck'].value_counts()`
- # C : 59를 사용해서 deck 를 채운다
- `titanic['deck'] = titanic['deck'].fillna(value='C') # 앞에서 나온 최빈값으로 치환`
- `print(f"titanic['deck'] : \r\n{titanic['deck']}")`
- # missing value 이 다 치환되었는지 확인하는 코드
- `print(titanic.isnull().sum())`
- # 타이타닉의 기본정보
- `titanic.info()`
- # 타이타닉의 생존자정보
- `titanic['survived'].value_counts()` # 0 는 (현재 죽음) 1 는 (현재 생존)

차트를 그려 데이터 시각화

- # 차트를 그려 데이터를 시각화하기
- `import matplotlib.pyplot as plt`
- `(f, ax) = plt.subplots(nrows=1, ncols=2, figsize=(10, 5))`
- `titanic['survived'][titanic['sex'] == 'male'].value_counts().plot.pie(explode=[0, 0.1], autopct='%1.1f%%', ax=ax[0], shadow=True)`
- `titanic['survived'][titanic['sex'] == 'female'].value_counts().plot.pie(explode=[0, 0.1], autopct='%1.1f%%', ax=ax[1], shadow=True)`
- `ax[0].set_title('Survived(Male)')`
- `ax[1].set_title('Survived(Female)')`
- `plt.show()`

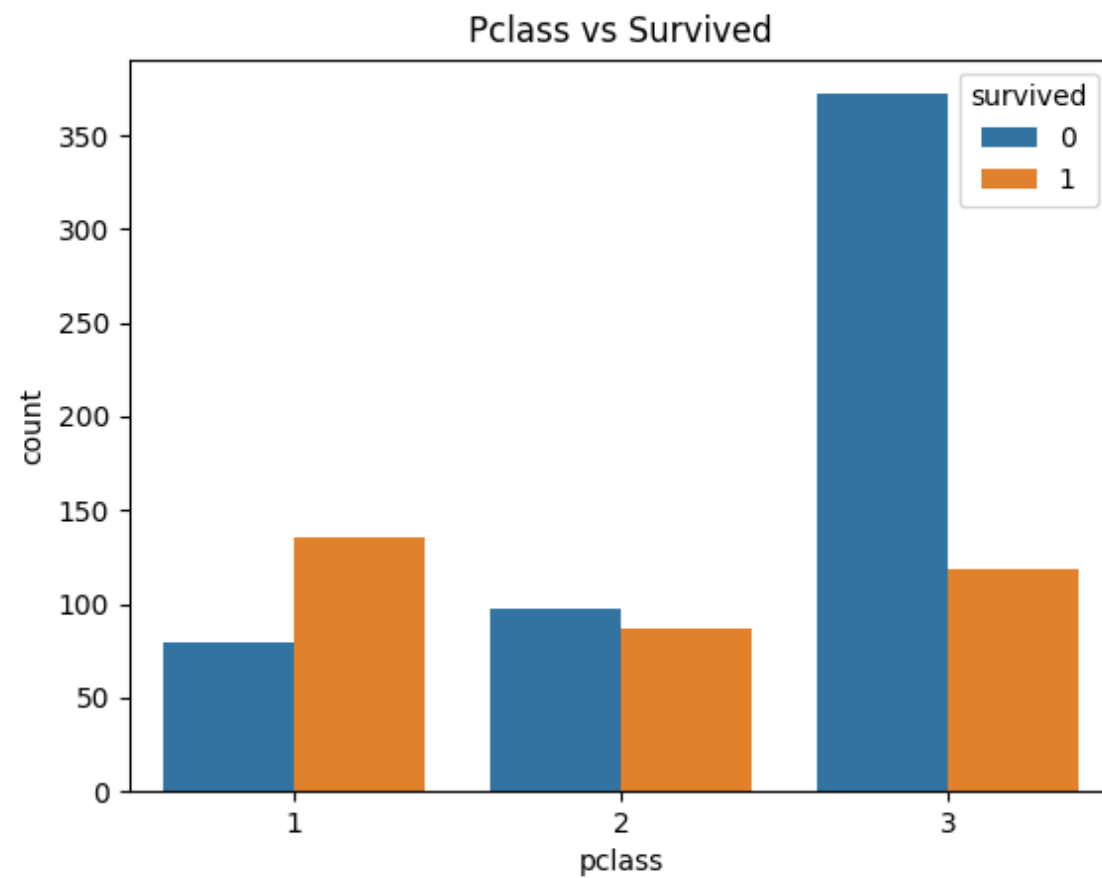
결과



등급별 생존자 수를 차트로 나타내기

- `sns.countplot(x='pclass', hue='survived', data=titanic)`
- `plt.title(label='Pclass vs Survived')`
- `plt.show()`

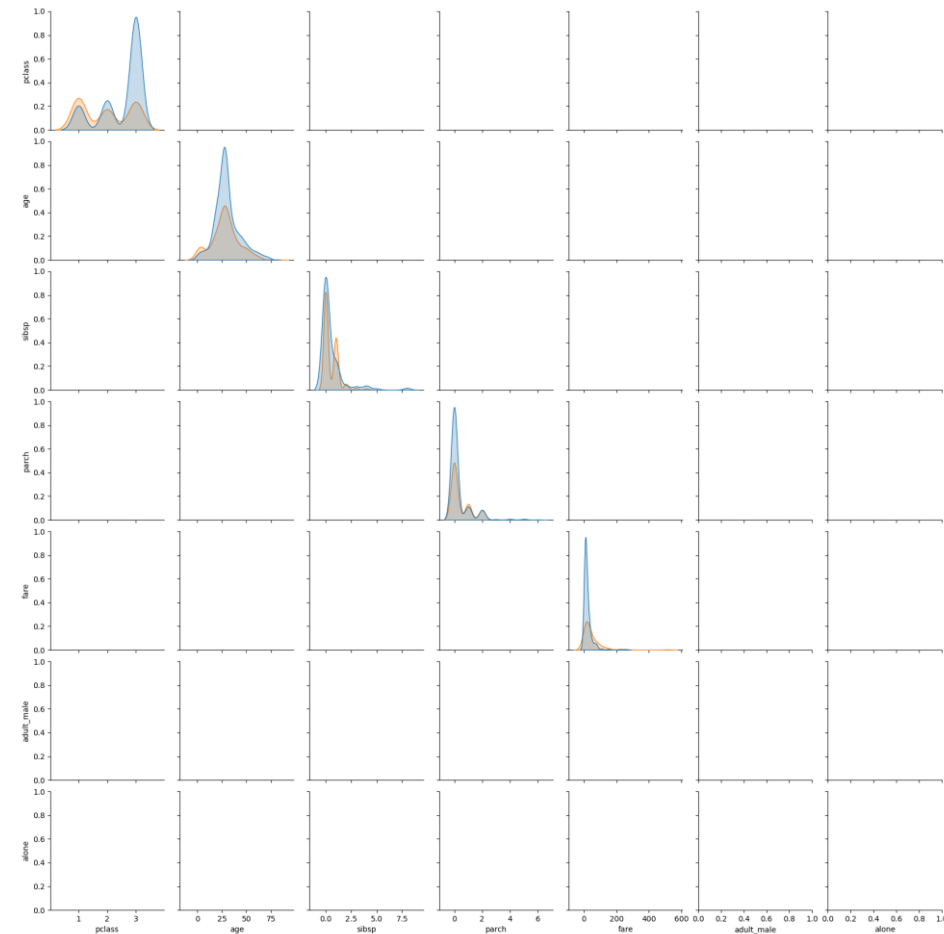
결과



데이터 모델링 - 상관분석을 위한 상관계수

- # 데이터 모델링
- # correlation 분석하기
- `titanic_correlation_analysis = titanic.corr(method='pearson')`
- `print(f'Pearson correlation : \r\n{titanic_correlation_analysis}')`
- `titanic_correlation_analysis.to_csv(path_or_buf='titanic_correlation.csv', index=False)`
- # 특정 변수 사이의 상관계수 구하기
- `titanic['survived'].corr(other=titanic['adult_male'], method='pearson')`
- `titanic['survived'].corr(other=titanic['fare'], method='pearson')`
- # 결과 시각화 (상관관계를 보고 싶을 때는 사용하는 방법)
- # 산점도 그리기
- `sns.pairplot(data=titanic, hue='survived')`
- `plt.show()`

데이터 모델링 - 결과

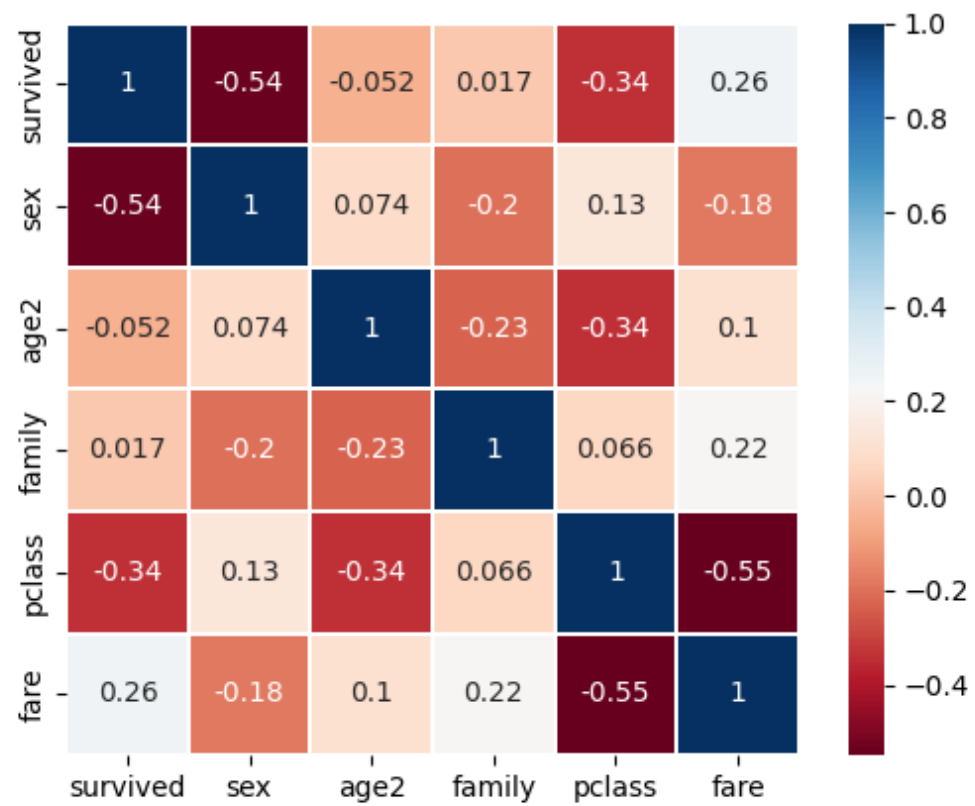


데이터 모델링 - 변수사이들의 상관계수를 히트맵으로 시각화하기

```
• def category_age(x): # 10살 단위로 등급을 나누어 0 ~ 7의 값으로 바꿔주는 category_age함수를 만듦
•     if x ≥ 70:
•         return 7
•     elif x ≥ 60:
•         return 6
•     elif x ≥ 50:
•         return 5
•     elif x ≥ 40:
•         return 4
•     elif x ≥ 30:
•         return 3
•     elif x ≥ 20:
•         return 2
•     elif x ≥ 10:
•         return 1
•     else:
•         return 0
•     pass

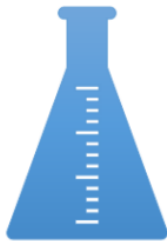
• titanic['age2'] = titanic['age'].apply(func=category_age)
• titanic['sex'] = titanic['sex'].map(arg={'male': 1, 'female': 0})
• titanic['family'] = titanic['sibsp'] + titanic['parch'] + 1
• titanic.to_csv(path_or_buf='C:\\titanic_heatmap.csv', index=False)
• heatmap_data = titanic[['survived', 'sex', 'age2', 'family', 'pclass', 'fare']]
• colormap = plt.cm.RdBu
• sns.heatmap(data=heatmap_data.astype(float).corr(), linewidths=0.1, vmax=1.0, square=True, cmap=colormap, linecolor='white', annot=True,
•             annot_kws={'size': 10})
• plt.show()
```

결과



데이터셋의 목적과 활용

데이터셋의
목적과 활용



연구



공공데이터



경연



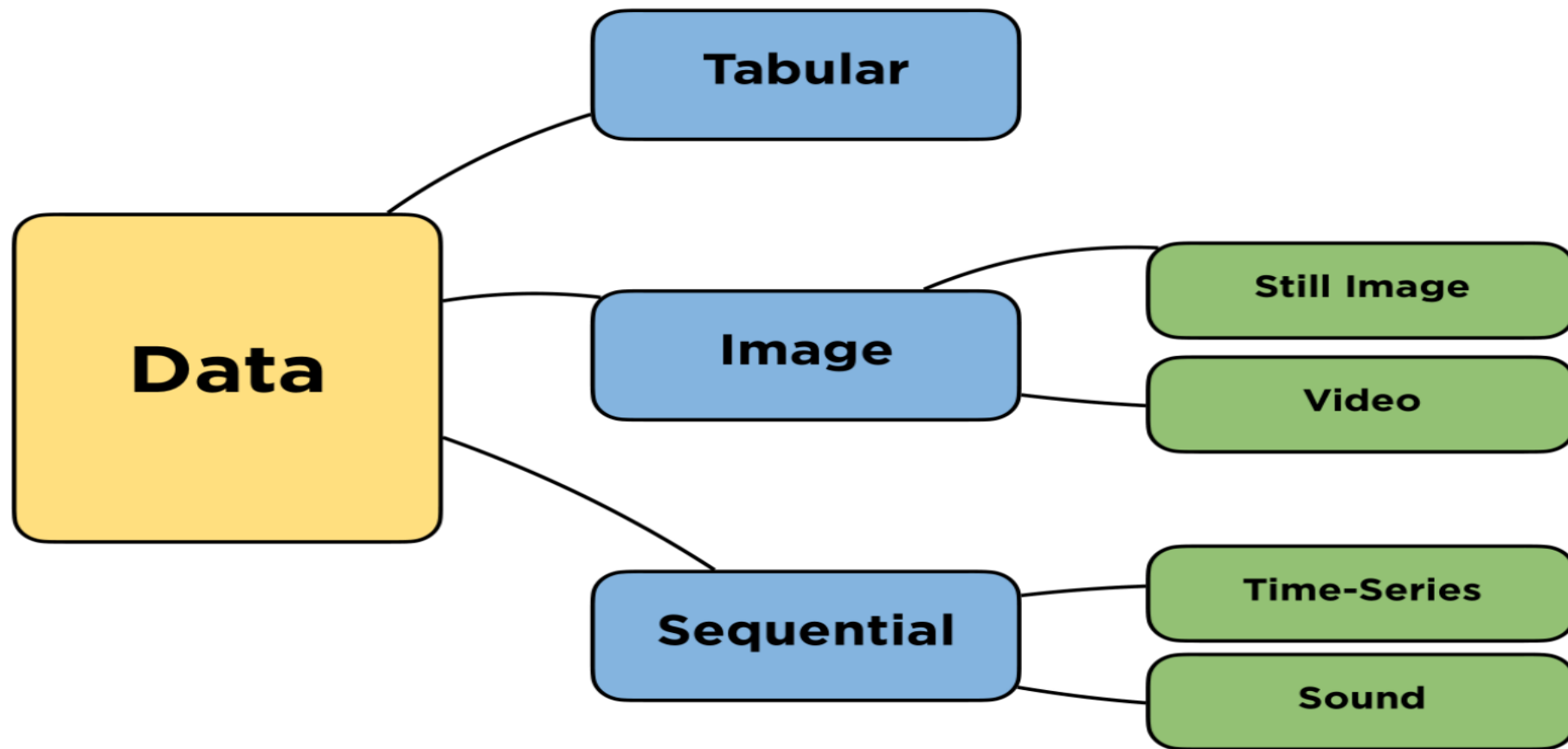
학술, 교육



사업, 서비스

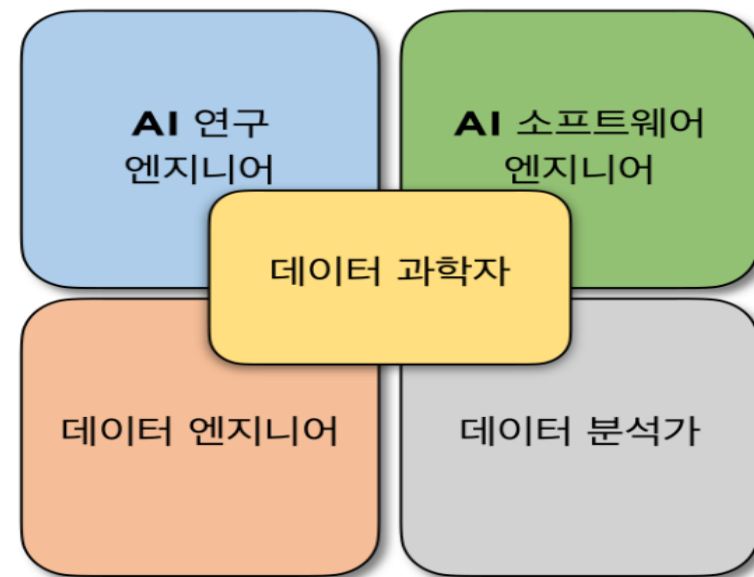
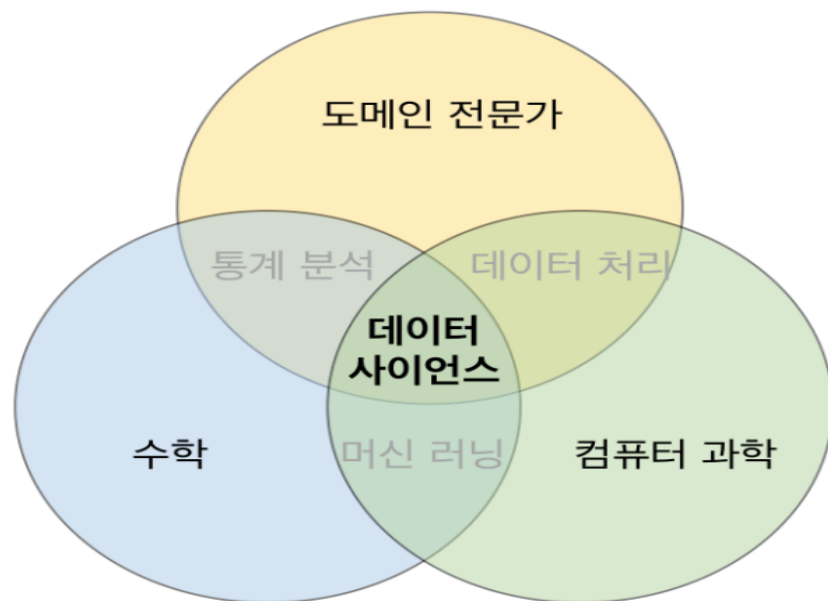
데이터 종류

데이터의 종류



데이터 사이언스

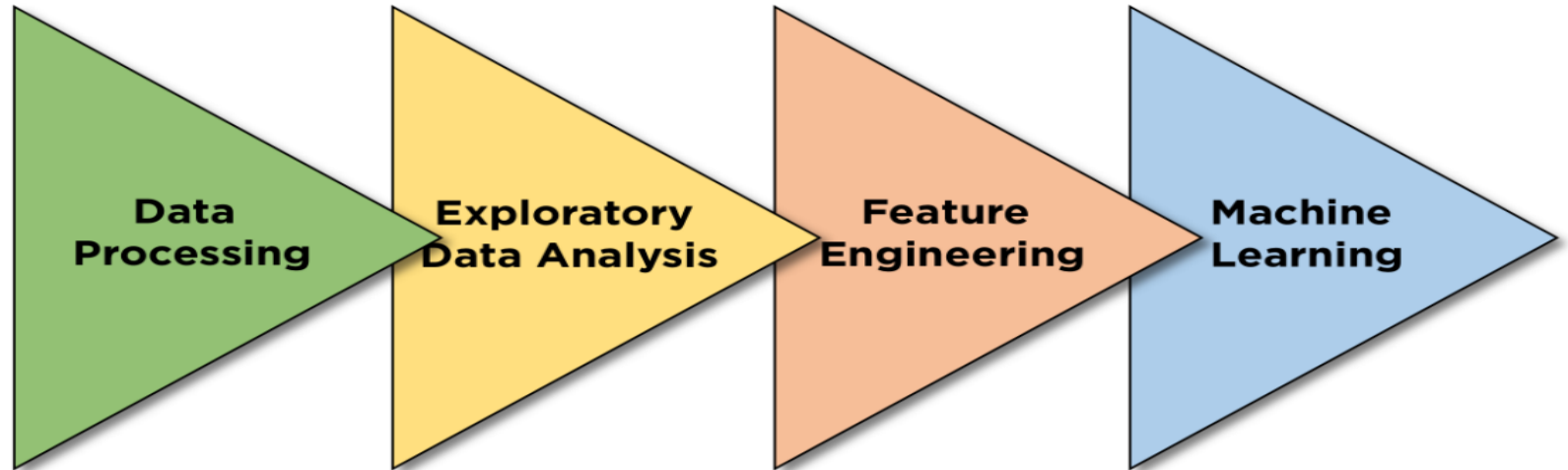
데이터 사이언스
소개



데이터 사이언스와 데이터 과학자가 담당하는 영역

데이터 분석 순서

데이터의 분석



데이터 분석의 일반적인 순서

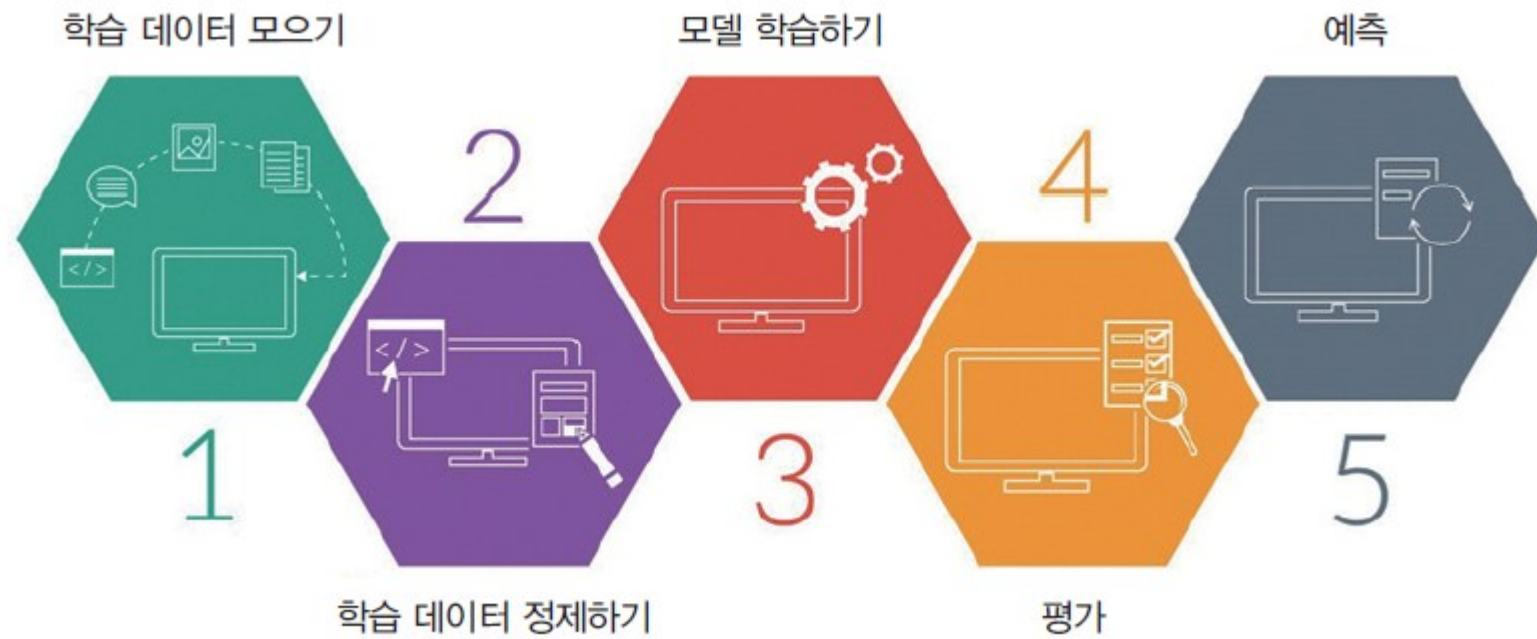
인공지능, 머신러닝, 딥러닝

인공지능
vs. 머신러닝
vs. 딥러닝



인공지능, 머신러닝, 그리고 딥러닝의 관계를 나타낸 벤 다이어그램

머신 러닝의 과정



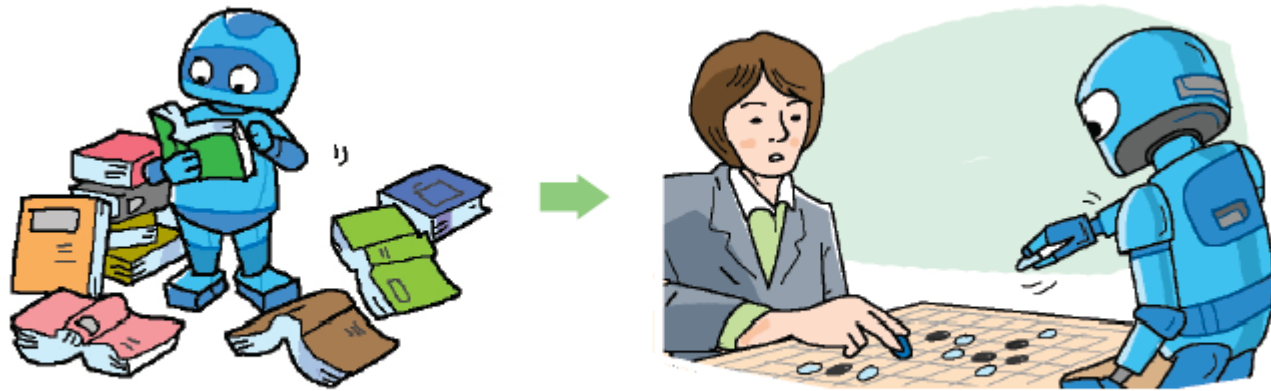
학습 목표

- 머신러닝과 전통적인 프로그래밍의 차이를 이해한다.
- 머신러닝의 과정을 이해한다.
- 붓꽃 데이터를 분류해본다.
- 숫자 이미지를 분류해본다.
- 머신러닝의 성능 측정 척도들을 살펴본다.



머신러닝이란?

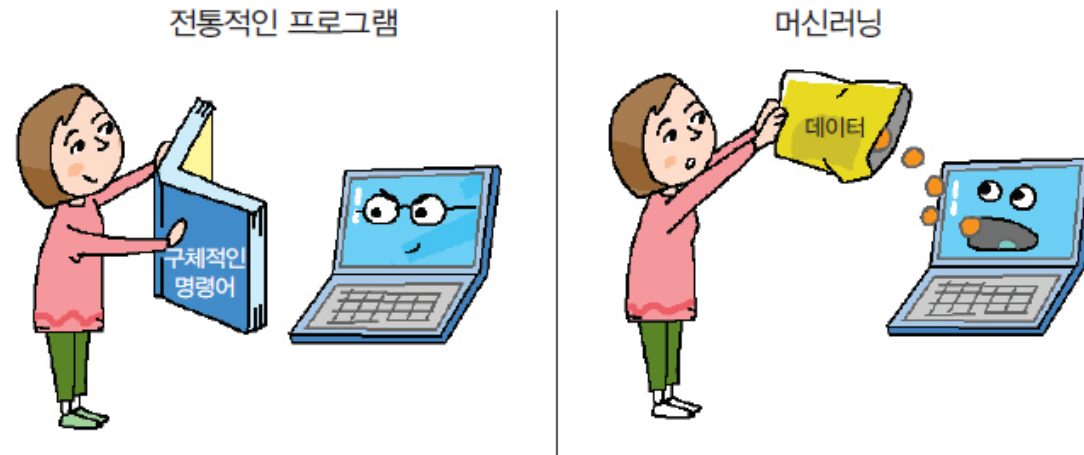
- 현재의 컴퓨터는 스스로 학습할 수 없기 때문에 우리가 컴퓨터에게 어떤 작업을 시키려면 반드시 프로그램을 작성하여 작업을 지시하여야 한다.
- 컴퓨터가 스스로 학습할 수 있다면 컴퓨터는 프로그램 없이도 여러 가지 일을 할 수 있을 것이다.



머신러닝

머신러닝과 전통적인 프로그래밍과의 차이점

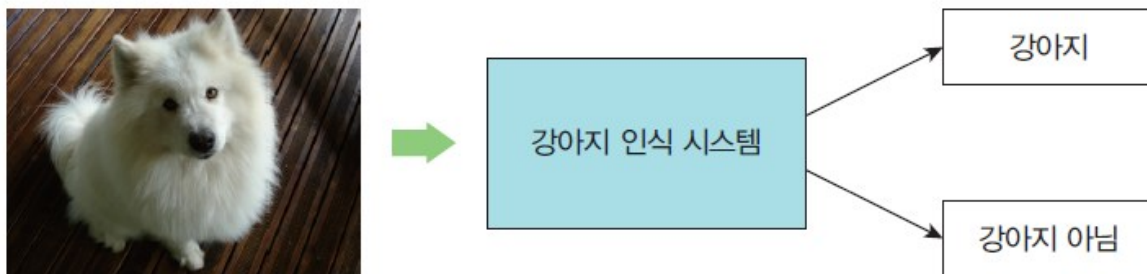
- 전통적인 접근 방식은 원하는 절차를 "프로그래밍"하는 것이다. 즉 인간이 컴퓨터에게 적절한 문제 해결 알고리즘을 만들어서 건네주어야 한다.
- 우리는 강아지를 인식하는 프로그램을 작성하지 않는다. 단순히 많은 수의 동물 사진을 머신러닝 시스템에 제공하고 어떤 사진이 강아지인지만 알려주면 된다. 이런 식으로 훈련이 진행된다면 머신러닝 시스템이 스스로 사진에서 강아지를 인식할 수 있다



전통적인 프로그래밍과 머신러닝의 차이점

예를 들어서 강아지와 고양이를 구별하는 문제를 생각해보자.

- 인간에게는 너무 쉬운 작업이지만 알고리즘으로 구성하기가 매우 어렵다.



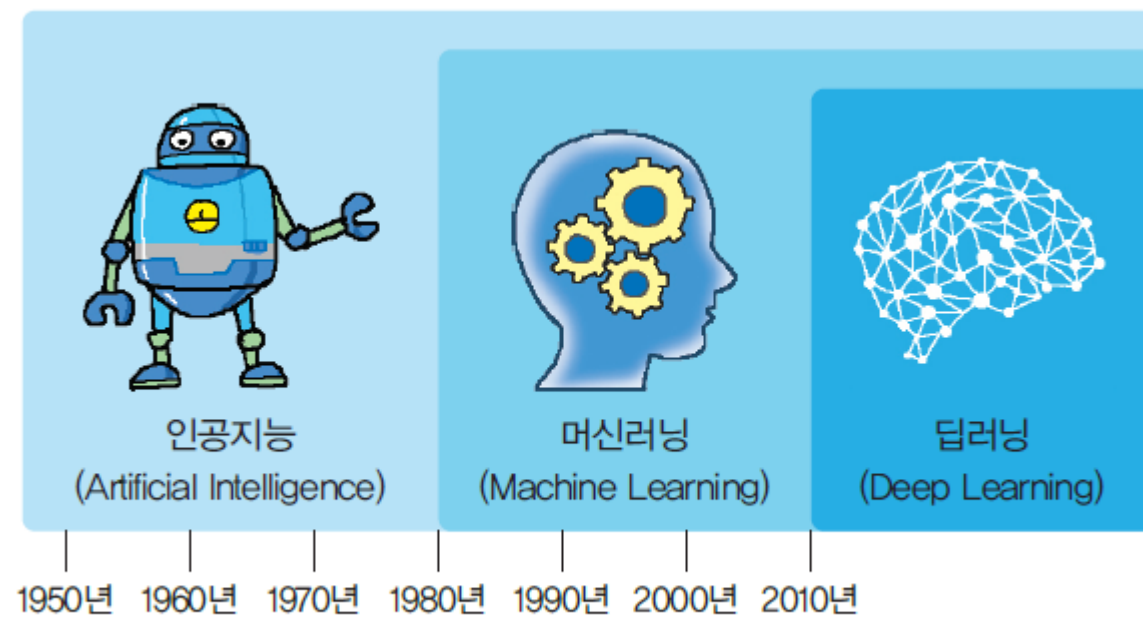
머신러닝 방법



전통적인 프로그래밍과 머신러닝을 비교

전통적인 프로그래밍	머신러닝
 <pre>graph LR; A[입력 데이터] --> C[컴퓨터]; B[프로그램 코드] --> C; C --> D[출력]</pre>	 <pre>graph LR; A[입력 데이터] --> C[컴퓨터]; D[출력] --> C; C --> B[프로그램 코드]</pre>
프로그램을 개발하여 컴퓨터로 입력 데이터를 처리하면 출력 데이터가 생성된다.	입력 데이터와 출력 데이터를 컴퓨터에 제공하면 컴퓨터가 스스로 프로그램 코드를 생성한다.

인공지능, 머신러닝, 딥러닝



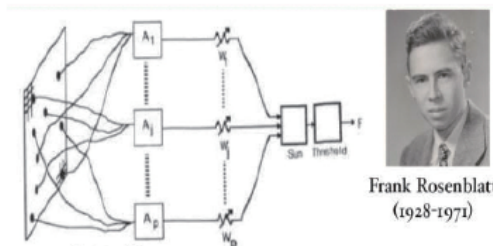
인공지능, 머신러닝, 딥러닝 간의 관계

머신러닝의 역사

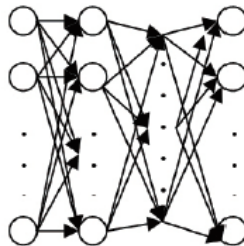
- IBM에서 근무하던 아서 사무엘(Arthur Samuel)이 컴퓨터 게임과 인공지능을 연구하면서 1959년에 "머신러닝"이란 용어를 만들었다.
- 그는 IBM 최초의 상용 컴퓨터인 IBM 701에서 최초의 체커 프로그램 만들었다.



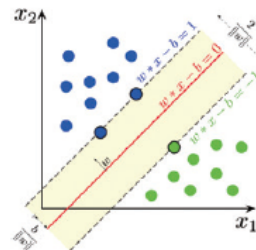
머신러닝의 역사



1957
Perceptron



1985
역전파 알고리즘



1995
SVM



1998
필기체 인식

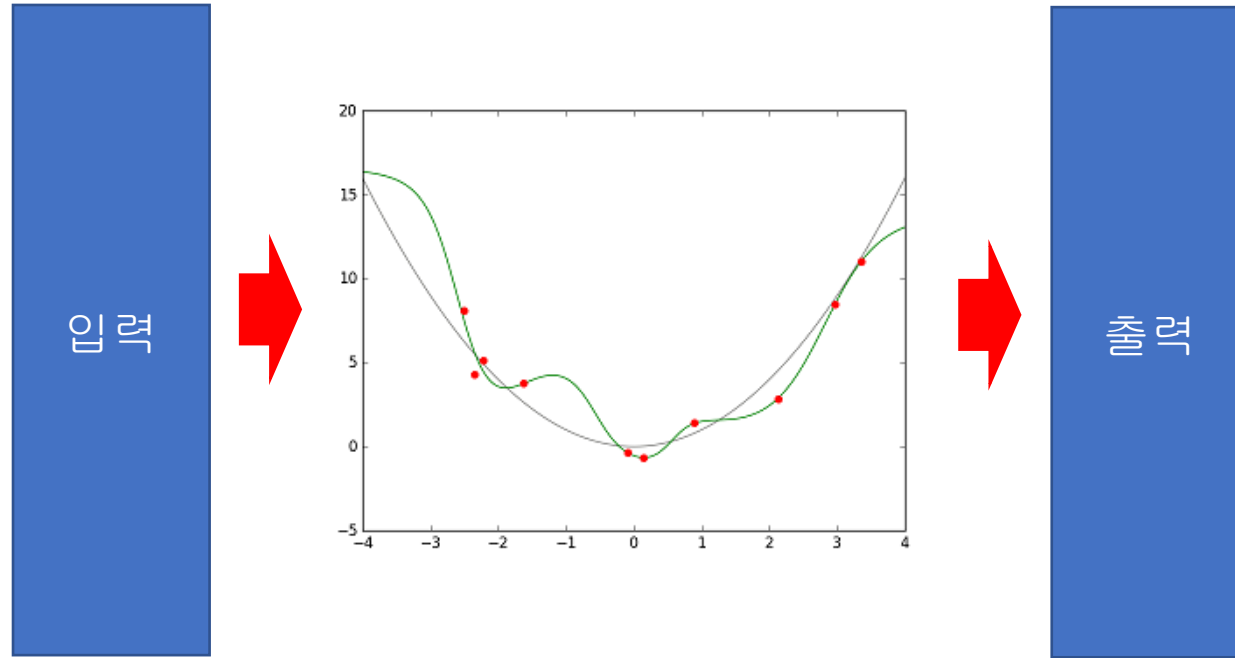
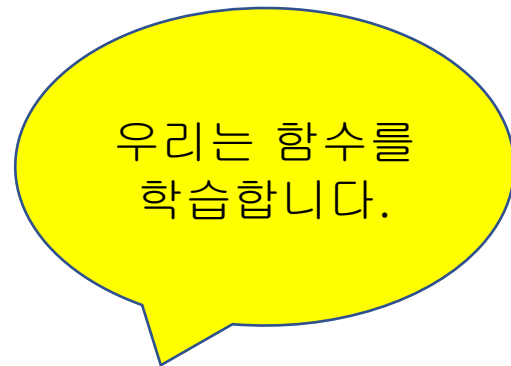


2012
ImageNet에서 우승

머신러닝의 역사

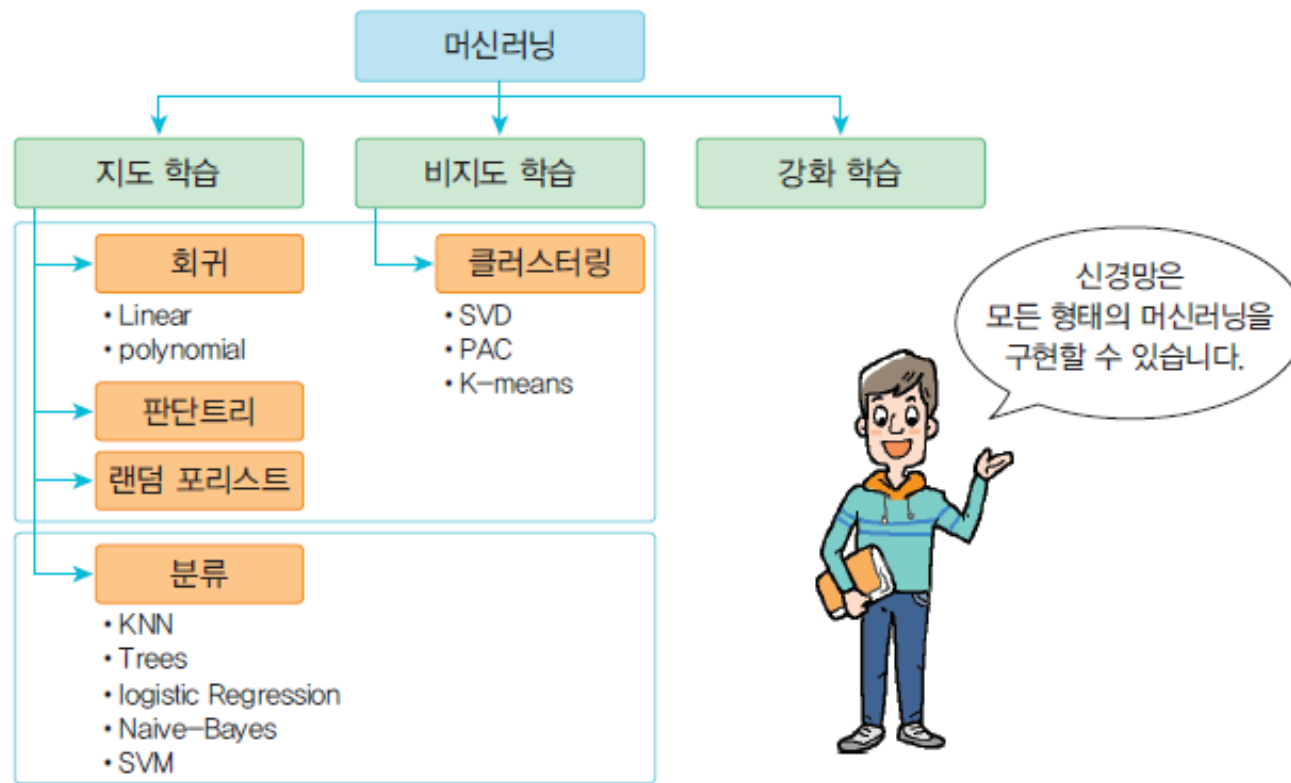
머신러닝

- 머신러닝은 입력을 받아서 출력하는 함수 $y=f(x)$ 를 학습한다고 생각할 수 있다. (함수 근사)



머신러닝(machine learning) == 함수 근사(function approximation)

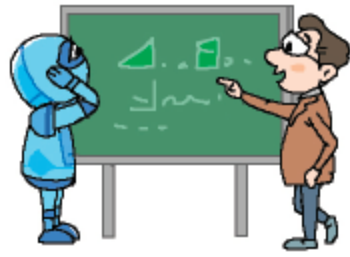
머신러닝의 종류



머신러닝의 종류

머신러닝의 종류

지도 학습



비지도 학습



강화 학습

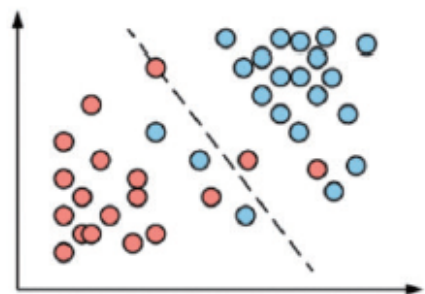


머신러닝의 종류

지도 학습(Supervised Learning)

컴퓨터는 “교사”에 의해 주어진 예제(샘플)와 정답(레이블)을 제공받는다. 지도 학습의 목표는 입력을 출력에 매핑하는 일반적인 규칙(함수, 패턴)을 학습하는 것이다.

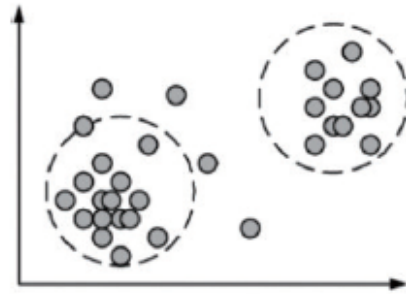
예를 들어서 강아지와 고양이를 구분하는 문제라면 강아지와 고양이에 대한 사진을 제공한 후에, 교사가 어떤 사진이 강아지인지, 어떤 사진이 고양이인지를 알려주는 것이다.



비지도 학습(Unsupervised learning)

외부에서 정답(레이블)이 주어지지 않고 학습 알고리즘이 스스로 입력 데이터에서 어떤 패턴을 발견하는 학습이다.

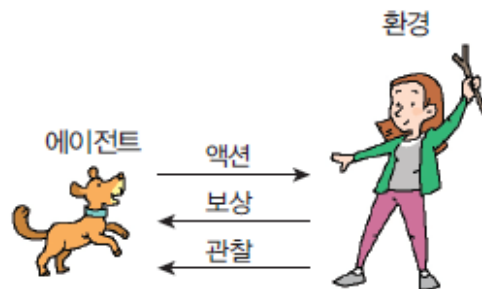
예를 들어 이름(레이블)이 붙어 있지 않은 과일을 분류하는 문제를 생각해 보자. 그래도 우리는 과일의 모양, 색상, 크기 등 다양한 특징을 이용하여 유사한 과일들을 분류할 수 있다.



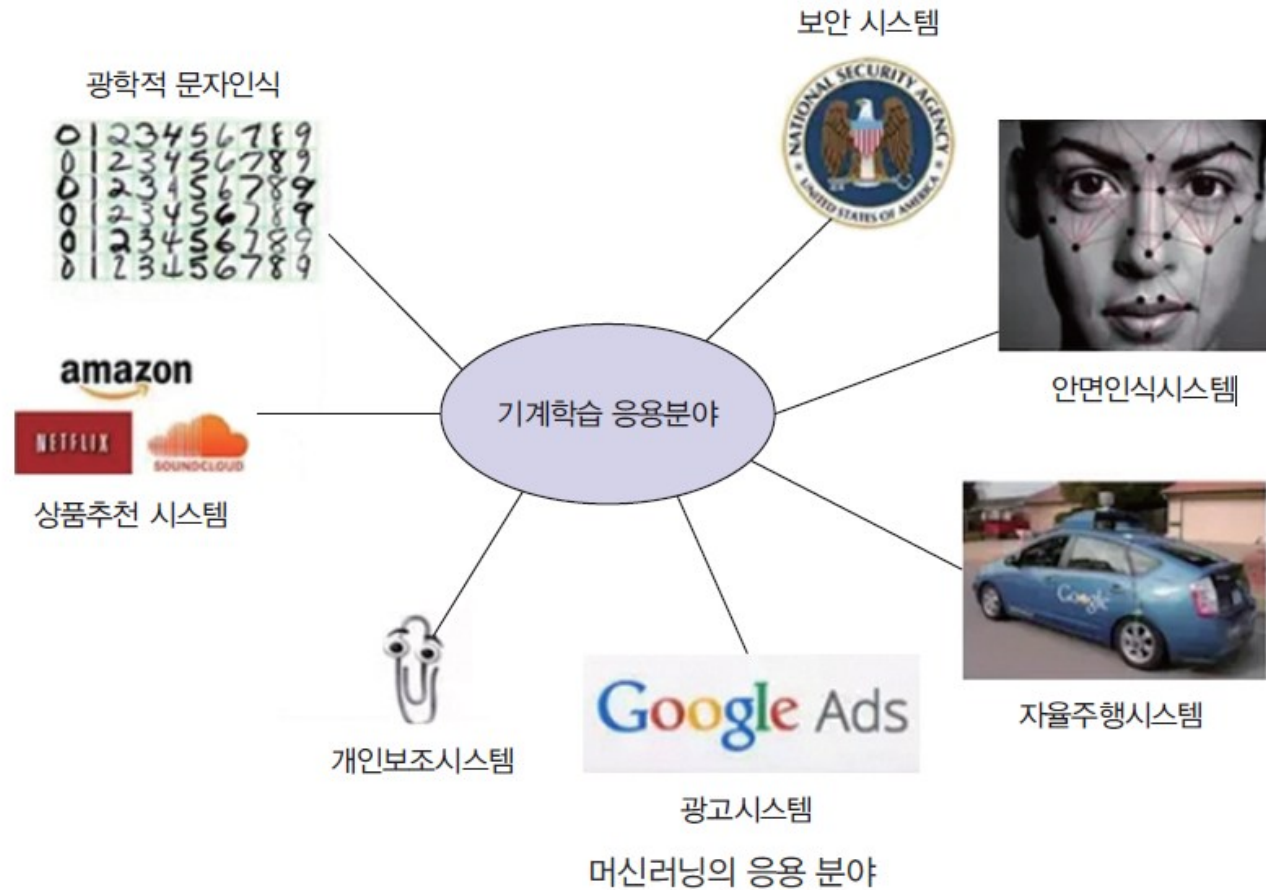
강화 학습(reinforcement Learning)

보상 및 처벌의 형태로 학습 데이터가 주어진다. 주로 차량 운전이나 상대방과의 경기 같은 동적인 환경에서 프로그램의 행동에 대한 피드백만 제공되는 경우이다.

예를 들어서 바둑에서 어떤 수를 두어서 승리하였다면 보상이 주어지는 식이다. 강화 학습에서는 보상과 처벌을 통하여 학습이 이루어진다.



머신러닝은 어디에 이용되는가?



머신러닝은 어디에 이용되는가?

- 이들 분야들은 살펴보면 복잡한 데이터들이 있고, 이들 데이터에 기반하여 결정을 내려야 하는 분야이다.
 - 영상 인식, 음성 인식처럼 프로그램으로 작성하기에는 규칙과 공식이 너무 복잡할 때
 - 보안 시스템에서 침입을 탐지하거나 신용 카드 거래 기록에서 사기를 감지하는 경우처럼 작업 규칙이 지속적으로 바뀌는 상황일 때
 - 주식 거래나 에너지 수요 예측, 쇼핑 추세 예측의 경우처럼 데이터 특징이 계속 바뀌고 프로그램을 계속해서 변경해야 하는 상황일 때
 - 전자 메일 메시지가 스팸인지 아닌지 여부
 - 신용 카드 거래가 허위인지 여부를 판별하는 시스템
 - 구매자가 클릭할 확률이 가장 높은 광고가 무엇인지를 알아내는 시스템

프로그래머로서 머신러닝의 실용적인 가치

- 첫 번째는 프로그래밍 시간을 줄일 수 있다는 점이다.
- 예를 들어서 맞춤법 오류를 수정하는 프로그램을 개발한다고 하자.
 - 전통적인 방법: 많은 맞춤법 규칙을 이용하여 작성할 수 있다. -> 상당한 시간이 필요
 - 머신러닝 이용: 많은 예제만 있다면 학습시켜서 빠른 시간 안에 신뢰성있는 프로그램을 완성할 수 있다.



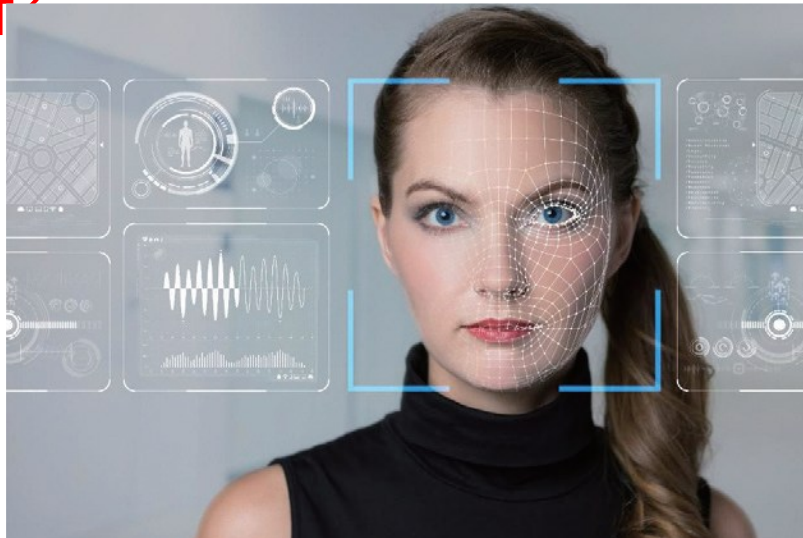
프로그래머로서 머신러닝의 실용적인 가치

- 두 번째로 맞춤형 제품을 쉽게 개발할 수 있다.
- 예를 들어서 여러분이 한국어 맞춤법 수정 프로그램이 작성하여 가지고 있다고 하자. 제품이 성공적이어서 30개국 언어 버전으로 확장하려고 한다.
 - 전통적인 방법: 각 언어마다 새로 작성하려면 수년 이상의 엄청난 시간이 필요하다.
 - 머신러닝 이용: 너무나도 쉽다 예제만 있으면 된다.



프로그래머로서 머신러닝의 실용적인 가치

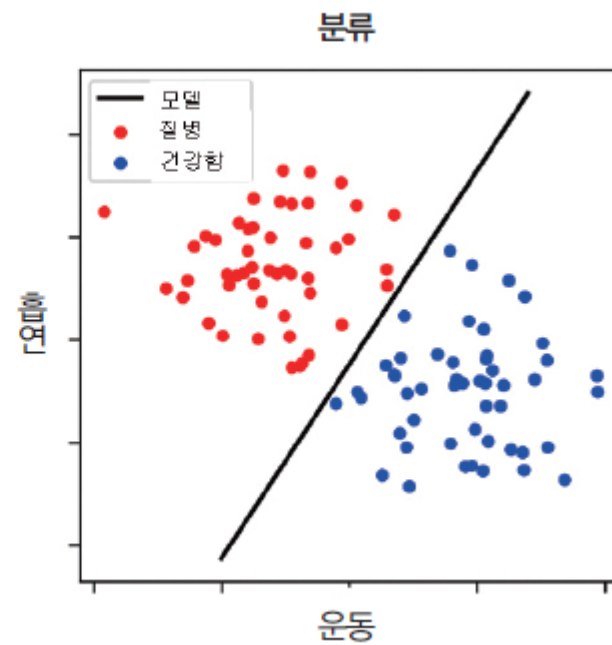
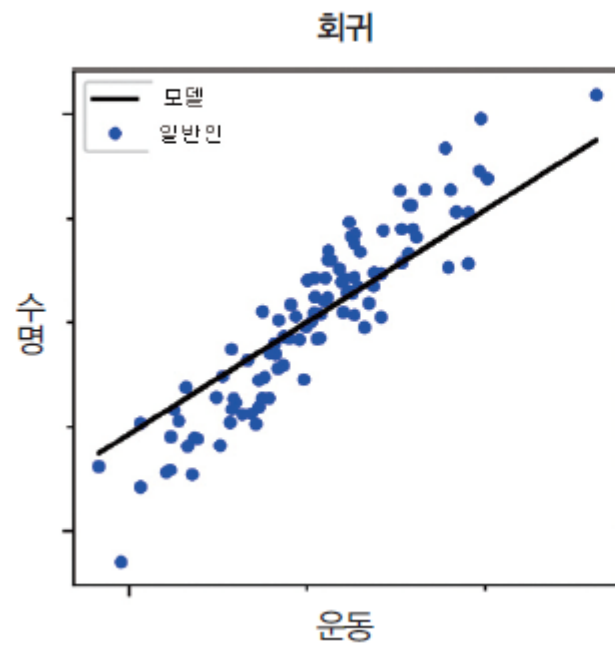
- 세 번째로 머신러닝은 프로그래머로 시도할 알고리즘이 떠오르지 않는 문제들을 해결할 수도 있다.
- 예를 들어서 컴퓨터가 사람의 얼굴을 인식하는 프로그램을 작성
 - 전통적인 방법: 이런 문제를 작성하려면 컴퓨터 시각 분야의 수많은 지식과 경험이 필요한 작업이다.
 - 머신러닝 이용: 프로그램에 수많은 예제만 보여주기만 하면 문제가 해결된다. 편리하지 않은가?



데이터 레이블링

- 머신러닝 시스템이 강아지와 고양이를 구별하게 학습시키려면 수천 장의 사진들이 필요하고 이들은 모두 강아지와 고양이로 구분되어 있어야 한다. 즉 사진에 라벨이 붙어 있어야 한다.
- 이들 구분 작업은 사람이 해줘야 한다. 이렇게 데이터에 라벨을 붙이는 작업을 데이터 라벨링(data labeling, data annotation)이라고 한다.
- “데이터 라벨러”라고 하는 새로운 직업이 생겨나고 있다고 한다.
-> 21세기의 ‘인형 눈알 붙이기’ 부업에 비유되기도 한다.

지도학습



지도학습

회귀(regression)

회귀는 주어진 입력-출력 쌍을 학습한 후에 새로운 입력값이 들어왔을 때, 합리적인 출력값을 예측하는 것이다. 회귀에서는 학습시키는 데이터가 이산적이 아니고 연속적이다. 즉 입력과 출력이 모두 실수이다. 회귀 모델은 연속적인 값을 예측한다. 예를 들어 운동과 수명에 대하여 학습한 후에, 회귀 모델은 다음과 같은 질문에 대한 답을 예측할 수 있다. "운동을 하루 5시간 한다면 그 사람의 예측 수명은 어떻게 될까?"

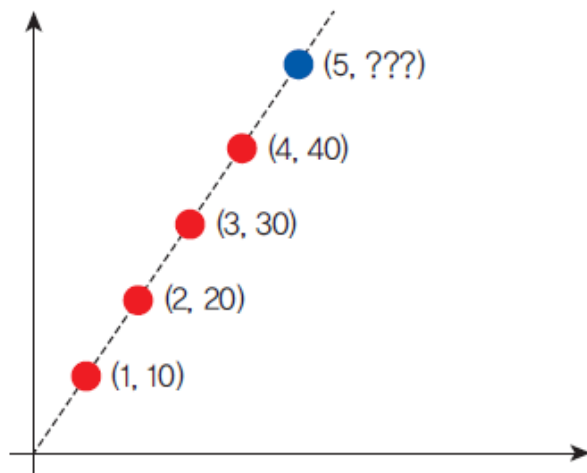
분류(classification)

입력을 두 개 이상의 레이블(유형)로 분할하는 것이다. 해당 모델을 학습시킬 때 우리는 레이블을 제공해야 한다. 학습 시에는 교사가 있어서 입력의 올바른 레이블을 알려준다. 학습이 끝나면 학습자가 한 번도 보지 못한 입력을 이들 레이블 중의 하나로 분류하는 시스템이다. 예를 들어서 운동시간을 입력하였을 때 "건강함", "질병상태"로 분류할 수 있다. 이때 입력은 운동시간이고 레이블은 "건강함", "질병상태"이다.

회귀

- 회귀(regression)는 주어진 입력-출력 쌍을 학습한 후에 새로운 입력값이 들어왔을 때, 합리적인 출력값을 예측
- 회귀는 입력(x)과 출력(y)이 주어질 때, 입력에서 출력으로의 매핑 함수를 학습하는 것이라 할 수 있다.

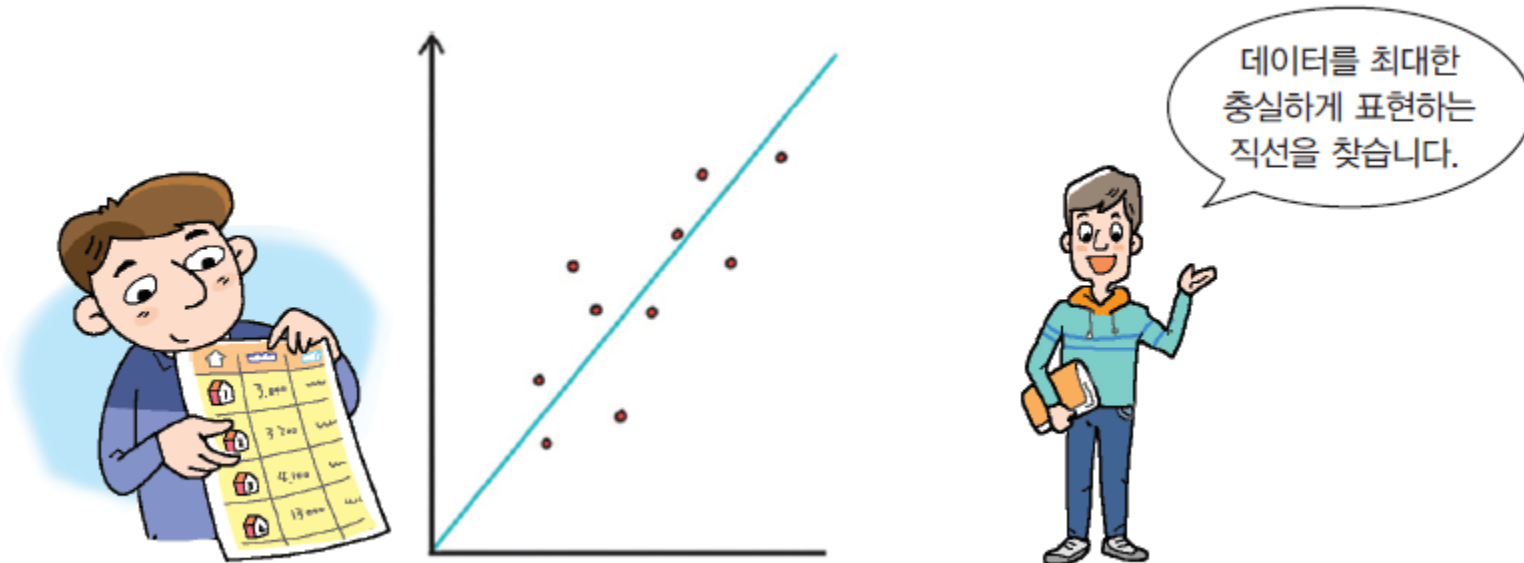
$$y = f(x)$$



회귀

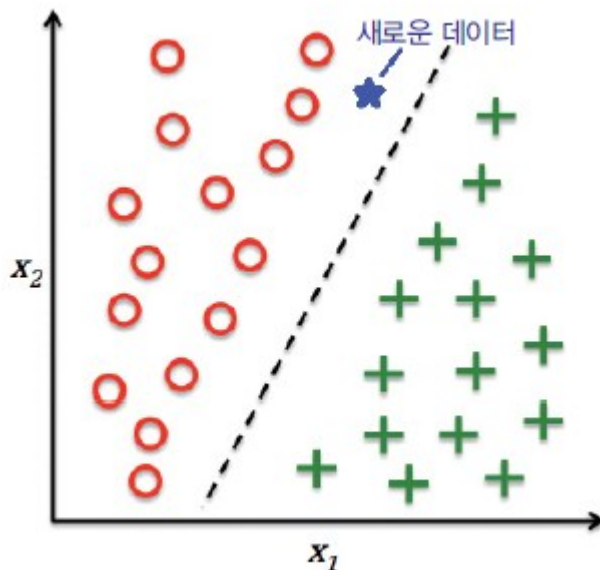
회귀

- 회귀(regression) : 회귀에서는 입력과 출력이 모두 실수이다.
 - "사용자가 이 광고를 클릭할 확률이 얼마인가요?"
 - "면적에 따른 각 아파트의 가격은 어떻게 되나요?"

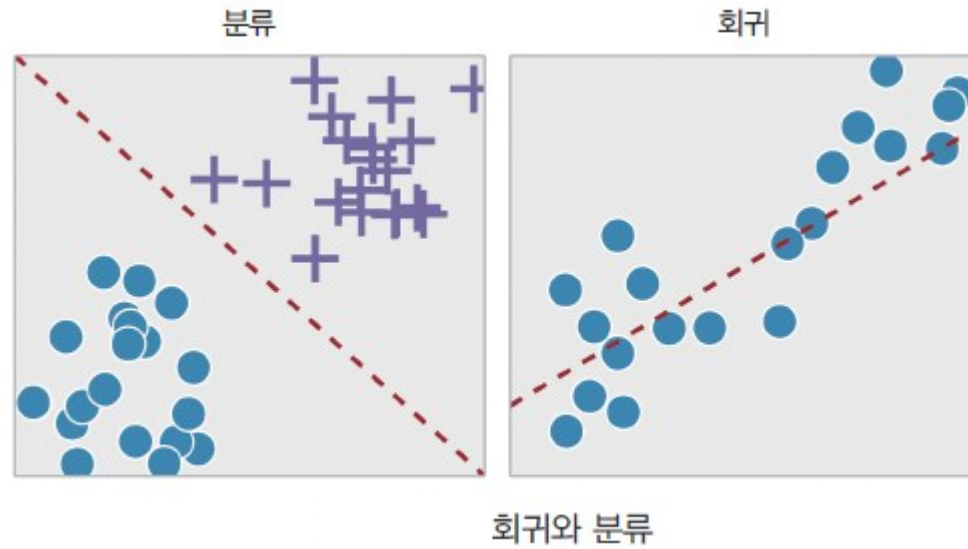


분류

- 앞에 나왔던 식 $y = f(x)$ 에서 출력 y 가 이산적(discrete)인 경우에 이것을 분류 문제(또는 인식 문제)라고 부른다.
- 분류에서는 입력을 2개 이상의 클래스(부류)로 나누는 것이다.
- 예를 들어서 사진을 보고 "강아지", 또는 "고양이"로 분류하는 것도 분류 문제이다.

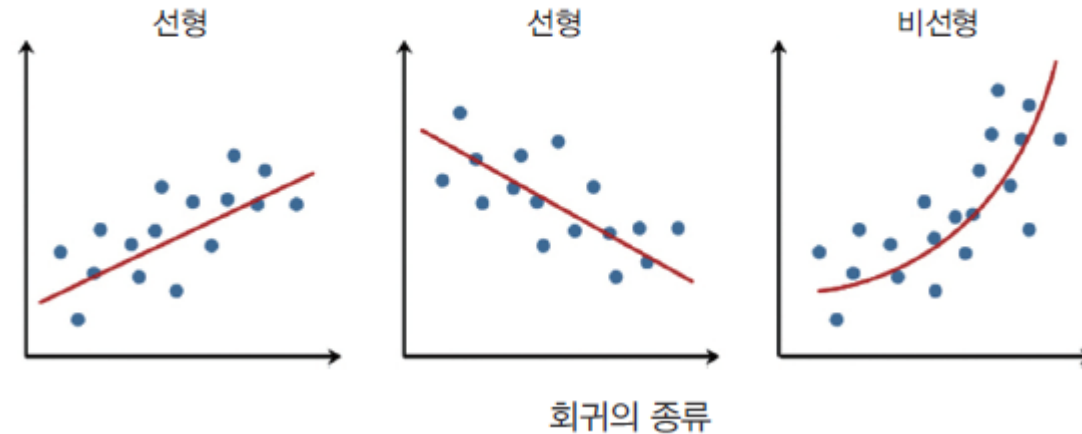


회귀(regression)와 분류(classification)



선형 회귀

- 회귀란 일반적으로 데이터들을 2차원 공간에 찍은 후에 이들 데이터들을 가장 잘 설명하는 직선이나 곡선을 찾는 문제라고 할 수 있다.
- $y = f(x)$ 에서 출력 y 가 실수이고 입력 x 도 실수일 때 함수 $f(x)$ 를 예측하는 것이 회귀이다.



선형 회귀의 예

- 부모의 키와 자녀의 키의 관계 조사
- 면적에 따른 주택의 가격
- 연령에 따른 실업율 예측
- 공부 시간과 학점과의 관계
- CPU 속도와 프로그램 실행 시간 예측

선형 회귀 소개

- 직선의 방정식: $f(x) = mx+b$
- 선형 회귀는 입력 데이터를 가장 잘 설명하는 기울기와 절편값을 찾는 문제이다
- 선형 회귀의 기본식: $f(x) = Wx+b$
 - 기울기->가중치
 - 절편->바이어스

선형 회귀의 종류

- 단순 선형 회귀: 단순 선형 회귀는 독립 변수(x)가 하나인 선형 회귀이다.

$$f(x) = wx + b$$

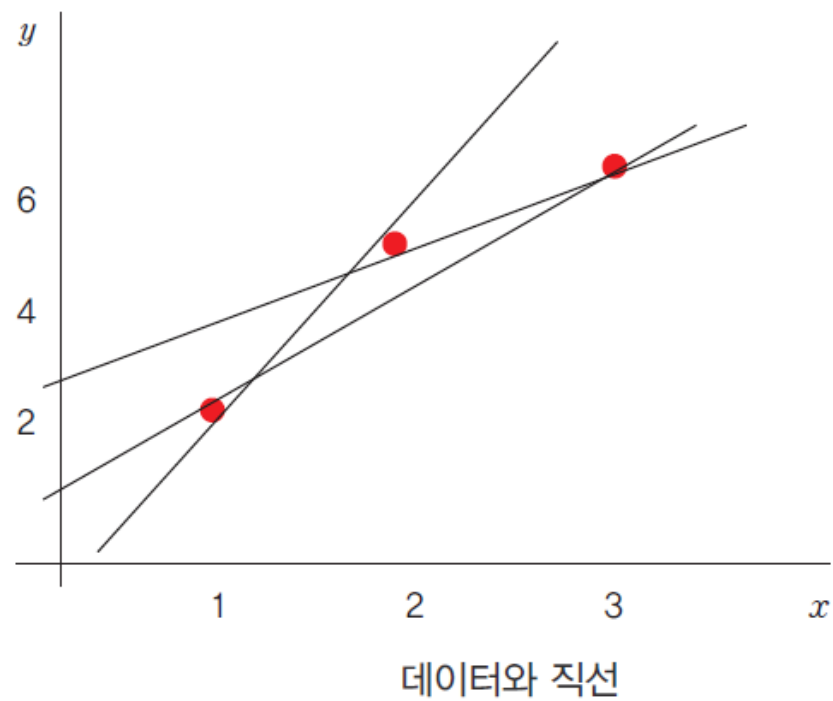
- 다중 선형 회귀: 독립 변수가 여러 개인 경우

$$f(x, y, z) = w_0 + w_1x + w_2y + w_3z$$

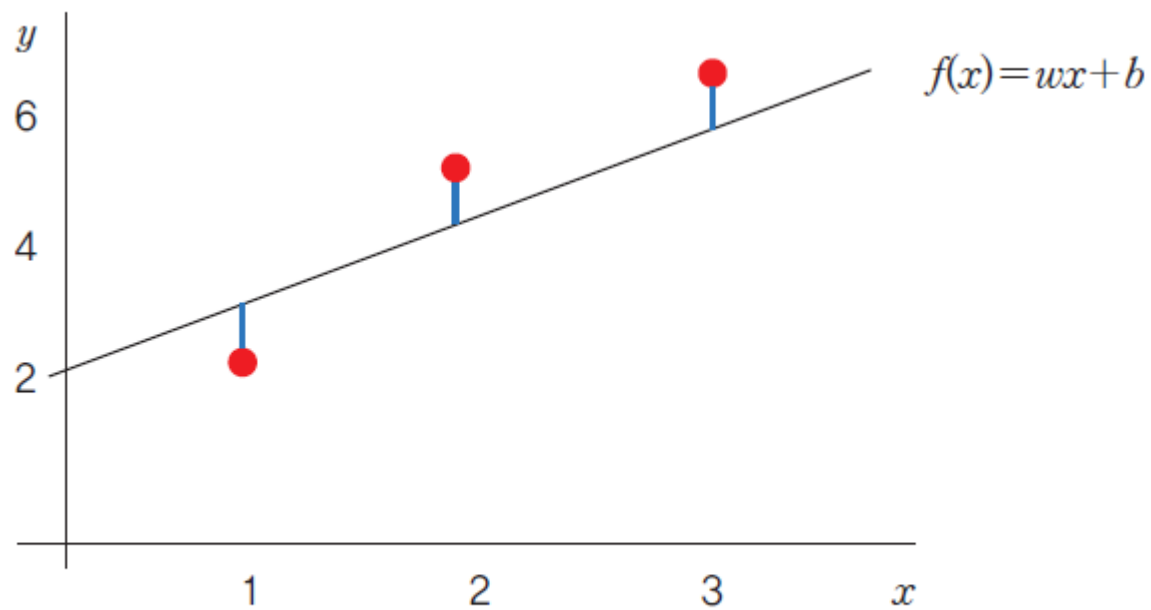
$$\text{매출} = w_0 + w_1 \times \text{인터넷 광고} + w_2 \times \text{신문광고} + w_3 \times \text{TV광고}$$

선형 회귀의 원리

x	y
1	2
2	5
3	6



직선과 데이터의 거리



데이터와 직선 간의 거리

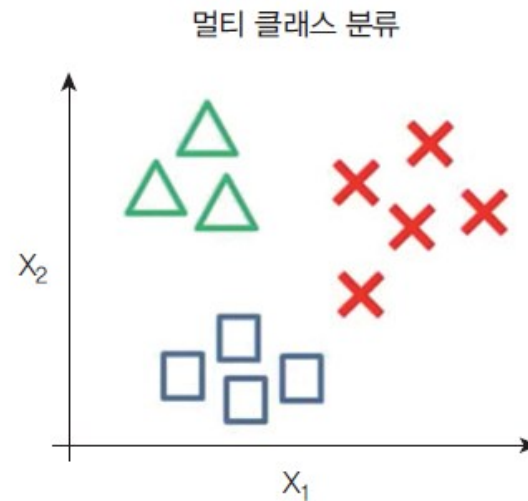
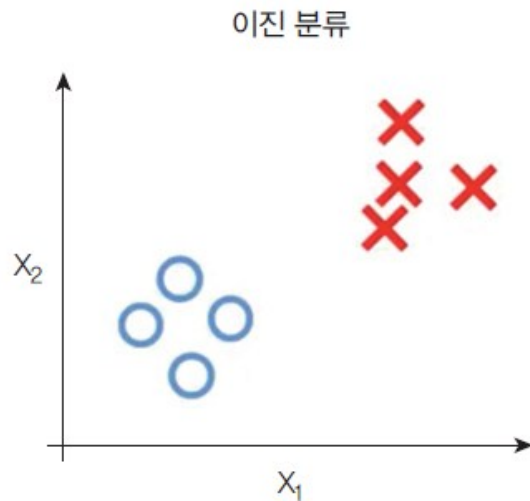
분류의 예

- 많은 과일로 채워진 과일 바구니를 보고, 프로그램이 바나나, 오렌지와 같은 올바른 레이블을 예측

번호	크기	색상	모양	과일 이름
1	크다.	빨강색	동근 모양에 꼭지가 있음	사과
2	작다.	빨강색	심장모양	체리
3	크다.	녹색	길고 곡선 형태의 원통 모양	바나나
4	작다.	녹색	타원형, 다발 형태	포도

분류

- 분류를 수행하기 위한 일반적인 알고리즘에는 신경망, kNN(k-nearest neighbor), SVM(Support Vector Machine), 의사 결정 트리 등이 포함된다.



비지도 학습

- 비지도 학습(unsupervised Learning)은 “교사” 없이 컴퓨터가 스스로 입력들을 분류하는 것을 의미한다.
식 $y = f(x)$ 에서 레이블 y 가 주어지지 않는 것이다.
- 데이터들의 상관도를 분석하여 유사한 데이터들을 모을 수는 있다.

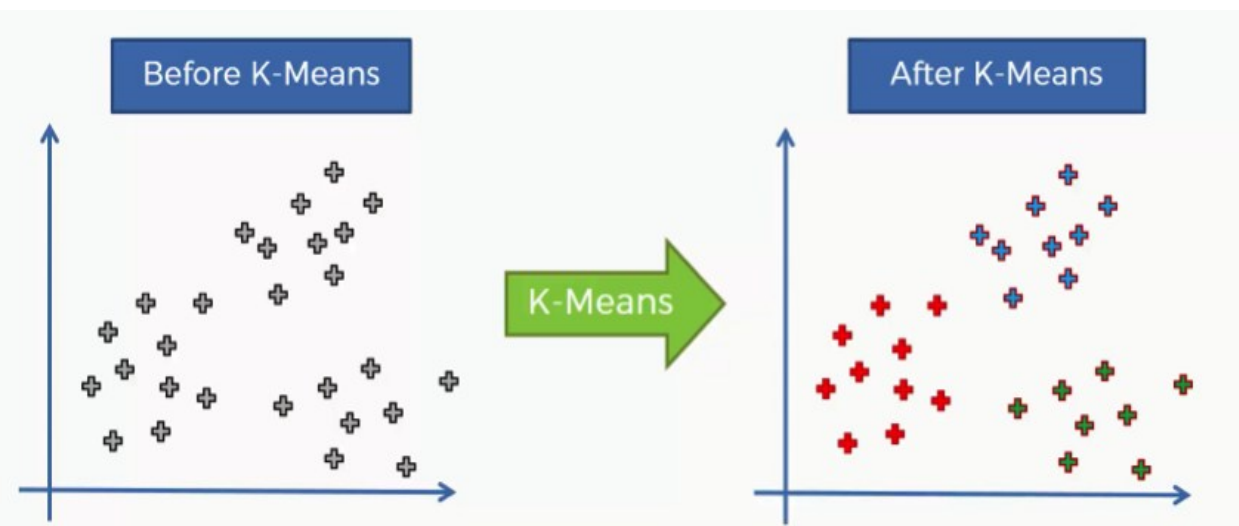


• 비지도학습

- ◆ K-means알고리즘은 목표변수(라벨,클래스) 없이, 정답을 모르는 상태로 학습을 하는 대표적인 비지도 학습(Unsupervised learning) 알고리즘 중 가장 쉽고 많이 사용되는 클러스터링 또는 군집분석 알고리즘 입니다.
- ◆ K-means알고리즘은 직관적으로 이해가 쉽고, 간단하게 구현할 수 있는 모델 입니다. 데이터 간의 거리를 구하는 유클리드(Euclidean distance)거리를 이용하는 군집 분석이나 다른 분석 방법들이 그렇듯 이상데이터에 민감합니다. 이를 극복하기 위해서 계산 비용이 늘어나는 것을 감수하고라도 다른 거리 지표 (median)를 사용하기도 합니다. 이 알고리즘은 데이터의 군집이 원형으로 나누어져 있을 때 잘 동작하기 쉬운 알고리즘이며 원형이 아니라 직선으로 분리된 데이터셋이나 그 외 다른 모양의 군집 모양을 가졌을 때 적절하지 않을 수 도 있습니다. 사전에 군집 대상인 K(하이퍼파라미터)의 수를 미리 알고 있어야 하며 K 값이 크면 클수록 군집 간의 에러가 줄어들기 때문에 무조건 큰 K 값을 추천할 가능성이 있습니다. 따라서 k의 값이 커질수록 일종의 패널티를 준다든지 해서 적절한 k의 수를 선택할 수 있게끔 하는 방법 등도 고려해야 합니다.

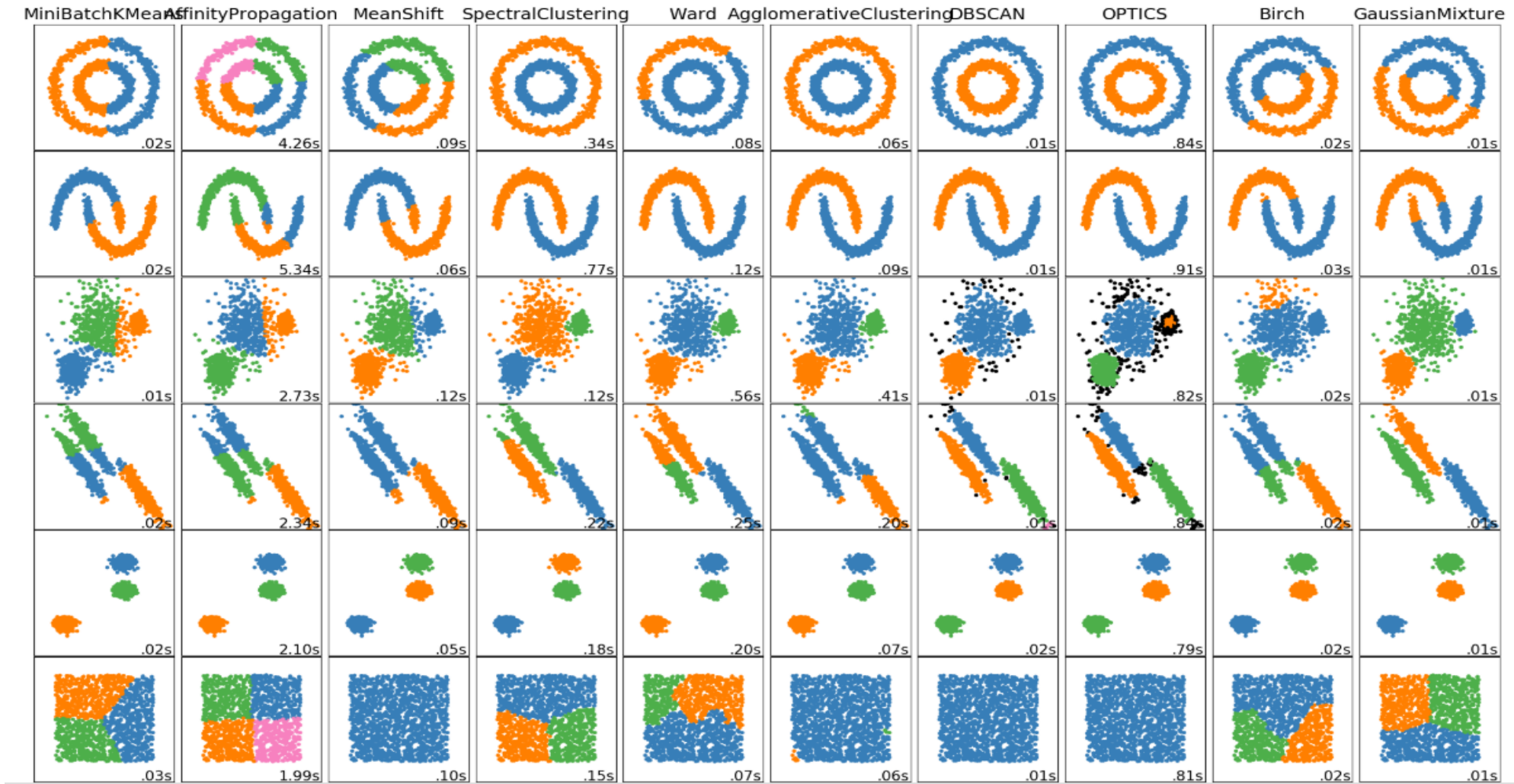
• K-means

- ① 초기화(Initialization): 데이터 집합에서 k 개의 초기 중심점을 무작위로 선택합니다. 이는 최초에 각 클러스터들의 중심이 어디인지 파악하지 못하기 때문입니다.
- ② 클러스터 할당(Cluster Assignment): 중심에 가장 가까운 모든 데이터 요소가 클러스터를 구성합니다. 데이터 포인트와 모든 중심점 사이의 유클리드 거리를 사용하여 두 점 사이의 직선이 그려지며 경계선을 기준으로 두 개의 클러스터로 나뉘어지게 됩니다.
- ③ 중심 이동(Move the centroid): 2번 과정까지 거치고 난 후, 센터가 필요한 새로운 클러스터가 남았다면 인접한 중심점과의 새로운 경계를 생성하여 새로운 클러스터를 생성합니다.
- ④ 2와 3의 과정을 더 이상 중심점이 이동하지 않을 때까지 반복적으로 수행합니다.



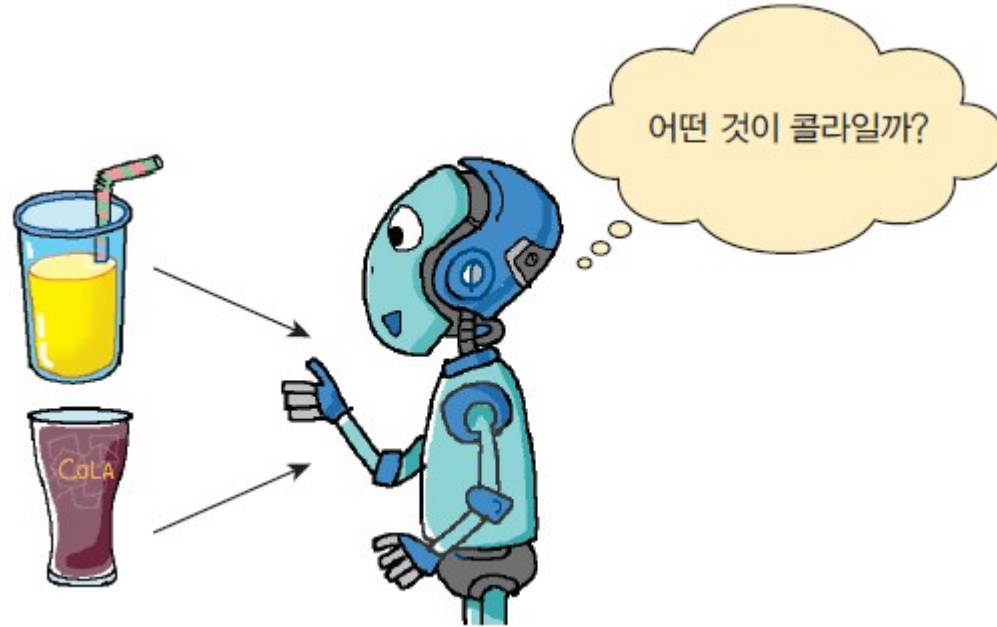
- 비지도학습모델

K-means외에도 DBSCAN, Hierarchical clustering, Spectral Clustering



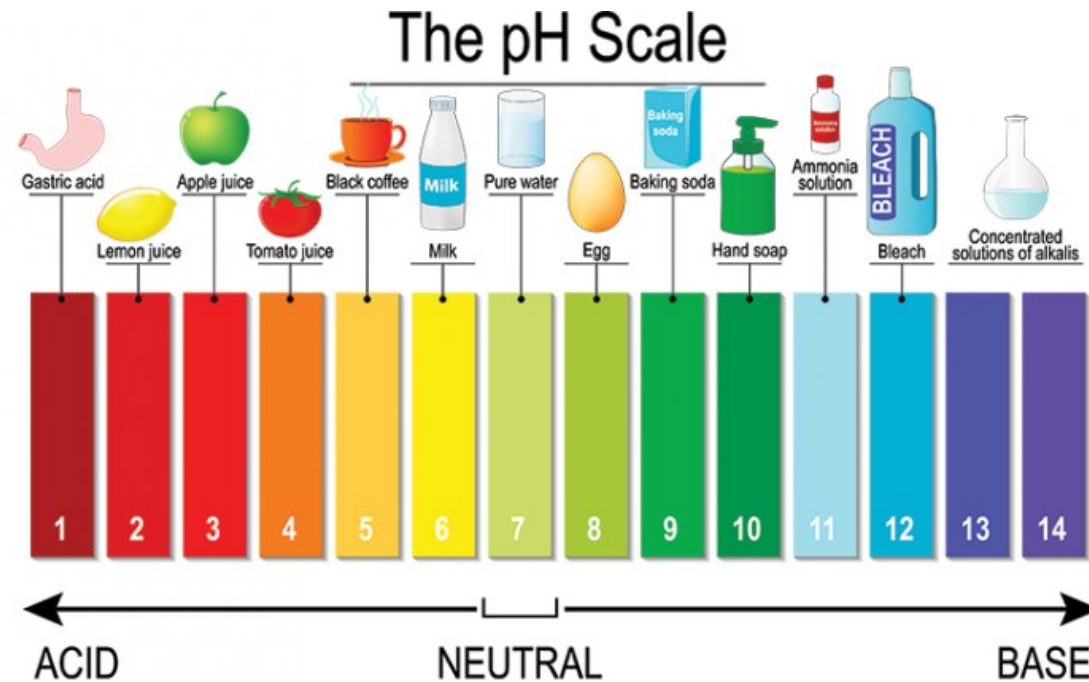
머신러닝의 예

- 우리가 음료수가 콜라인지 주스인지를 판별하는 시스템을 만들어 달라는 요청을 받았다고 가정하자. 머신러닝에서는 이러한 시스템을 "모델(model)"이라고 하며, 이 모델은 "학습(train)"이라는 과정을 통해 생성된다.



어떤 특징?

- 우리는 색상(파장)과 산성도(실수)의 두 가지를 선택하도록 하자. 우리는 이 두 가지 요소만으로 구별할 수 있기를 희망한다. 이것을 "특징(feature)"라고 부른다.



데이터 수집

색상(nm)	산성도(pH)	라벨
610	3.8	오렌지주스
380	2.5	콜라
390	2.6	콜라
...	...	...

훈련 데이터의 구조

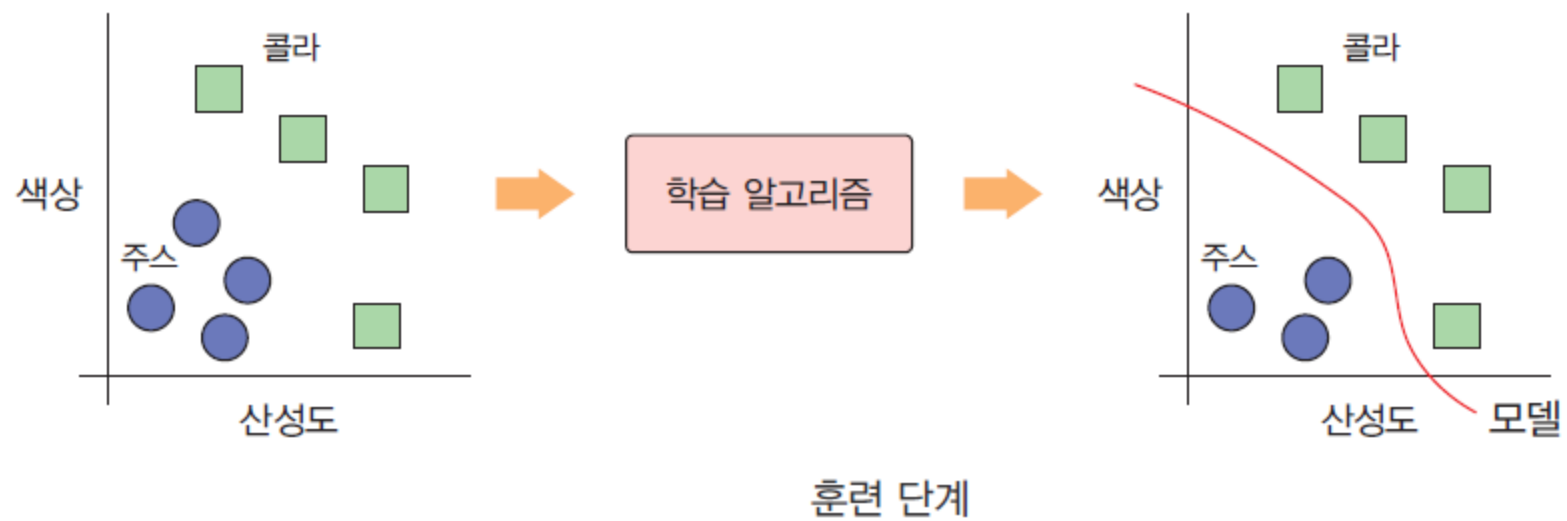
특징 #1	특징 #2	특징 #3	...	특징 #n	
x_1	x_2	x_3	...	x_n	샘플 #1
x_1	x_2	x_3	...	x_n	샘플 #2
x_1	x_2	x_3	...	x_n	샘플 #3
...					
x_1	x_2	x_3	...	x_n	샘플 #k

학습 데이터와 테스트 데이터

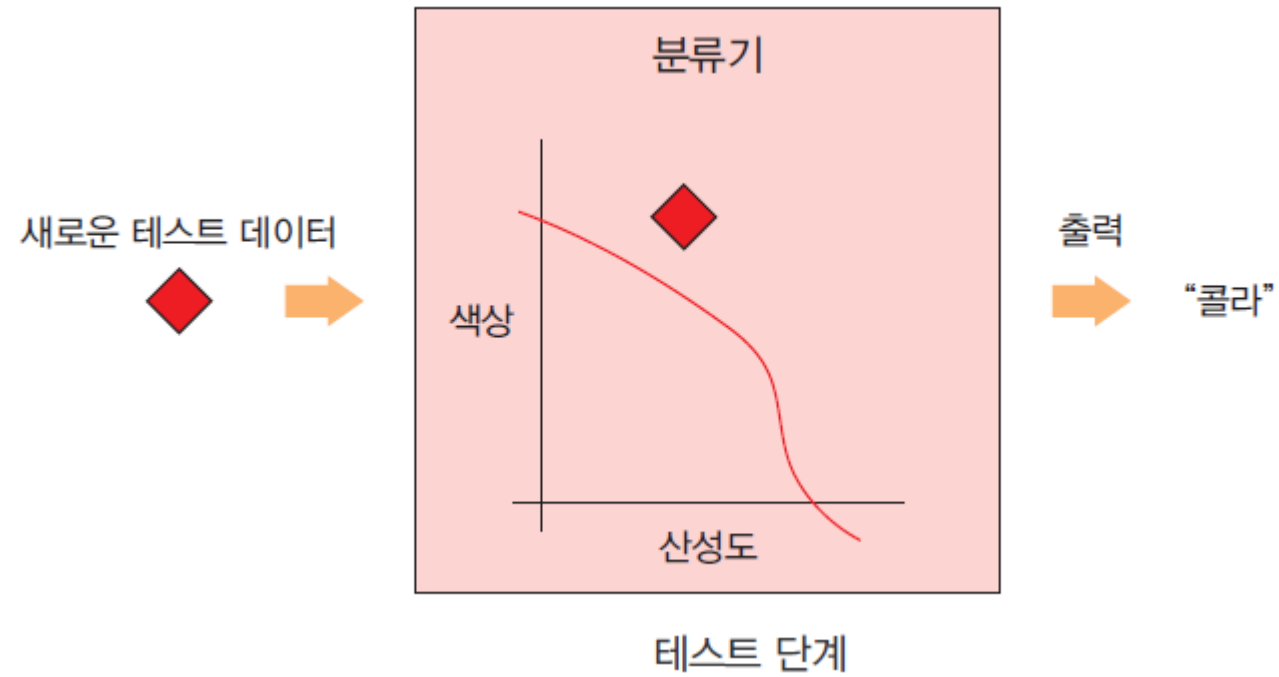
- 머신러닝에는 항상 훈련 데이터와 테스트 데이터가 있어야 한다.



훈련 단계

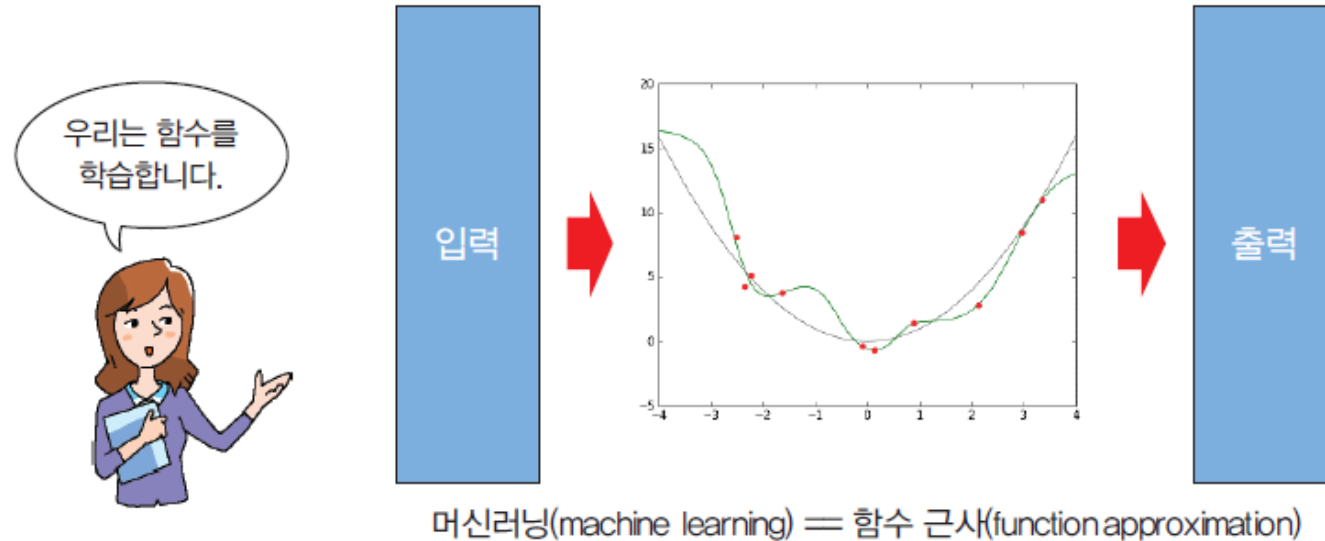


테스트 단계

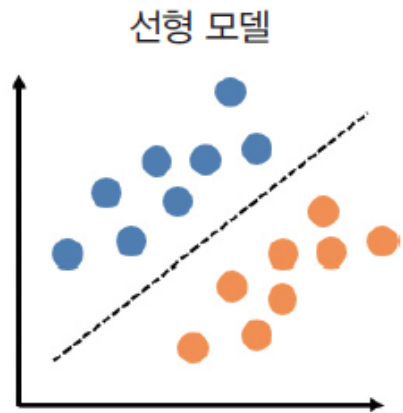


모델 선택

- 다음 단계는 머신러닝 모델을 선택하는 것이다.
- 많은 연구자들이 수년에 걸쳐 만든 많은 모델이 있다.
- 우리의 경우에는 색상과 산성도라는 두 가지 특성만 있으므로 매우 간단한 선형 모델을 사용하자.



선형 모델



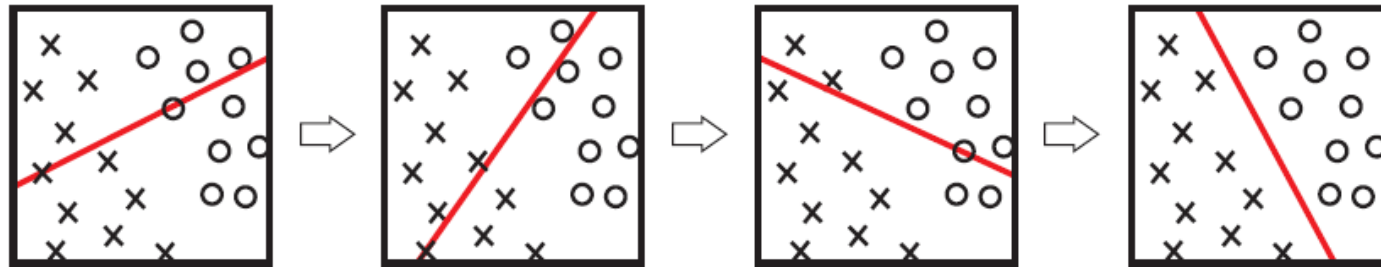
$$y = m * x + b$$

출력 기울기 입력 y-절편

학습 과정

학습

- 맨 처음에는 W 와 b 를 임의의 값을 초기화하고 입력값으로 출력을 예측해본다.
- 이 출력값을 정확한 값과 비교하여 더 정확한 예측을 갖도록 W 및 b 의 값을 조정

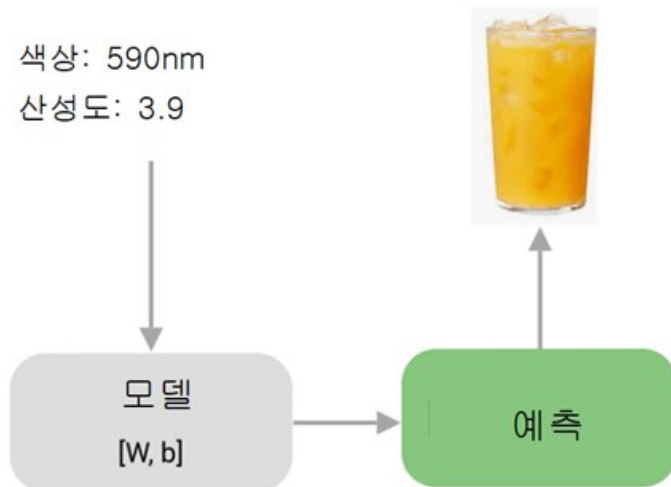


평가

- 학습이 완료되면 모델을 평가하여 모델이 좋은지 나쁜지를 확인해야 한다.
- 테스트 데이터 사용
- 일반적인 훈련 데이터와 테스트 데이터의 비율은 80:20 또는 70:30이다.

예측

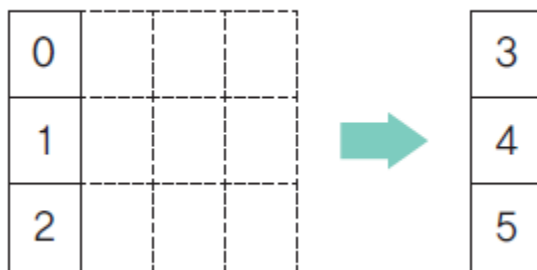
- 이 단계에서 머신러닝 시스템은 우리의 질문에 답한다.
- 예를 들어서 색상이 600nm이고 산성도가 1.5인 음료가 무엇인지를 머신러닝 시스템에 물어볼 수 있다.
- 머신러닝 시스템은 훈련된 대로 색상과 산성도를 고려하여 주어진 음료가 콜라인지 주스인지 예측할 수 있다.



학습 데이터 만들기

`pip install scikit-learn`

- 학습 데이터는 반드시 2차원 배열이어야 한다
(한 열만 있어도 반드시 2차원 배열 형태로 만들어야 한다). 따라서
리스트의 리스트를 만들어서 다음과 같은 2차원 배열을 생성한다.



선형 회귀 예제

```
import matplotlib.pyplot as plt
from sklearn import linear_model

# 선형 회귀 모델을 생성한다.
reg = linear_model.LinearRegression()

# 데이터는 파이썬의 리스트로 만들어도 되고 아니면 넘파이의 배열로 만들어도 됨
X = [[0], [1], [2]]          # 2차원으로 만들어야 함

y = [3, 4, 5]                #  $y = x + 3$ 

# 학습을 시킨다.
reg.fit(X, y)
```

선형 회귀 예제

```
>>> reg.coef_          # 직선의 기울기
array([1.])

>>> reg.intercept_     # 직선의 y-절편
3.0

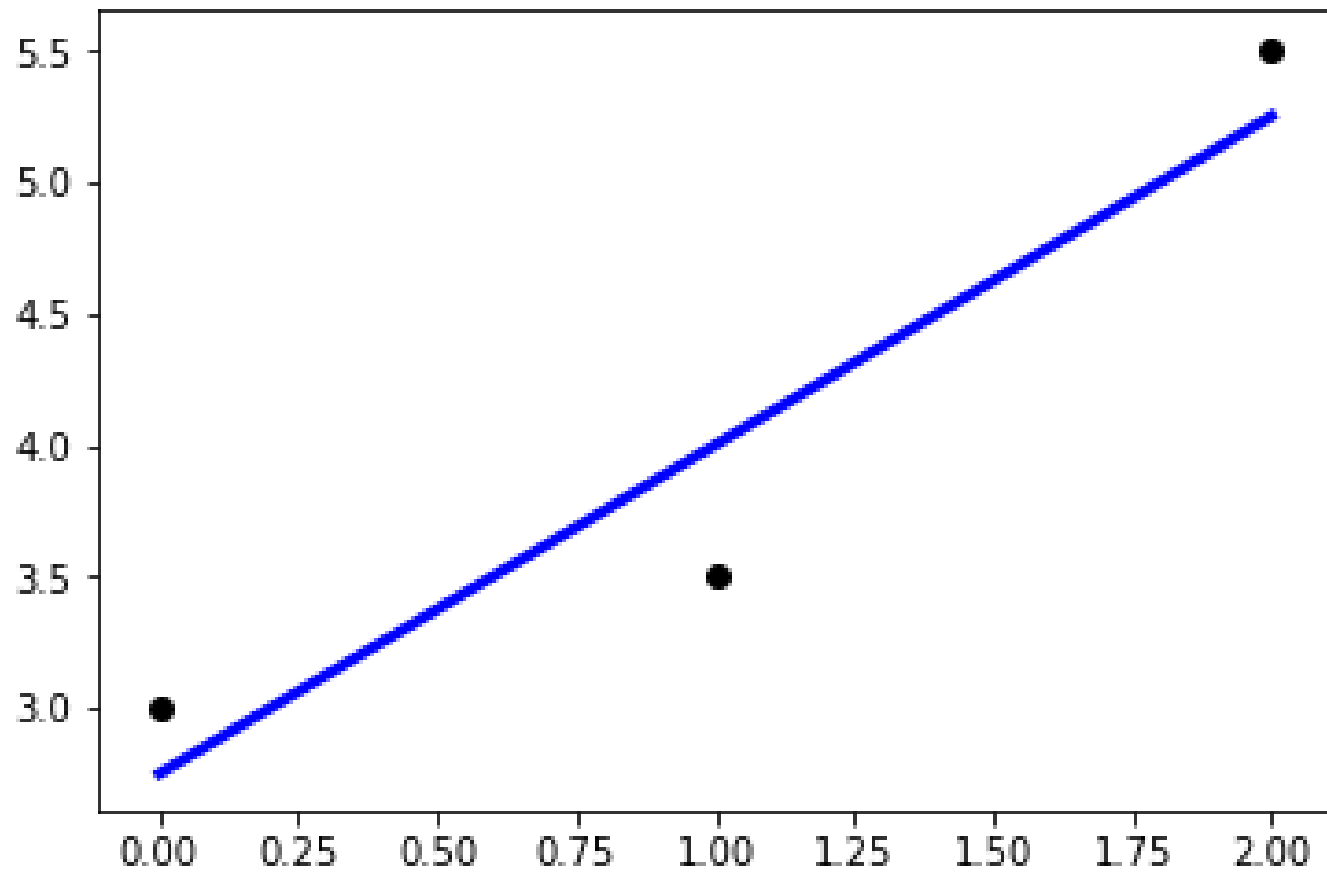
>>> reg.score(X, y)
1.0

>>> reg.predict([[5]])
array([8.])
```

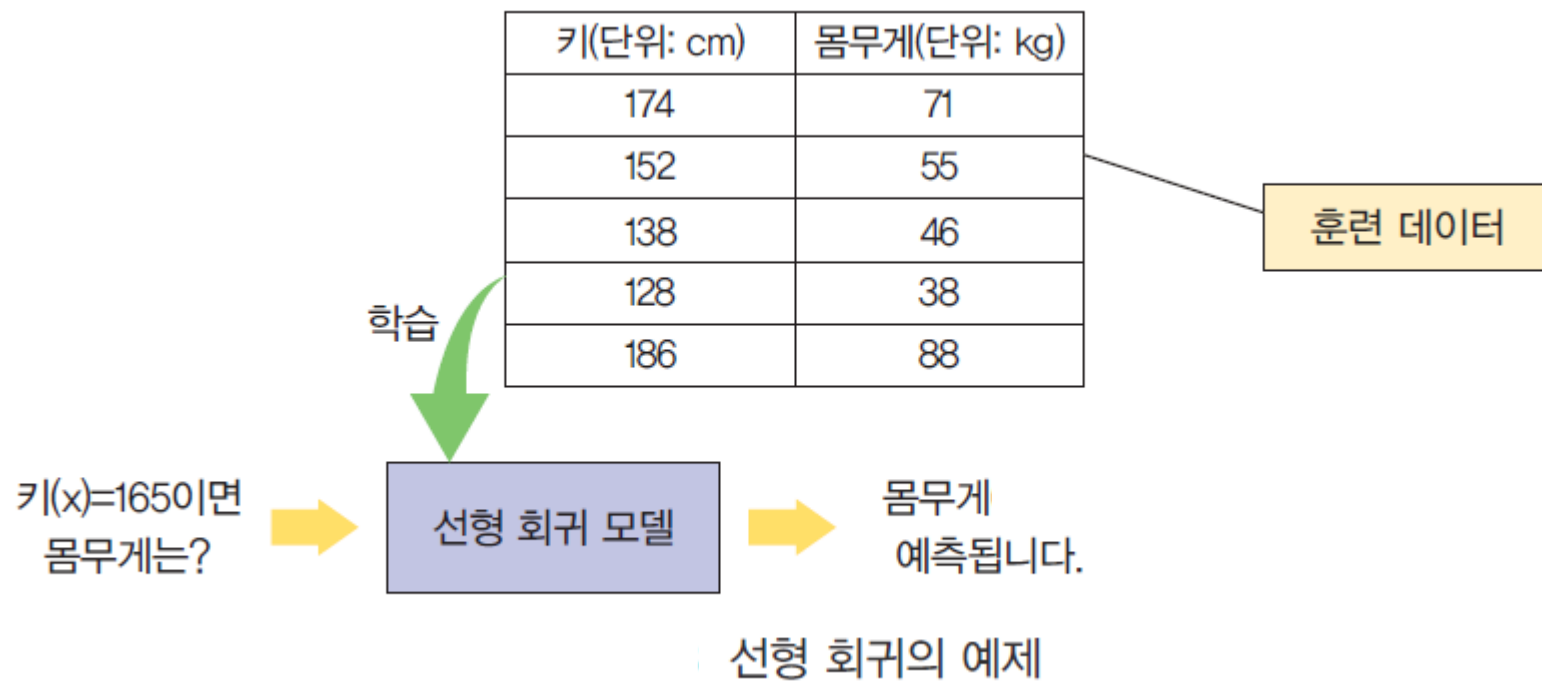
그래프를 그려보자.

```
# 학습 데이터와 y 값을 산포도로 그린다.  
plt.scatter(X, y, color='black')  
  
# 학습 데이터를 입력으로 하여 예측값을 계산한다.  
y_pred = reg.predict(X)  
  
# 학습 데이터와 예측값으로 선그래프로 그린다.  
# 계산된 기울기와 y 절편을 가지는 직선이 그려진다.  
plt.plot(X, y_pred, color='blue', linewidth=3)  
  
plt.show()
```

실행 결과

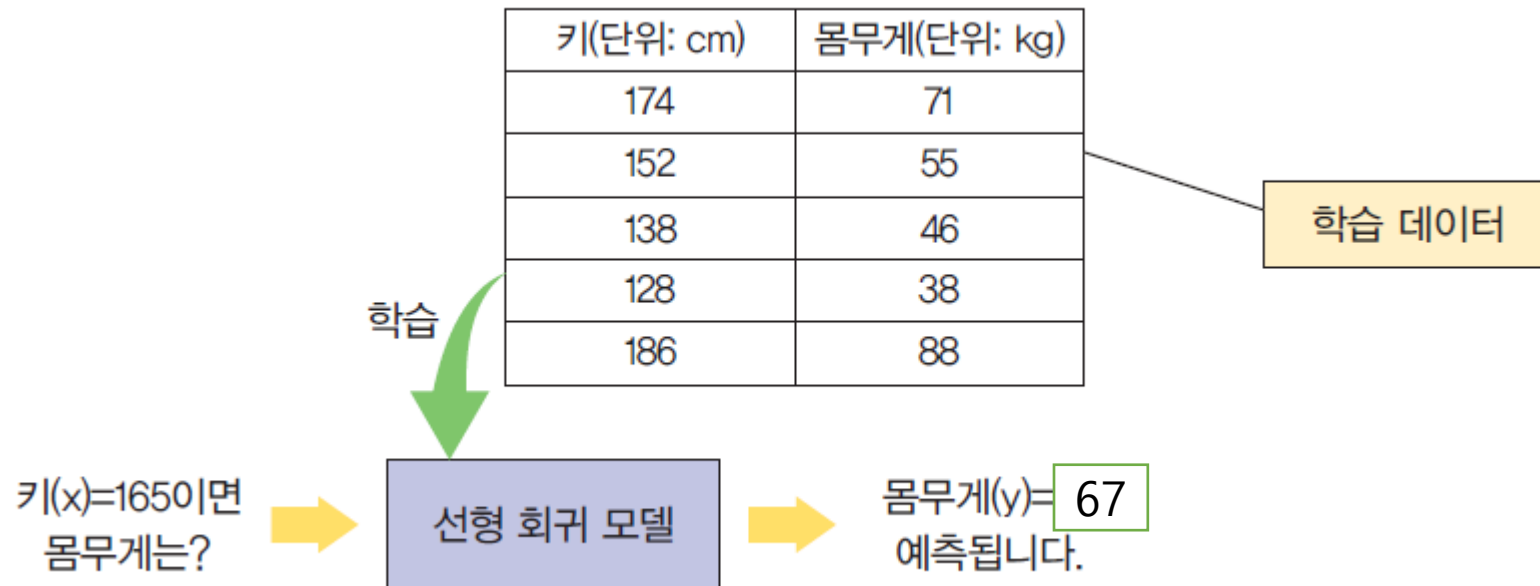


선형 회귀 예제



선형 회귀 실습

- 인간의 **키**와 **몸무게**는 어느 정도 비례할 것으로 예상된다. 아래와 같은 데이터가 있을 때, 선형 회귀를 이용하여 학습시키고 키가 165cm일 때의 예측값을 얻어보자. 파이썬으로 구현하여 본다.



선형 회귀 실습

```
import matplotlib.pyplot as plt
from sklearn import linear_model

reg = linear_model.LinearRegression()

X = [[174], [152], [138], [128], [186]]
y = [71, 55, 46, 38, 88]

reg.fit(X, y)                                # 학습

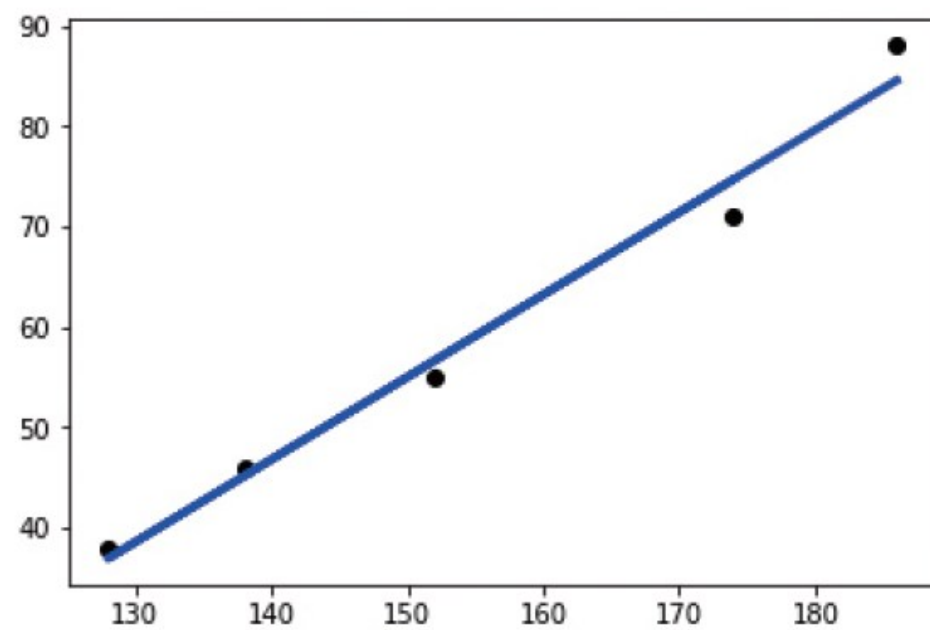
print(reg.predict([[165]]))

# 학습 데이터와 y 값을 산포도로 그린다.
plt.scatter(X, y, color='black')

# 학습 데이터를 입력으로 하여 예측값을 계산한다.
y_pred = reg.predict(X)

# 학습 데이터와 예측값으로 선그래프로 그린다.
# 계산된 기울기와 y 절편을 가지는 직선이 그려진다.
plt.plot(X, y_pred, color='blue', linewidth=3)
plt.show()
```

[67.30998637]



잘 알려진 데이터 세트

데이터 세트 이름	함수
보스톤 지역의 집값	<code>load_boston(*[, return_X_y])</code>
붓꽃 데이터	<code>load_iris(*[, return_X_y, as_frame])</code>
당뇨병 데이터	<code>load_diabetes(*[, return_X_y, as_frame])</code>
숫자 이미지 데이터	<code>load_digits(*[, n_class, return_X_y, as_frame])</code>
운동 데이터	<code>load_linnerud(*[, return_X_y, as_frame])</code>
와인 데이터	<code>load_wine(*[, return_X_y, as_frame])</code>
유방암 데이터	<code>load_breast_cancer(*[, return_X_y, as_frame])</code>

당뇨병 예제 (BMI와 혈당치의 상관관계)

- 훈련데이터 : 환자들의 나이, 성별, BMI, 혈압정보들이 있음.



선형 회귀 예제

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn import datasets

# 당뇨병 데이터 세트를 적재한다.
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)

# 하나의 특징(BMI)만 추려내서 2차원 배열로 만든다. BMI 특징의 인덱스가 2이다.
diabetes_X_new = diabetes_X[:, np.newaxis, 2]

# 학습 데이터와 테스트 데이터를 분리한다.
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(diabetes_X_new, diabetes_y,
test_size=0.1, random_state=0)
```


선형 회귀 예제

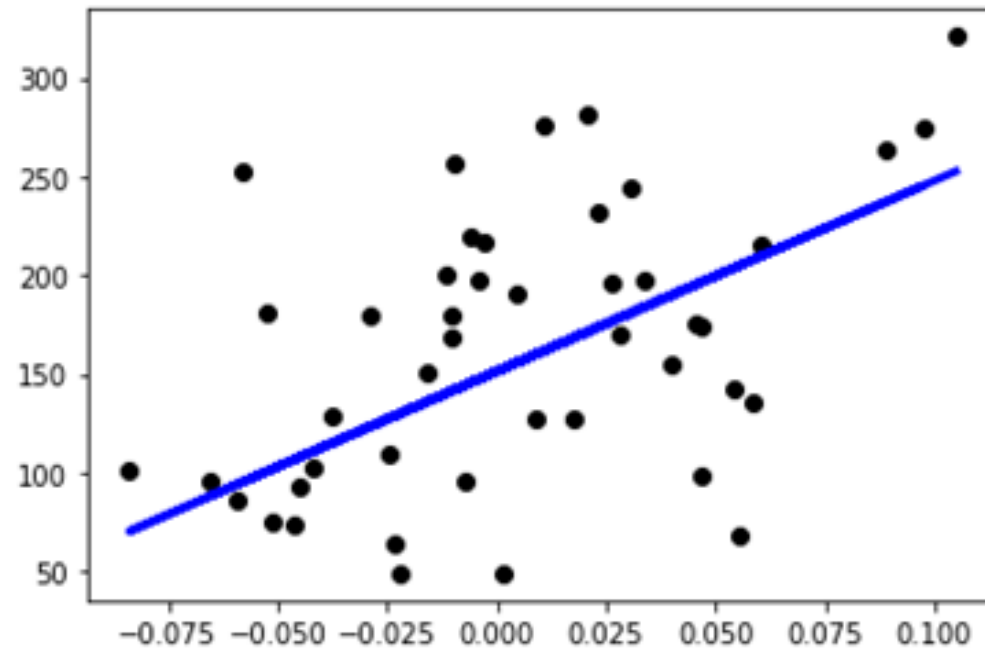
```
regr = linear_model.LinearRegression()
regr.fit(X_train, y_train)

# 테스트 데이터로 예측해보자.
y_pred = model.predict(X_test)

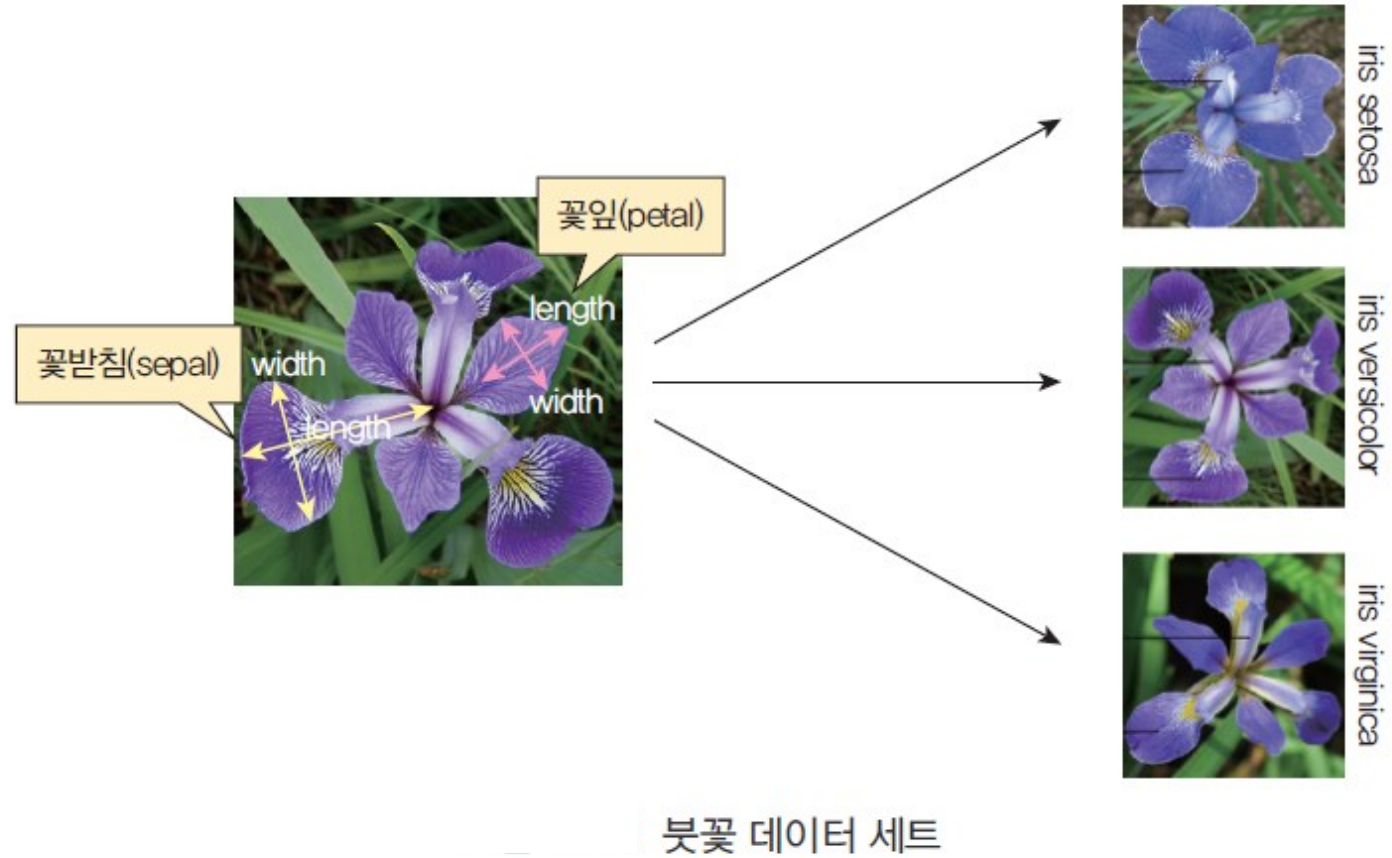
# 실제 데이터와 예측 데이터를 비교해보자.
plt.plot(y_test, y_pred, '.')

plt.scatter(X_test, y_test, color='black')
plt.plot(X_test, y_pred, color='blue', linewidth=3)
plt.show()
```

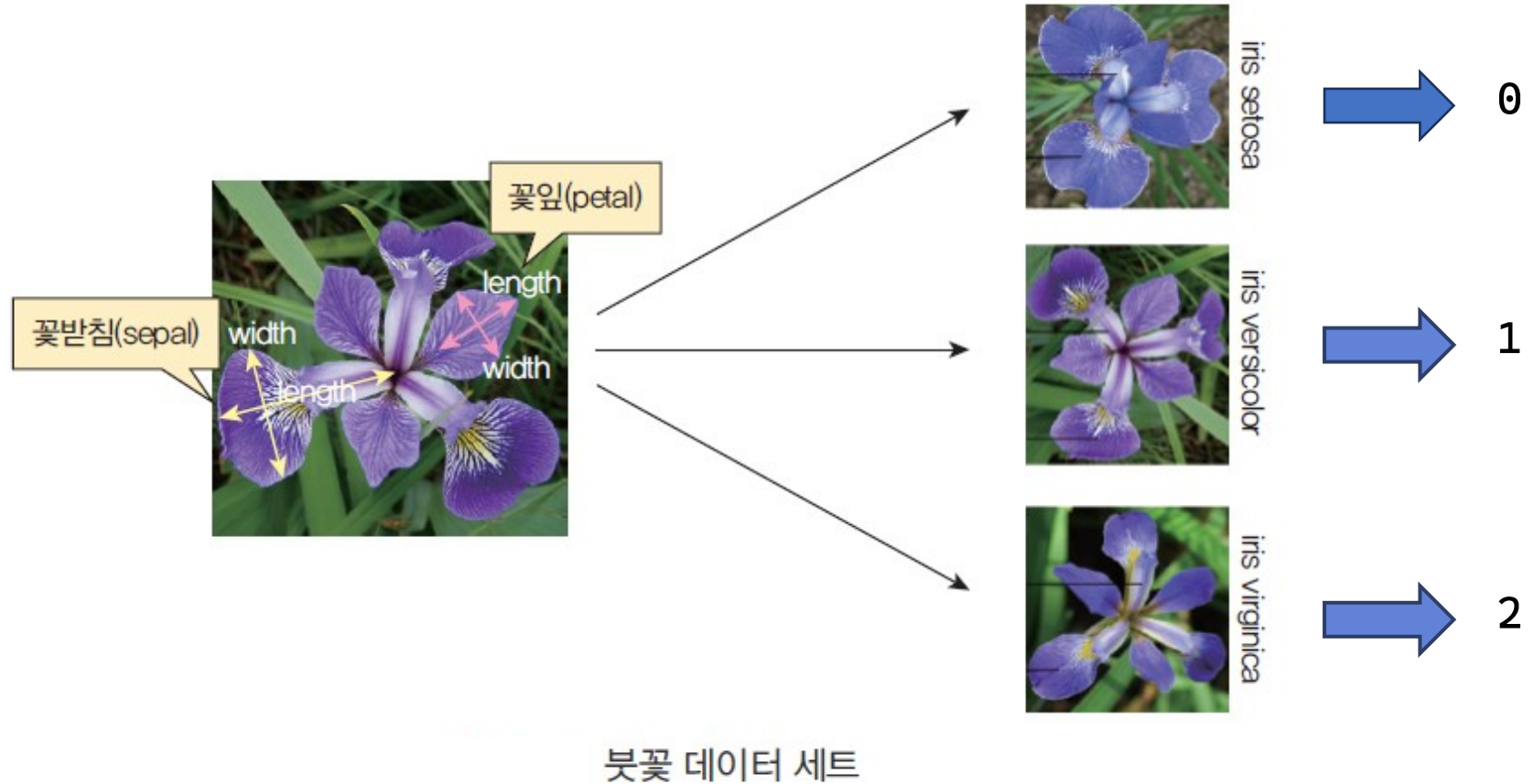
실행 결과



붓꽃 데이터 세트

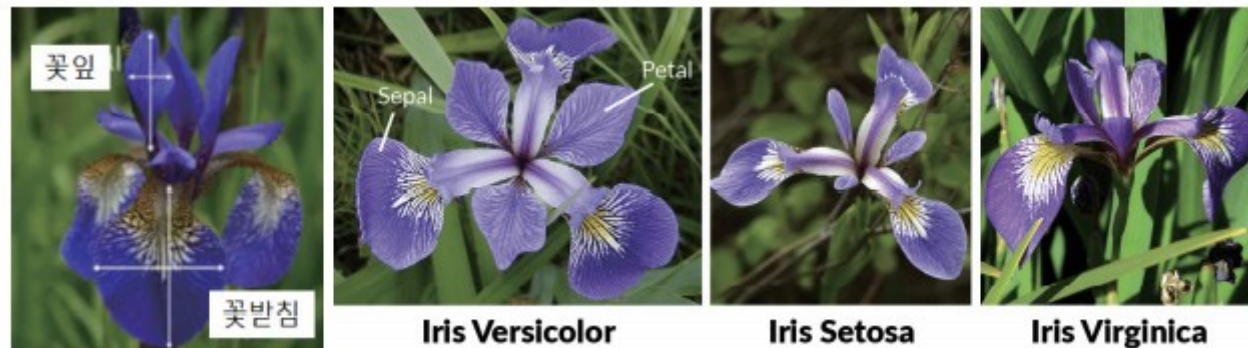


붓꽃(Iris)종류와 데이터 세트



k-NN 활용 예제 - 붓꽃 데이터 준비하기

- **붓꽃**^{iris}는 관상용으로도 재배되는 아름다운 꽃이며 크기와 색상이 다른 여러 종이 있음
- 예제에서는 사이킷런 라이브러리에서 제공하는 붓꽃 데이터 세트를 사용해 볼 예정



붓꽃 데이터 세트

순번	sepal length (꽃받침 길이)	sepal width (꽃받침 너비)	petal length (꽃잎 길이)	petal width (꽃잎 너비)	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.3	0.2	0
2	4.7	3.2	1.3	0.2	0
...					
					1
					2
149					

data

target

특징과 레이블의 구조

붓꽃 (Iris) 을 머신러닝 해보기

- 현재 많은 머신러닝 교육분야에서 유명한 scikit-learn 라이브러리를 이용하기
- 딥러닝 학습에서 많이 사용되는 검증된 데이터와 라이브러리를 가지고 있음.
- 논문자료등에서 많이 인용되고 있음.
- 데이터셋은 세가지 붓꽃종류(Iris setosa, Iris virginica, Iris versicolor) 150개 샘플
- 각 샘플에서 꽃받침(sepal) 길이(length)와 너비(width), 꽃잎(petal) 길이(length)와 너비(width) 4가지의 특성으로 분류된다.

붓꽃 데이터 세트

```
from sklearn import datasets
iris = datasets.load_iris()
print(iris)
```

[illegible]

훈련 데이터와 테스트 데이터 분리

```
from sklearn.model_selection import train_test_split

X = iris.data
y = iris.target

# (80:20)으로 분할한다.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4)

print(X_train.shape)
print(X_test.shape)
```

```
(120, 4)
(30, 4)
```

학습

```
from sklearn.neighbors import KNeighborsClassifier  
  
knn = KNeighborsClassifier(n_neighbors=6)  
knn.fit(X_train, y_train)
```

평가

```
y_pred = knn.predict(X_test)
from sklearn import metrics

scores = metrics.accuracy_score(y_test, y_pred)
```

```
0.9666666666666667
```

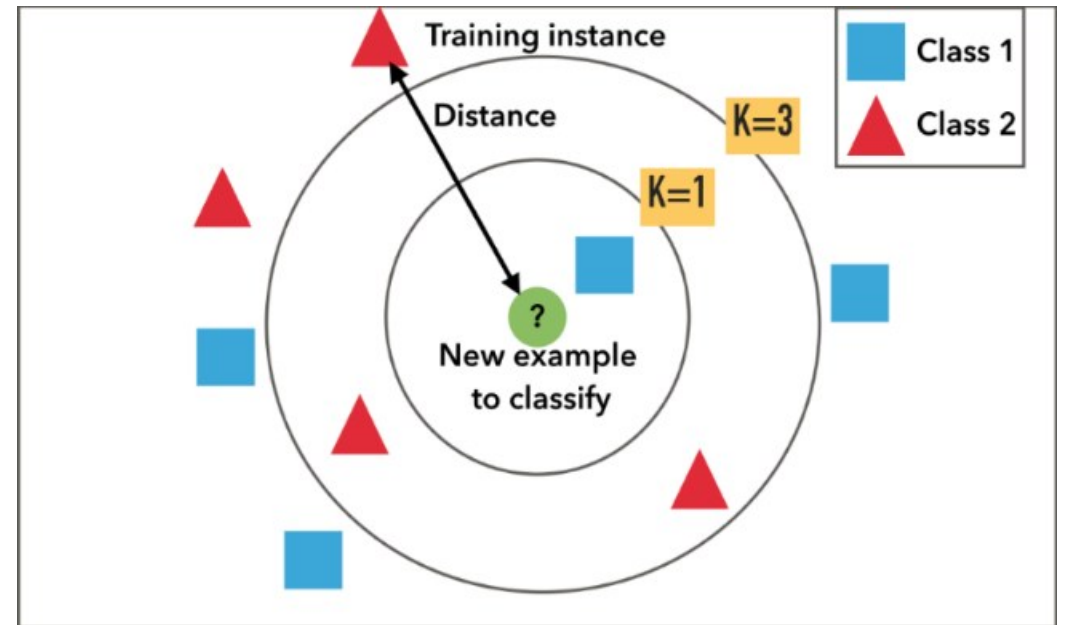
예측

```
classes = {0:'setosa',1:'versicolor',2:'virginica'}  
  
# 전혀 보지 못한 새로운 데이터를 제시해보자.  
x_new = [[3,4,5,2], [5,4,2,2]]  
  
y_predict = knn.predict(x_new)  
  
print(classes[y_predict[0]])  
print(classes[y_predict[1]])
```

```
versicolor  
setosa
```

• 지도학습

- ◆ K-최근접 이웃(K-Nearest Neighbor, K-NN)은 새로운 데이터가 주어졌을 때 기존 데이터 가운데 가장 가까운 K개 이웃의 정보로 새로운 데이터를 예측하는 모델입니다.
- ◆ 분류하고자 하는 데이터는 연두색으로 아직 미정입니다. 이 데이터는 파랑 사각형과 빨강 삼각형 중 어디로 분류될까요?
- ◆ 자신과 가장 가까운 거리에 있는 것으로 분류를 하면 파랑 사각형으로 분류 될 것이고 거리를 $K=3$ 으로 정한다면 $K=3$ 이내에 가장 많은 있는 빨강 삼각형으로 분류 될 것입니다. 만약 $K=3$ 거리에 동일한 숫자의 사각형, 삼각형이 있다면, 거리를 $K=2.5$ 로 다시 계산하면 둘 중 하나가 더 많아 질것입니다

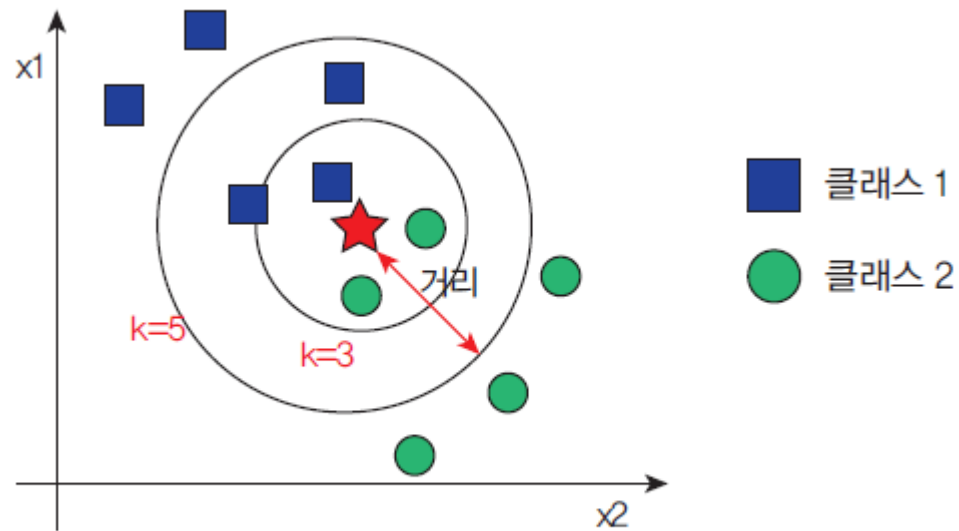


• K-NN

- ◆ K-NN은 특별한 학습이라고 할 만한 절차가 없습니다. 새로운 데이터가 들어왔을 때, 기존 데이터 사이 거리를 계산해 이웃들을 선택하기 때문입니다. K-NN은 모델을 별도로 구축하지 않는다는 뜻으로 게으른 모델(Lazy model)이라고 하기도 하고 **사례중심학습(Instance-based learning)**이라고 하기도 합니다.
- ◆ 데이터로부터 모델을 생성해 주어진 임무를 수행하는 대부분 **머신러닝 모델기반 학습(Model-Based Learning)**과 대비되는 개념으로, 별도 모델 생성과정 없이 각각의 관측치(instance)만을 이용하여 분류/회귀 등을 수행한다는 뜻입니다.
- ◆ K-NN의 하이퍼파라미터 는 탐색할 이웃 수(K), 거리 측정 방법 두 가지뿐입니다. K가 작을 경우 데이터의 지역적 특성을 지나치게 반영하게 되고(Overfitting). 반대로 매우 클 경우 모델이 과하게 정규화 되는 경향이 있습니다(Underfitting). 최근 빅데이터기반 머신 러닝에서는 많이 사용되지는 않는 전통적인 머신러닝 모델입니다.

모델 선택

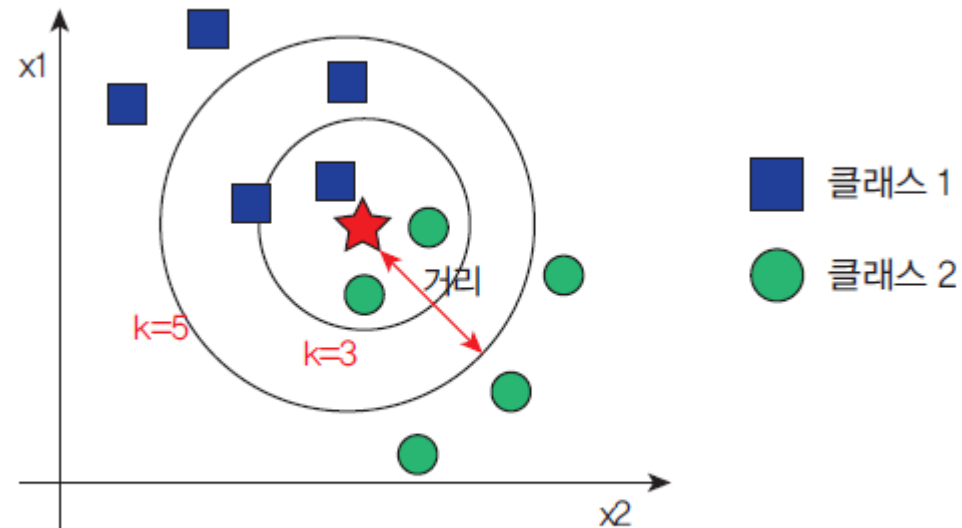
- k-Nearest Neighbor(kNN) 알고리즘은 모든 머신러닝 알고리즘 중에서도 가장 간단하고 이해하기 쉬운 분류 알고리즘이다.
- kNN은 학습 시에 교사가 존재하는 “지도 학습”이다.



kNN 알고리즘에서의 분류

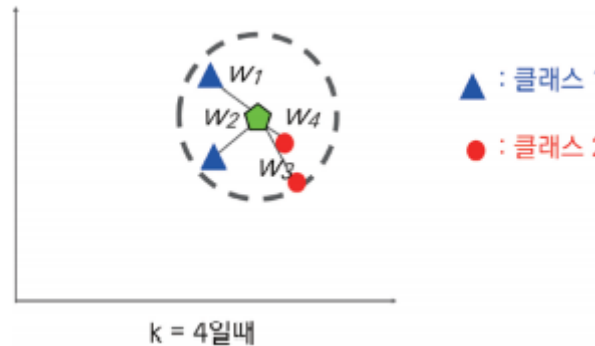
모델 선택

- k-Nearest Neighbor(kNN) 알고리즘은 모든 머신러닝 알고리즘 중에서도 가장 간단하고 이해하기 쉬운 분류 알고리즘이다.
- kNN은 학습 시에 교사가 존재하는 “지도 학습”이다.
- k값을 잘 설정하면 클래스분류에 좋은 성능을 발휘할 수 있다.
- k=5로 설정하면, 가장 가까운 이웃은 파란색 사각형임. 따라서 별표는 파란색 사각형으로 예측됨.



kNN 알고리즘에서의 분류

- k 가 4와 같이 짝수이고 클래스 1과 클래스 2의 개수가 같은 경우에도 새 데이터의 클래스를 판정하는 방법
- 예를 들어 2개의 빨간색 원은 다른 2개의 파란색 삼각형보다 더 가까이 있다는 것을 고려해 클래스 2로 분류하는 것



k-NN 알고리즘과 분류문제

- 선형회귀에 이어 **분류**classification와 **군집화**clustering에 대해서 살펴 보도록 하자.
- 두 가지 방법 모두 데이터를 비슷한 집단으로 묶는다는 점에서는 유사
- **분류는 소속 집단의 정보를 이미 알고 있는 상태에서 비슷한 집단으로 묶는 방법**
- **군집화란 소속 집단의 정보가 없는 상태에서 비슷한 집단으로 묶는 방법**

- k-NN 방법은 특징 공간에 있는 모든 데이터에 대한 정보가 필요
- 데이터 인스턴스, 클래스, 특징의 요소들의 개수가 많다면, 많은 메모리 공간과 계산 시간이 필요하다는 단점
- 알고리즘이 매우 단순하고 직관적이며, 사전 학습이나 특별한 준비 시간이 필요 없다는 점은 장점

- 세 붓꽃 종의 이름은 **Versicolor, Setosa, Virginica**
- 각 종에 따라 꽃받침의 길이와 너비, 꽃잎의 길이와 너비가 약간씩 차이
- 꽃받침과 꽃잎의 크기를 측정한 데이터를 기반으로 새로운 종을 분류하는 k-NN 모델을 구축

```
from sklearn.datasets import load_iris

iris = load_iris()
iris.data[:5]      # 최초 5개 데이터의 값을 출력

array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2]])
```

- 각각의 속성은 **꽃받침 길이** sepal length와 **너비** sepal width 및 **꽃잎 길이** petal length와 **너비** petal width를 cm 단위로 측정한 값

- 넘파이 다차원 배열인 iris.data를 조금 다루기 쉽게 판다스의 데이터프레임으로 만들어서 보도록 하자
- 다섯 번째 속성이 0, 1, 2의 소속 클래스의 값을 갖는 target 속성임을 알 수 있음



```
import pandas as pd
```

```
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)  
iris_df['target'] = pd.Series(iris.target)  
iris_df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

k-NN 활용 예제 - 붓꽃 데이터로 학습하기

- describe() 메소드를 통해서 그 속성을 상세히 살펴보고 다음과 같은 결과를 얻을 수 있음

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

5개의 속성값으로 이루어진
다차원 배열



```
iris_df['target'].value_counts()
```

```
2    50
1    50
0    50
```

```
Name: target, dtype: int64
```

붓꽃 데이터에는 3개의
종에 대하여 각각 50개의
특성값 데이터들이 저장
되어 있어요.



```
iris_df.values
```

```
array([[5.1, 3.5, 1.4, 0.2, 0. ],
       [4.9, 3. , 1.4, 0.2, 0. ],
       [4.7, 3.2, 1.3, 0.2, 0. ],
       ...])
```

새로운 꽃에 대해서 모델을 적용하고 분류해 보자

- 이번에는 훈련 데이터와 검증 데이터를 나누어서 훈련하지 않고 사용가능한 모든 데이터를 사용해서 모델을 학습



```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
k = 3
knn = KNeighborsClassifier(n_neighbors = k)
knn.fit(iris.data, iris.target)
```

- knn.fit()을 통해서 k-NN 분류기 모델을 얻었으며 이 분류기 모델이 전혀 다른 적이 없는 새로운 데이터를 가지고 예측



```
classes = {0:'setosa', 1:'versicolor', 2:'virginica'}
```

```
# 새로운 데이터를 제시해 보자.
```

```
X = [[4, 2, 1.3, 0.4],  
      [4, 3, 3.2, 2.2]]
```

```
y = knn.predict(X)
```

```
print('{} 특성을 가지는 품종: {}'.format(X[0], classes[y[0]]))
```

```
print('{} 특성을 가지는 품종: {}'.format(X[1], classes[y[1]]))
```

```
[4, 2, 1.3, 0.4] 특성을 가지는 품종: setosa
```

```
[4, 3, 3.2, 2.2] 특성을 가지는 품종: versicolor
```


- 테스트 데이터의 규모가 적은 편이므로 전체 데이터에 대해 분류를 실시
- iris.data를 분류기에 넣고, 결과 y_pred_all을 구한 뒤에 정답인 iris.target과 비교할 수 있도록 준비



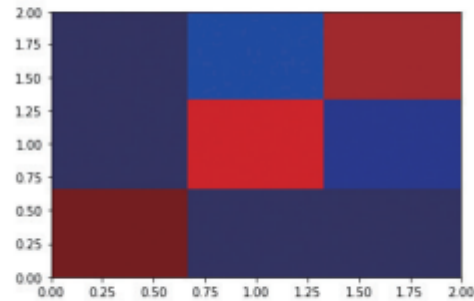
```
y_pred_all = knn.predict(iris.data)
scores = metrics.accuracy_score(iris.target, y_pred_all)
print('n_neighbors가 {0:d}일때 정확도: {1:.3f}'.format(k, scores))
```

```
n_neighbors가 3일때 정확도: 0.960
```

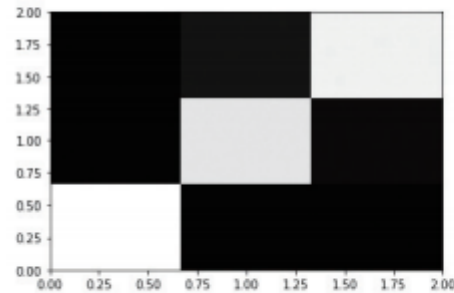
- 4%의 잘못된 분류가 있음을 확인
- 결과에 대해 (정답, 예측)의 쌍으로 데이터를 만들어 2차원 히스토그램



```
import matplotlib.pyplot as plt  
plt.hist2d(iris.target, y_pred_all, bins=(3,3), cmap=plt.cm.jet)
```



cmap = plt.cm.jet



cmap = plt.cm.gray

- 사이킷런 모듈에서는 이러한 시각화 결과를 **혼동 행렬** `confusion matrix`이라는 이름으로 아래와 같이 구할 수 있음



```
from sklearn.metrics import confusion_matrix  
conf_mat = confusion_matrix(iris.target, y_pred_all)  
conf_mat
```

```
array([[50,  0,  0],  
       [ 0, 47,  3],  
       [ 0,  3, 47]])
```

훈련 데이터와 테스트 데이터를 분류해서 훈련하고 훈련 검증하기

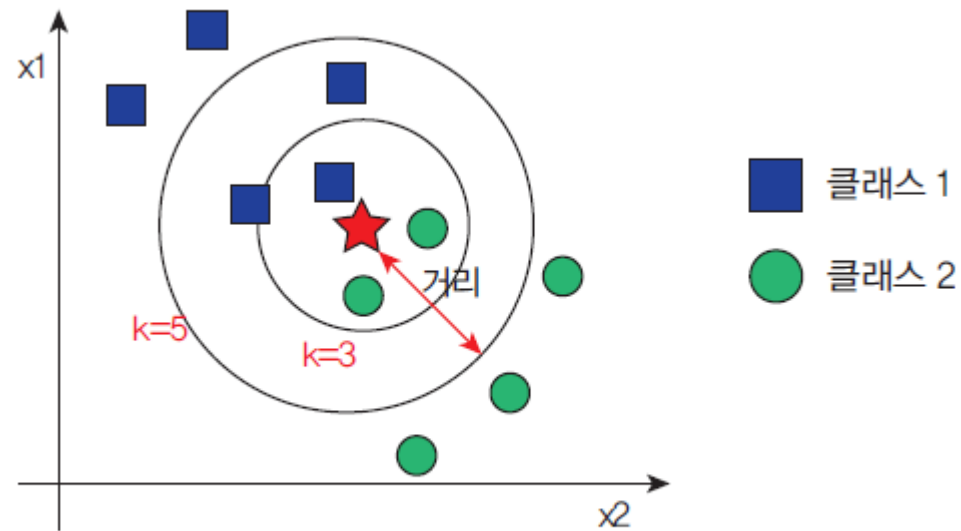
```
from sklearn.datasets import load_iris
iris = load_iris()
data = iris.data
target = iris.target
(X_train, X_test, y_train, y_test) = train_test_split(data, target, test_size=0.2)
```



모델과 학습

```
from sklearn.neighbors import KNeighborsClassifier  
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X_train, y_train)
```



KNN 알고리즘에서의 분류

학습, 평가, 그리고 예측

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=5) # 3, 5, 7, 9  
knn.fit(X_train, y_train)
```

```
y_pred = knn.predict(X_test)  
from sklearn import metrics  
scores = metrics.accuracy_score(y_test, y_pred)
```

```
0.9666666666666667
```

```
classes = {0:'setosa',1:'versicolor',2:'virginica'}  
# 전혀 보지 못한 새로운 데이터를 제시해보자.  
x_new = [[3,4,5,2], [5,4,2,2]]  
y_predict = knn.predict(x_new)  
print(classes[y_predict[0]])  
print(classes[y_predict[1]])
```

```
versicolor  
setosa
```

평가

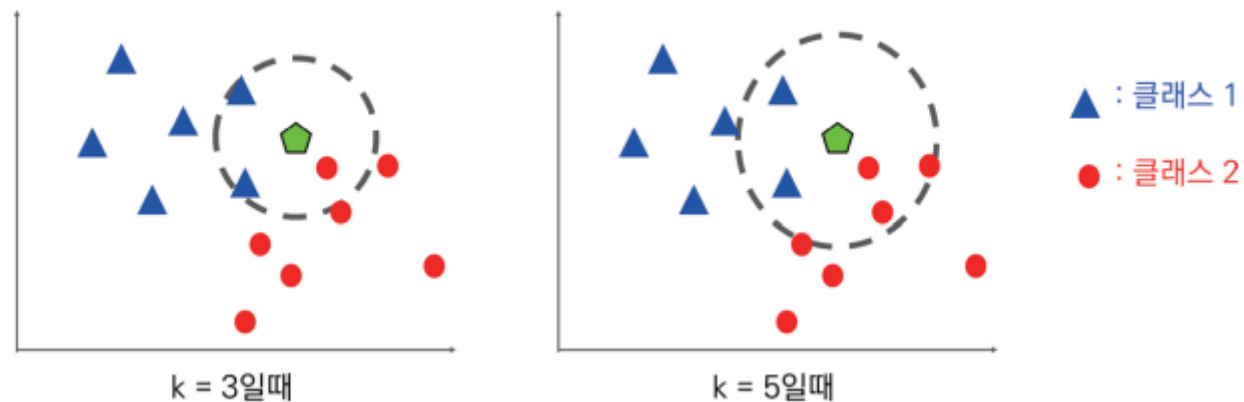
```
# 테스트 데이터로 예측해본다.  
y_pred = knn.predict(X_test)  
# 정확도를 계산한다.  
  
scores = metrics.accuracy_score(y_test, y_pred)  
print(scores)
```

```
0.9532814238042269
```

- 우리 주변에서 볼 수 있는 개의 품종 중에는 **닥스훈트** dachshund와 **사모예드** samoyed라는 종이 있는데, 두 종은 몸의 높이와 길이 비율이 서로 다름
- 일반적으로 닥스훈트는 몸통 길이에 비해서 높이가 낮고, 사모예드의 경우 몸통 길이와 높이가 비슷한 특징을 가짐
- 몸통의 길이와 높이 **특징** feature을 각각 x_1, x_2 라고 두고 이 개들의 표본 집합에 대하여 x_1, x_2 를 측정하면 아래 그림의 오른쪽과 같은 산포도 그래프를 얻을 수 있음

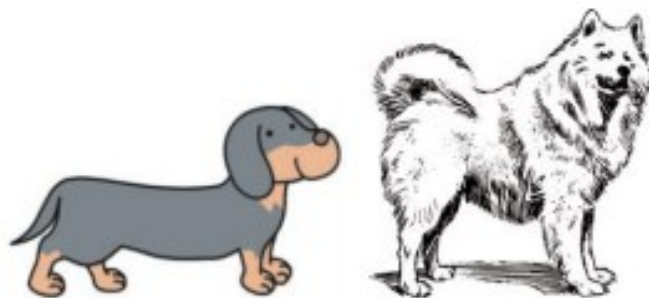


- 산포도 그래프의 위쪽에 분포한 파란색 점은 사모예드 종이고, 아래 쪽의 붉은 점들은 닥스훈트 종
- 데이터를 효과적으로 분류하려면 어떤 기법을 적용하는 것이 바람직 할까?
- k-NN 알고리즘에서 k-NN은 **k-최근접 이웃**(k-Nearest Neighbor)의 약자로 특징 공간에 분포하는 데이터에 대하여 k개의 가장 가까운 이웃을 살펴보고 다수결 방식으로 데이터의 레이블을 할당 하는 분류방식



k-NN 알고리즘을 위한 데이터 준비하기

- 이제 다음과 같이 닥스훈트와 사모예드의 데이터를 바탕으로 k-NN 알고리즘을 익혀보도록 하자



닥스훈트 8마리의 길이와 높이								
길이	77	78	85	83	73	77	73	80
높이	25	28	29	30	21	22	17	35
사모예드 8마리의 길이와 높이								
길이	75	77	86	86	79	83	83	88
높이	56	57	50	53	60	53	49	61



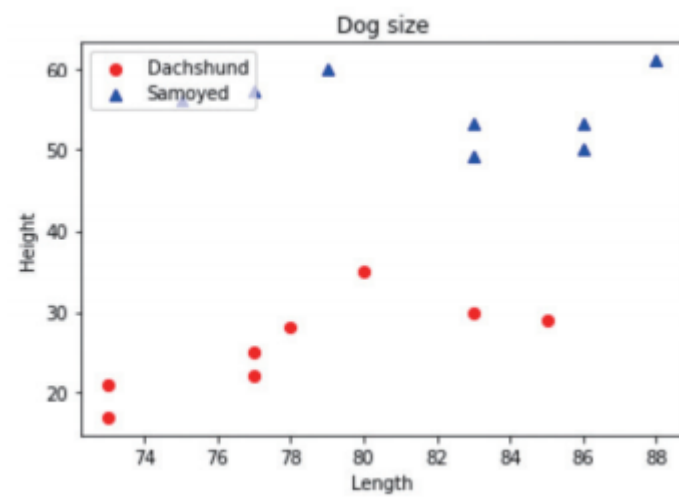
```
import matplotlib.pyplot as plt
import numpy as np

# 닥스훈트의 길이와 높이 데이터
dach_length = [77, 78, 85, 83, 73, 77, 73, 80]
dach_height = [25, 28, 29, 30, 21, 22, 17, 35]
# 사모예드의 길이와 높이 데이터
samo_length = [75, 77, 86, 86, 79, 83, 83, 88]
samo_height = [56, 57, 50, 53, 60, 53, 49, 61]

plt.scatter(dach_length, dach_height, c='red', label='Dachshund')
plt.scatter(samo_length, samo_height, c='blue', marker='^', label='Samoyed')

plt.xlabel('Length')
plt.ylabel('Height')
plt.title("Dog size")
plt.legend(loc='upper left')

plt.show()
```



- 이제 길이와 높이가 각각 79, 35인 새로운 데이터가 들어왔고 이 특징을 가진 개의 종류를 모를 경우, 이 데이터는 닥스훈트와 사모예드 중에서 어떤 개로 분류하는 것이 바람직할지 살펴 보자



...코드 생략

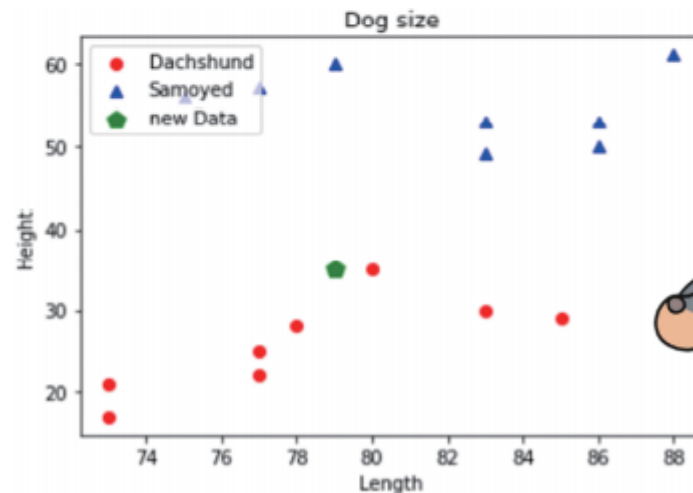
```
newdata_length = [79]    # 새로운 데이터의 길이
```

```
newdata_height = [35]    # 새로운 데이터의 높이
```

...코드 생략

새 데이터의 표식은 오각형(pentagon)으로 설정하고, 레이블은 new Data로

```
plt.scatter(newdata_length, newdata_height, s=100,  
            marker='p', c='green', label='new Data')
```



new Data는 두 클래스 사이에 위치하네요. 하지만 주위에 있는 가까운 점들이 대부분 닥스훈트 클래스예요



k-NN 분류기를 실행하자

- 준비된 데이터에 k-NN 알고리즘을 적용해 보자
- 우선 모델의 훈련을 위해서 다음과 같은 방법으로 닥스훈트의 몸 길이 `dach_length`와 몸 높이 `dach_height` 배열 데이터를 묶어서 `[몸길이1, 몸높이1], [몸 길이2, 몸높이2], ...`와 같은 형태의 항목을 가지는 `d_data` 배열을 만들어 보자
- 다음으로 이 `d_data` 배열의 크기와 같은 0으로 이루어진 배열 `[0, 0, ..]`을 만들어 보자



```
d_data = np.column_stack((dach_length, dach_height))  
d_label = np.zeros(len(d_data)) # 닥스훈트는 0으로 레이블링
```

- 동일한 명령으로 사모예드 데이터 s_data 배열을 만들고 s_data 배열과 크기가 같은 1로 이루어진 s_label 배열 [1, 1, ..]을 만들도록 하자



```
s_data = np.column_stack((samo_length, samo_height))  
s_label = np.ones(len(s_data)) # 사모예드는 1로 레이블링
```

- 이렇게 생성된 데이터와 레이블을 기반으로 k-NN 모델을 만들어 학습을 진행하자
- 각 클래스의 특징을 바탕으로 분류를 실시할 데이터는 newdata라는 이름으로 생성



```
newdata = [[79, 35]]
```

- 아래의 코드에서 dog_classes는 **키**key 값이 0일때 'Dachshund', 1일때 'Samoyed'라는 레이블을 가지도록 생성한 딕셔너리
- 딕셔너리는 fit() 함수를 통해 생성한 모델이 잘 적용되는가를 보기 쉽게 출력하기 위하여 사용



```
dog_classes = {0:'Dachshund', 1:'Samoyed'}
```

```
k = 3      # k를 3으로 두고 kNN 분류기를 만들어 보자
```

```
knn = KNeighborsClassifier(n_neighbors = k)
```

```
knn.fit(dogs, labels)
```

```
y_pred = knn.predict(newdata)
```




```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics

# 닥스훈트의 몸 길이와 몸 높이
dach_length = [77, 78, 85, 83, 73, 77, 73, 80]
dach_height = [25, 28, 19, 30, 21, 22, 17, 35]

# 사모예드의 몸 길이와 몸 높이
samo_length = [75, 77, 86, 86, 79, 83, 83, 88]
samo_height = [56, 57, 50, 53, 60, 53, 49, 61]

d_data = np.column_stack((dach_length, dach_height))
d_label = np.zeros(len(d_data)) # 닥스훈트는 0으로 레이블링
s_data = np.column_stack((samo_length, samo_height))
s_label = np.ones(len(s_data)) # 사모예드는 1로 레이블링

newdata = [[79, 35]]

dogs = np.concatenate((d_data, s_data))
labels = np.concatenate((d_label, s_label))

dog_classes = {0:'Dachshund', 1:'Samoyed'}

k = 3 # k를 3으로 두고 kNN 분류기를 만들어 보자
knn = KNeighborsClassifier(n_neighbors = k)
knn.fit(dogs, labels)
y_pred = knn.predict(newdata)
print('데이터', newdata, ', 판정 결과:', dog_classes[y_pred[0]])
```

데이터 [[79, 35]] , 판정 결과: Dachshund



```
X = iris_df.iloc[:, :4]
y = iris_df.iloc[:, -1]
```

제일 마지막 속성에는
0, 1, 2 중 하나의 레이블
값이 들어 있어요.

- X, y를 훈련 데이터와 검증 데이터로 구분하여 k-NN 알고리즘을 적용시키고 정확도를 측정



```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics

def iris_knn(X, y, k):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
    knn = KNeighborsClassifier(n_neighbors = k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    return metrics.accuracy_score(y_test, y_pred)

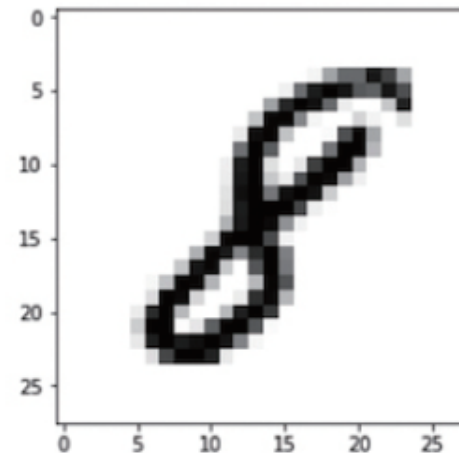
k = 3
scores = iris_knn(X, y, k)
print('n_neighbors가 {0:d}일때 정확도: {1:.3f}'.format(k, scores))
```

n_neighbors가 3일때 정확도: 0.956

k가 3일때 95.6% 정확도로 예측

sklearn의 dataset의 필기체 숫자를 분류

- MNIST가 배포하는 필기체 숫자 이미지
- 우리는 sklearn을 사용하여 필기체 숫자 이미지를 인식하는 프로그램을 작성해보자.

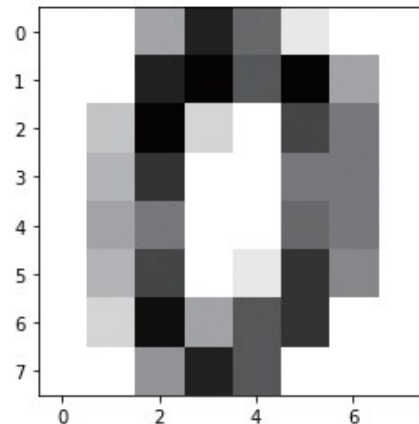


MNIST 데이터

데이터 세트(dataset) 읽기

```
import matplotlib.pyplot as plt
from sklearn import datasets, metrics
from sklearn.model_selection import train_test_split

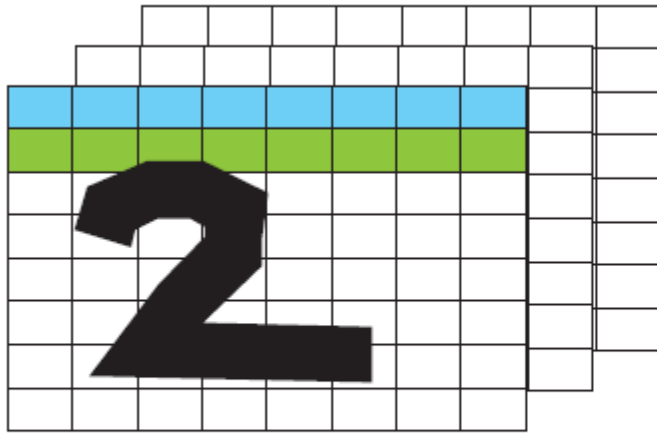
digits = datasets.load_digits()
plt.imshow(digits.images[0], cmap=plt.cm.gray_r, interpolation='nearest')
```



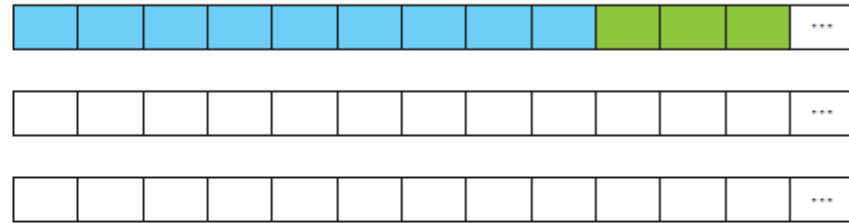
이미지 평탄화

```
n_samples = len(digits.images)
```

```
data = digits.images.reshape((n_samples, -1))
```



$n_samples \times 8 \times 8$



$n_samples \times 64$

평탄화

훈련 데이터와 테스트 데이터

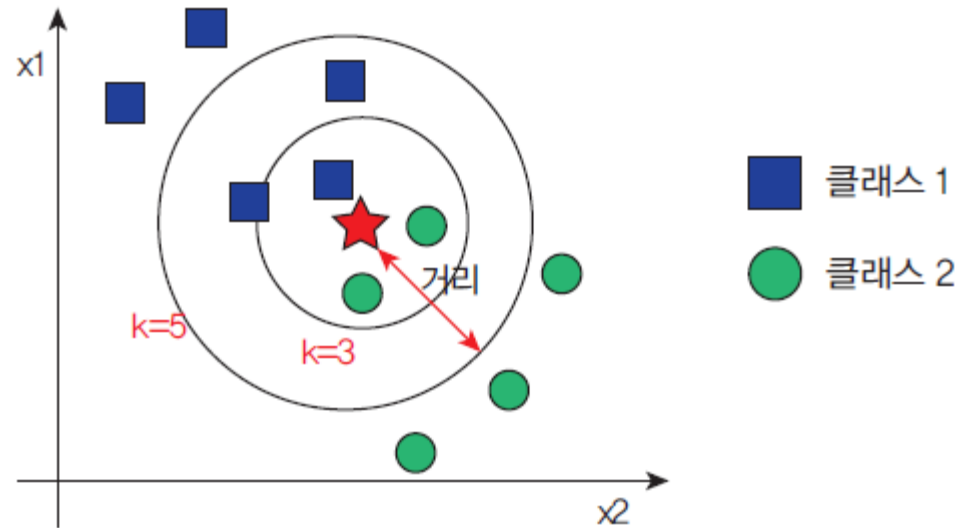
```
X_train, X_test, y_train, y_test = train_test_split(data, digits.target, test_size=0.2)
```



모델과 학습 (Kneighbors Classifier)

```
from sklearn.neighbors import KNeighborsClassifier  
clf = KNeighborsClassifier(n_neighbors=5)
```

```
clf.fit(X_train, y_train)
```



KNN 알고리즘에서의 분류

예측 (score)

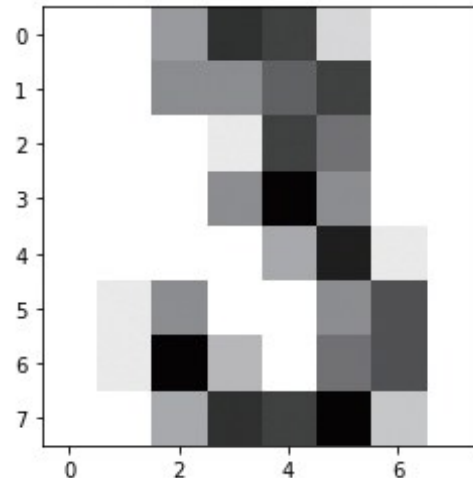
```
# 테스트 데이터로 예측해본다.  
y_predict = clf.predict(X_test)  
# 정확도를 계산한다.  
  
score = metrics.accuracy_score(y_test, y_predict)  
print(score)
```

```
0.9532814238042269
```


예측 (predict)

```
# 이미지를 출력하기 위하여 평탄화된 이미지를 다시 8×8 형상으로 만든다.  
plt.imshow(X_test[10].reshape(8,8), cmap=plt.cm.gray_r, interpolation='nearest')  
  
y_pred = knn.predict([X_test[10]]) # 입력은 항상 2차원 행렬이어야 한다.  
print(y_pred)
```

[3]



머신러닝 알고리즘의 성능평가

- 정확도

$$\text{정확도(accuracy)} = \frac{\text{올바르게 분류한 샘플 수}}{\text{전체 샘플 수}}$$

- 혼동행렬

		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

혼동 행렬

- 긍정을 긍정으로 올바르게 예측하면 TP(True Positive)라고 한다. 앞에 True가 붙으면 예측과 실재가 같다는 의미이다.
- 긍정을 부정으로 잘못 예측하면 FN(False Negative)라고 한다. 앞에 False가 붙으면 예측과 실재가 틀리다는 의미이다.
- 부정을 긍정으로 잘못 예측하면 FP(False Positive)라고 한다.
- 부정을 부정으로 올바르게 예측하면 TN(True Negative)라고 한다.

$$\text{민감도} = \frac{TP}{TP + FN}, \quad \text{특이도} = \frac{TN}{TN + FP}$$

혼동 행렬 출력

```
import matplotlib.pyplot as plt

from sklearn import datasets, metrics
from sklearn.model_selection import train_test_split

digits = datasets.load_digits()
n_samples = len(digits.images)
data = digits.images.reshape((n_samples, -1))

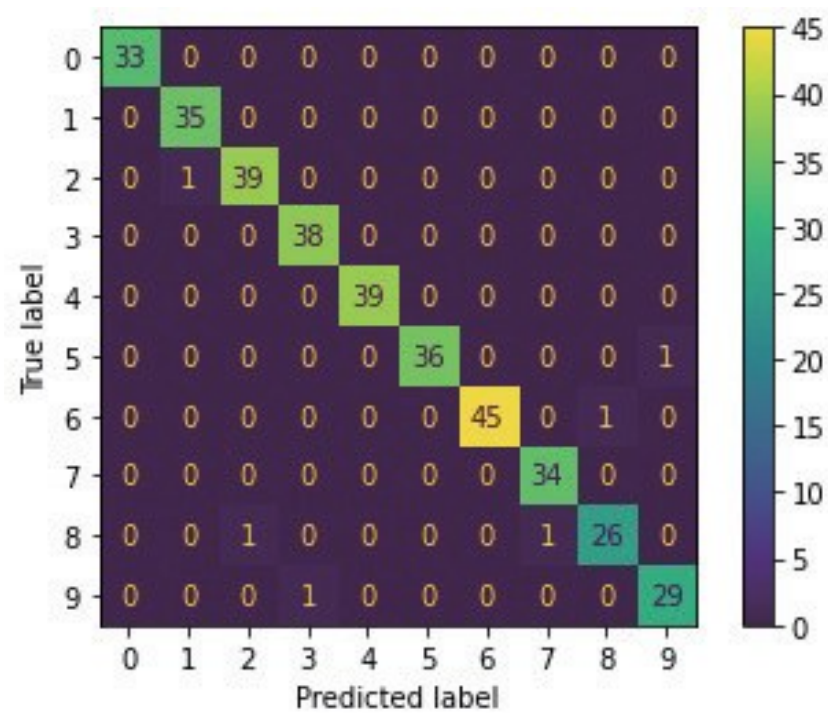
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)

(X_train, X_test, y_train, y_test) = train_test_split(data, digits.target, test_size=0.2)

knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

display_confusion_matrix = metrics.plot_confusion_matrix(knn, X_test, y_test)
print(display_confusion_matrix)
plt.show()
```

혼동 행렬 출력



분류 리포트

- 사이킷런에서는 분류 리포트를 생성하는 기능이 있다. 대표적인 성능 척도들을 계산해준다

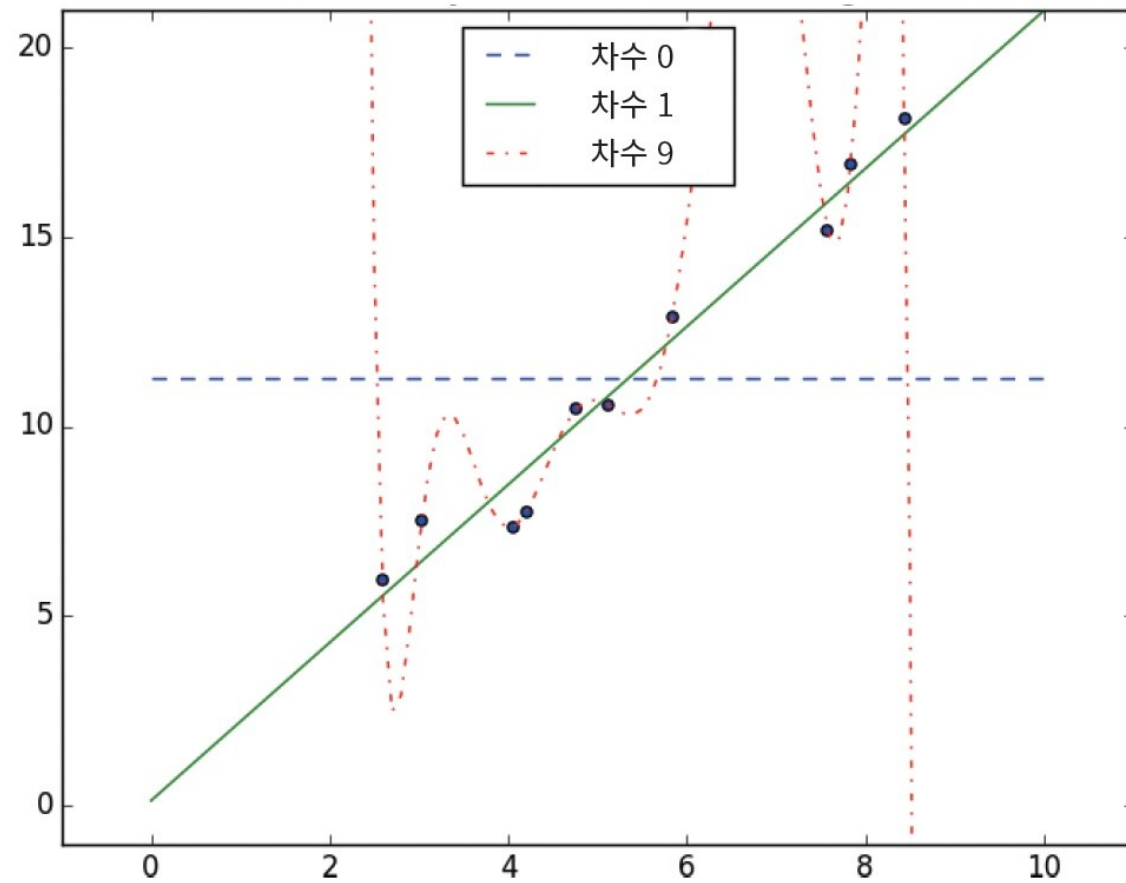
```
print(f'{metrics.classification_report(y_test, y_pred)}\n')
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	28
1	0.95	1.00	0.97	39
2	1.00	1.00	1.00	33
3	0.95	1.00	0.97	36
4	1.00	1.00	1.00	35
5	1.00	1.00	1.00	35
6	1.00	1.00	1.00	45
7	1.00	1.00	1.00	40
8	0.97	0.93	0.95	40
9	1.00	0.93	0.96	29
accuracy			0.99	360
macro avg	0.99	0.99	0.99	360
weighted avg	0.99	0.99	0.99	360

과잉 적합 vs 과소 적합

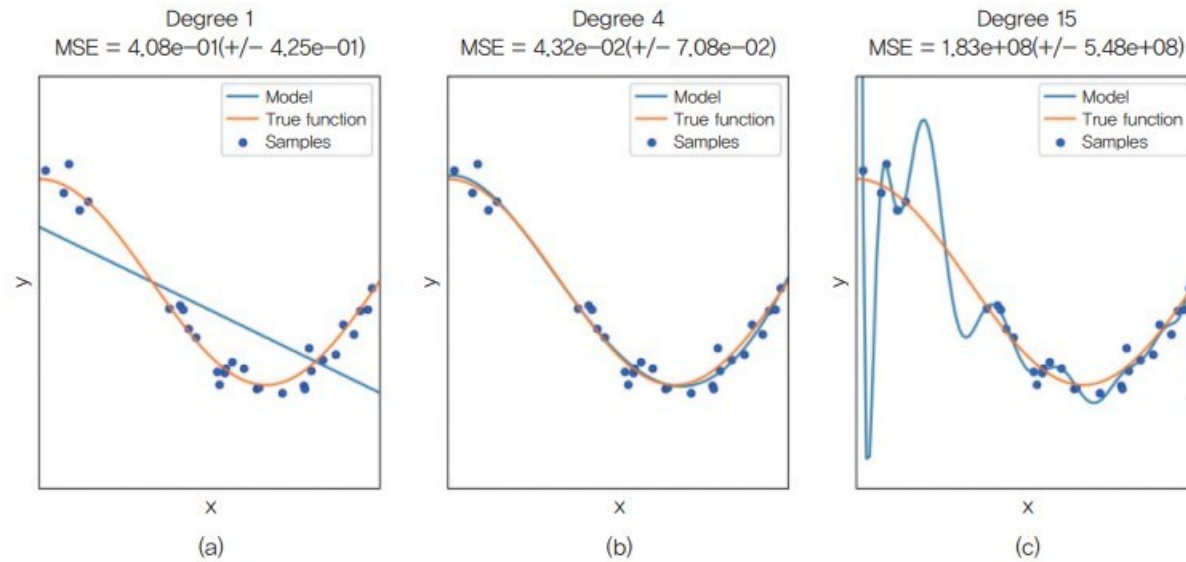
- 과잉 적합(overfitting)이란 학습하는 데이터에서는 성능이 뛰어나지만 새로운 데이터(일반화)에 대해서는 성능이 잘 나오지 않는 모델을 생성하는 것이다.
- 과소 적합(underfitting)이란 학습 데이터에서도 성능이 좋지 않은 경우이다. 이 경우에는 모델 자체가 적합하지 않은 경우가 많다. 더 나은 모델을 찾아야 한다.

과잉 적합 vs 과소 적합



회귀에서 과잉 적합의 예

과잉 적합 vs 과소 적합



회귀에서 과잉 적합의 예(출처: 사이킷런 홈페이지)

평가

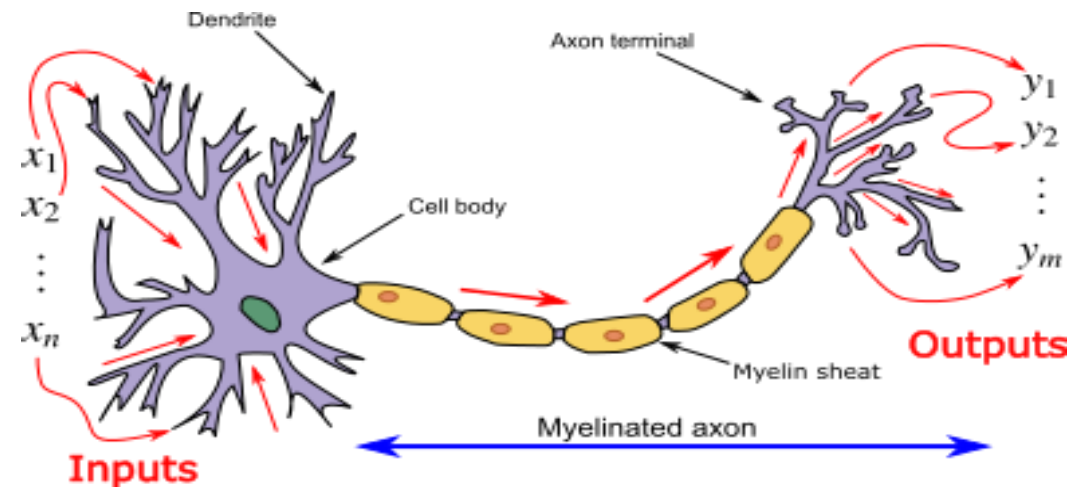
- 학습이 완료되면 모델을 평가하여 모델이 좋은지 나쁜지를 확인해야 한다.
- 테스트 데이터 사용
- 일반적인 훈련 데이터와 테스트 데이터의 비율은 80:20 또는 70:30이다.

Summary

- 지도 학습에는 회귀(regression)와 분류(classification)가 있다. 전자는 연속적인 값을 예측하고 후자는 입력을 어떤 카테고리 중의 하나로 예측한다.
- 선형 회귀는 입력 데이터를 가장 잘 설명하는 직선의 기울기와 절편값을 찾는 문제이다.
- 손실 함수(loss function)란 실제 데이터와 직선 간의 차이를 제공하는 값이다.
- 회귀란 손실 함수를 최소화 하는 직선의 기울기와 절편값을 계산하는 것이다.
- 손실 함수의 값이 작아지는 방향을 알려면 일반적으로 경사 하강법 (gradient descent method)과 같은 방법을 많이 사용한다.

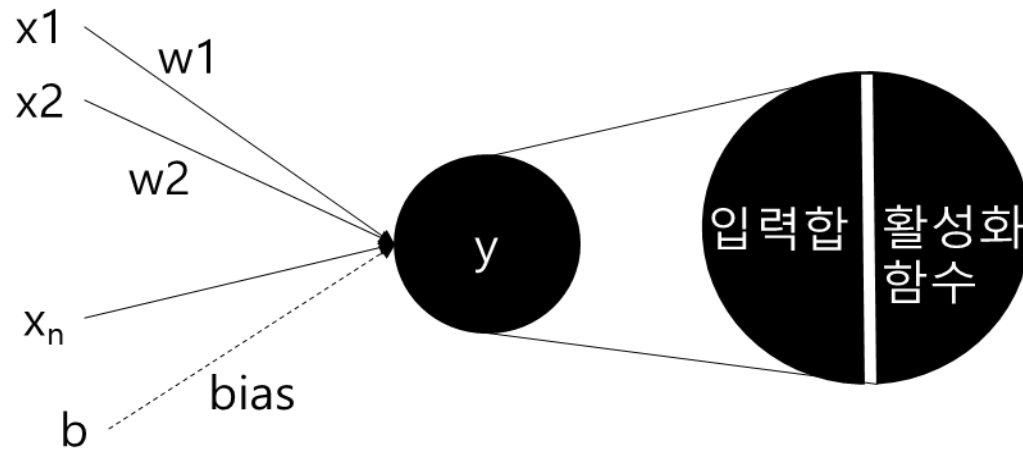
• 인공 신경망

- ◆ 퍼셉트론(Perceptron)은 프랑크 로젠블라트(Frank Rosenblatt)가 1957년에 제안한 초기 형태의 인공 신경망으로 다수의 입력으로부터 하나의 결과를 내보내는 알고리즘입니다. 퍼셉트론은 실제 뇌를 구성하는 신경 세포 뉴런의 동작과 유사한데, 신경 세포 뉴런의 그림을 먼저 보도록 하겠습니다. 뉴런은 가지돌기에서 신호를 받아들이고, 이 신호가 일정치 이상의 크기를 가지면 축삭돌기를 통해서 신호를 전달합니다



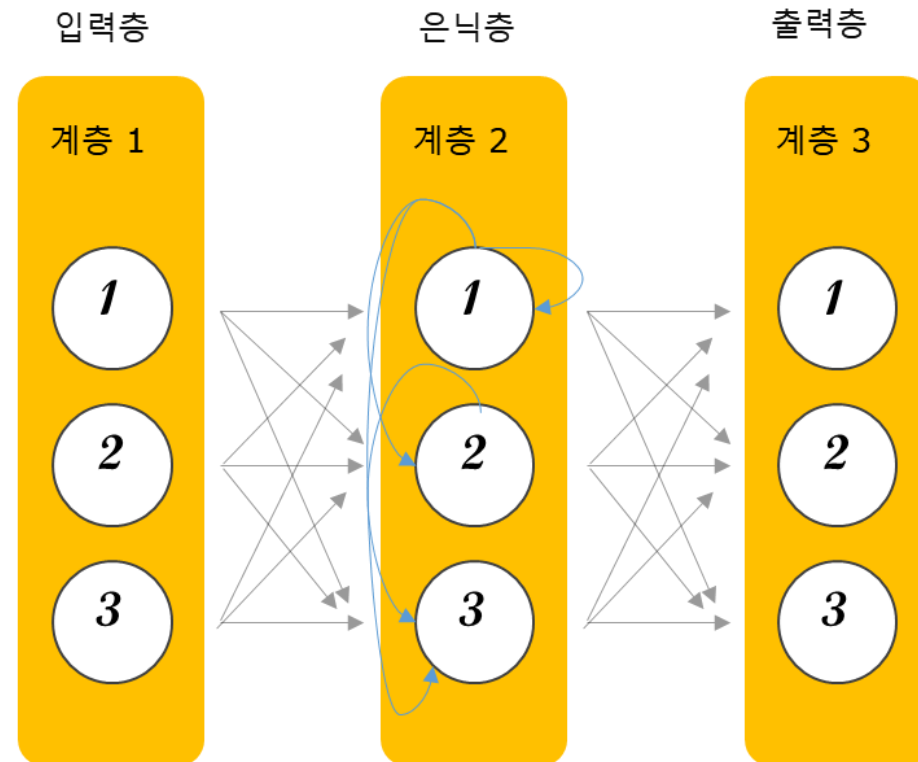
• 인공 신경망

- ◆ 인간 신경망을 퍼셉트론으로 만들어 보면 x 는 입력값, w 는 가중치(weight), y 는 출력값이 됩니다. 그림 원은 인공뉴런(y)입니다. 실제 신경 세포 뉴런에서 신호를 전달하는 축삭돌기의 역할을 퍼셉트론에서는 가중치가 담당합니다. 각각의 인공 뉴런에서 보내진 입력값 x 는 각각의 가중치 W 와 함께 인공 뉴런(y)에 전달되고 있습니다. 각각의 입력값에는 각각의 가중치가 존재하는데, 이때 가중치의 값이 크면 클수록 해당 입력 값이 중요하다는 것을 의미합니다.
- ◆ 각 입력값이 가중치와 곱해져서 인공 뉴런에 보내지고, 각 입력값과 그에 해당되는 가중치의 곱의 전체 합이 임계치(threshold)를 넘으면 종착지에 있는 인공 뉴런(y)은 출력 신호로서 1을 출력하고, 그렇지 않을 경우에는 0을 출력합니다. 이렇게 뉴런에서 출력값을 변경시키는 함수를 '활성화 함수(Activation Function)'라고 합니다.



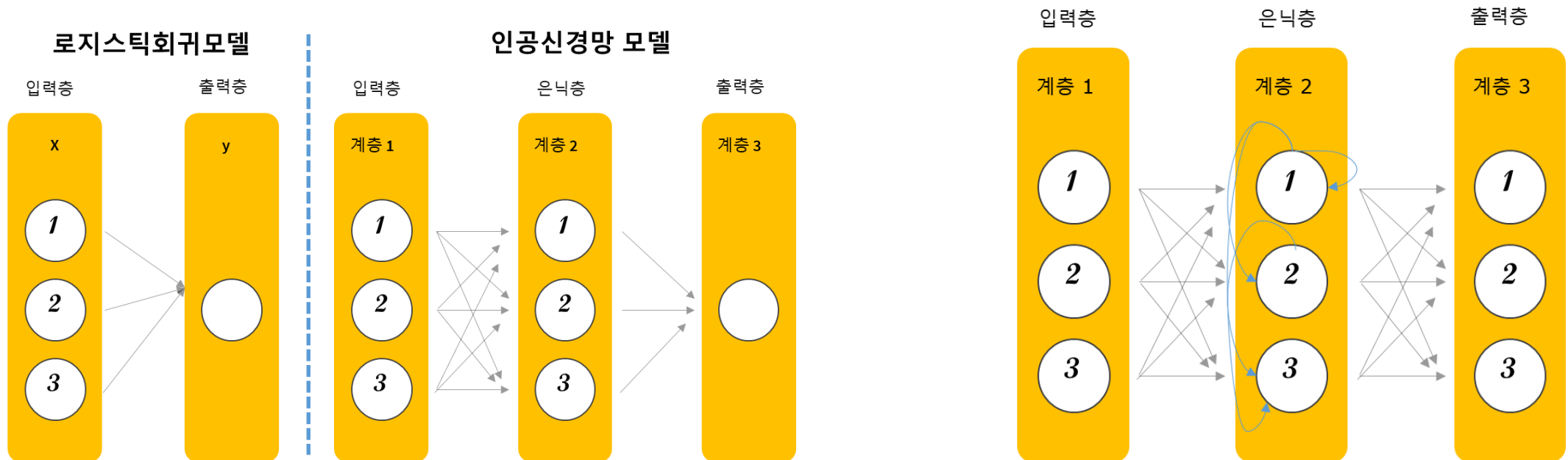
• 인공지능망

- ◆ 인공 신경망(ANN, Artificial Neural Networks)은 인간 두뇌의 신경 체계를 모방한 퍼셉트론으로 부터 발전 되었습니다. 각 인공 신경 체계는 시스템 내부에서 다른 뉴런과 연결되며 신경망은 계층(Layer)들로 배열되어, 한 계층의 뉴런이 다음 계층의 다수 뉴런 에게 데이터를 전달하는 구조 입니다. 초기 ANN 은 은닉 계층이 하나 였습니다. 과거 데이터 양의 한계와 가중치 최적화가 어려워 신경망의 성능을 보장 할 수 없었기 때문입니다. 이런 이유로 단일 은닉 계층의 ANN은 얇은 신경망(shallow neural networks)으로 불리기도 했습니다.



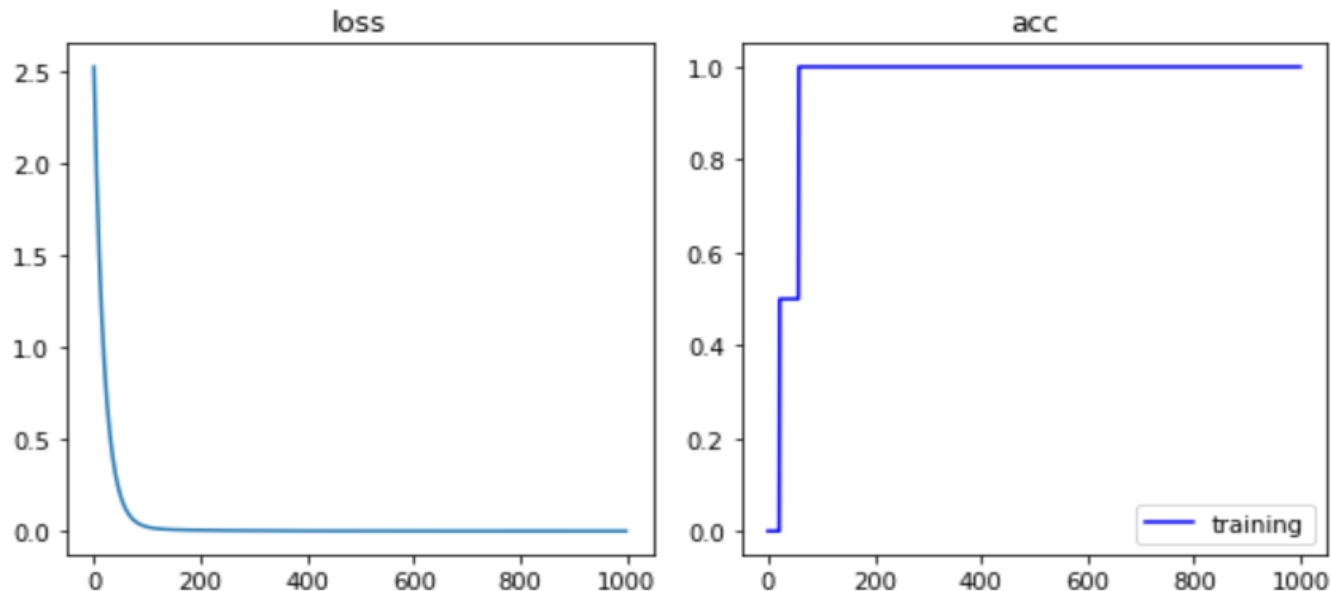
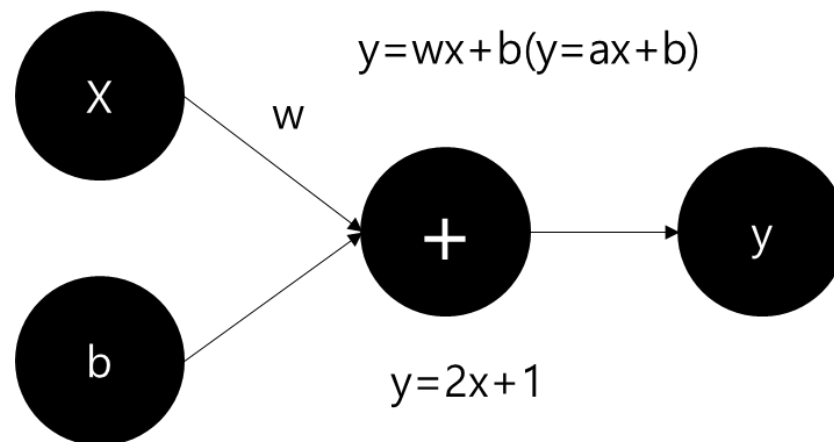
• 순환신경망(RNN)

- ◆ 인공 신경망을 회귀모델과 비교 하면 회귀모델은 은닉층(중간층)없이 입력값과 출력값 간의 관계가 직접 연결되어 있습니다. 또한 인공 신경망 모델은 입력층에서 출력층 방향으로 연산이 전개되는 피드 포워드 신경망(Feed-Forward Neural Network, FFNN)이라고 합니다. 이렇게 별도로 부르는 이유는 FFNN이 아닌 신경망이 존재하기 때문입니다. 아래 그림은 순환 신경망(RNN, Recurrent Neural Network) 사례 입니다. RNN은 은닉층의 출력값을 출력층으로도 값을 보내지만, 동시에 은닉층의 출력값이 다시 은닉층의 입력으로 복잡하게 사용되기도 합니다.



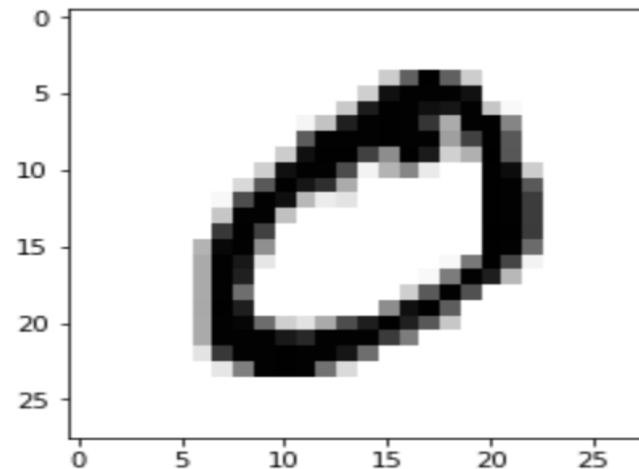
• 기본 신경망

- ◆ 신경망 모델은 x 입력 에서 W 가중치와 b 바이어스를 학습을 통해 y 결과 값을 찾는 과정입니다.

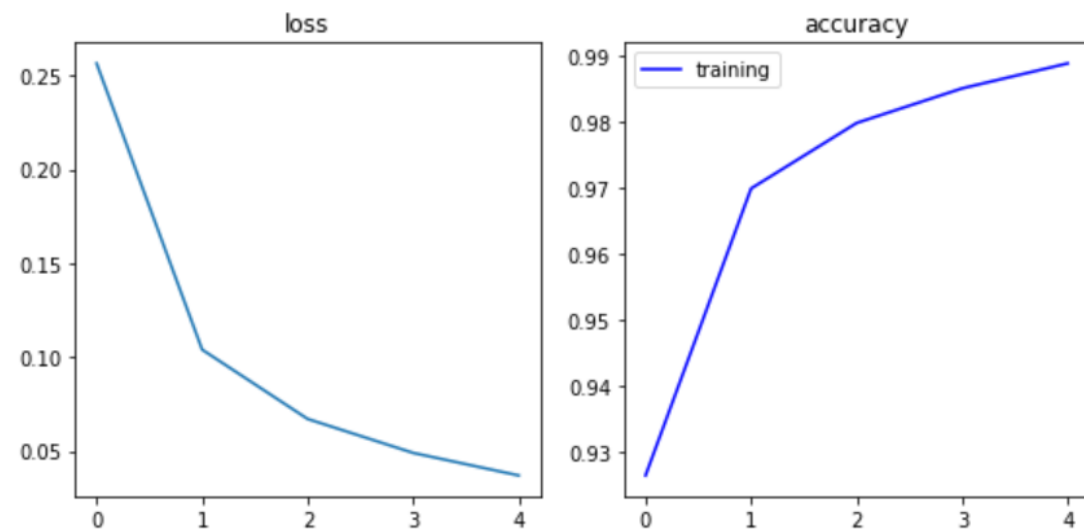


X	Y	구분
0	1	학습
1	3	학습
2	?	예측
3	?	예측
4	?	예측
5	?	예측

- 케라스 ANN 이미지 모델

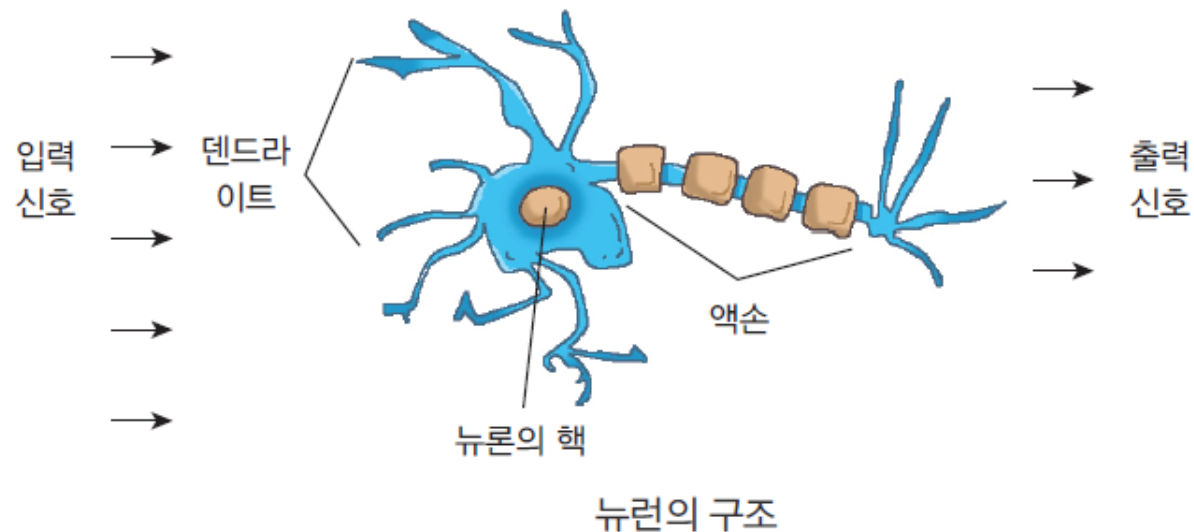
[illegible]

Keras MNIST

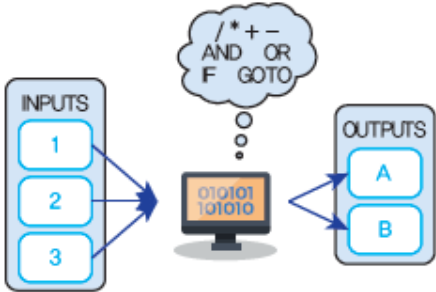


신경망 (Neuron Network)

- 최근에 많은 인기를 끌고 있는 딥러닝(deep learning)의 시작은 1950년대부터 연구되어 온 인공 신경망 (artificial neural network: ANN)이다.
- 인공 신경망은 생물학적인 신경망에서 영감을 받아서 만들어진 컴퓨팅 구조이다.



전통적인 컴퓨터 vs 인공신경망

	기존의 컴퓨터	인간의 두뇌
처리소자의 개수	10^8 개의 트랜지스터	10^{10} 개의 뉴런
처리소자의 속도	10^{12} Hz	10^2 Hz
학습기능	없음	있음
계산 스타일	중앙집중식, 순차적인 처리	분산 병렬 처리
	 The diagram illustrates a traditional computer architecture. On the left, a box labeled 'INPUTS' contains three numbered slots (1, 2, 3). Arrows point from these slots to a central computer monitor. The monitor displays binary code '010101' and '101010'. Above the monitor is a thought bubble containing logical operations: '/ * + -', 'AND OR', and 'F GOTO'. An arrow points from the monitor to a box on the right labeled 'OUTPUTS', which contains two slots labeled 'A' and 'B'.	 The diagram shows a human brain filled with a dense, interconnected network of neurons. Each neuron is represented by a small colored circle (red, blue, green, yellow, purple, etc.), and they are all connected by a complex web of lines, representing the distributed and parallel nature of neural processing.

신경망의 장점

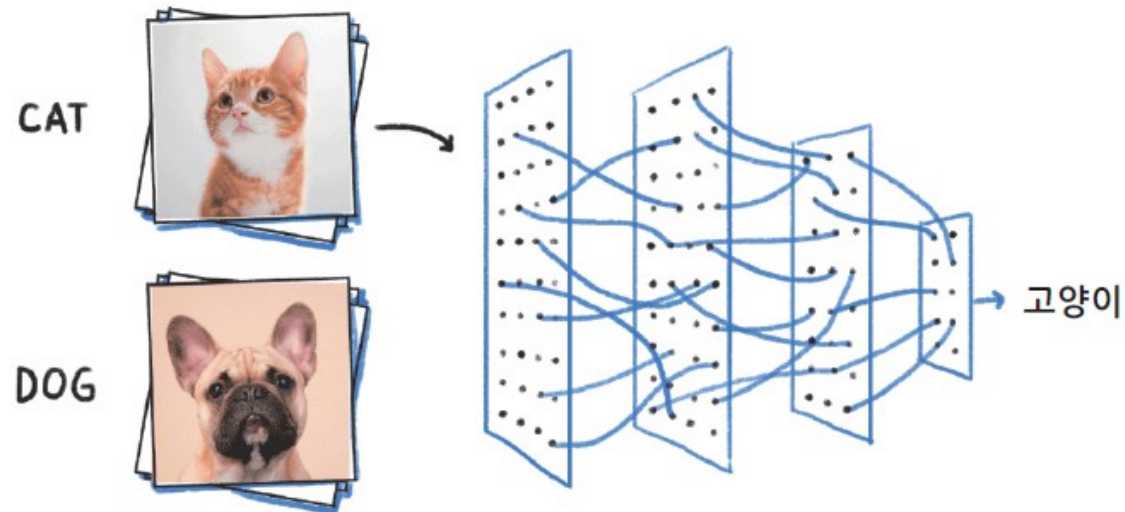
- 첫 번째는 학습이 가능하다는 점이다. 데이터만 주어진다면 신경망은 예제로부터 배울 수 있다.
- 두 번째는 몇 개의 소자가 오동작하더라도 전체적으로는 큰 문제가 발생하지 않는다는 점이다.



학습하는 컴퓨터

신경망이 필요한 분야

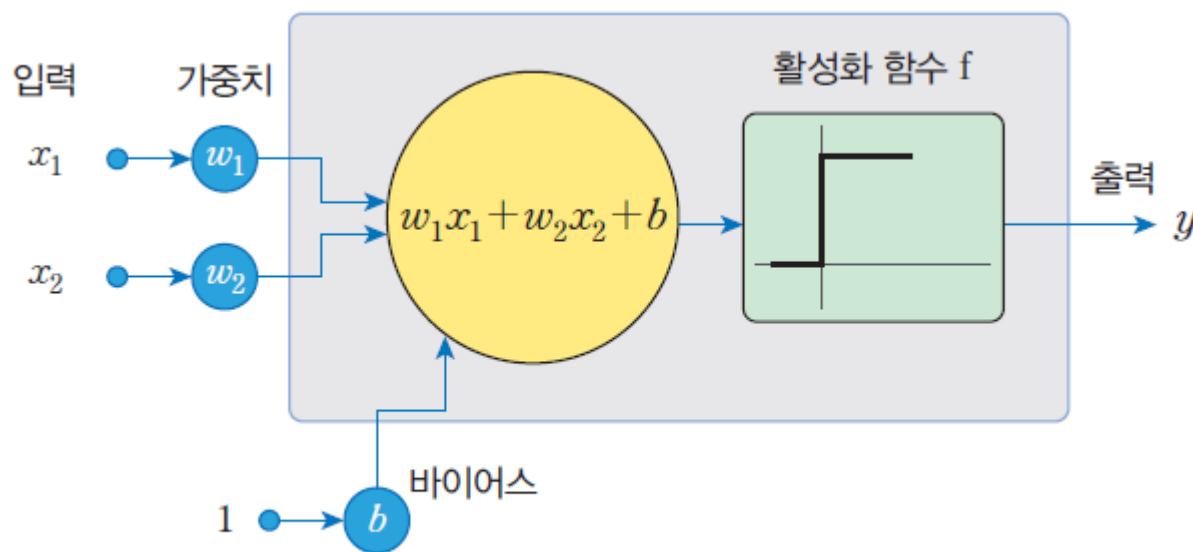
- 예를 들어 강아지 이미지와 고양이 이미지를 식별하는 작업을 생각해보자.
인간은 쉽게 이미지를 인식하지만 인간도 인식의 메커니즘을 정확히 모르기 때문에 인식 알고리즘을 명시적으로 만드는 것은 아주 어려운 일이다.



신경망을 이용한 이미지 인식

퍼셉트론

- 퍼셉트론(perceptron)은 1957년에 로젠블라트(Frank Rosenblatt)가 고안한 인공 신경망이다.

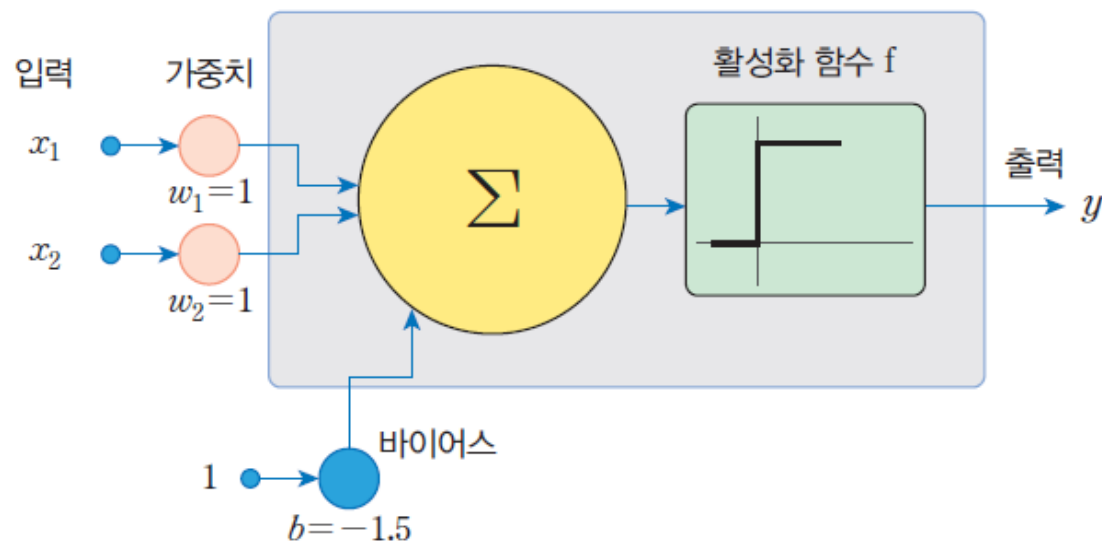


퍼셉트론에서의 뉴런의 모델

퍼셉트론

- 뉴런에서는 입력 신호의 가중치 합이 어떤 임계값을 넘는 경우에만 뉴런이 활성화되어서 1을 출력한다. 그렇지 않으면 0을 출력한다.

$$y = \begin{cases} 1 & \text{if } (w_1x_1 + w_2x_2 + b \geq 0) \\ 0 & \text{otherwise} \end{cases}$$



논리 연산을 하는 퍼셉트론

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

논리 연산을 학습해보자

퍼셉트론 학습 알고리즘

```
import numpy as np

epsilon = 0.0000001          # 부동소수점 오차 방지 (0 으로 나누어지는 문제 해결을 위해서)

def step_func(t):             # 퍼셉트론의 활성화 함수
    if t > epsilon:
        return 1
    else:
        return 0

X = np.array([               # 훈련 데이터 세트
    [0, 0, 1],               # 맨 끝의 1은 바이어스를 위한 입력 신호 1이다.
    [0, 1, 1],               # 맨 끝의 1은 바이어스를 위한 입력 신호 1이다.
    [1, 0, 1],               # 맨 끝의 1은 바이어스를 위한 입력 신호 1이다.
    [1, 1, 1]])              # 맨 끝의 1은 바이어스를 위한 입력 신호 1이다.

y = np.array([0, 0, 0, 1])   # 정답을 저장하는 넘파이 행렬
W = np.zeros(len(X[0]))      # 가중치를 저장하는 넘파이 행렬
```


퍼셉트론 학습 알고리즘

```
def perceptron_fit(X, Y, epochs=10): # 퍼셉트론 학습 알고리즘 구현
    global W
    eta = 0.2                        # 학습률

    for t in range(epochs):
        print("epoch=", t, "=====")
        for i in range(len(X)):
            predict = step_func(np.dot(X[i], W))
            error = Y[i] - predict    # 오차 계산
            W += eta * error * X[i]   # 가중치 업데이트
            print("현재 처리 입력=", X[i], "정답=", Y[i], "출력=", predict, "변경된 가중치=", W)
        print("=====")
```

```
def perceptron_predict(X):          # 예측
    global W
    for x in X:
        print(x[0], x[1], "->", step_func(np.dot(x, W)))

perceptron_fit(X, y, 20)
perceptron_predict(X)
```

학습 결과

epoch= 0

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0. 0. 0.]
현재 처리 입력= [0 1 1] 정답= 0 출력= 0 변경된 가중치= [0. 0. 0.]
현재 처리 입력= [1 0 1] 정답= 0 출력= 0 변경된 가중치= [0. 0. 0.]
현재 처리 입력= [1 1 1] 정답= 1 출력= 0 변경된 가중치= [0.2 0.2 0.2]

epoch= 1

현재 처리 입력= [0 0 1] 정답= 0 출력= 1 변경된 가중치= [0.2 0.2 0.]
현재 처리 입력= [0 1 1] 정답= 0 출력= 1 변경된 가중치= [0.2 0. -0.2]
현재 처리 입력= [1 0 1] 정답= 0 출력= 0 변경된 가중치= [0.2 0. -0.2]
현재 처리 입력= [1 1 1] 정답= 1 출력= 0 변경된 가중치= [0.4 0.2 0.]

epoch= 2

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 0.]
현재 처리 입력= [0 1 1] 정답= 0 출력= 1 변경된 가중치= [0.4 0. -0.2]
현재 처리 입력= [1 0 1] 정답= 0 출력= 1 변경된 가중치= [0.2 0. -0.4]
현재 처리 입력= [1 1 1] 정답= 1 출력= 0 변경된 가중치= [0.4 0.2 -0.2]

epoch= 3

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.2]
현재 처리 입력= [0 1 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.2]
현재 처리 입력= [1 0 1] 정답= 0 출력= 1 변경된 가중치= [0.2 0.2 -0.4]
현재 처리 입력= [1 1 1] 정답= 1 출력= 0 변경된 가중치= [0.4 0.4 -0.2]

epoch= 4

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.4 -0.2]
현재 처리 입력= [0 1 1] 정답= 0 출력= 1 변경된 가중치= [0.4 0.2 -0.4]
현재 처리 입력= [1 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.4]
현재 처리 입력= [1 1 1] 정답= 1 출력= 1 변경된 가중치= [0.4 0.2 -0.4]

epoch= 5

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.4]
현재 처리 입력= [0 1 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.4]
현재 처리 입력= [1 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.4]
현재 처리 입력= [1 1 1] 정답= 1 출력= 1 변경된 가중치= [0.4 0.2 -0.4]

0 0 → 0

0 1 → 0

1 0 → 0

1 1 → 1

sklearn으로 퍼셉트론 실습하기

```
from sklearn.linear_model import Perceptron

# 샘플과 레이블이다.
X = [[0,0],[0,1],[1,0],[1,1]]
y = [0, 0, 0, 1]

# 퍼셉트론을 생성한다. tol는 종료 조건이다. random_state는 난수의 시드이다.
clf = Perceptron(tol=1e-3, random_state=0)

# 학습을 수행한다.
clf.fit(X, y)

# 테스트를 수행한다.
print(clf.predict(X))
```

```
[0 0 0 1]
```

퍼셉트론의 한계점

- XOR 연산

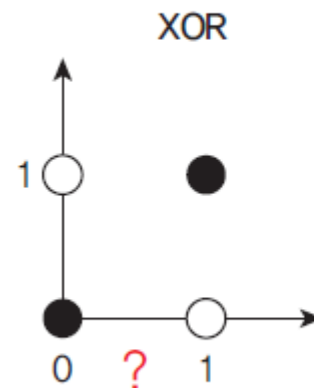
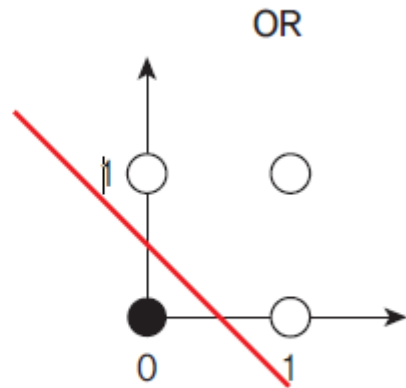
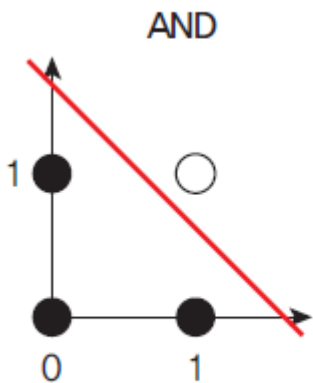
x1	x2	y
0	0	0
1	0	1
0	1	1
1	1	0

...
0 0 -> 1
0 1 -> 1
1 0 -> 0
1 1 -> 0

원하는 출력이 나오지 않는다.

선형 분류 가능 문제

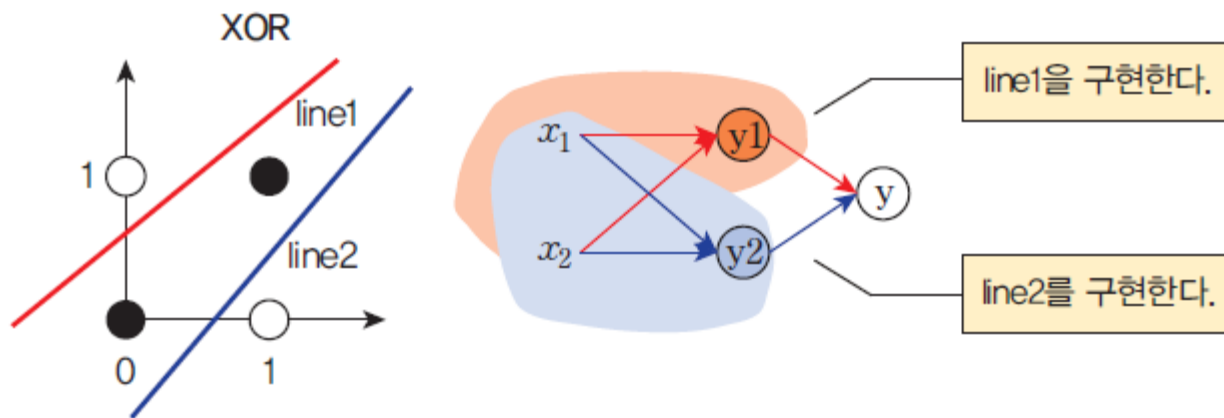
- Minsky와 Papert는 1969년에 발간된 책 "Perceptrons"에서 1개의 레이어(layer, 계층)으로 구성된 퍼셉트론은 XOR 문제를 학습할 수 없다는 것을 수학적으로 증명



선형 분리 가능한 문제

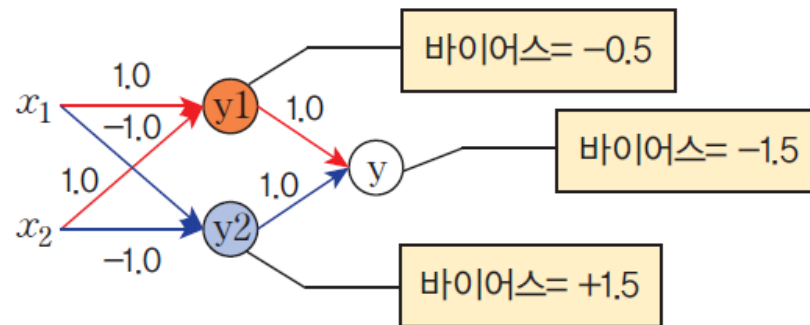
선형 분리 불가능한 문제

다층 퍼셉트론으로 XOR 문제를 해결



다층을 사용하는 퍼셉트론

다층 퍼셉트론으로 XOR 문제를 해결

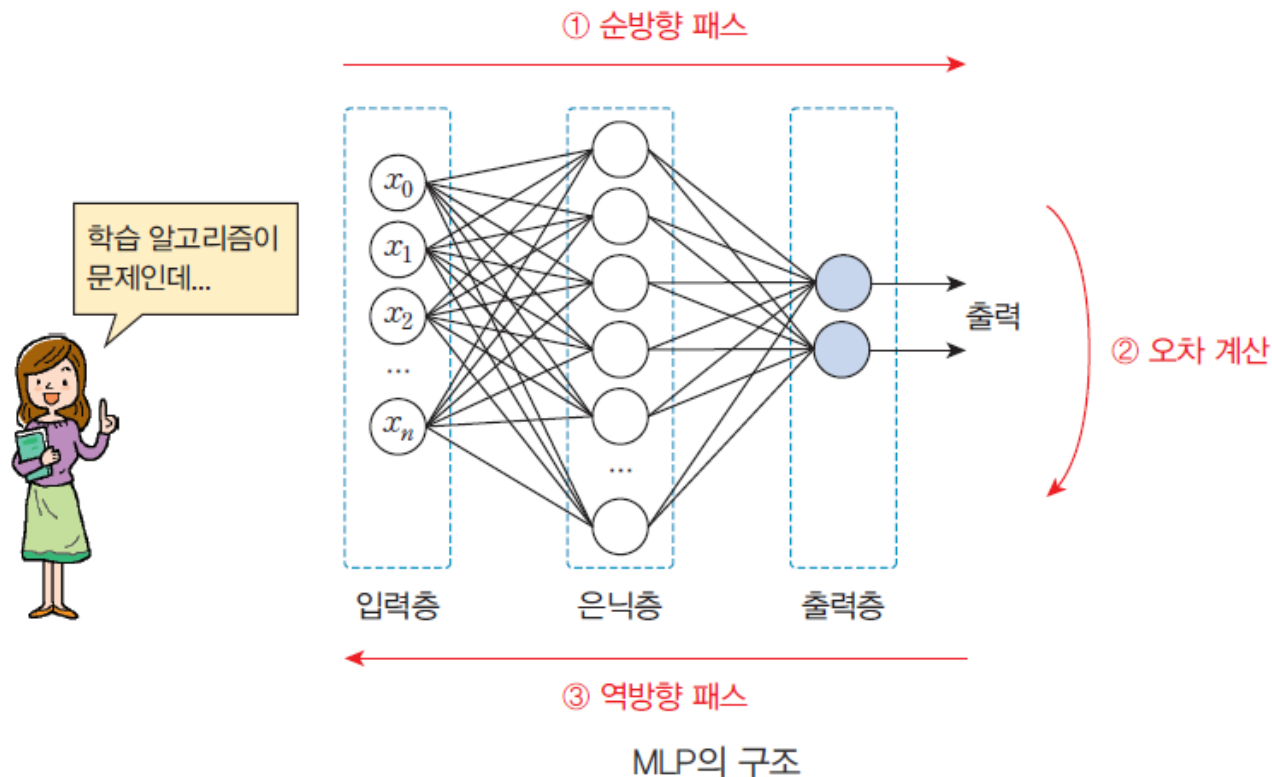


다층 퍼셉트론에서 XOR 문제 해결

x_1	x_2	y_1	y_2	y	XOR 출력
0	0	0	1	0	0
1	0	1	1	1	1
0	1	1	1	1	1
1	1	1	0	0	0

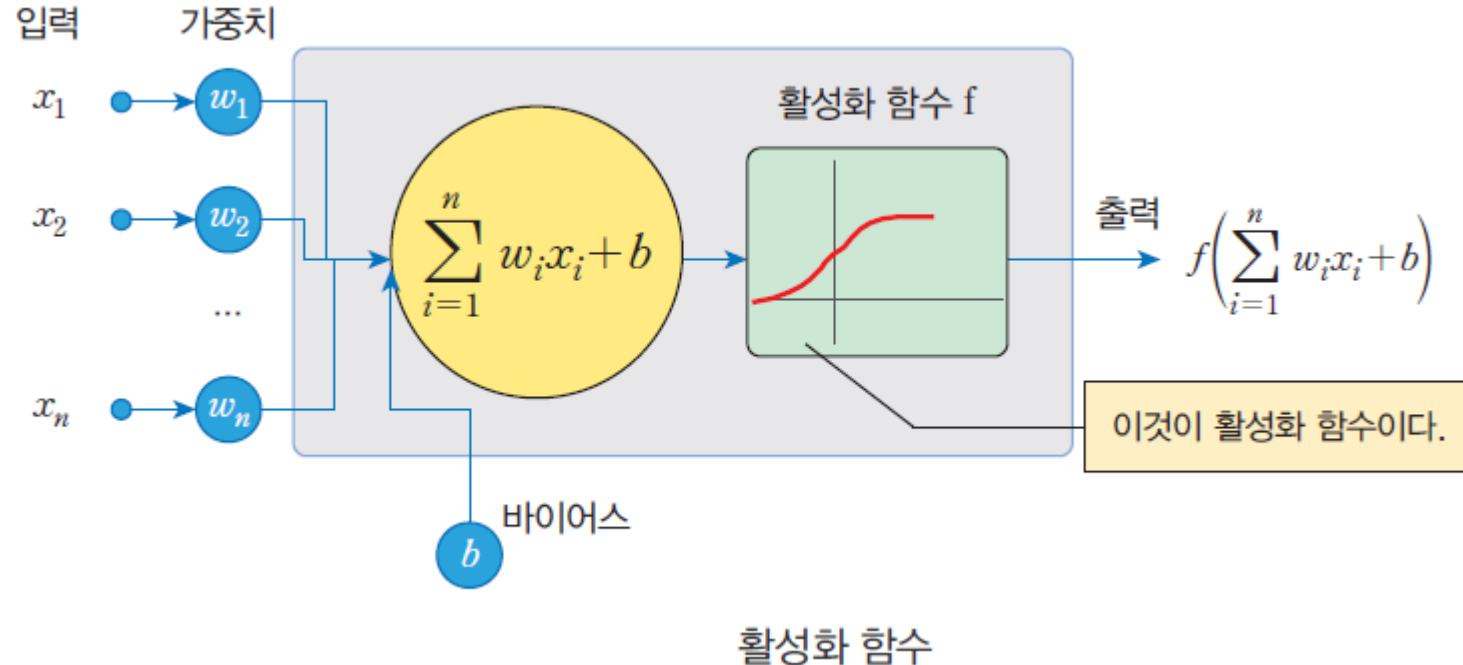
MLP (Multi Layer Perceptron)

- 다층 퍼셉트론(Multi Layer Perceptron: MLP): 입력층과 출력층 사이에 은닉층(hidden layer)을 가지고 있는 퍼셉트론

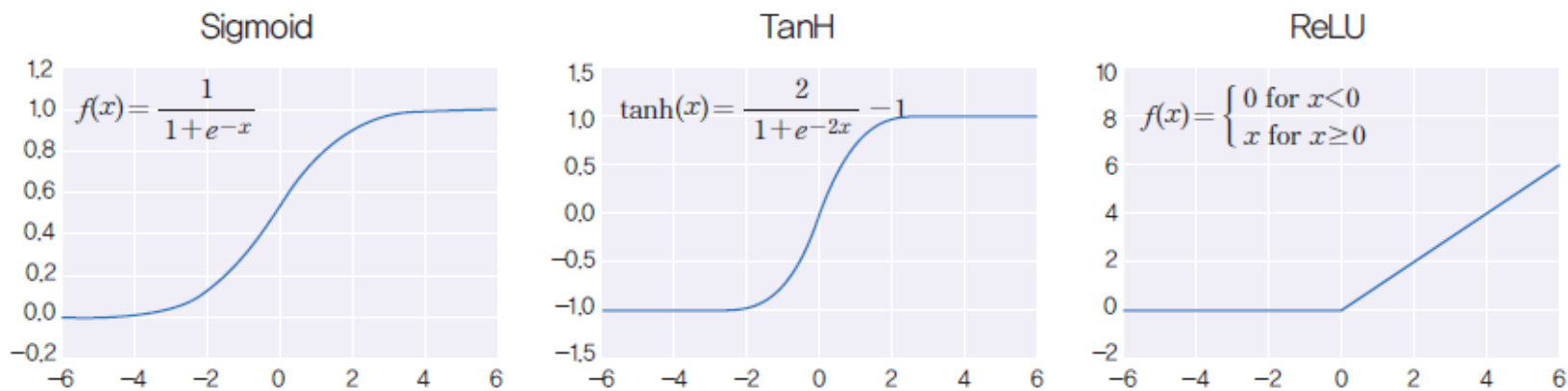


활성화 함수

- 활성화 함수(activation function)은 입력의 총합을 받아서 출력 값을 계산하는 함수이다.
- MLP에서는 다양한 활성화 함수를 사용한다.



일반적으로 많이 사용되는 활성화 함수



많이 사용되는 활성화 함수

$$f(x) = \frac{1}{1+e^{-x}}$$

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{2}{1+e^{-2x}} - 1$$

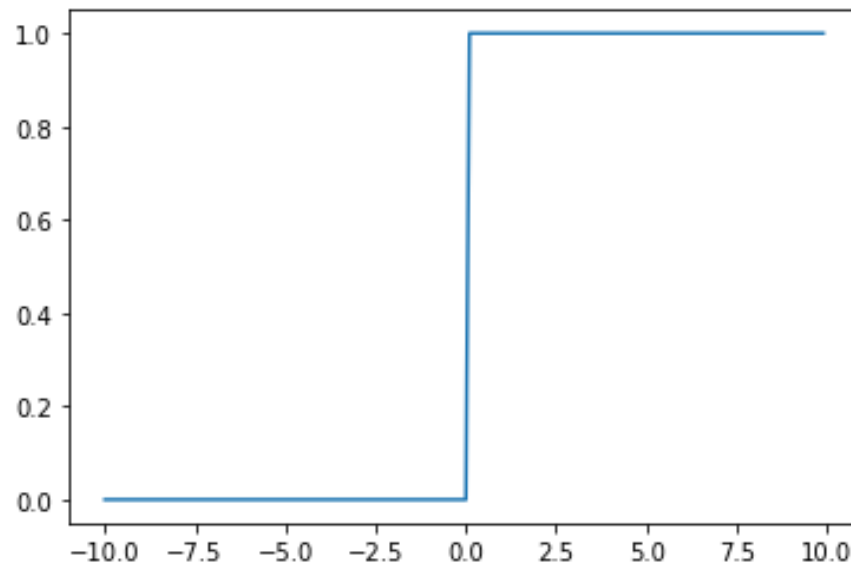
$$f(x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

그래프 그리기

```
import numpy as np
import matplotlib.pyplot as plt

def step(x):          # 활성화 함수
    if x > 0.0001:
        return 1
    else:
        return 0

x = np.arange(-10.0, 10.0, 0.1)
y = step(x)
plt.plot(x, y)
plt.show()
```

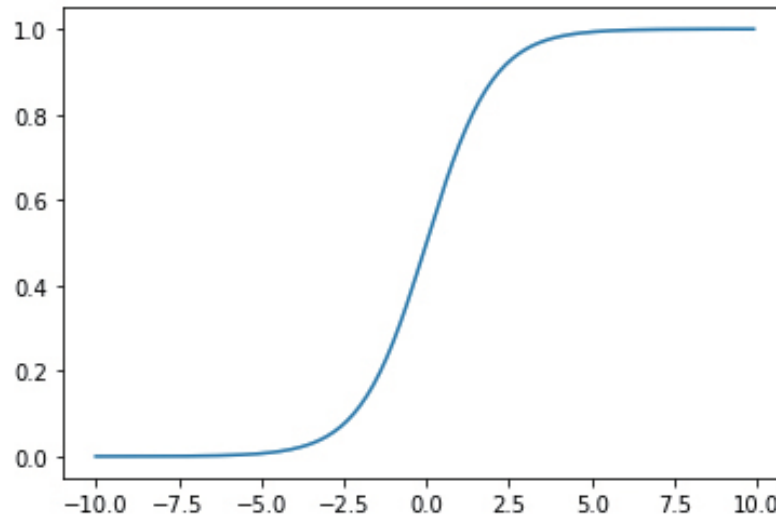


시그모이드 함수

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

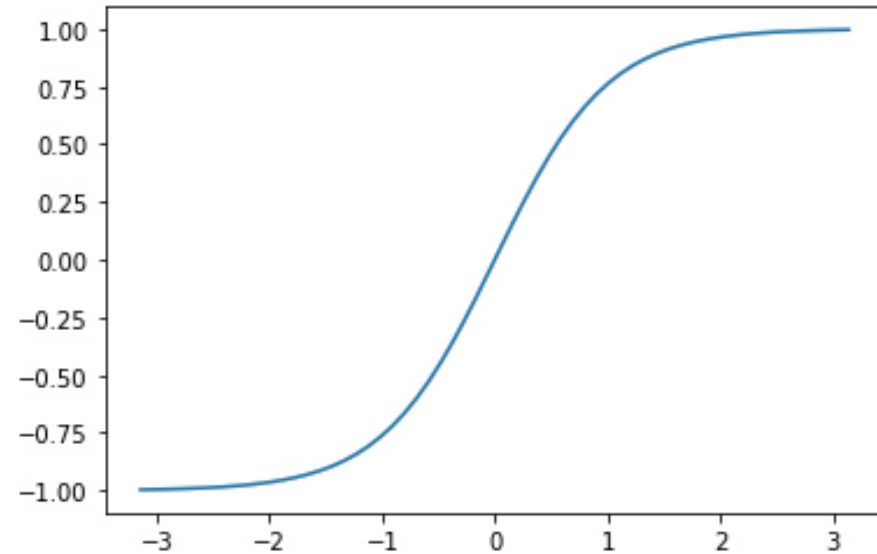
x = np.arange(-10.0, 10.0, 0.1)
y = sigmoid(x)
plt.plot(x, y)
plt.show()
```



tanh 함수

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-np.pi, np.pi, 60)
y = np.tanh(x)
plt.plot(x, y)
plt.show()
```

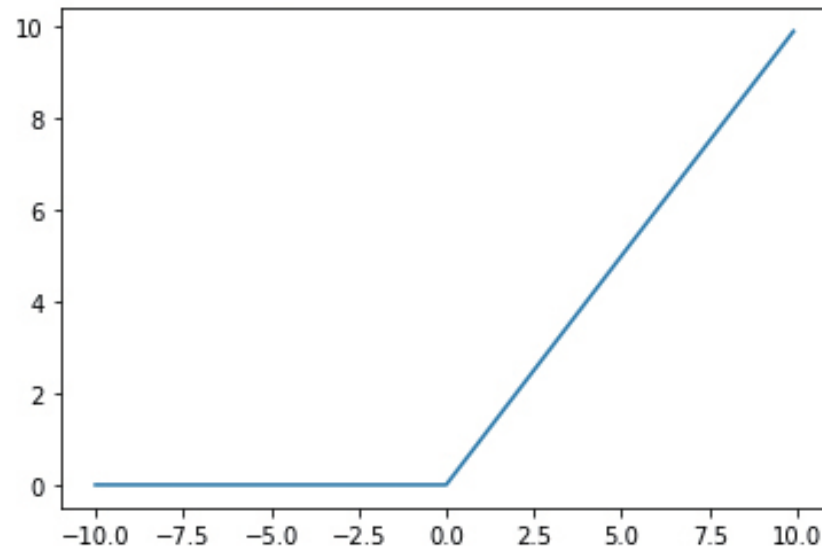


ReLU 함수

```
import numpy as np
import matplotlib.pyplot as plt

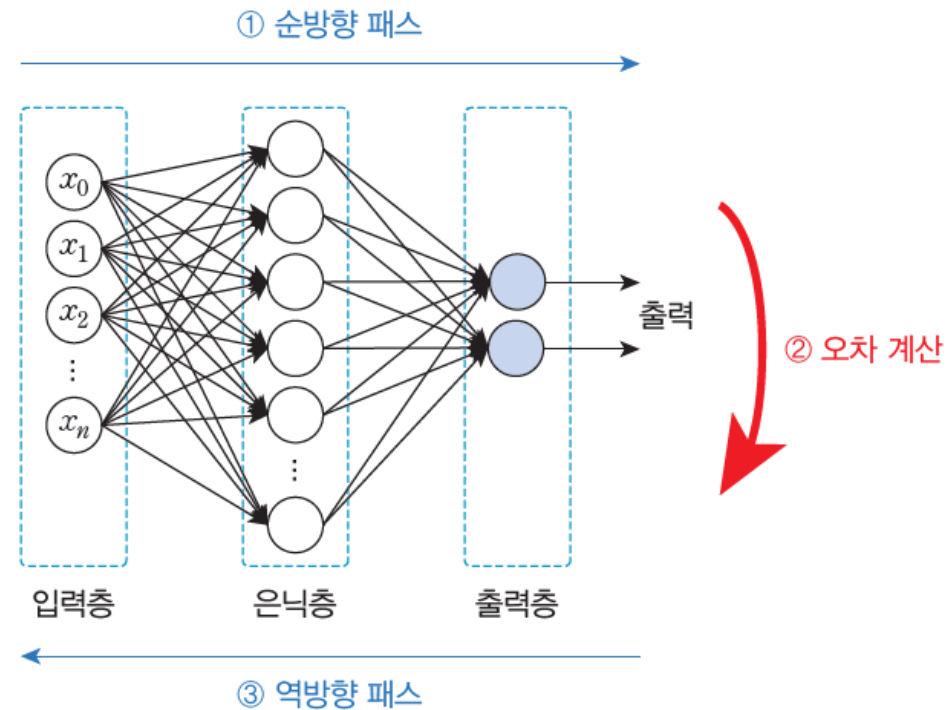
def relu(x):
    return np.maximum(x, 0)

x = np.arange(-10.0, 10.0, 0.1)
y = relu(x)
plt.plot(x, y)
plt.show()
```



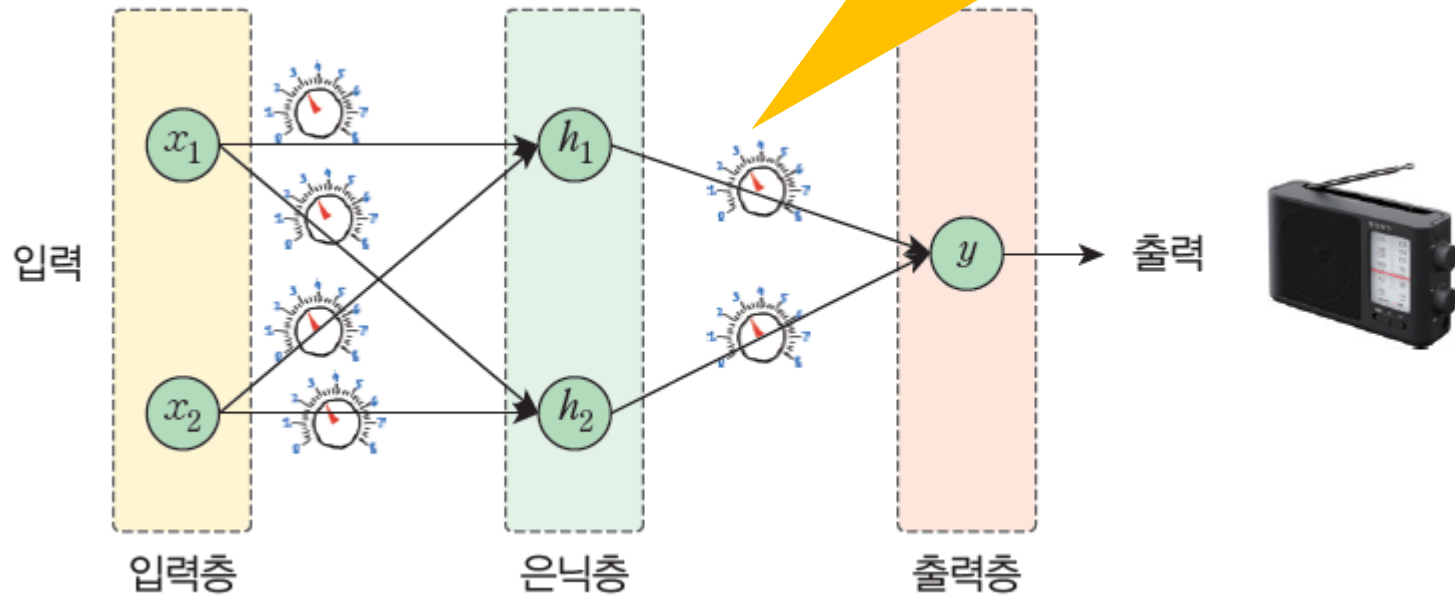
손실 함수 계산

- 신경망에서 학습을 시킬 때는 실제 출력과 원하는 출력 사이의 오차를 이용한다. 오차를 계산하는 함수를 손실함수(loss function)라고 한다.



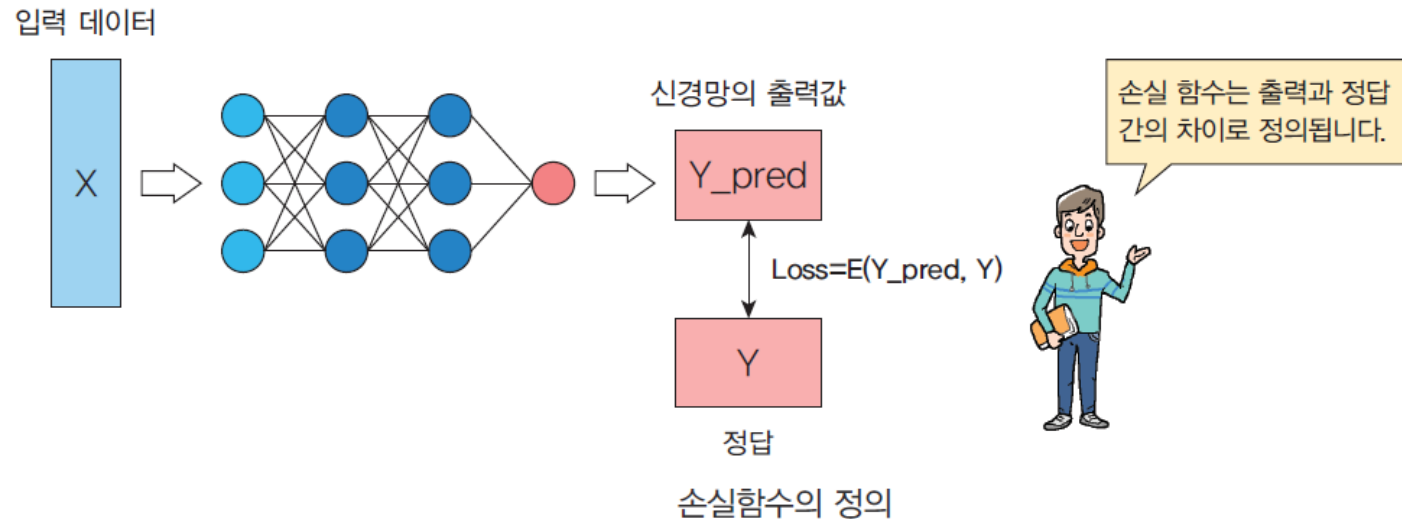
가중치 == 다이얼

가중치를 조절한다는 것은 스피커에서
나는 소리를 들으면서 튜너 다이얼을 돌
리는 것과 같다.



손실 함수

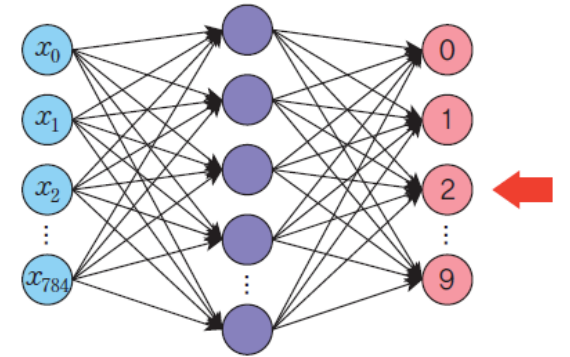
- 신경망에서도 학습의 성과를 나타내는 지표가 있어야 한다. 이것을 손실함수(loss function)이라고 한다



평균 제곱 오차(MSE)

- 예측값과 정답 간의 평균 제곱 오차

$$E(w) = \frac{1}{2} \sum_i (y_i - t_i)^2$$



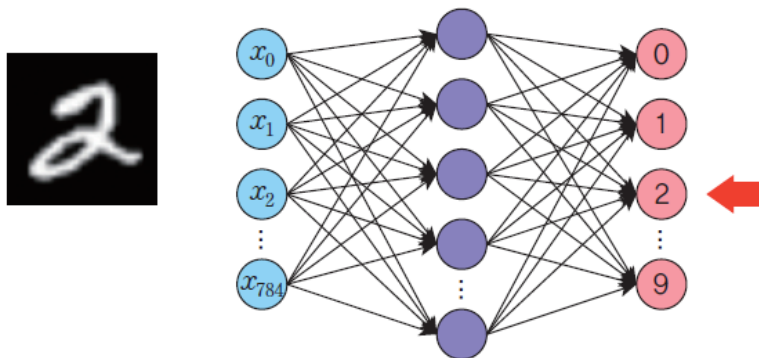
MNIST 숫자 이미지를 분류하는 신경망

```
>>> y = np.array([ 0.0, 0.0, 0.8, 0.1, 0.0, 0.0, 0.0, 0.1, 0.0, 0.0 ])
>>> target = np.array([ 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 ])

>>> def MSE(target, y):
    return 0.5 * np.sum((y-target)**2)

>>> MSE(target, y)
0.029999999999999992
```

예측값과 정답이 많이 차이는 경우



MNIST 숫자 이미지를 분류하는 신경망

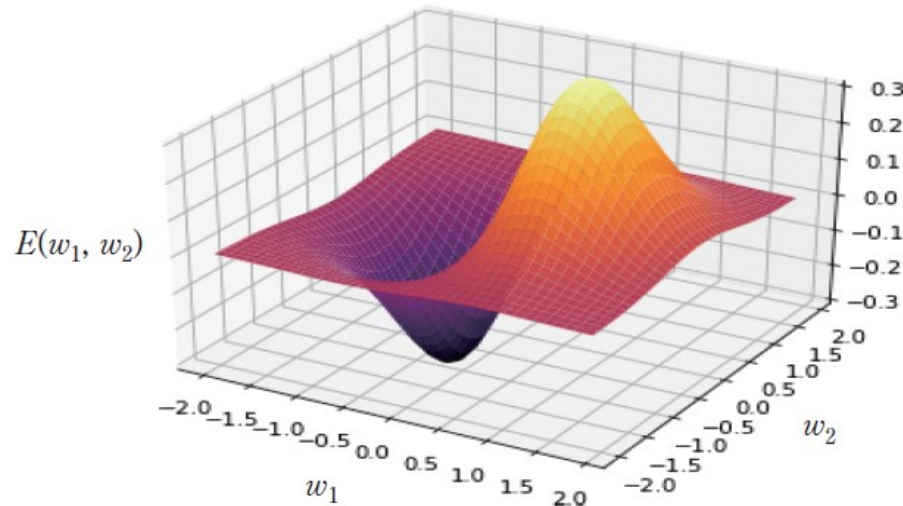
```
>>> y = np.array([ 0.9, 0.0, 0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 ] )
>>> target = np.array([ 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 ] )

>>> def MSE(target, y):
>>>     return 0.5 * np.sum((y-target)**2)
>>> MSE(target, y)
0.81
```

경사하강법

- 역전파 알고리즘은 신경망 학습 문제를 최적화 문제 (optimization)로 접근한다. 우리는 손실함수 값을 최소로 하는 가중치를

$$W^* = \operatorname{argmin}_W E(W)$$

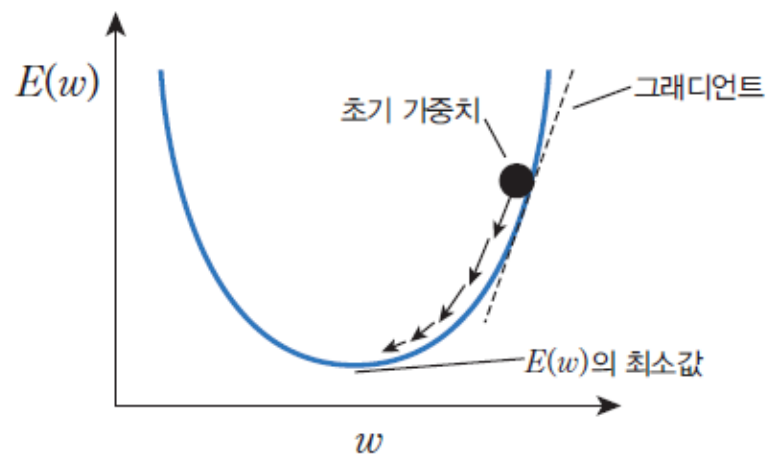


옆의 평면에서 가장 낮은 값을 찾으면 됩니다.



경사 하강법

경사하강법



그래디언트는 접선의 기울기로 이해
해도 됩니다. 접선의 기울기가 양수
이면 반대로 w 를 감소시킵니다.



경사 하강법

손실함수를 가중치로 미분한
값이 양수이면



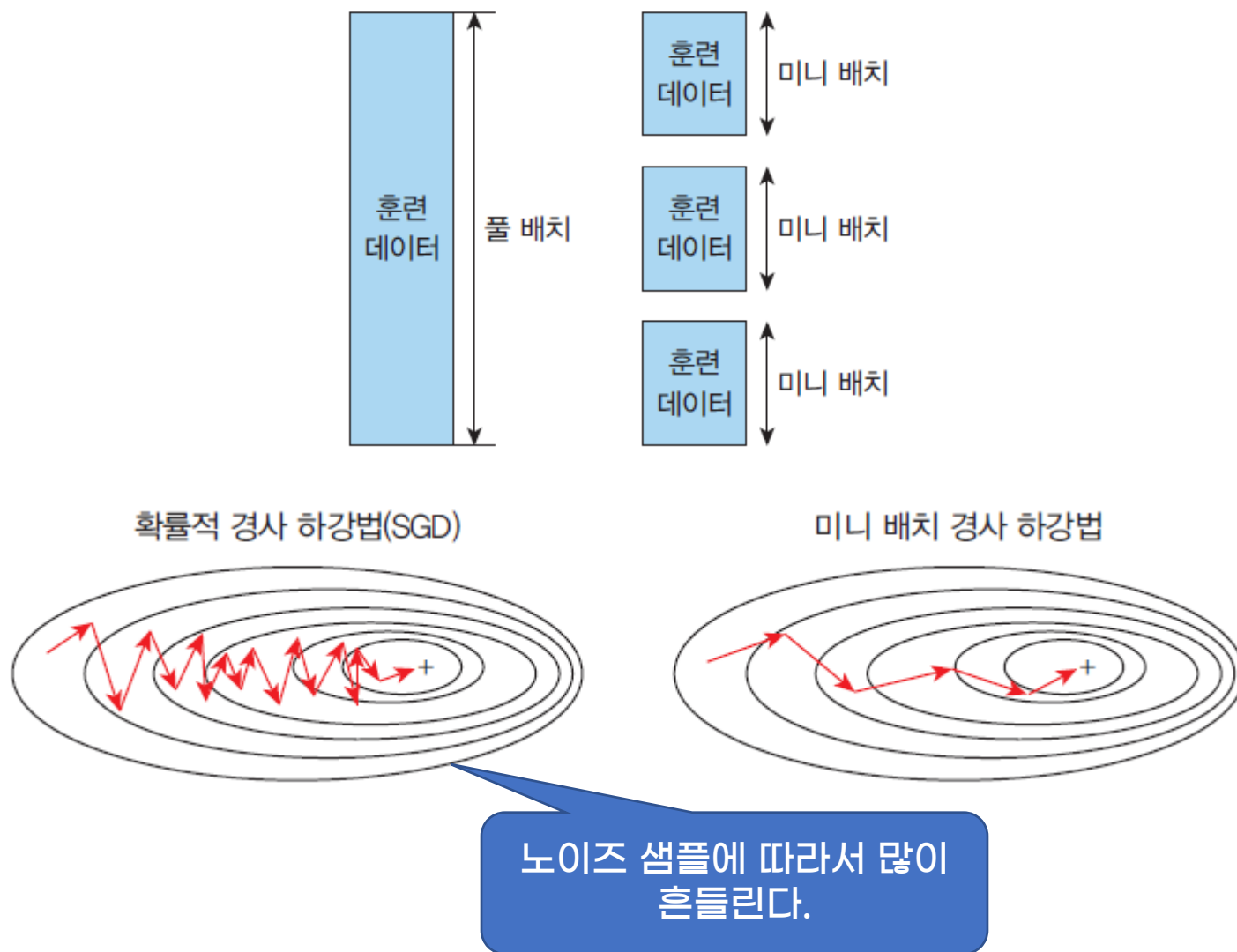
가중치를 감소시킨다.

손실함수를 가중치로 미분한
값이 음수이면



가중치를 증가시킨다.

경사하강법과 미니배치



미니 배치 실습

```
import numpy as np
import tensorflow as tf

# 데이터를 학습 데이터와 테스트 데이터로 나눈다.
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

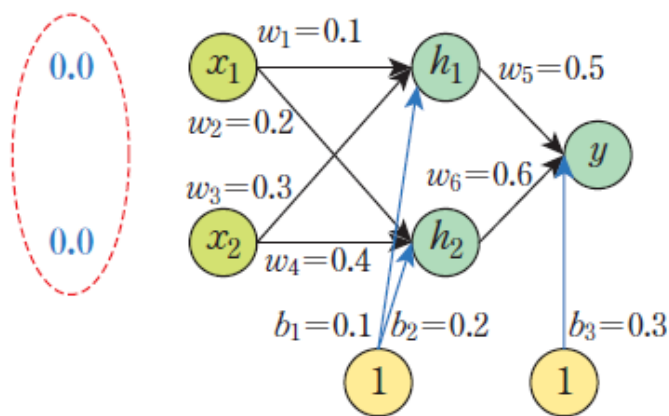
data_size = x_train.shape[0]
batch_size = 12          # 배치 크기

selected = np.random.choice(data_size, batch_size)
print(selected)
x_batch = x_train[selected]
y_batch = y_train[selected]
```

배치 크기만큼 랜덤하게 선택

```
[58298  3085 27743 33570 35343 47286 18267 25804  4632 10890 44164 18822]
```

행렬로 미치 배치 구현하기 (결론 : 행렬)



$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} + [b_1 \ b_2]$$

여기에 다른 예제를 붙이면 어떨까?

$$Z_1 = X \times W + B = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} \\ x_1^{(3)} & x_2^{(3)} \\ x_1^{(4)} & x_2^{(4)} \end{bmatrix} \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} + [b_1 \ b_2]$$

각 샘플별로 출력이 계산된다.

$$A_1 = f(Z_1) = \begin{bmatrix} f(z_1^{(1)}) & f(z_2^{(1)}) \\ f(z_1^{(2)}) & f(z_2^{(2)}) \\ f(z_1^{(3)}) & f(z_2^{(3)}) \\ f(z_1^{(4)}) & f(z_2^{(4)}) \end{bmatrix}$$

$$Z_2 = A_1 \times W_2 + B_2 = \begin{bmatrix} h_1^{(1)} & h_2^{(1)} \\ h_1^{(2)} & h_2^{(2)} \\ h_1^{(3)} & h_2^{(3)} \\ h_1^{(4)} & h_2^{(4)} \end{bmatrix} \times \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} + [b_3] = \begin{bmatrix} z_y^{(1)} \\ z_y^{(2)} \\ z_y^{(3)} \\ z_y^{(4)} \end{bmatrix}$$

$$Y = f(Z_2) = \begin{bmatrix} f(z_y^{(1)}) \\ f(z_y^{(2)}) \\ f(z_y^{(3)}) \\ f(z_y^{(4)}) \end{bmatrix} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ y^{(4)} \end{bmatrix}$$

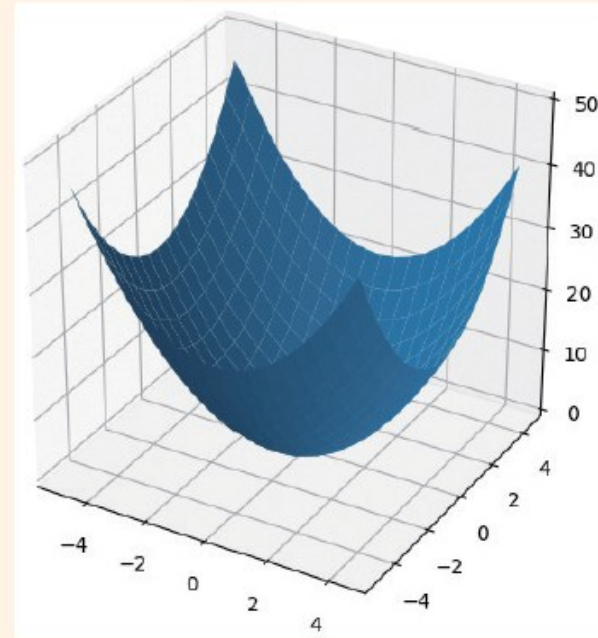
2차원 그래디언트 시각화

```
from mpl_toolkits.mplot3d import axes3d
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(-5, 5, 0.5)
y = np.arange(-5, 5, 0.5)
X, Y = np.meshgrid(x, y) # 참고 박스
Z = X**2 + Y**2           # 넘파이 연산

fig = plt.figure(figsize=(6,6))
ax = fig.add_subplot(111, projection='3d')

# 3차원 그래프를 그린다.
ax.plot_surface(X, Y, Z)
plt.show()
```



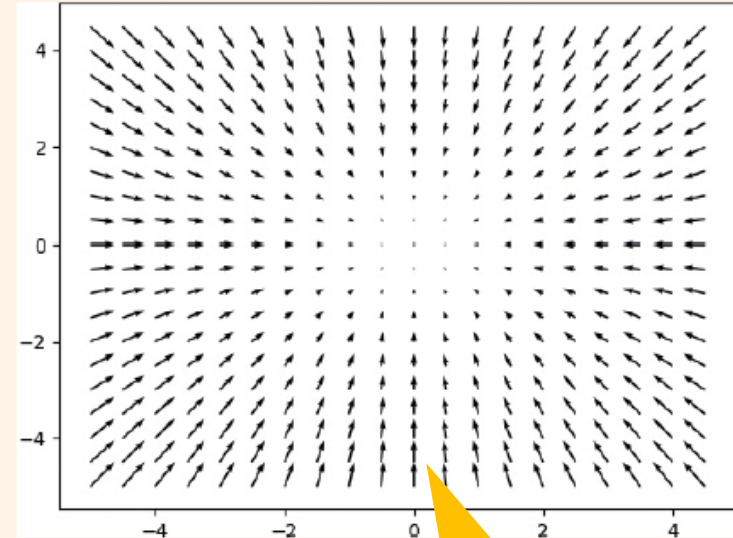
2차원 그래디언트 시각화

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(-5,5,0.5)
y = np.arange(-5,5,0.5)
X, Y = np.meshgrid(x,y)
U = -2*X
V = -2*Y

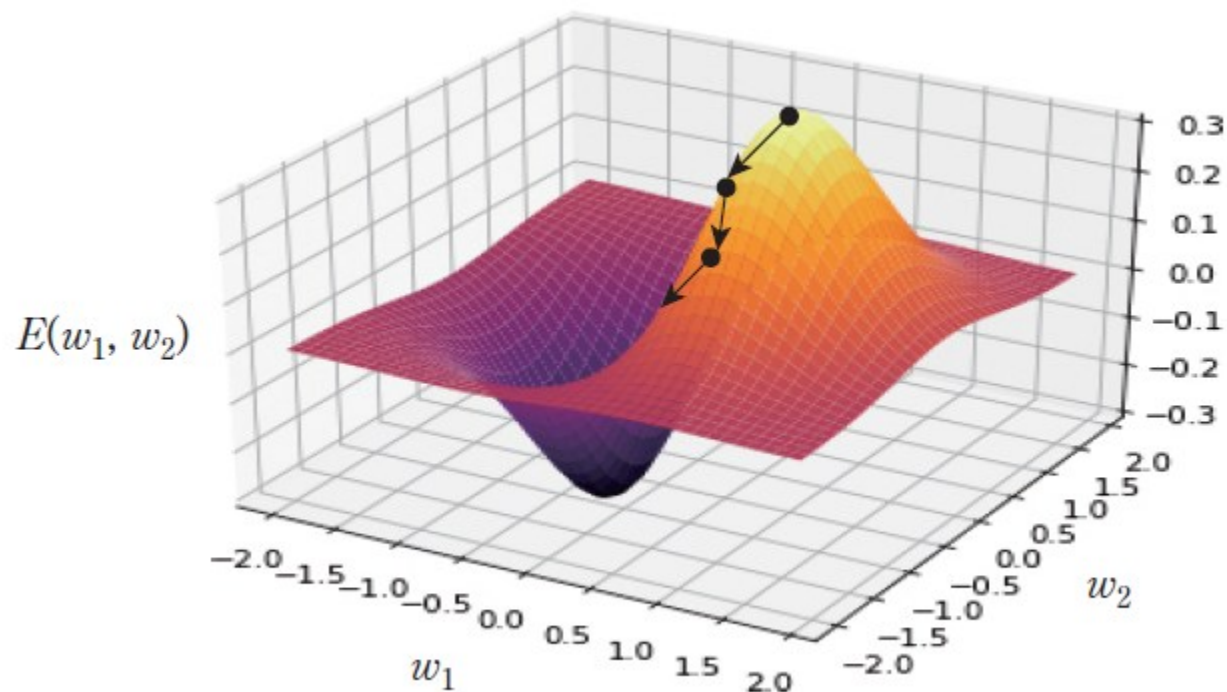
plt.figure()
Q = plt.quiver(X, Y, U, V, units='width')
plt.show()
```

그래디언트의 음수



화살표가 최소값을
가리키고 있음을 알
수 있다.

역전파 학습 알고리즘



넘파이를 이용한 MLP 구현

```
import numpy as np

# 시그모이드 함수
def actf(x):
    return 1 / (1 + np.exp(-x))

# 시그모이드 함수의 미분값
def actf_deriv(x):
    return x * (1-x)

# 입력유닛의 개수, 은닉유닛의 개수, 출력유닛의 개수
(inputs, hiddens, outputs) = (2, 2, 1)
learning_rate=0.2

# 훈련 샘플과 정답
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
T = np.array([[1], [0], [0], [1]]) # target
```

순방향 전파 구현

```
W1 = np.array([[0.10, 0.20], [0.30, 0.40]])
W2 = np.array([[0.50], [0.60]])
B1 = np.array([0.1, 0.2])
B2 = np.array([0.3])

# 순방향 전파 계산
def predict(x):
    layer0 = x                    # 입력을 layer0에 대입한다.
    Z1 = np.dot(layer0, W1) + B1  # 행렬의 곱을 계산한다.
    layer1 = actf(Z1)             # 활성화 함수를 적용한다.
    Z2 = np.dot(layer1, W2) + B2  # 행렬의 곱을 계산한다.
    layer2 = actf(Z2)             # 활성화 함수를 적용한다.
    return (layer0, layer1, layer2)
```

오차 역전파 구현

```
# 역방향 전파 계산
def fit():
    global W1, W2, B1, B2          # 우리는 외부에 정의된 변수를 변경해야 한다.
    for i in range(90_000):        # 9만번 반복한다.
        for (x, y) in zip(X, T):  # T : Target # 학습 샘플을 하나씩 꺼낸다.
            x = np.reshape(x, (1, -1)) # 2차원 행렬로 만든다. ①
            y = np.reshape(y, (1, -1)) # 2차원 행렬로 만든다.

            layer0, layer1, layer2 = predict(x)          # 순방향 계산
            layer2_error = layer2 - y                    # 오차 계산
            layer2_delta = layer2_error * actf_deriv(layer2) # 출력층의 델타 계산
            layer1_error = np.dot(layer2_delta, W2.T)      # 은닉층의 오차 계산 ②
            layer1_delta = layer1_error * actf_deriv(layer1) # 은닉층의 델타 계산 ③

            W2 += -learning_rate * np.dot(layer1.T, layer2_delta) # ④
            W1 += -learning_rate * np.dot(layer0.T, layer1_delta) #
            B2 += -learning_rate * np.sum(layer2_delta, axis=0)    # ⑤
            B1 += -learning_rate * np.sum(layer1_delta, axis=0)    #
```

오차 역전파 구현

```
def test():  
    for x, y in zip(X, T):  
        x = np.reshape(x, (1, -1))      # 하나의 샘플을 꺼내서 2차원 행렬로 만든다.  
        layer0, layer1, layer2 = predict(x)  
        print(x, y, layer2)             # 출력층의 값을 출력해본다.  
  
fit()  
test()
```

```
[[0 0]] [1] [[0.99196032]]  
[[0 1]] [0] [[0.00835708]]  
[[1 0]] [0] [[0.00836107]]  
[[1 1]] [1] [[0.98974873]]
```


손실 함수

- 직선과 데이터 사이의 간격을 제공하여 합한 값을 손실 함수(loss function) 또는 비용 함수(cost function)라고 한다.

$$Loss = \frac{1}{3} ((f(x_1) - y_1)^2 + (f(x_2) - y_2)^2 + (f(x_3) - y_3)^2)$$

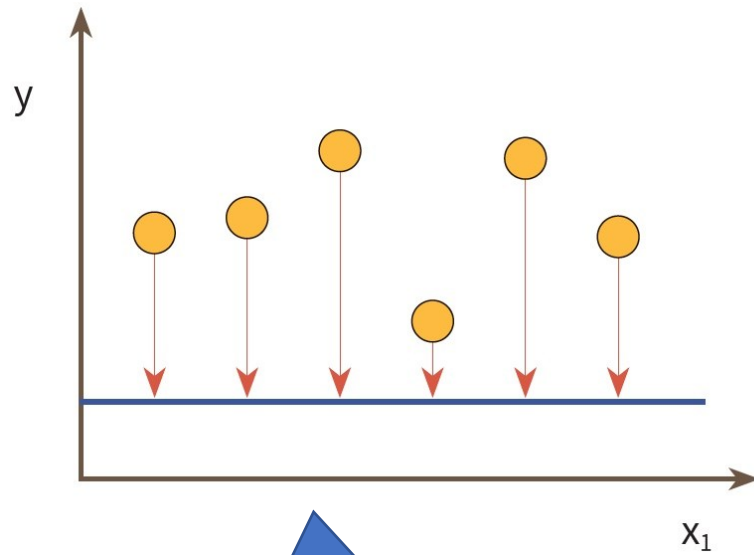


$$Loss(W, b) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2 = \frac{1}{n} \sum_{i=1}^n ((Wx_i + b) - y_i)^2$$

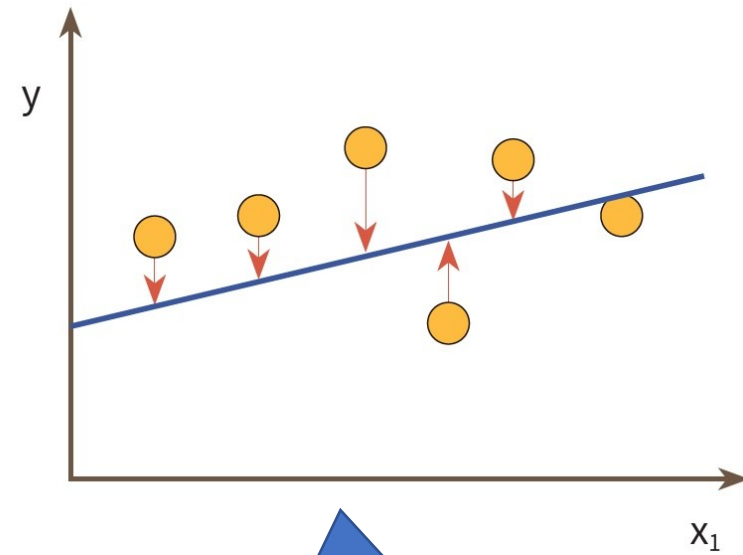


$$\underset{W, b}{\operatorname{argmin}} Loss(W, b)$$

손실 함수



손실이 큰 경우



손실이 작은 경우

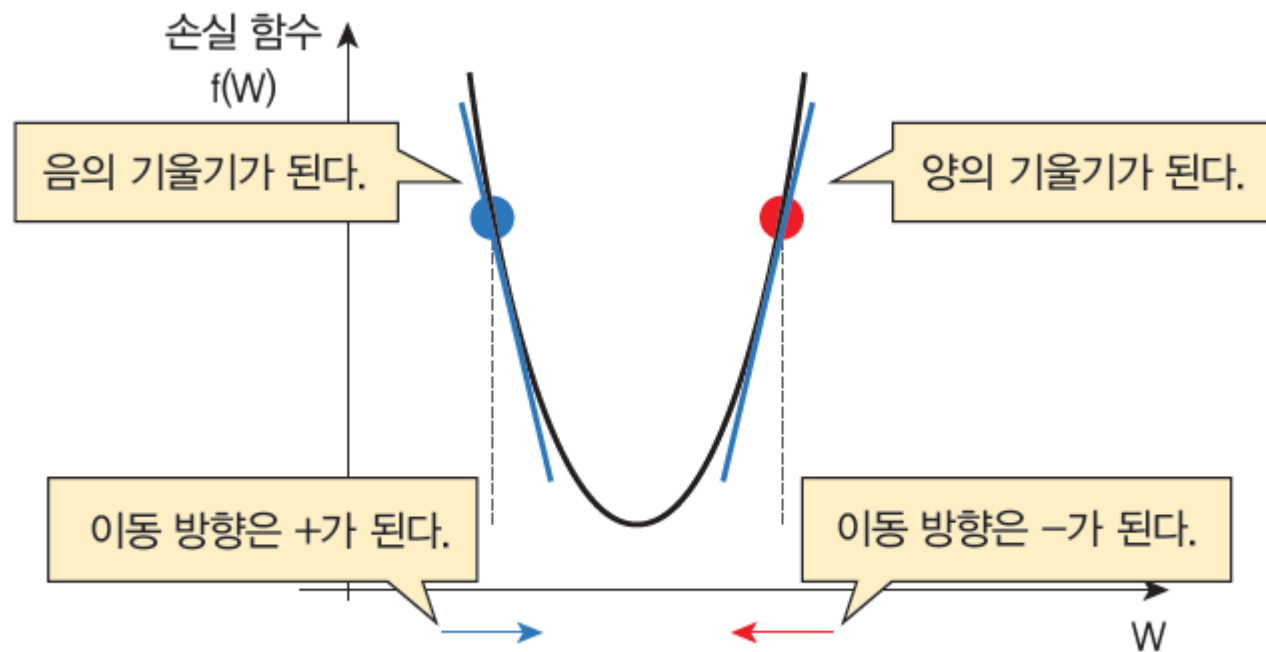
선형 회귀에서 손실 함수 최소화 방법

- 분석적인 방법: 독립 변수와 종속 변수가 각각 하나인 선형 회귀

$$w = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad b = \bar{y} - w\bar{x}$$

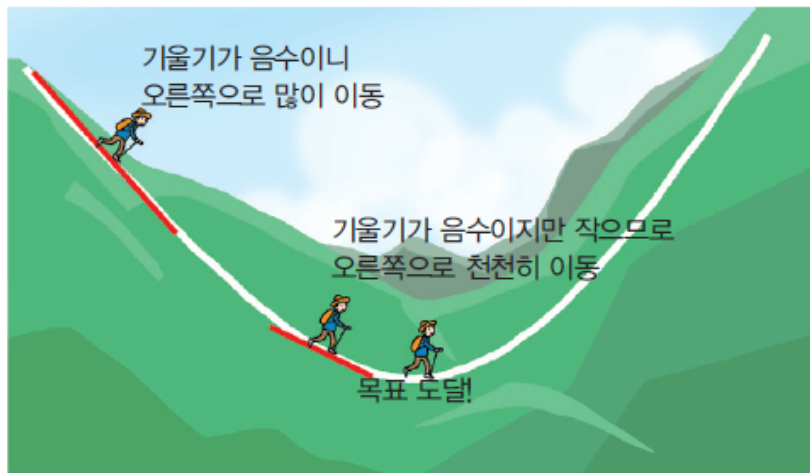
- 경사 하강법(gradient descent method):
경사 하강법은 손실 함수가 어떤 형태이러도, 또 매개 변수가 아무리 많아도 적용할 수 있는 일반적인 방법이다.
- 점진적인 학습이 가능하다

경사하강법



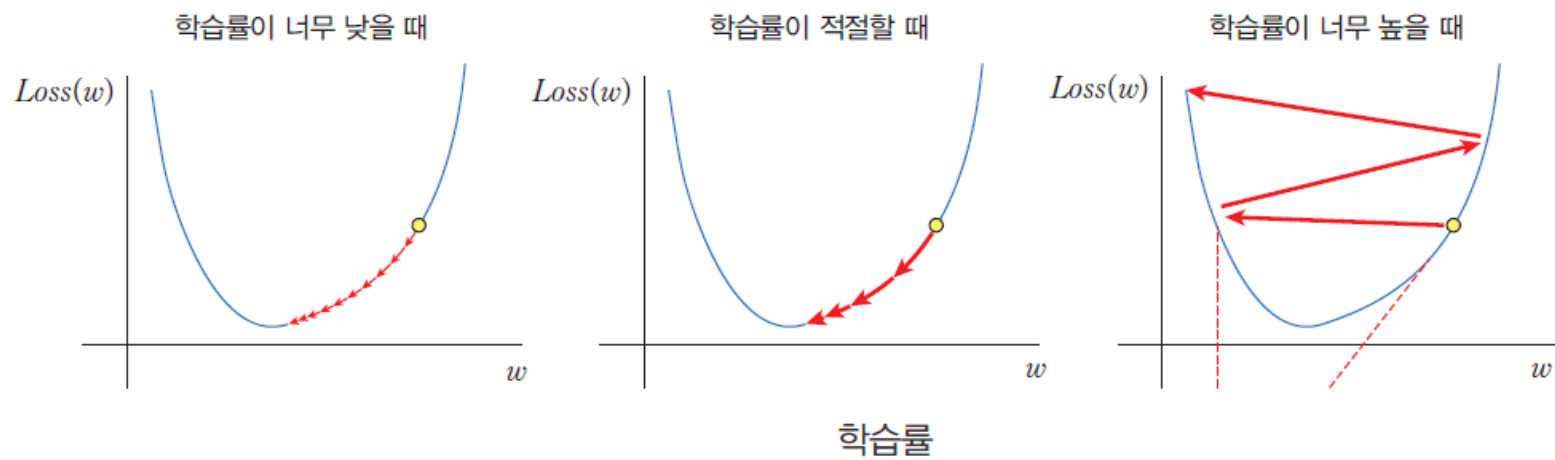
경사 하강법의 이해

경사하강법

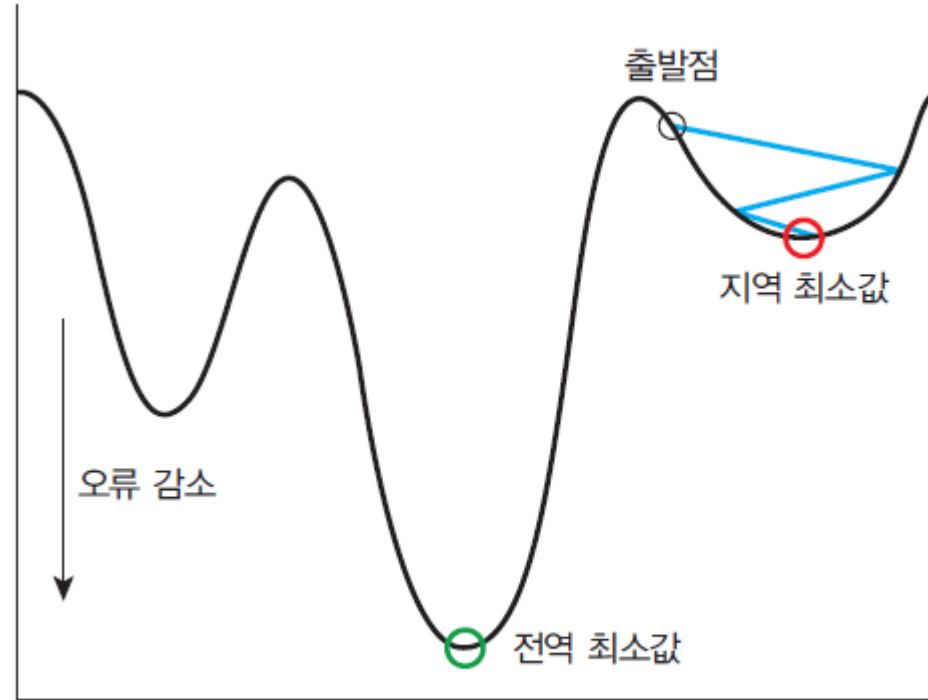


이것은 마치 산에서 내려오는 것과 유사합니다. 현재 위치에서 산의 기울기를 계산하여서 기울기의 반대 방향으로 이동하면 산에서 내려오게 됩니다.

학습률



지역최소값 문제



지역 최소값과 전역 최소값

선형 회귀에서 경사하강법

$$Loss(W, b) = \frac{1}{n} \sum_{i=1}^n ((Wx_i + b) - y_i)^2$$

$$\frac{\partial Loss(W, b)}{\partial W} = \frac{1}{n} \sum_{i=1}^n 2((Wx_i + b) - y_i)(x_i) = \frac{2}{n} \sum_{i=1}^n x_i((Wx_i + b) - y_i)$$

$$\frac{\partial Loss(W, b)}{\partial b} = \frac{2}{n} \sum_{i=1}^n ((Wx_i + b) - y_i)$$

$$W = W - 0.01 * \frac{\partial Loss}{\partial W}$$

$$b = b - 0.01 * \frac{\partial Loss}{\partial b}$$

경사 하강법 구현

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([0.0, 1.0, 2.0])
y = np.array([3.0, 3.5, 5.5])

W = 0          # 기울기
b = 0          # 절편

lr = 0.01      # 학습률
epochs = 1000  # 반복 횟수

n = float(len(X)) # 입력 데이터의 개수

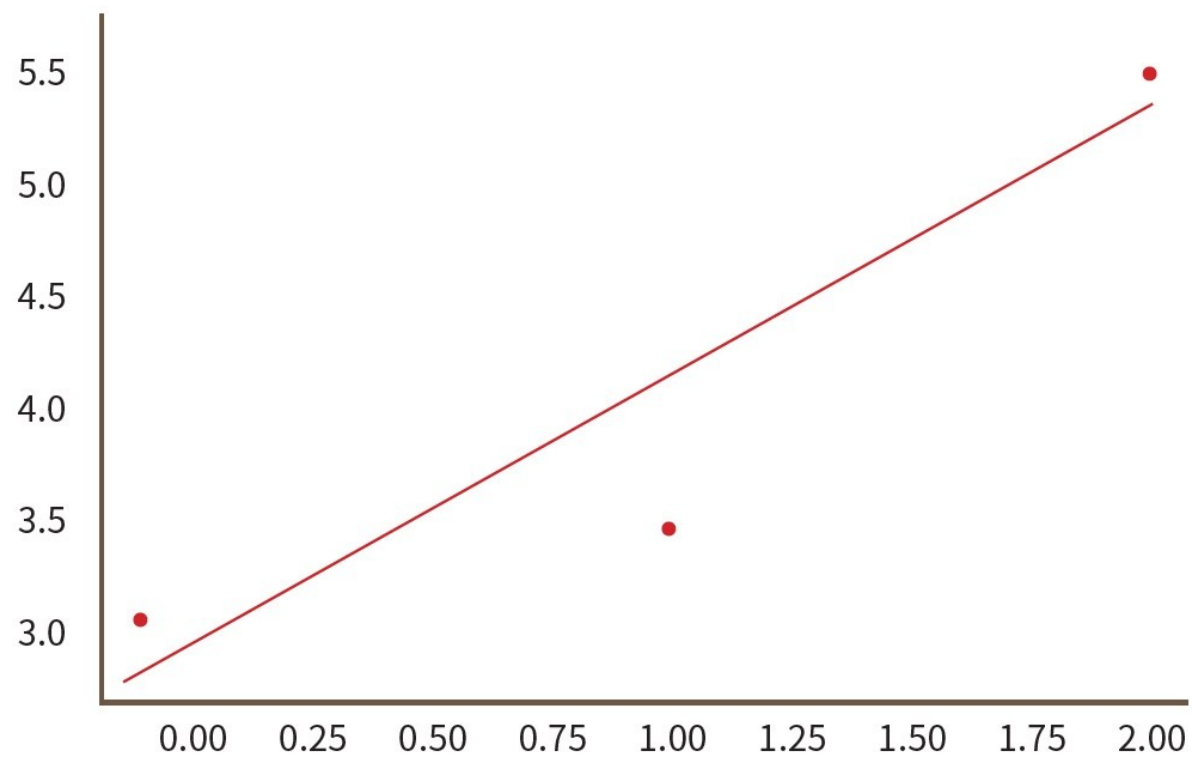
# 경사 하강법
for i in range(epochs):
    y_pred = W*X + b # 예측값
    dW = (2/n) * sum(X * (y_pred-y))
    db = (2/n) * sum(y_pred-y)
    W = W - lr * dW # 기울기 수정
    b = b - lr * db # 절편 수정
```

경사 하강법 구현

```
# 기울기와 절편을 출력한다.  
print (W, b)  
  
# 예측값을 만든다.  
y_pred = W*X + b  
  
# 입력 데이터를 그래프 상에 찍는다.  
plt.scatter(X, y)  
  
# 예측값은 선그래프로 그린다.  
plt.plot([min(X), max(X)], [min(y_pred), max(y_pred)],  
color='red')  
plt.show()
```

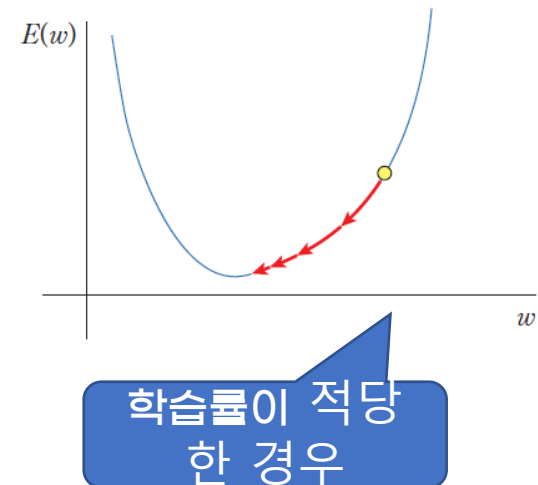
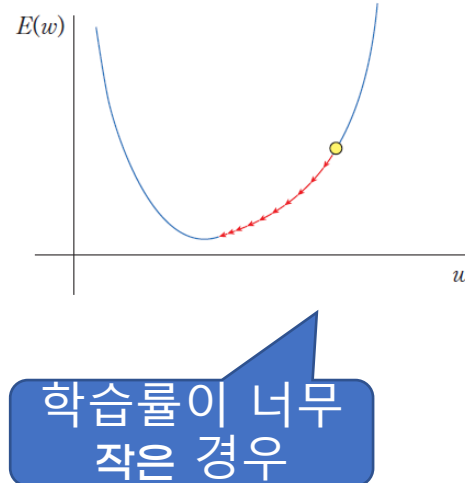
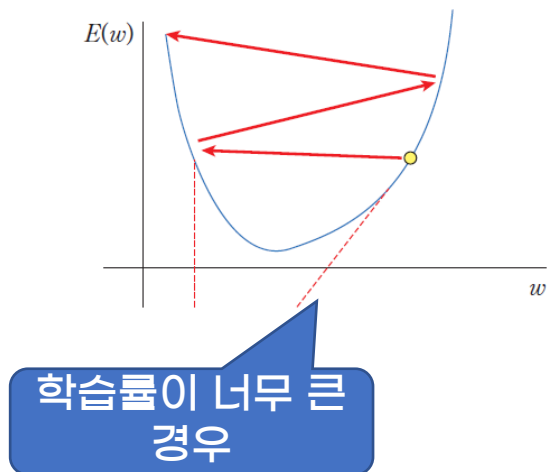
1.2532418085611319 2.745502230882486

경사 하강법 구현



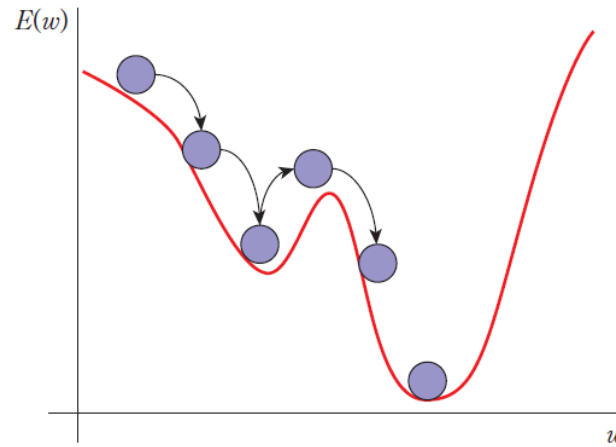
학습률

- 학습률이란 한 번에 가중치를 얼마나 변경할 것인가를 나타낸다.
- 학습률이 모델의 성능에 심대한 영향을 끼치지만 설정하기가 아주 어렵다는 것을 발견하였다.



모멘텀 (학습률 옵션)

- 모멘텀(momentum)은 운동량으로 학습 속도를 가속시킬 목적으로 사용한다



$$W_{t+1} = W_t - \eta \frac{\partial E}{\partial W} + \text{momentum} * W_t$$

학습률 설정 방법

- Adagrad: Adagrad는 가변 학습률을 사용하는 방법으로 SGD 방법을 개량한 최적화 방법이다.
주된 방법은 학습률 감쇠(learning rate decay)이다.
Adagrad는 학습률을 이전 단계의 기울기들을 누적한 값에 반비례하여서 설정한다

$$W_{t+1} = W_t - \eta \frac{1}{\sqrt{G_t + \epsilon}} \frac{\partial E}{\partial W}$$

학습률이 점점 줄어든다.

학습률 설정 방법

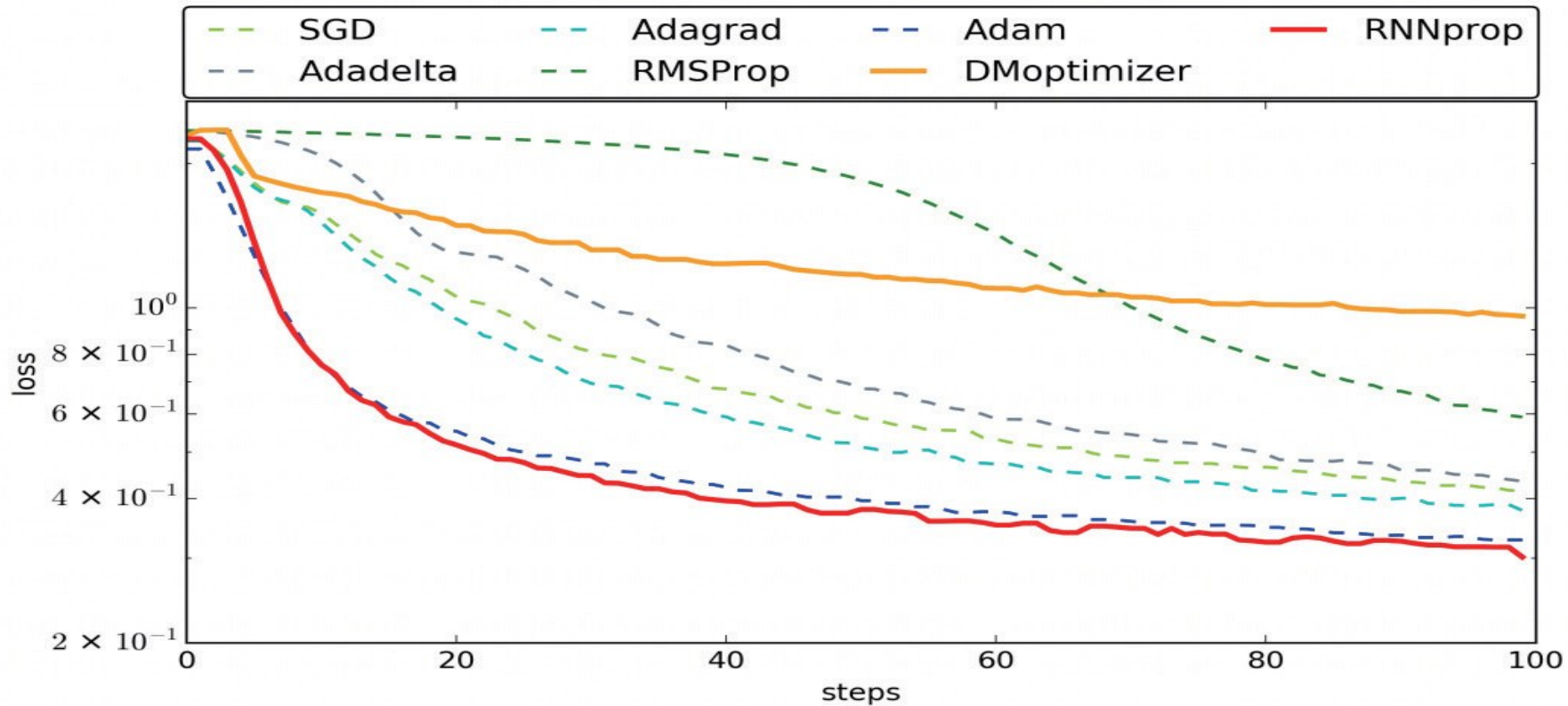
- RMSprop : RMSprop은 Adagrad에 대한 수정판이다. Geoffy Hinton이 Coursea 강의 과정에서 제안했다.
- Adadelta와 유사하지만 한 가지 차이점이 있다.
- 그래디언트 누적 대신에 지수 가중 이동 평균을 사용한다

$$v(t) = \rho v(t-1) + (1-\rho) * \left[\frac{\partial E}{\partial W} \right]^2$$

$$W_{t+1} = W_t - \eta \frac{1}{\sqrt{v_t + \epsilon}} \frac{\partial E}{\partial W}$$

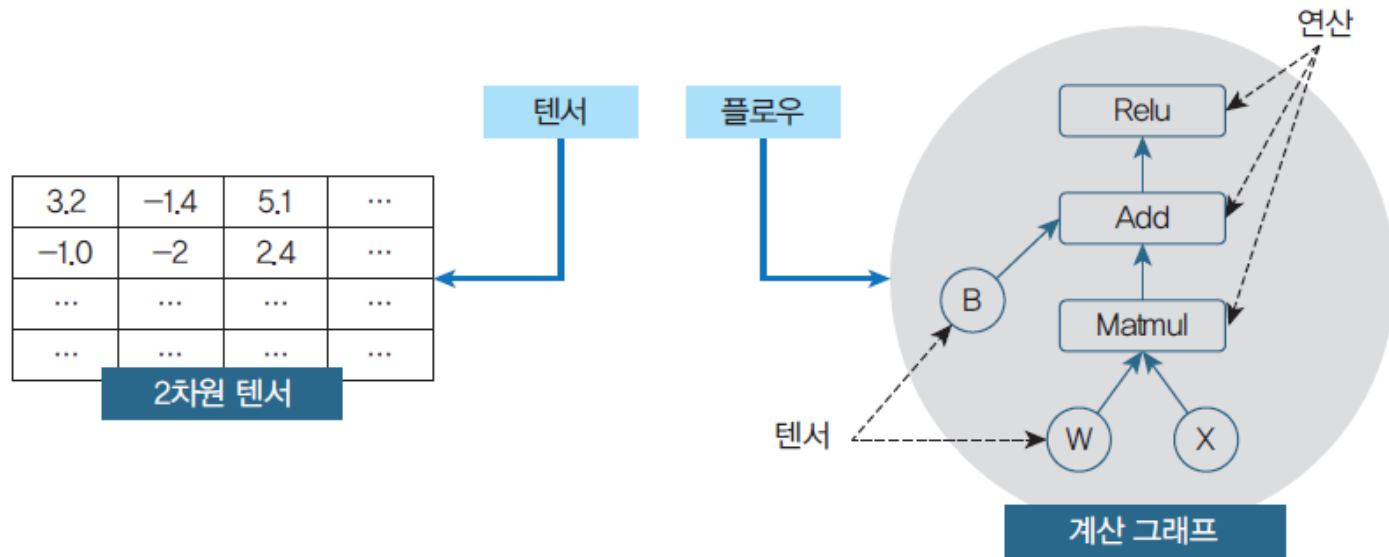
학습률 설정 방법

- Adam: Adam은 Adaptive Moment Estimation의 약자이다. Adam은 기본적으로 (RMSprop+ 모멘텀)이다.



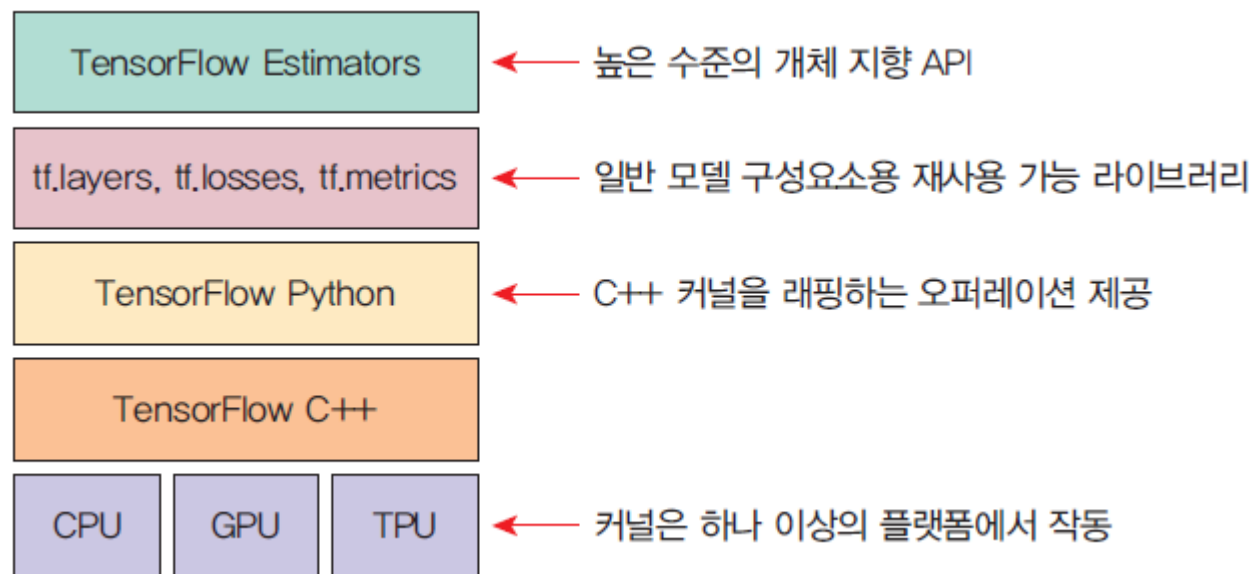
텐서플로우와 케라스

- 텐서플로우(TensorFlow)는 딥러닝 프레임워크의 일종이다. 텐서플로우는 내부적으로 C/C++로 구현되어 있고 파이썬을 비롯하여 여러 가지 언어에서 접근할 수 있도록 인터페이스를 제공한다.



텐서플로우의 개념

텐서플로우의 구조

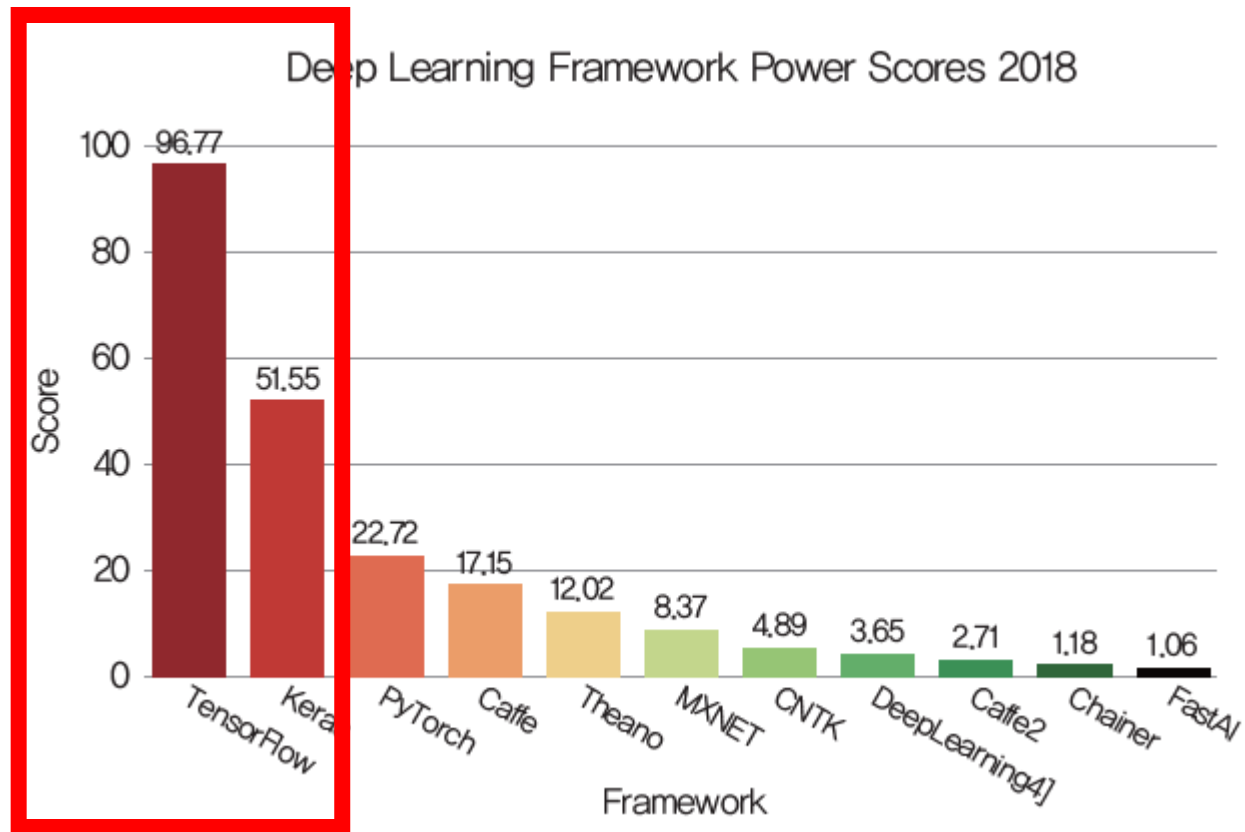


케라스

- 케라스는 파이썬으로 작성되었으며, **고수준 딥러닝 API**이다. 케라스에서는 여러 가지 백엔드를 선택할 수 있지만, 아무래도 가장 많이 선택되는 백엔드는 텐서플로우이다
- 쉽고 빠른 프로토타이핑이 가능하다.
- 피드포워드 신경망, 컨볼루션 신경망과 순환 신경망은 물론, 여러 가지의 조합도 지원한다.
- CPU 및 GPU에서 원활하게 실행된다.

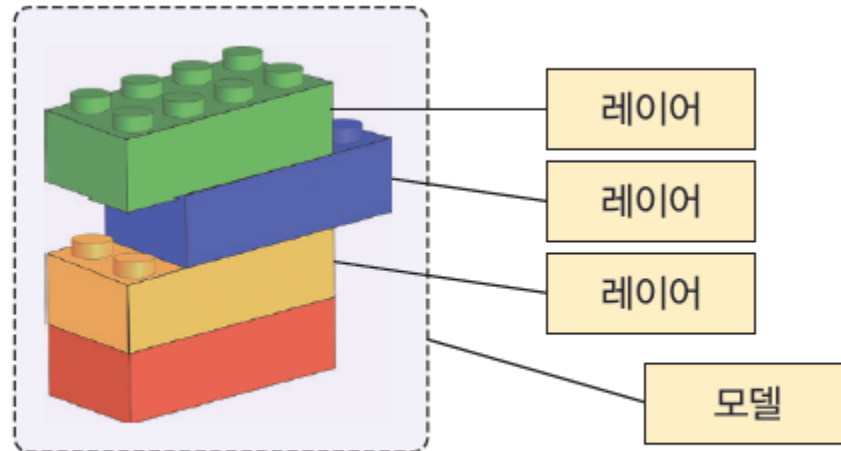


딥러닝 프레임워크



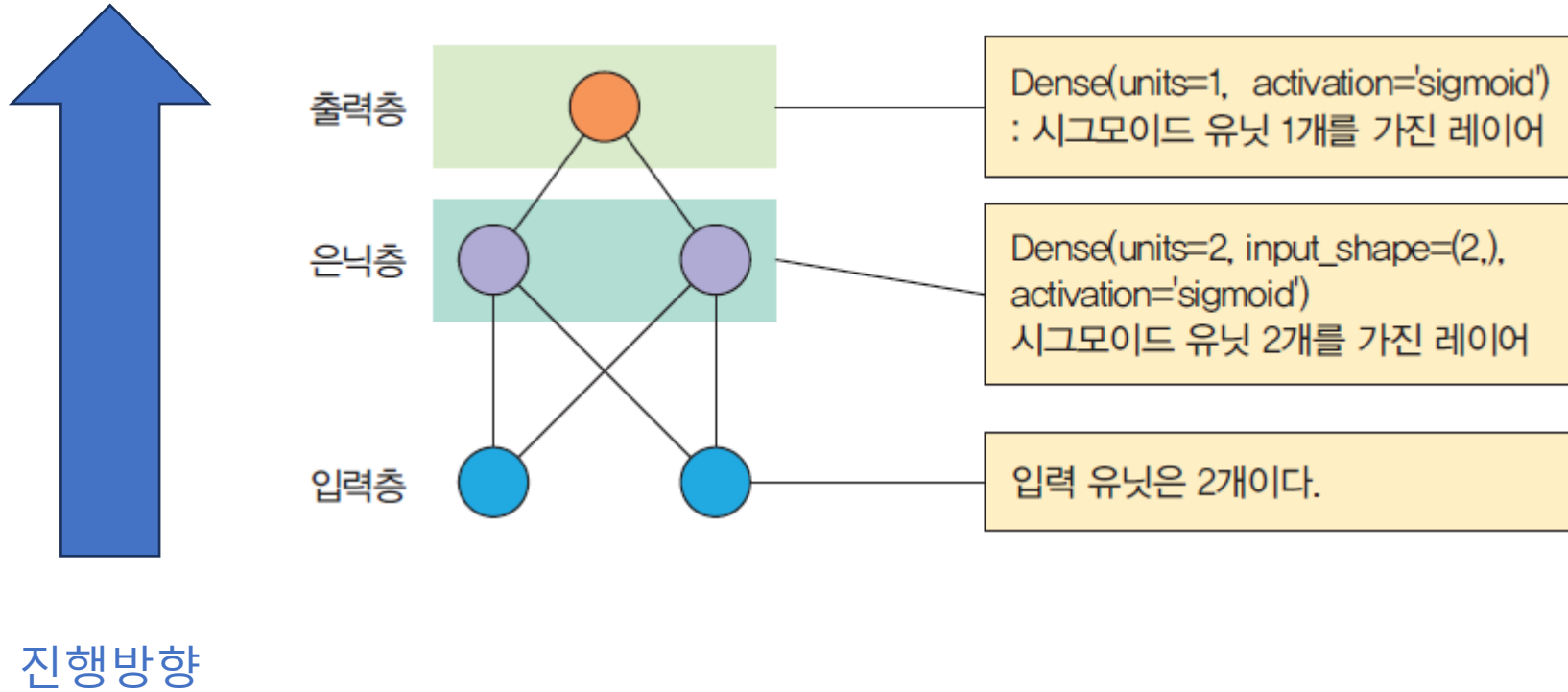
케라스로 신경망을 작성하는 절차

- 케라스의 핵심 데이터 구조는 모델(model)이며 이것은 레이어를 구성하는 방법을 나타낸다.
- 가장 간단한 모델 유형은 Sequential 선형 스택 모델이다. Sequential 모델은 레이어를 선형으로 쌓을 수 있는 신경망 모델이다



케라스의 기본 개념

예제: XOR를 학습하는 MLP를 작성

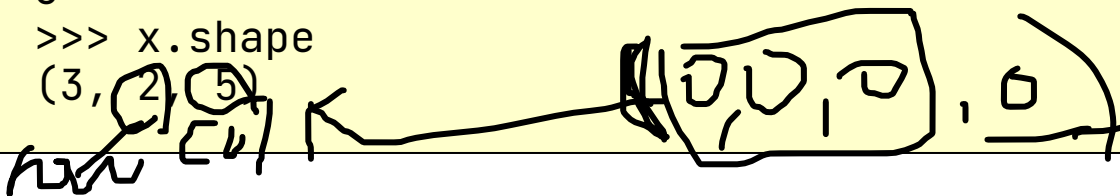


텐서 (Tensor)

- 텐서는 다차원 넘파이 배열이다.
- 텐서는 데이터(실수)를 저장하는 컨테이너라고 생각하면 된다. 텐서에서는 배열의 차원을 축(axis)이라고 부른다.
- 예를 들어서 3차원 텐서는 다음과 같이 생성할 수 있다.

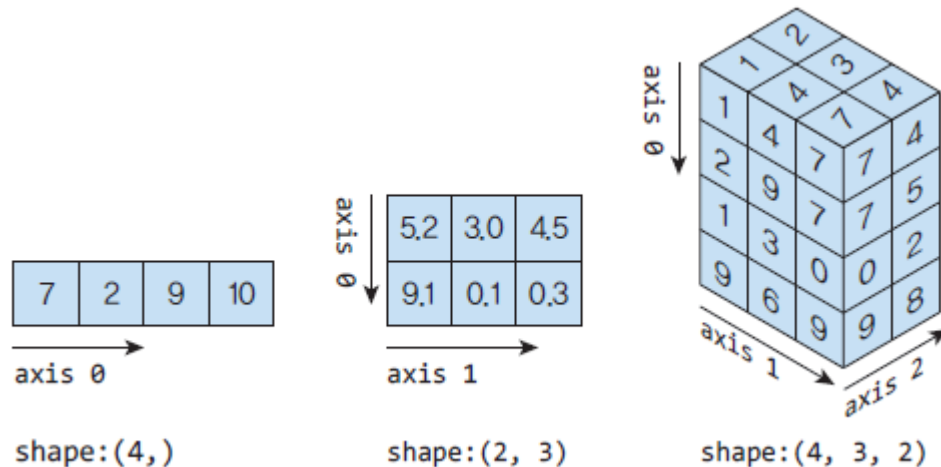
```
>>> import numpy as np
x = np.array(
    [
        [[0, 1, 2, 3, 4],
         [5, 6, 7, 8, 9]],
        [[10, 11, 12, 13, 14],
         [15, 16, 17, 18, 19]],
        [[20, 21, 22, 23, 24],
         [25, 26, 27, 28, 29]],
    ])
>>> x.ndim
3
>>> x.shape
(3, 2, 5)
```

Axis
1 - gray scale
3 - color



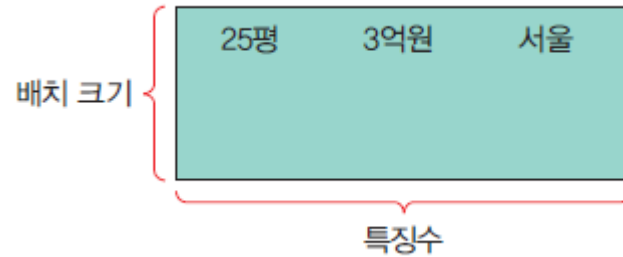
텐서의 속성

- 텐서의 차원(축의 개수): 텐서에 존재하는 축의 개수이다. 3차원 텐서에는 3개의 축이 있다. ndim 속성
- 형상(shape): 텐서의 각 축으로 얼마나 데이터가 있는지를 파이썬의 튜플로 나타낸 것이다.
- 데이터 타입(data type): 텐서 요소의 자료형.

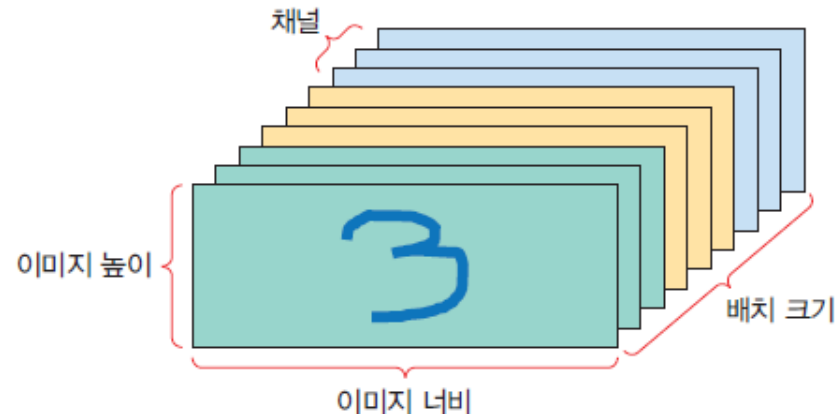


훈련 데이터의 형상

- 벡터 데이터: (배치 크기, 특징수)의 형상을 가진다.

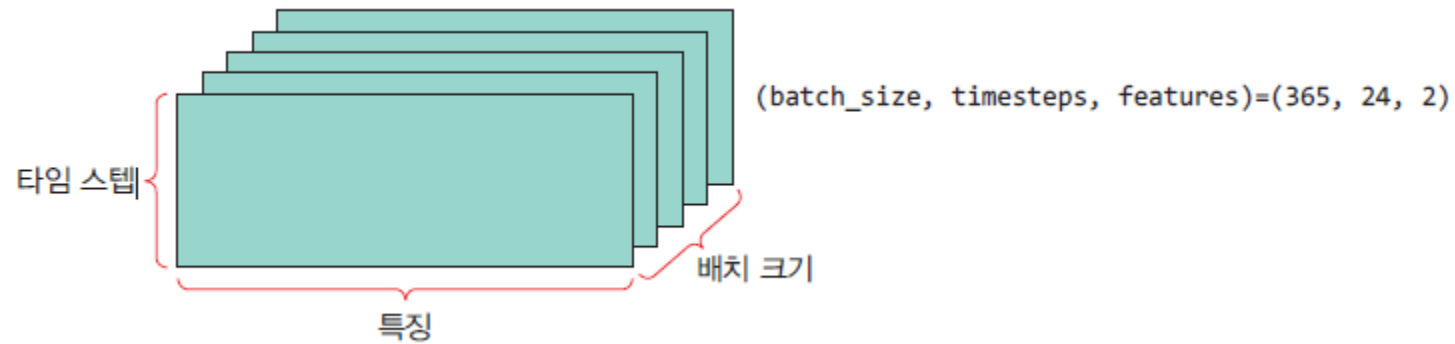


- 이미지 데이터: (배치 크기, 이미지 높이, 이미지 너비, 채널수) 형상의 4차원 넘파이 텐서에 저장된다



훈련 데이터의 형상

- 시계열 데이터: (배치 크기, 타임 스텝, 특징수) 형상의 3차원 넘파이 텐서에 저장된다.



예제: XOR를 학습하는 MLP를 작성

훈련 데이터

	x1	x2		y
샘플 #1	0	0		0
샘플 #2	0	1		1
샘플 #3	1	0		1
샘플 #4	1	1		0

예제: XOR를 학습하는 MLP를 작성

Keras Sequential Modeling

```
import tensorflow as tf
model = tf.keras.models.Sequential()
```

Sequential 모델을 생성

```
model.add(tf.keras.layers.Input(shape=(2,)))
model.add(tf.keras.layers.Dense(units=16, activation='relu'))
model.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

Sequential 모델에 add() 함수를 이용하여 필요한 레이어를 추가

```
X = tf.constant([[0, 0], [0, 1], [1, 0], [1, 1]])
y = tf.constant([0, 1, 1, 0])
model.compile(loss='binary_crossentropy', optimizer=keras.optimizers.SGD(learning_rate=0.5))
```

compile() 함수를 호출하여서 Sequential 모델을 컴파일한다

```
model.fit(X, y, batch_size=4, epochs=500)
```

fit()를 호출하여서 학습을 수행한다.

```
print(model.predict(X))
```

predict()를 호출하여서 모델을 테스트한다

예제: XOR를 학습하는 MLP를 작성

model.summary()

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 2)	6
dense_1 (Dense)	(None, 1)	3

Total params: 9

Trainable params: 9

Non-trainable params: 0

예제: XOR를 학습하는 MLP를 작성

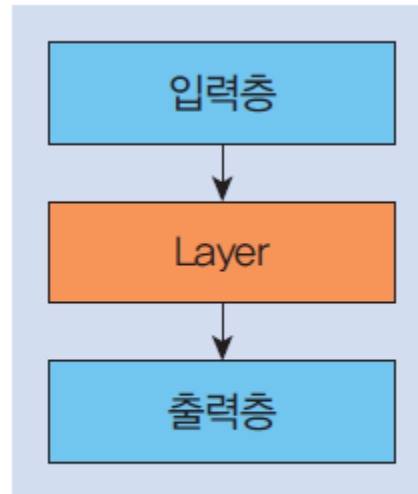
실행 결과

```
Epoch 498/500  
1/1 _____ 0s 47ms/step - loss: 0.0105  
  
Epoch 499/500  
1/1 _____ 0s 45ms/step - loss: 0.0105  
  
Epoch 500/500  
1/1 _____ 0s 46ms/step - loss: 0.0105  
  
1/1 _____ 0s 49ms/step  
  
[[0.02744537]  
 [0.9959539 ]  
 [0.99374884]  
 [0.00363332]]
```

케라스를 사용하는 3가지 방법

(1) Sequential 모델을 만들고 모델에 필요한 레이어를 추가하는 방법이다.

```
model = Sequential()  
model.add(Input(shape=(2,)))  
model.add(Dense(units=4, activation='relu'))  
model.add(Dense(units=1, activation='sigmoid'))
```



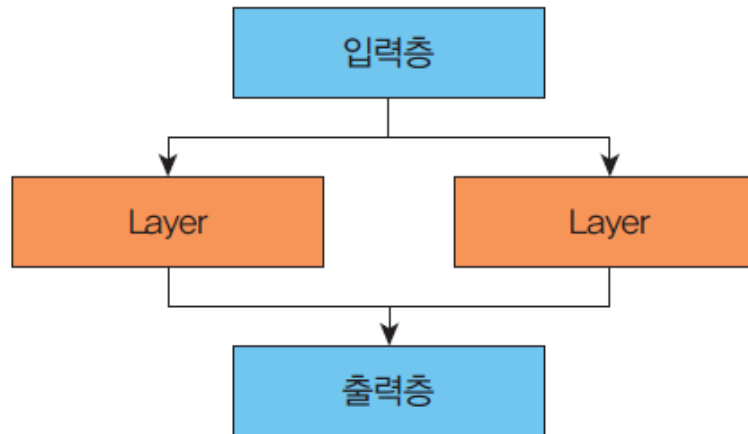
Sequential

케라스를 사용하는 3가지 방법

(2) 함수형 API를 사용하는 방법

```
inputs = Input(shape=(2,))  
x = Dense(4, activation="sigmoid")(inputs)  
prediction = Dense(1, activation="sigmoid")(x)  
  
model = Model(inputs=inputs, outputs=prediction)
```

입력층
은닉층 ①
출력층



케라스를 사용하는 3가지 방법

(3) Model 클래스를 상속받아서 사용자 나름대로의 클래스를 정의하는 방법

```
class SimpleMLP(Model):  
  
    def __init__(self, num_classes): # 생성자 작성  
        super(SimpleMLP, self).__init__(name='mlp')  
        self.num_classes = num_classes  
  
        self.dense1 = Dense(32, activation='sigmoid')  
        self.dense2 = Dense(num_classes, activation='sigmoid')  
  
    def call(self, inputs): # 순방향 호출을 구현한다.  
        x = self.dense1(inputs)  
        return self.dense2(x)  
  
model = SimpleMLP()  
model.compile(...)  
model.fit(...)
```

케라스의 입력 데이터

- 넘파이 배열 : `numpy.ndarray`
- TensorFlow Dataset 객체: 크기가 커서, 메모리에 한 번에 적재될 수 없는 경우에 디스크 또는 분산 파일 시스템에서 스트리밍될 수 있다.
- 파이썬 제너레이터: 예를 들어서 `keras.utils.Sequence` 클래스는 하드 디스크에 위치한 파일을 읽어서 순차적으로 케라스 모델로 공급할 수 있다.

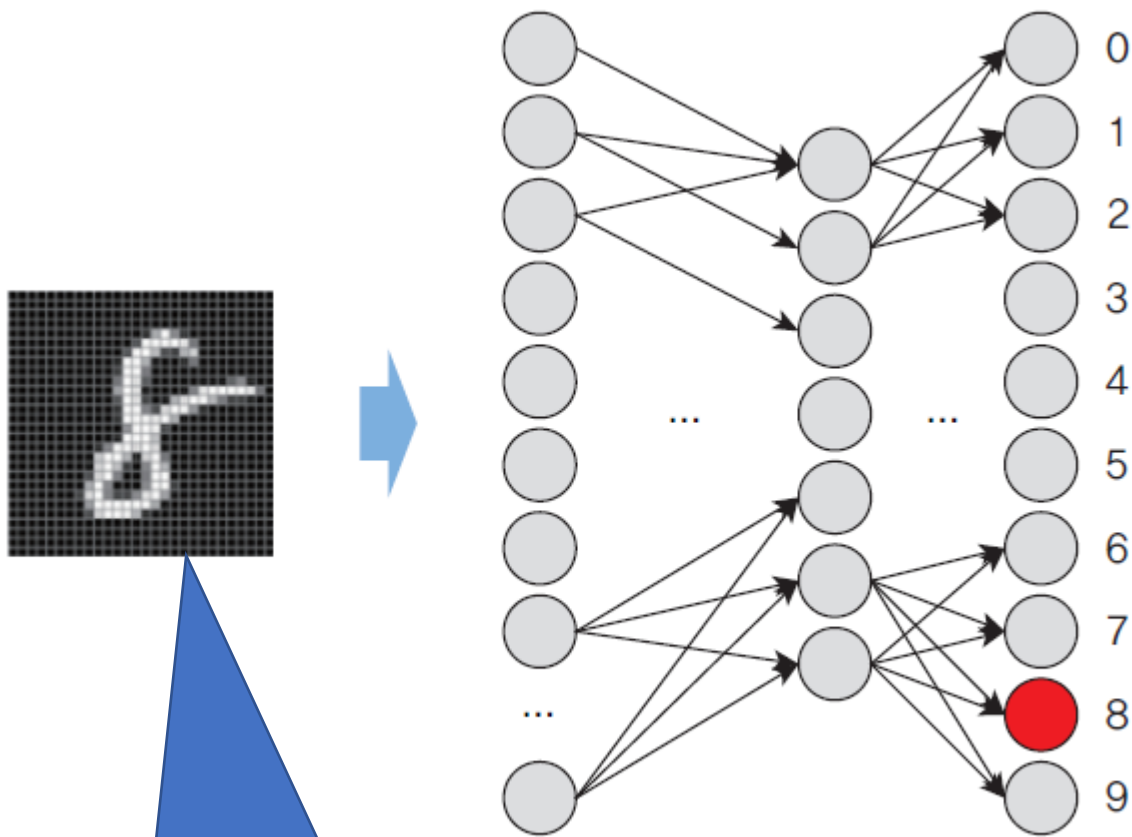
MNIST 필기체 숫자 데이터 세트

- 1980년대에 미국의 국립표준 연구소(NIST)에서 수집한 데이터 세트로 6만개의 훈련 이미지와 1만개의 테스트 이미지로 이루어져 있다

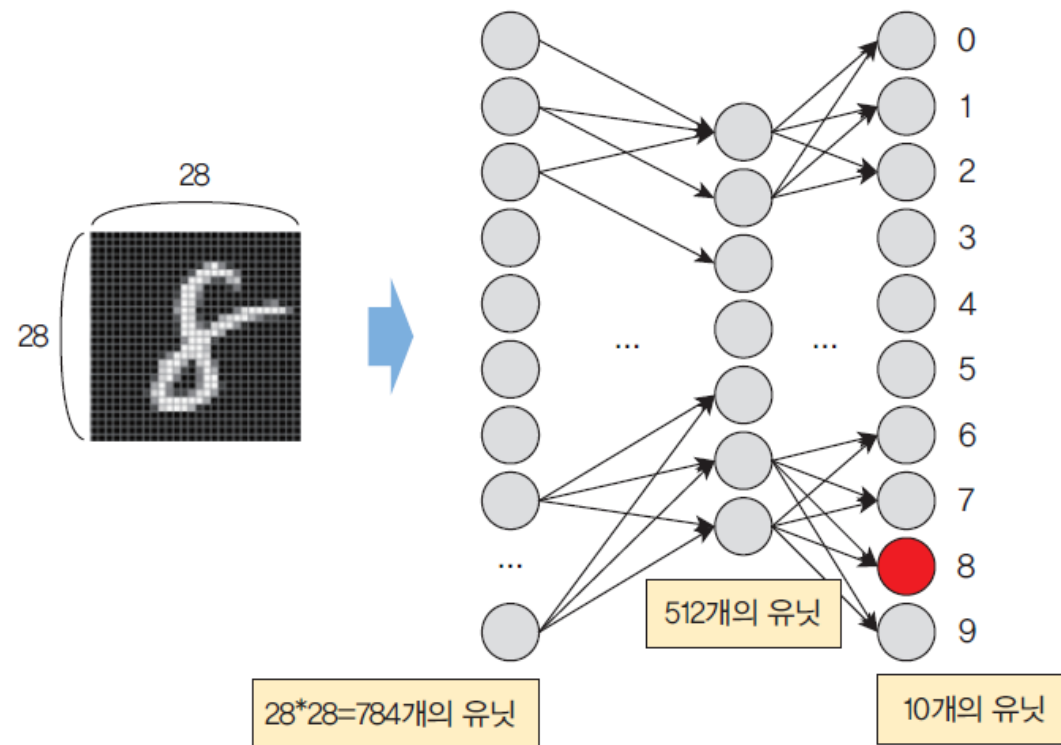


MNIST 데이터

케라스를 이용한 MNIST 숫자 인식



딥러닝의 “필기체 인식” 예제



예제 : MNIST 필기체 숫자 인식 (MLP)

숫자 데이터 가져오기

```
import matplotlib.pyplot as plt
import tensorflow as tf

(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()

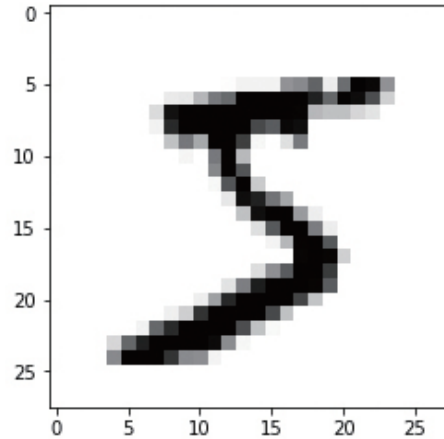
>>> train_images.shape
(60000, 28, 28)

>>> train_label
array([5, 0, 4, ..., 5, 6, 8], dtype=uint8)

>>> test_images.shape
(10000, 28, 28)
```

숫자 데이터 표시하기

```
>>> plt.imshow(train_images[0], cmap="Greys")
```



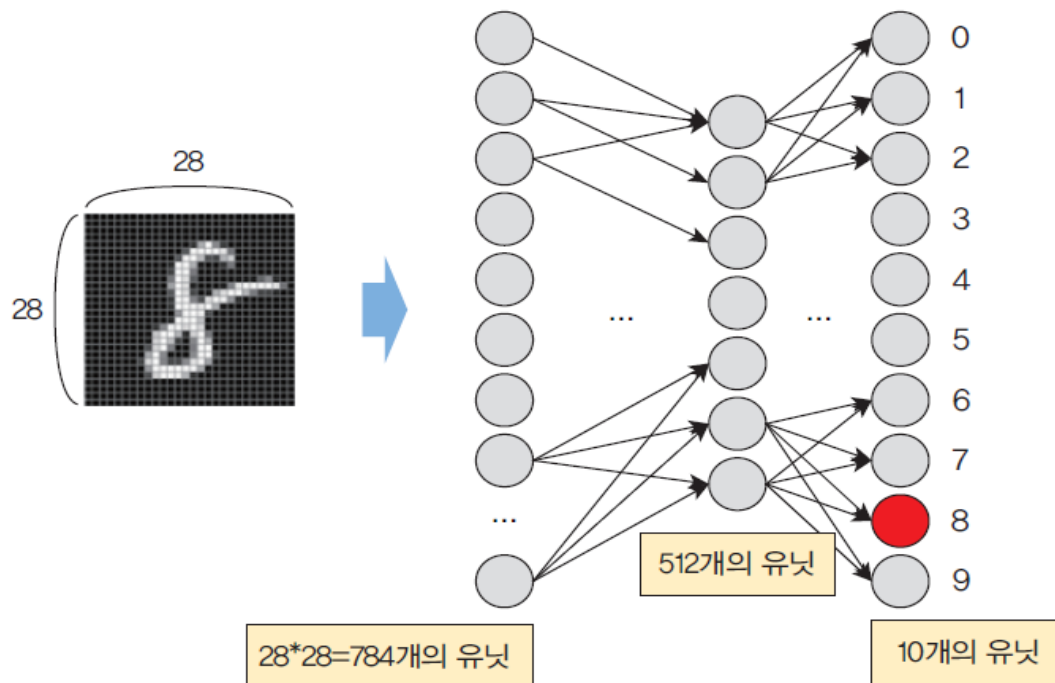
신경망 구축하기

```
model = tf.keras.models.Sequential()
```

```
model.add(tf.keras.models.Input(shape=(784,)))
```

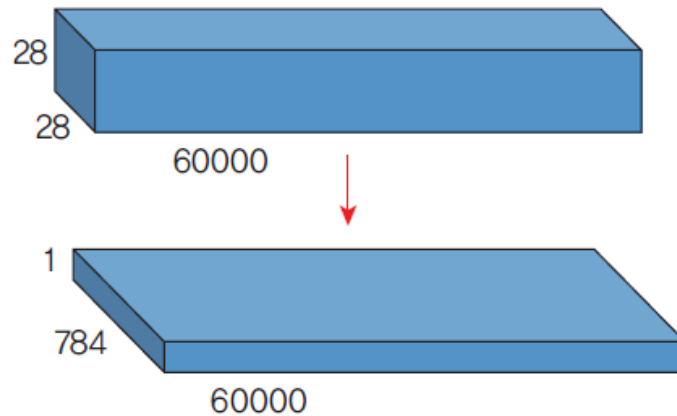
```
model.add(tf.keras.layers.Dense(512, activation='relu'))
```

```
model.add(tf.keras.layers.Dense(10, activation='softmax'))
```



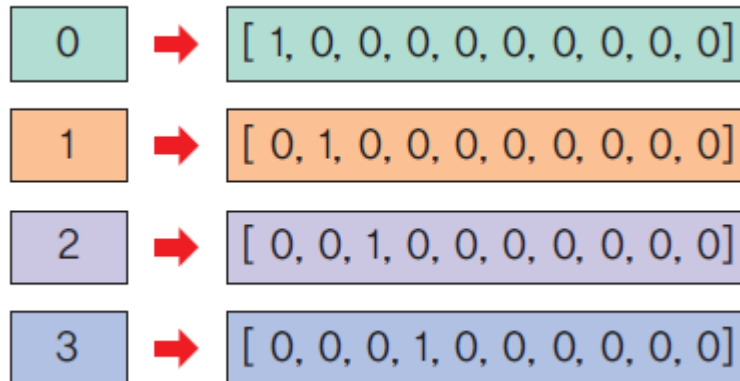
데이터 전처리

```
train_images = train_images.reshape((60000, 784))  
train_images = train_images.astype('float32') / 255.0  
  
test_images = test_images.reshape((10000, 784))  
test_images = test_images.astype('float32') / 255.0
```



정답 레이블 형태 변경(원핫 인코딩)

```
train_labels = tf.keras.utils.to_categorical(train_labels)
test_labels = tf.keras.utils.to_categorical(test_labels)
```



옵티마이저와 손실함수, 지표 등을 정의하는 컴파일 단계

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

- 손실함수(loss function): 신경망의 출력과 정답 간의 오차를 계산하는 함수
- 옵티마이저(optimizer): 손실 함수를 기반으로 신경망의 파라미터를 최적화하는 알고리즘
- 지표(metric): 훈련과 테스트 과정에서 사용되는 척도

학습

```
model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

Epoch 1/5

469/469 [=====] - 2s 3ms/step - loss: 0.0158 - accuracy: 0.9168

...

Epoch 5/5

469/469 [=====] - 2s 3ms/step - loss: 0.0027 - accuracy: 0.9867

테스트

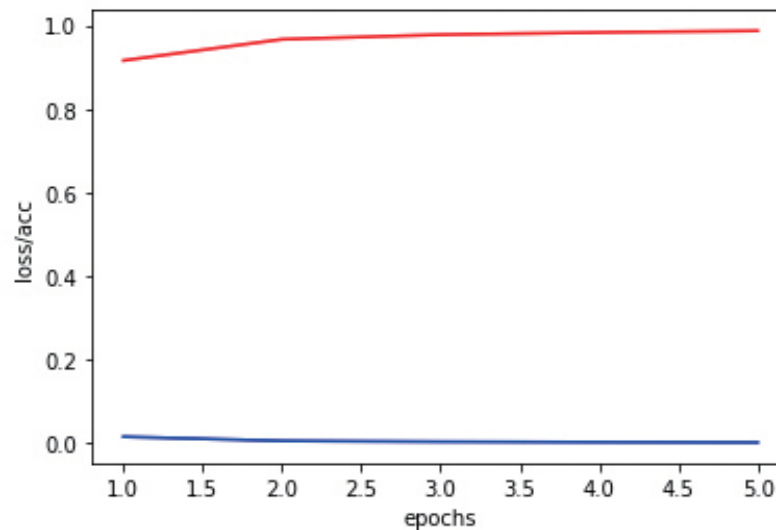
```
test_loss, test_acc = model.evaluate(test_images, test_labels)
print('테스트 정확도:', test_acc)
```

```
313/313 [=====] - 0s 892us/step - loss:
0.0039 - accuracy: 0.9788
테스트 정확도: 0.9787999987602234
```

그래프 그리기

```
history = model.fit(train_images, train_labels, epochs=5, batch_size=128) # 다시 학습하기
loss = history.history['loss']
acc = history.history['accuracy']
epochs = range(1, len(loss)+1)

plt.plot(epochs, loss, 'b', label='Training Loss')
plt.plot(epochs, acc, 'r', label='Accuracy')
plt.xlabel('epochs')
plt.ylabel('loss/acc')
plt.show()
```

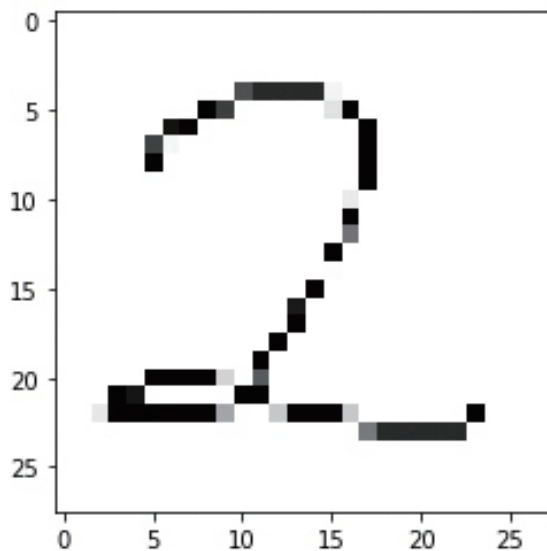


실제 이미지로 테스트하기

```
import cv2 as cv

image = cv.imread('test.png', cv.IMREAD_GRAYSCALE)
image = cv.resize(image, (28, 28))
image = image.astype('float32')
image = image.reshape(1, 784)
image = 255-image
image /= 255.0

plt.imshow(image.reshape(28, 28), cmap='Greys')
plt.show()
```



테스트

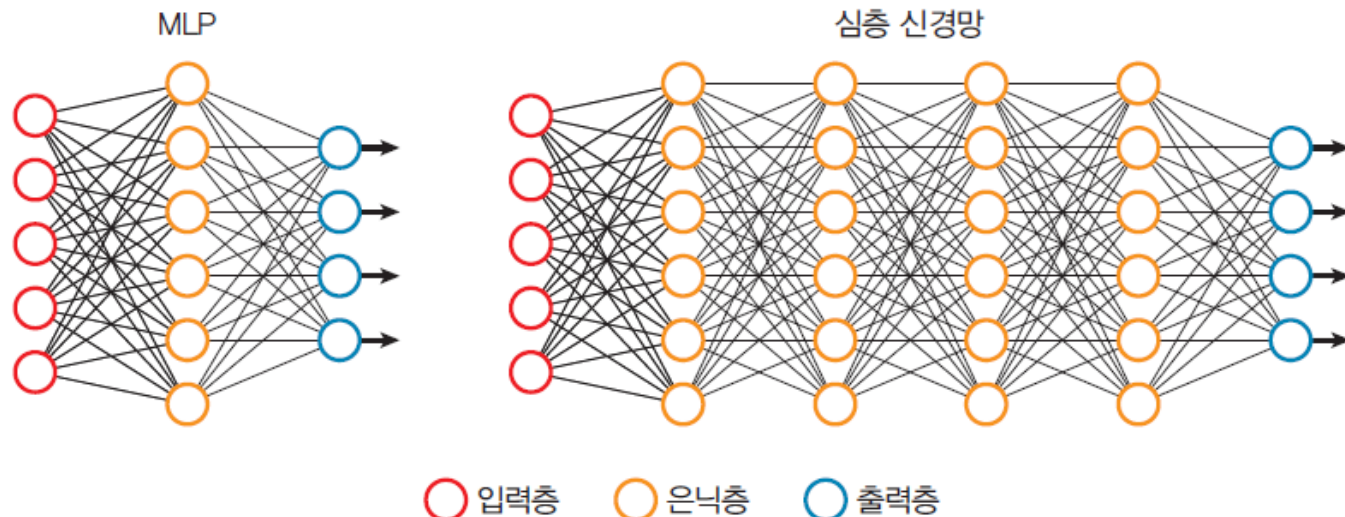
```
pred = model.predict(image.reshape(1, 784), batch_size=1)  
print("추정된 숫자=", pred.argmax())
```

추정된 숫자= 2

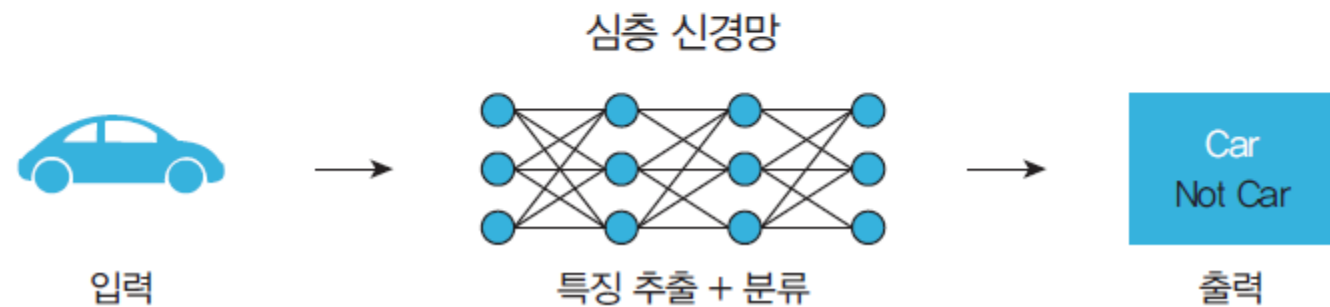
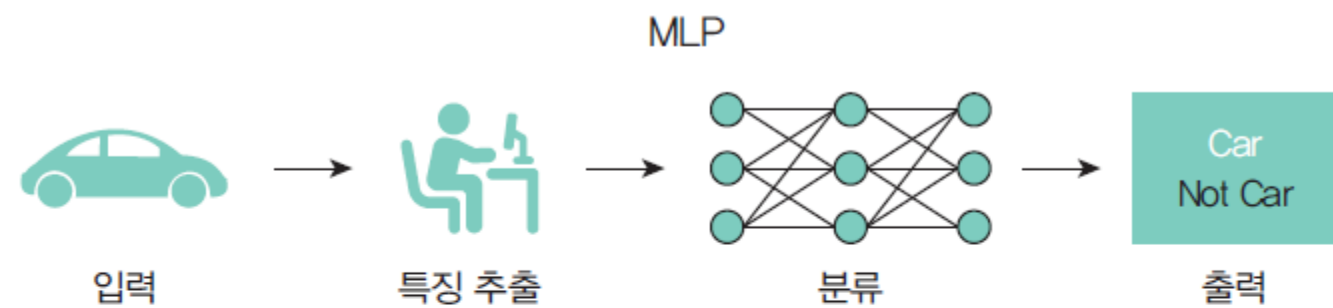
심층신경망 (Deep Neuron Network)

MLP 가 최소 3개 이상

- 심층 신경망(DNN: Deep Neural Networks)은 MLP(다층 퍼셉트론)에서 은닉층의 개수를 증가시킨 것이다.
- 은닉층을하나만 사용하는 것이 아니고 여러 개를 사용한다.
- 최근에 딥러닝은 컴퓨터 시각, 음성 인식, 자연어 처리, 소셜 네트워크 필터링, 기계 번역 등에 적용되어서 인간 전문가에 필적하는 결과를 얻고 있다.

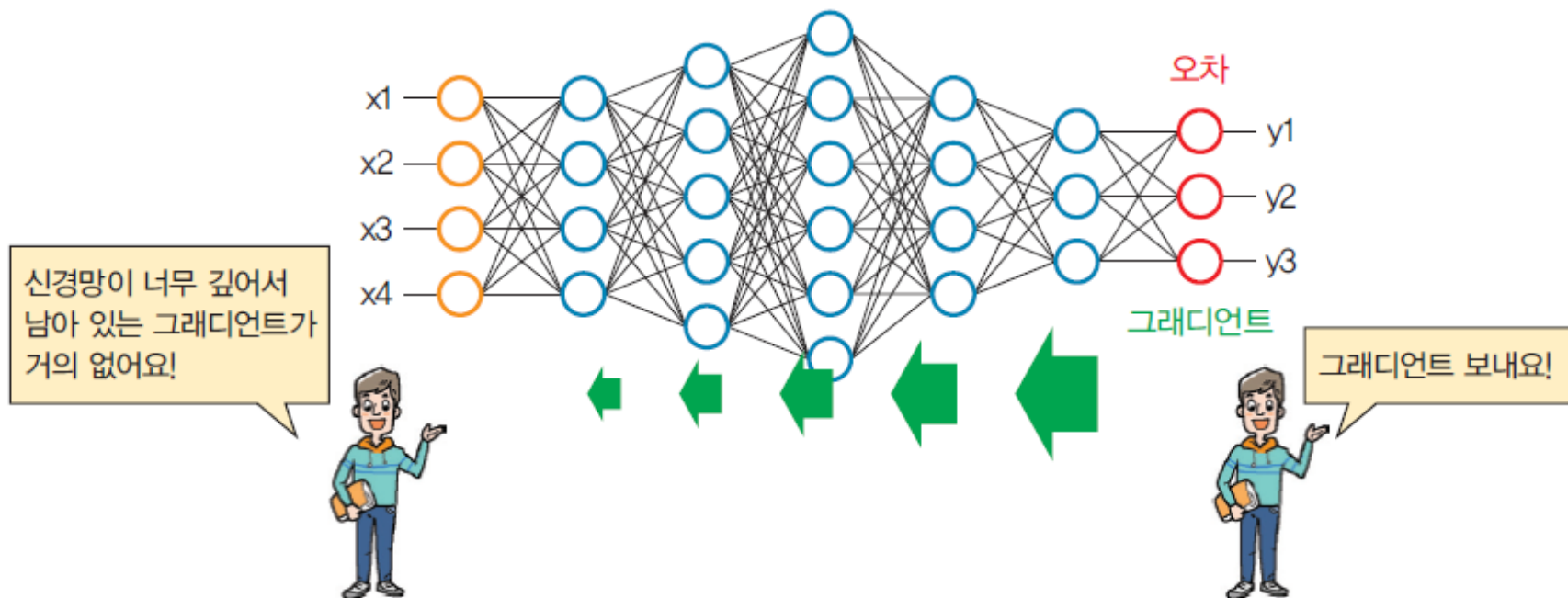


MLP vs DNN



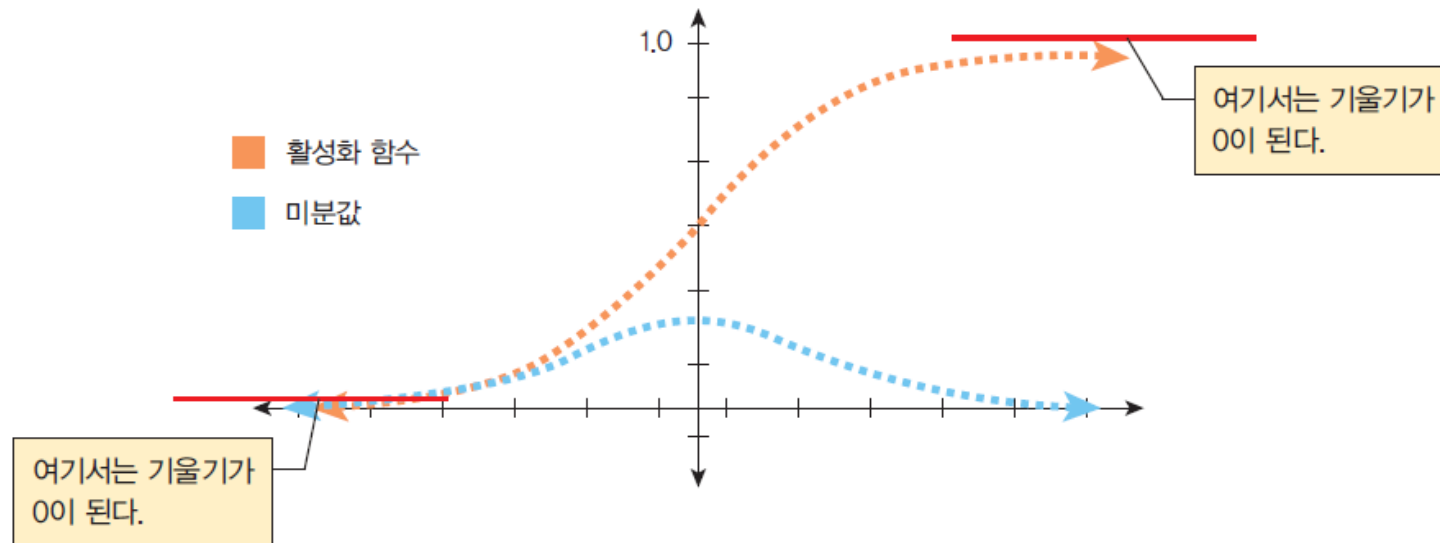
그래디언트 소실 문제

- 심층 신경망에서 그래디언트가 전달되다가 점점 0에 가까워지는 현상이다



범인은?

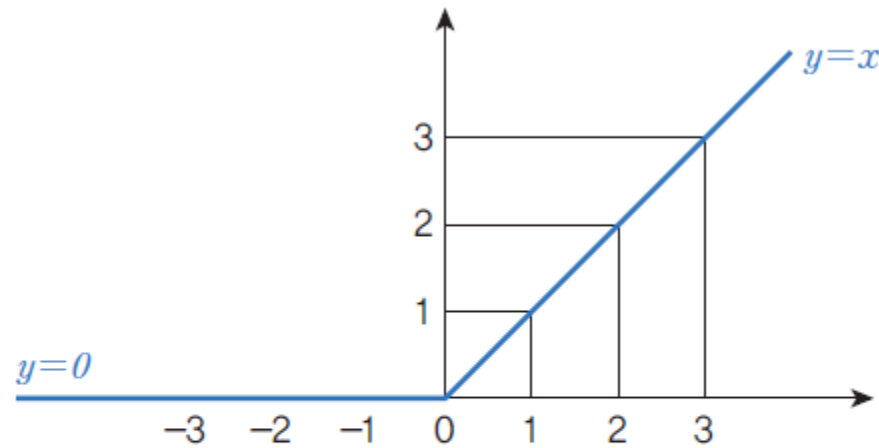
- 범인은 시그모이드 활성화 함수로 드러난다.
- 시그모이드 함수의 특성상, 아주 큰 양수나 음수가 들어오면 출력이 포화되어서 거의 0이 된다.



시그모이드 함수와 그래디언트 소실

새로운 활성화 함수

- 그래디언트 소실 문제를 해결하기 위하여 심층 신경망에서는 활성화 함수로 ReLU 함수를 많이 사용한다.

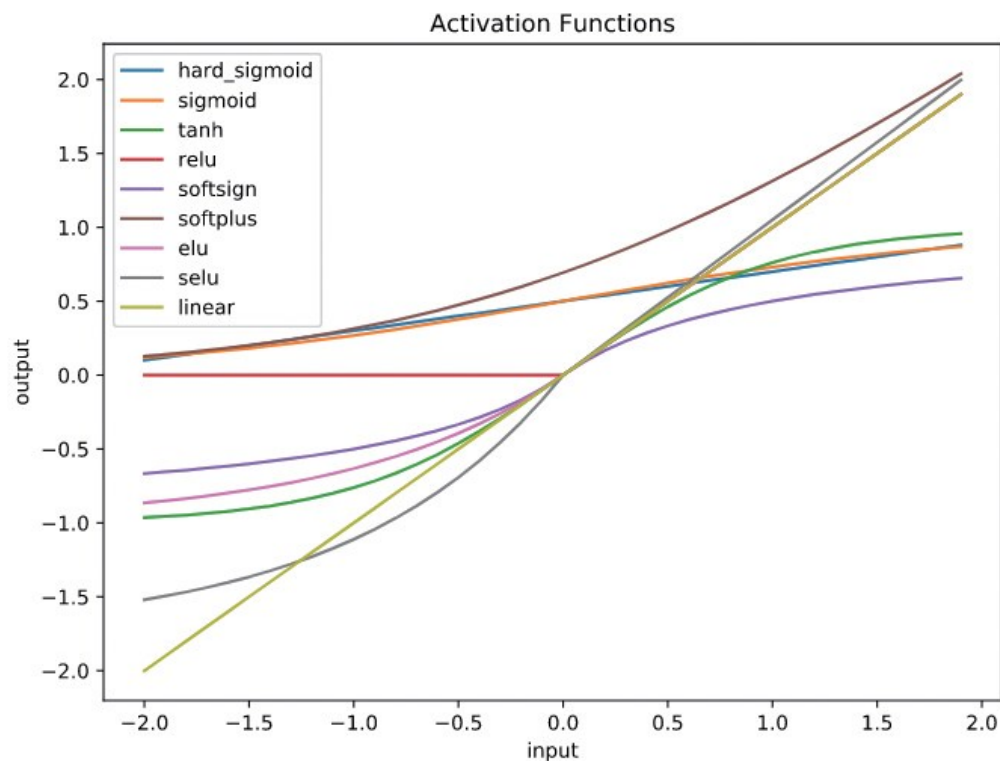


$$f(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$

ReLU 함수

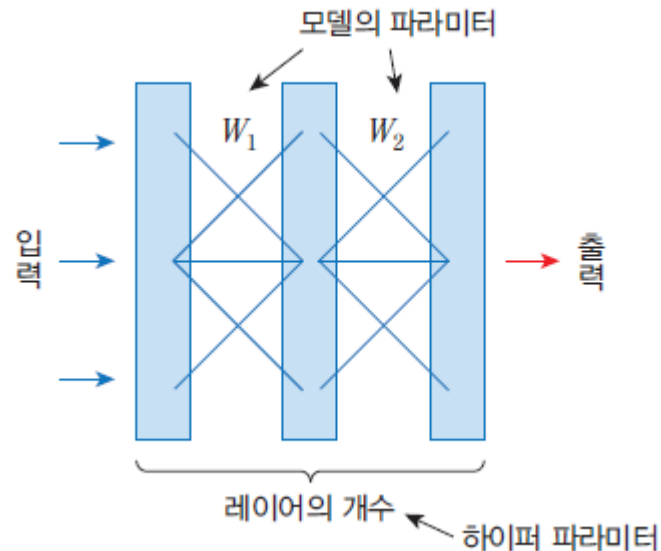
활성화 함수들

- Sigmoid
- Relu(Rectified Linear Unit)
- Softmax
- Tanh
- Selu(Scaled Exponential Linear Unit)
- softplus



하이퍼 매개변수

- 학습률이나 은닉층을 몇 개로 할 것이며, 은닉층의 개수나 유닛의 개수는 누가 정하는 것일까? ==> **하이퍼 매개변수**
- 즉 신경망의 학습률이나 모멘텀의 가중치, 은닉층의 개수, 유닛의 개수, 미니 배치의 크기 등이 하이퍼 매개변수이다

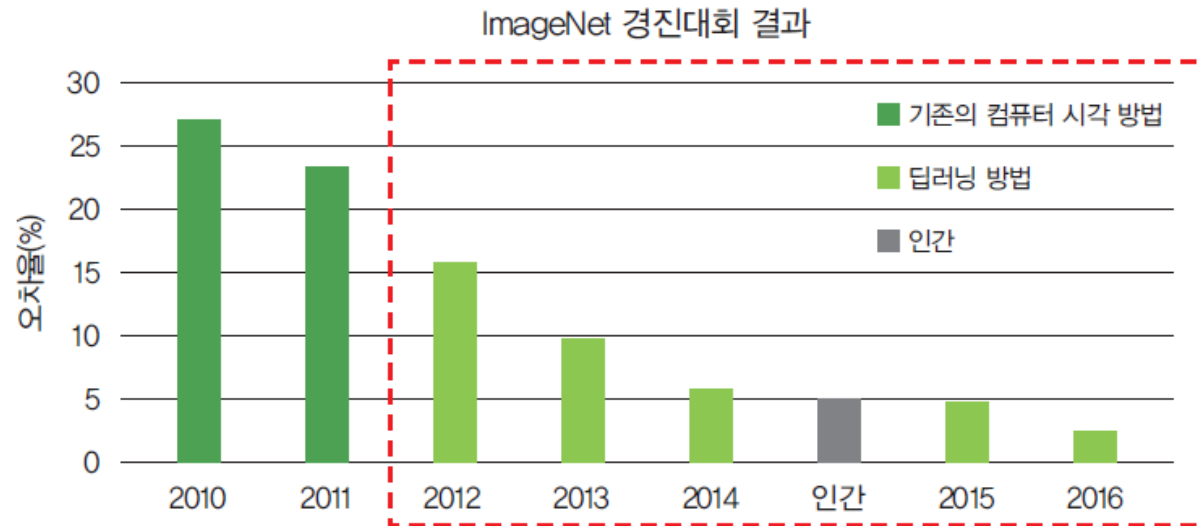


하이퍼 매개 변수를 찾는 방법

- 기본값 사용: 라이브러리 개발자가 설정한 기본값을 그대로 사용한다.
- 수동 검색: 사용자가 하이퍼 매개변수를 지정한다.
- 그리드 검색: 격자 형태로 하이퍼 매개변수를 변경하면서 성능을 측정하는 방법이다.
- 랜덤 검색: 랜덤으로 검색한다.

MLP의 문제점 해결

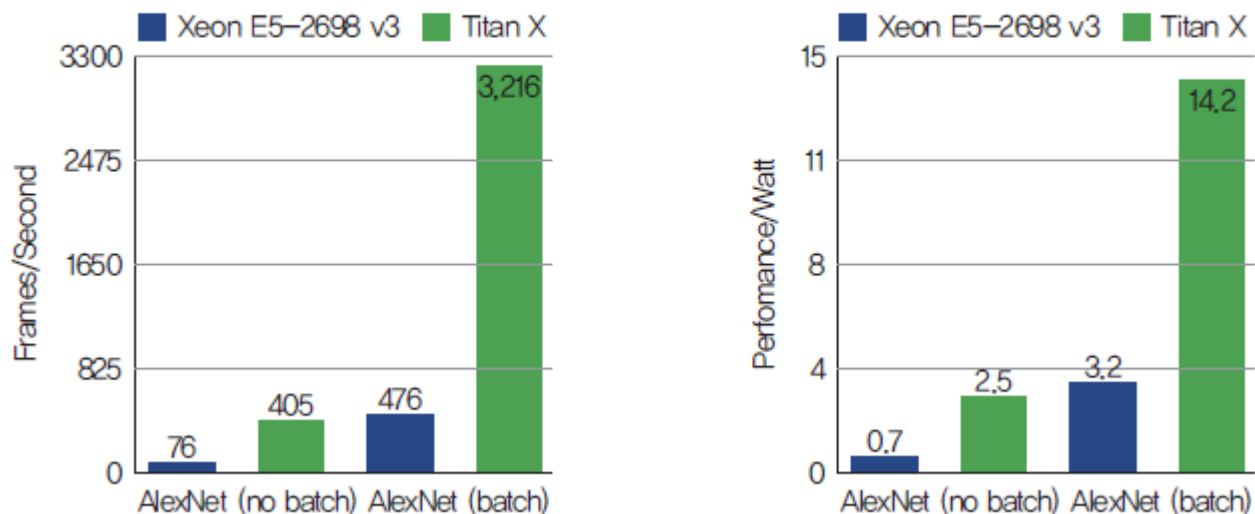
- 은닉층이 많아지면 출력층에서 계산된 그래디언트가 역전파되다가 값이 점점 작아져서 없어지는 문제점 → 해결
- 훈련 데이터가 충분하지 못하면, 과잉 적합(over fitting)이 될 가능성도 높아진다. → 해결
- 2012년에는 AlexNet이 다른 머신러닝 방법들을 큰 차이로 물리치고 ImageNet 경진대회에서 우승한다.



GPU의 도움

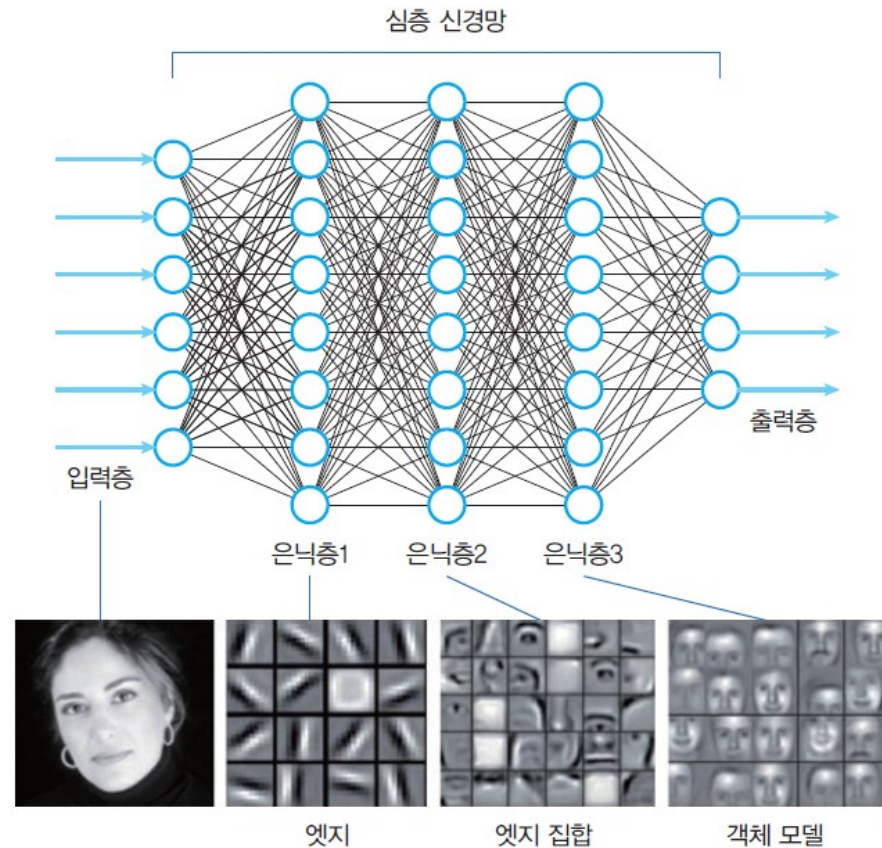
- DNN의 학습 속도는 상당히 느리고 계산 집약적이기 때문에 학습에 시간과 자원이 많이 소모되었다.
- 최근 GPU(Graphic Processor) 기술이 엄청나게 발전하면서 GPU가 제공하는 데이터 처리 기능을 딥러닝도 사용할 수 있게 되었다. 딥러닝 혁명에는 게이머들의 도움도 컸다.

CPU와 GPU의 성능비교



은닉층의 역할

- 은닉층은 무슨 역할을 하고 있는 것일까? -> 여러 개의 은닉층 중에서 앞단은 경계선(에지)과 같은 저급 특징들을 추출하고 뒷단은 코너와 같은 고급 특징들을 추출한다.



케라스의 클래스들

- 모델: 하나의 신경망을 나타낸다.
- 레이어: 신경망에서 하나의 층이다.
- 입력 데이터: 텐서플로우 텐서 형식이다.
- 손실 함수: 신경망의 출력과 정답 레이블 간의 차이를 측정하는 함수이다.
- 옵티마이저: 학습을 수행하는 최적화 알고리즘이다. 학습률과 모멘텀을 동적으로 변경한다

Keras Layer 클래스들

- `Input(shape, batch_size, name):`
입력을 받아서 케라스 텐서를 생성하는 객체
- `Dense(units, activation=None, use_bias=True):`
유닛들이 전부 연결된 레이어
- `Embedding(input_dim, output_dim):`

Keras Sequential 모델

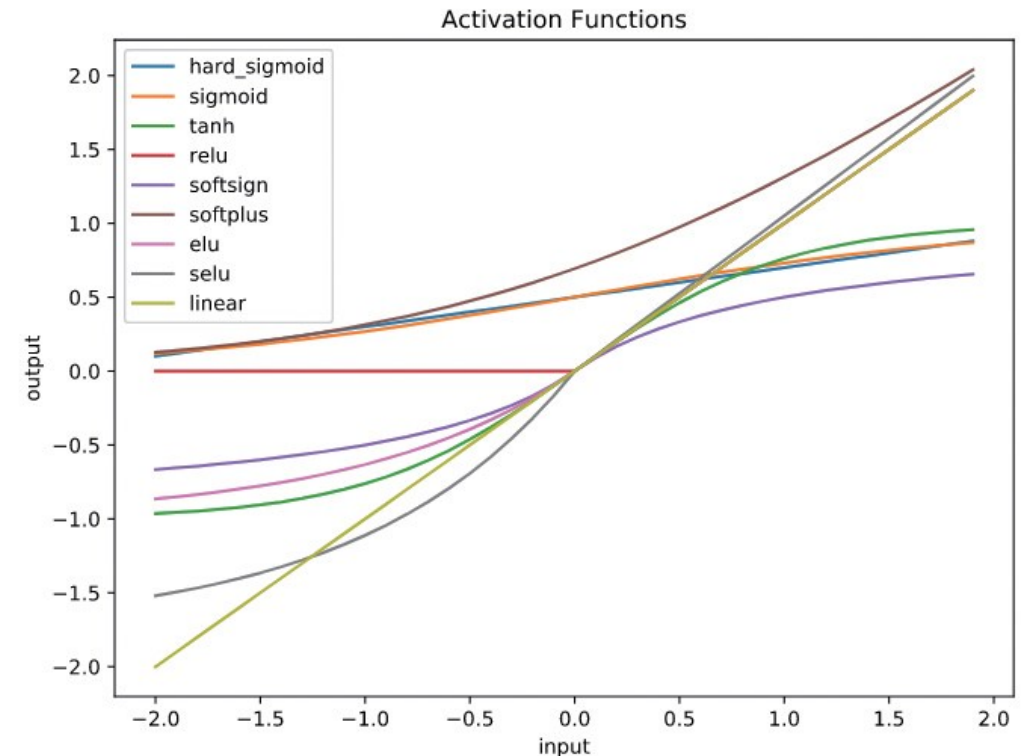
- `compile(optimizer, loss=None, metrics=None)`: 훈련을 위해서 모델을 구성하는 메소드
- `fit(x=None, y=None, batch_size=None, epochs=1, verbose=1)`: 훈련 메소드
- `evaluate(x=None, y=None)`: 테스트 모드에서 모델의 손실 함수 값과 측정 항목 값을 반환
- `predict(x, batch_size=None)`: 입력 샘플에 대한 예측값을 생성
- `add(layer)`: 레이어를 모델에 추가한다.

Optimizers

- **SGD**: 확률적 경사 하강법 (Stochastic Gradient Descent, SGD) . Nesterov 모멘텀을 지원한다.
- Adagrad: Adagrad는 가변 학습률을 사용하는 방법
- Adadelta: Adadelta는 모멘텀을 이용하여 감소하는 학습률 문제를 처리하는 Adagrad의 변형이다.
- **RMSprop**: RMSprop는 Adagrad에 대한 수정판이다.
- **Adam**: Adam은 기본적으로 (RMSprop + 모멘텀)이다.

Karas 에서 지원하는 활성화 함수

- sigmoid
- relu(Rectified Linear Unit)
- softmax
- tanh
- selu(Scaled Exponential Linear Unit)
- softplus



손실 함수

- **MeanSquaredError**: 정답 레이블과 예측값 사이의 평균 제곱 오차를 계산한다. (입력과 정답이 0.0 ~ 1.0 숫자값들)
- **BinaryCrossentropy**: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을 계산한다(예를 들어서 강아지, 강아지 아님).
- **CategoricalCrossentropy**: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을 계산한다(예를 들어서 강아지, 고양이, 호랑이).
정답 레이블은 원핫 인코딩으로 제공되어야 한다.
- **SparseCategoricalCrossentropy**: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을 계산한다(예를 들어서 강아지, 고양이, 호랑이). 정답 레이블은 정수로 제공되어야 한다.

측정 항목

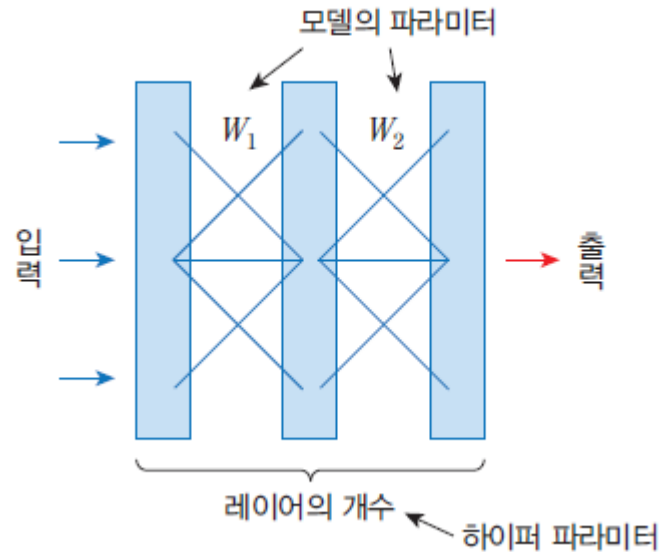
- accuracy: 정확도이다. 예측값이 정답 레이블과 같은 횟수를 계산한다.
- categorical_accuracy: 범주형 정확도이다. 신경망의 예측값이 원-핫 레이블과 일치하는 빈도를 계산한다.

옵티마이저 (학습률)

- SGD: 확률적 경사 하강법(Stochastic Gradient Descent, SGD). Nesterov 모멘텀을 지원한다.
- Adagrad: Adagrad는 가변 학습률을 사용하는 방법
- Adadelat: Adadelat는 모멘텀을 이용하여 감소하는 학습률 문제를 처리하는 Adagrad의 변형이다.
- RMSprop: RMSprop는 Adagrad에 대한 수정판이다.
- Adam: Adam은 기본적으로 (RMSprop + 모멘텀)이다.

하이퍼 매개변수

- 학습률이나 은닉층을 몇 개로 할 것이며, 은닉층의 개수나 유닛의 개수는 누가 정하는 것일까? -> **하이퍼 매개변수**
- 즉 신경망의 학습률이나 모멘텀의 가중치, 은닉층의 개수, 유닛의 개수, 미니 배치의 크기 등이 하이퍼 매개변수이다

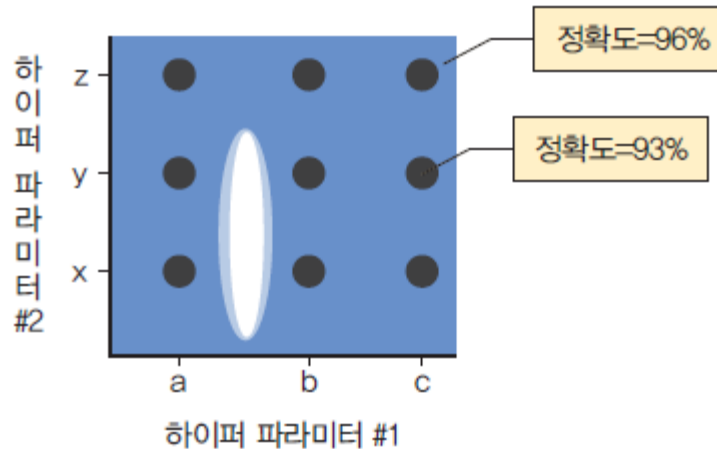


하이퍼 매개변수를 찾는 방법

- 기본값 사용: 라이브러리 개발자가 설정한 기본값을 그대로 사용한다.
- 수동 검색: 사용자가 하이퍼 매개변수를 지정한다.
- 그리드 검색: 격자 형태로 하이퍼 매개변수를 변경하면서 성능을 측정하는 방법이다.
- 랜덤 검색: 랜덤으로 검색한다.

그리드 검색 (알고리즘)

- 각 하이퍼 매개변수에 대하여 몇 개의 값을 지정하면 이 중에서 가장 좋은 조합을 찾아주는 알고리즘이다.
- sklearn 패키지에서 제공해주고 있기 때문에 손쉽게 사용할 수 있다



예제

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.wrappers.scikit_learn import KerasClassifier

# 데이터 세트 준비
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()

train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype('float32') / 255

test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype('float32') / 255

train_labels = tf.keras.utils.to_categorical(train_labels)
test_labels = tf.keras.utils.to_categorical(test_labels)
```

예제

케라스를 sklearn처럼 보이게 한다.

```
# 신경망 모델 구축
def build_model():
    network = tf.keras.models.Sequential()

    network.add(tf.keras.layers.Input(shape=(28*28,)))
    network.add(tf.keras.layers.Dense(512, activation='relu'))
    network.add(tf.keras.layers.Dense(10, activation='softmax'))

    network.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
    return network

# 하이퍼 매개변수 딕셔너리
param_grid = {
    'epochs':[1, 2, 3],
    'batch_size':[32, 64]
}
```

에포크 수: 1, 2, 3
배치 크기: 32, 64

예제

```
# 케라스 모델을 sklearn에서 사용하도록 포장한다.
model = KerasClassifier(build_fn = build_model, verbose=1)

# 그리드 검색
gs = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=3,
    n_jobs=-1
)

# 그리드 검색 결과 출력
grid_result = gs.fit(train_images, train_labels)
print(grid_result.best_score_)
print(grid_result.best_params_)
```


실행결과

```
...  
Epoch 3/3  
938/938 [=====] - 3s 4ms/step - loss: 0.0664 - accuracy:  
0.9799  
157/157 [=====] - 0s 939us/step  
0.968666672706604  
{'batch_size': 64, 'epochs': 3}
```

손실 함수 선택 문제 (회귀)

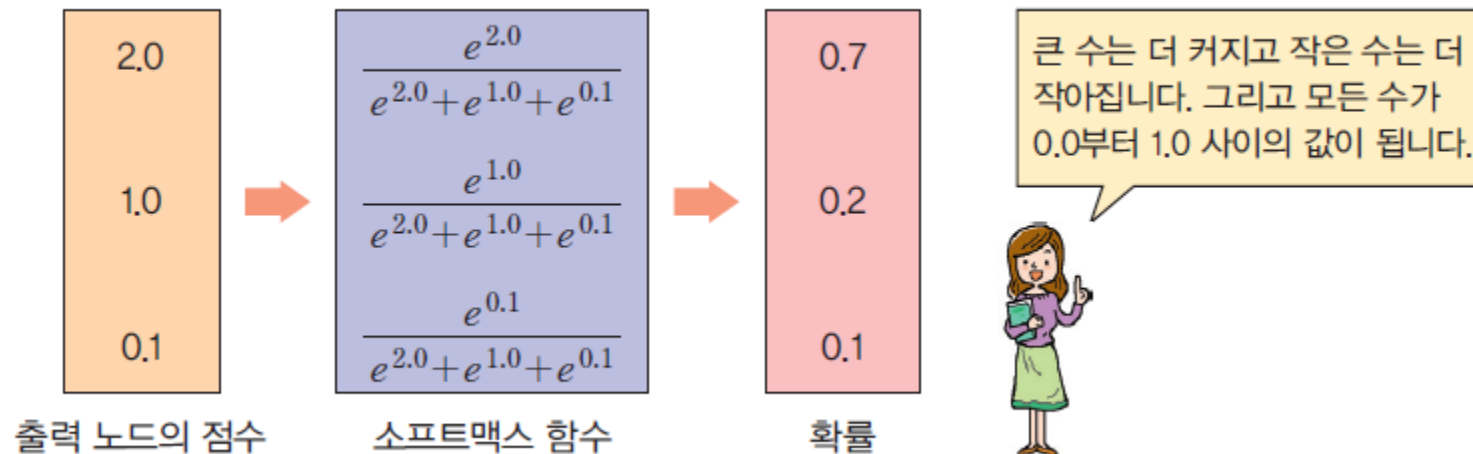
- 예전에는 보통 손실 함수로는 평균 제곱 오차 (Mean Squared Error: MSE)를 주로 사용했음.

$$E = \frac{1}{m} \sum (y_i - \hat{y}_i)^2$$

- 정답과 예측값의 차이가 커지면 MSE도 커지므로 많이 사용했던 손실 함수는 MSE이다.
- 회귀문제는 반드시 MSE를 손실함수를 사용해야 함.
- 출력노드가 1개 인 경우는 sigmoid 활성화 함수를 사용함.
- 하지만 분류 문제를 위해서는 MSE보다는 더 성능이 좋은 손실 함수가 개발되어 있다. -> 교차 엔트로피 함수 (xxxxx-Crossentropy)
- 딥러닝에서는 출력층노드에는 softmax 활성화함수를 사용하고, 손실함수로는 교차엔트로피를 많이 사용함.

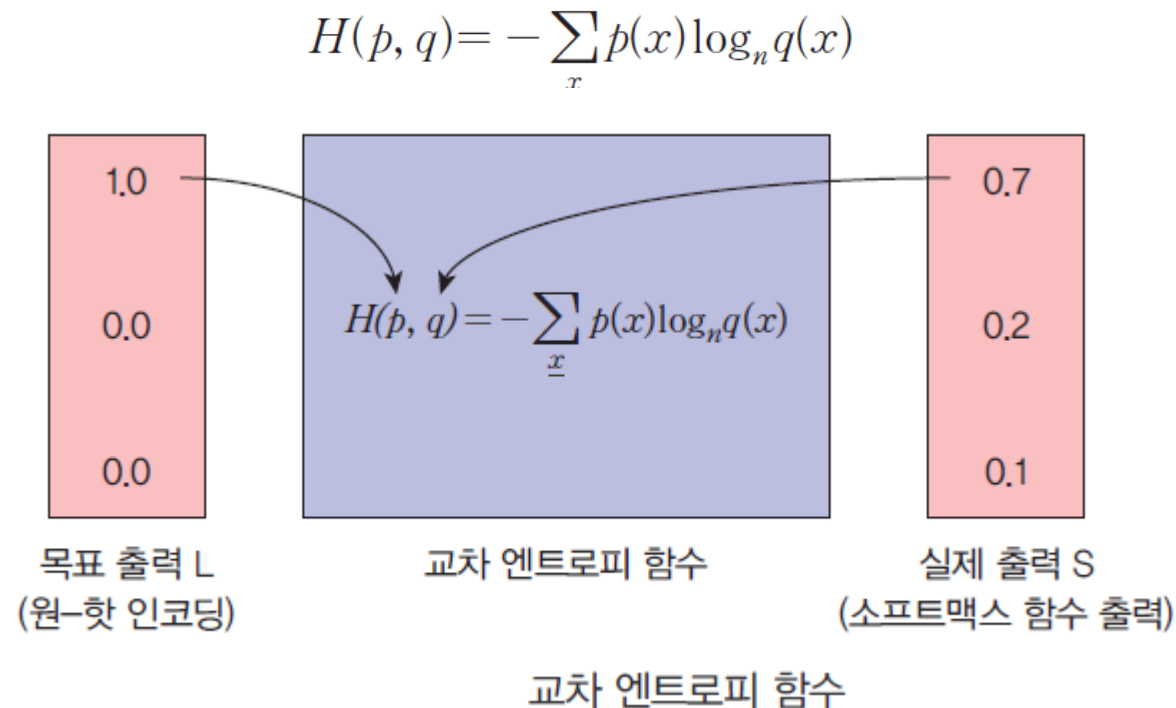
소프트맥스(softmax) 활성화 함수 (분류)

- 최근의 신경망에서는 출력층에 소프트맥스(softmax) 활성화 함수를 많이 사용함 (**출력노드가 2개 이상 경우**).
- “소프트맥스”라는 이름이 붙은 이유는 이것이 max 함수의 소프트한 버전이기 때문이다. Max 함수의 출력은 전적으로 최대 입력값에 의하여 결정된다

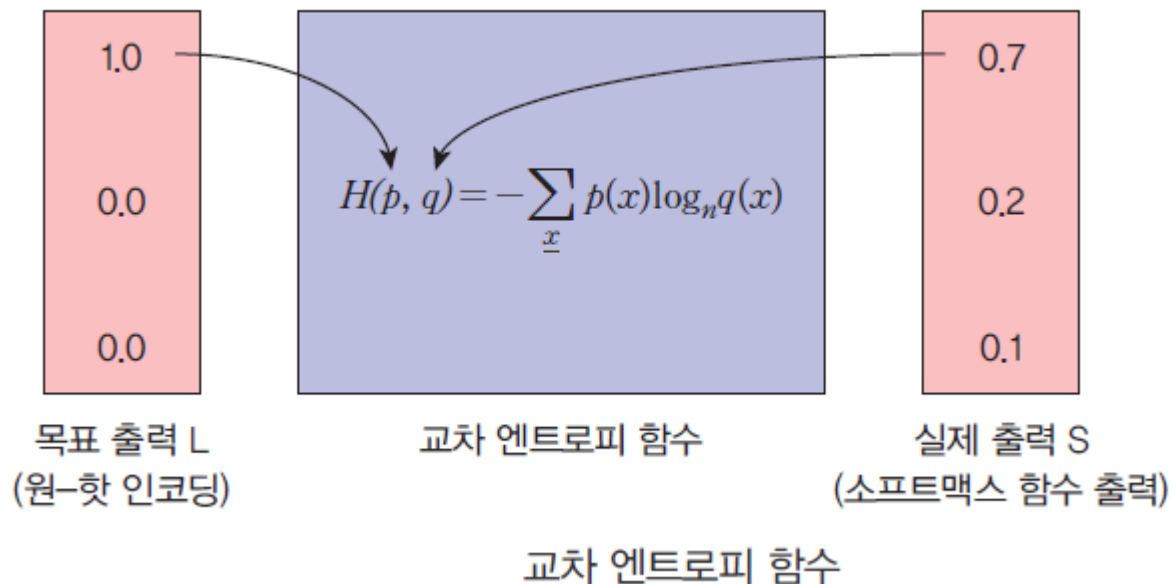


교차 엔트로피 손실 함수 (분류) (One-hot encoding)

- 교차 엔트로피는 2개의 확률 분포 p , q 에 대해서 다음과 같이 정의된다.
- 하나면 1.0, 나머지는 0.0

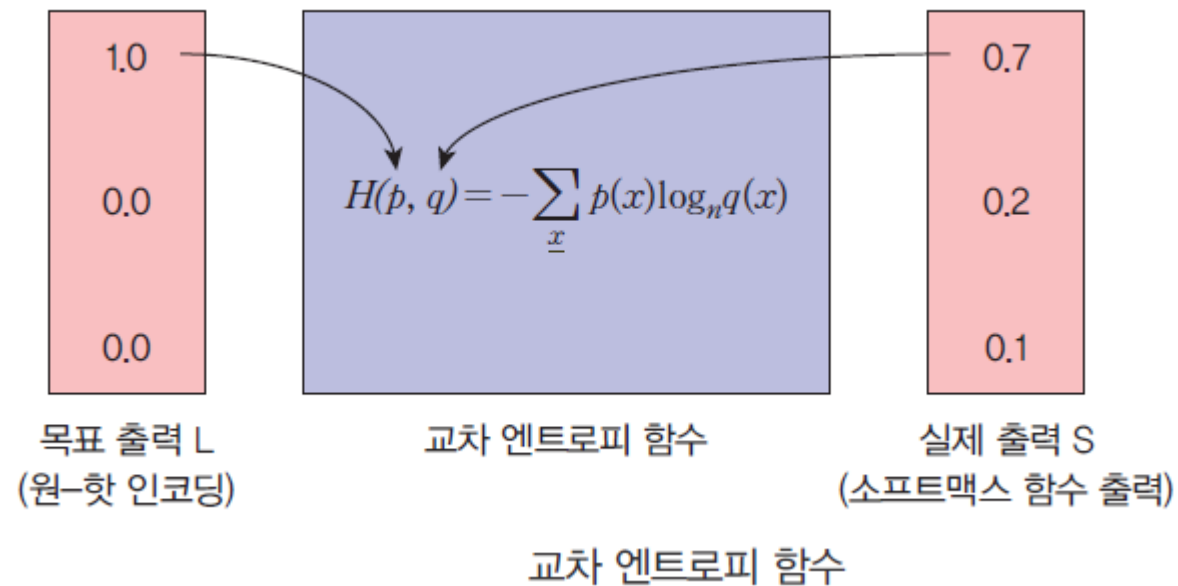


교차 엔트로피 손실 함수



$$\begin{aligned} H(p, q) &= - \sum_x p(x) \log_n q(x) \\ &= -(1.0 * \log 0.7 + 0.0 * \log 0.2 + 0.0 * \log 0.1) \\ &= 0.154901 \end{aligned}$$

완벽하게 일치한다면



$$\begin{aligned} H(p, q) &= -\sum_x p(x) \log_n q(x) \\ &= -(1.0 * \log 1.0 + 0.0 * \log 0.0 + 0.0 * \log 0.0) \\ &= 0 \end{aligned}$$

교차 엔트로피의 계산

입력 샘플	실제 출력			목표 출력		
샘플 #1	0.1	0.3	0.6	0	0	1
샘플 #2	0.2	0.6	0.2	0	1	0
샘플 #3	0.3	0.4	0.3	1	0	0

▷ 첫 번째 샘플에 대하여 교차 엔트로피를 계산해보자. $-(\log(0.1) * 0 + \log(0.3) * 0 + \log(0.6) * 1) = -(0 + 0 - 0.51) = 0.51$ 이 된다.

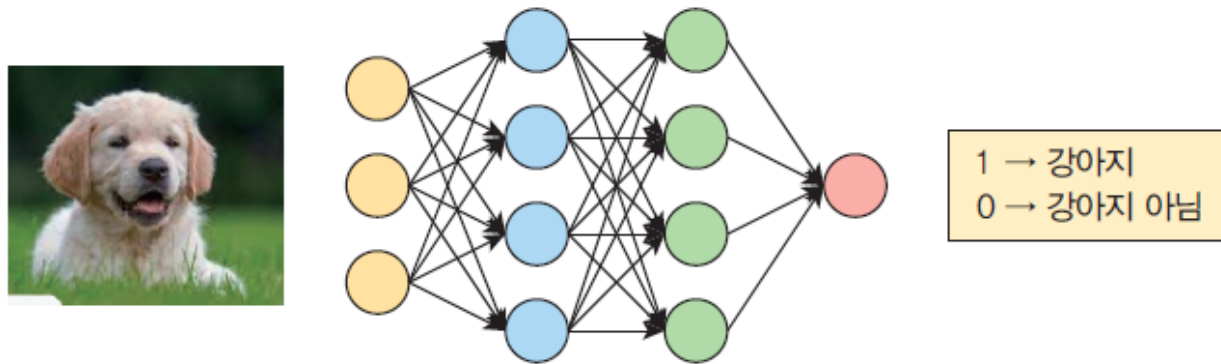
▷ 두 번째 샘플의 교차 엔트로피는 $-(\log(0.2) * 0 + \log(0.6) * 1 + \log(0.2) * 0) = -(0 - 0.51 + 0) = 0.51$ 이다.

▷ 세 번째 샘플의 교차 엔트로피는 $-(\log(0.3) * 1 + \log(0.4) * 0 + \log(0.3) * 0) = -(-1.2 + 0 + 0) = 1.20$ 이다.

▷ 따라서 3개 샘플의 평균 교차 엔트로피 오류는 $(0.51 + 0.51 + 1.20) / 3 = 0.74$ 가 된다.

케라스에서 손실함수 (분류)

- BinaryCrossentropy: 이진 교차 엔트로피(BCE)는 이진 분류 문제를 해결하는 데 사용되는 손실 함수이다.
- 출력노드가 한 개인 경우: **sigmoid 활성화 함수 (default)**
- 즉 우리가 분류해야 하는 부류가 두 가지뿐일 때 사용한다. 예를 들어 이미지를 “강아지”, “강아지 아님”의 두 부류로 분류할 때 BinaryCrossentropy를 사용한다.



$$BCE = -\frac{1}{n} \sum_{i=1}^n (y_i \log_n(\hat{y}_i) + (1 - y_i) \log_n(1 - \hat{y}_i))$$

실제 계산 결과

실제 레이블 y	1	0	0	1
예측 값 $\hat{y}$	0.8	0.3	0.5	0.9

샘플 1: $BCE1 = -(1 \cdot \log(0.8) + (1-1) \cdot \log(1-0.8))$

샘플 2: $BCE2 = -(0 \cdot \log(0.3) + (1-0) \cdot \log(1-0.3))$

샘플 3: $BCE3 = -(0 \cdot \log(0.5) + (1-0) \cdot \log(1-0.5))$

샘플 4: $BCE4 = -(1 \cdot \log(0.9) + (1-1) \cdot \log(1-0.9))$

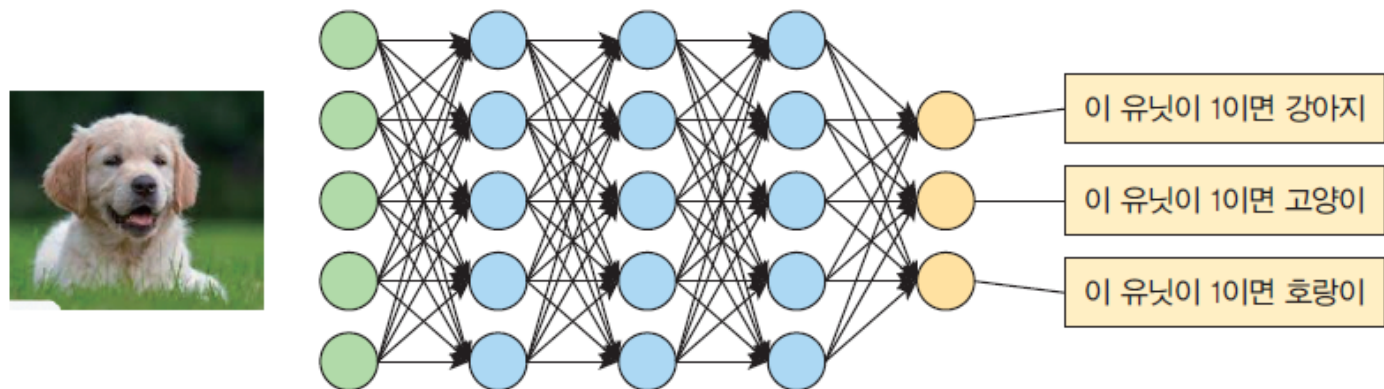
$$BCE = \frac{BCE1 + BCE2 + BCE3 + BCE4}{4} \approx 0.345$$

```
y_true = [ [1], [0], [0], [1]]  
y_pred = [[0.8], [0.3], [0.5], [0.9]]  
bce = tf.keras.losses.BinaryCrossentropy()  
print(bce(y_true, y_pred).numpy())
```

0.3445814

케라스: CategoricalCrossEntropy (분류)

- CategoricalCrossentropy: 우리가 분류해야 할 부류가 두 개 이상이라면(즉 다중 분류 문제라면) 을 사용
- 정답 레이블은 원-핫 인코딩으로 제공한다. 예를 들어서 입력 이미지를 “강아지(1,0, 0)”, “고양이(0, 1, 0)”, “호랑이(0, 0, 1)” 중의 하나로 분류

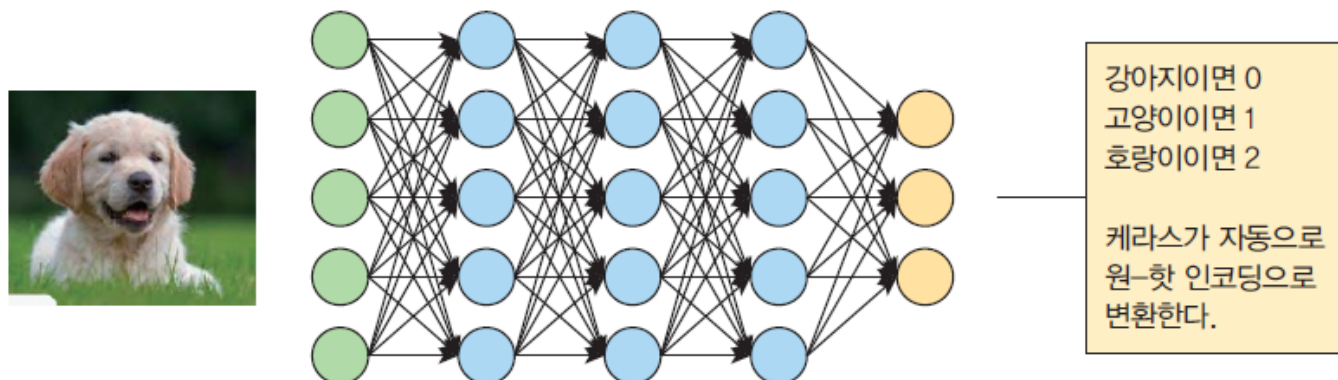


```
y_true = [[0.0, 1.0, 0.0], [0.0, 0.0, 1.0], [1.0, 0.0, 0.0]] # 고양이, 호랑이, 강아지
y_pred = [[0.6, 0.3, 0.1], [0.3, 0.6, 0.1], [0.1, 0.7, 0.2]]
cce = tf.keras.losses.CategoricalCrossentropy ()
print(cce(y_true, y_pred).numpy ())
```

1.936381

케라스: SparseCategoricalCrossEntropy(분류)

- SparseCategoricalCrossentropy: 정답 레이블이 원-핫 인코딩이 아니고 정수로 주어지면 SparseCategoricalCrossentropy를 사용
- 예를 들어서 정답 레이블이 0(강아지), 1(고양이), 2(호랑이)로 주어지면 SparseCategoricalCrossentropy를 사용한다.



```
y_true = [1, 2, 0] # 고양이, 호랑이, 강아지
y_pred = [[0.6, 0.3, 0.1], [0.3, 0.6, 0.1], [0.1, 0.7, 0.2]]
scce = tf.keras.losses.SparseCategoricalCrossentropy()
print(scce(y_true, y_pred).numpy())
```

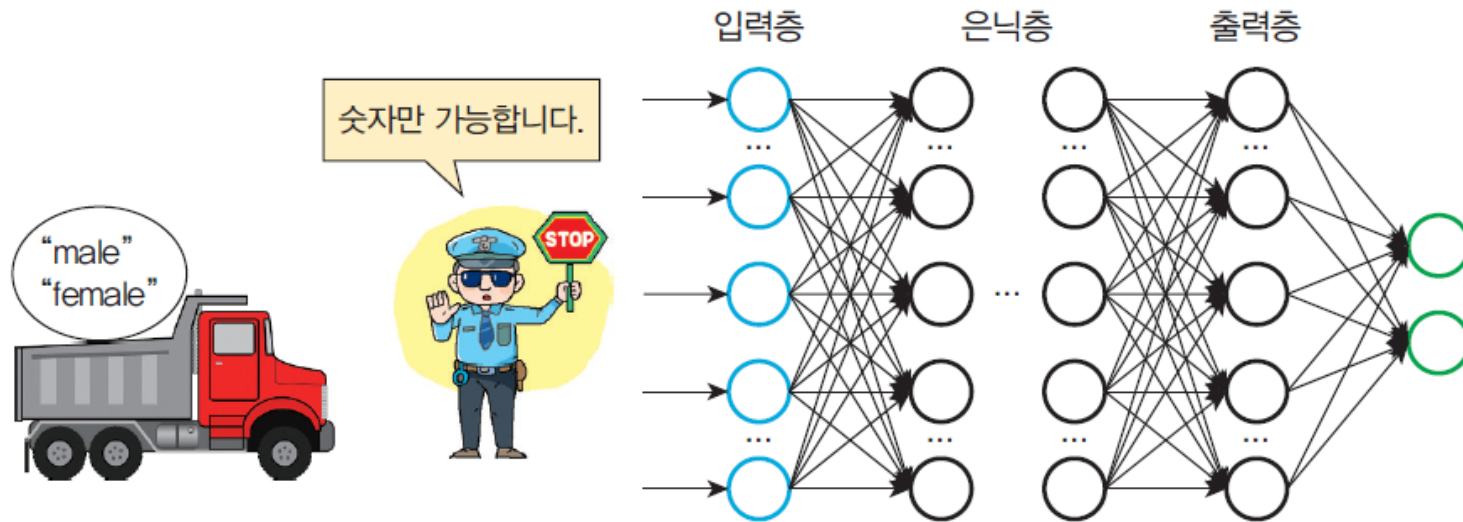
1.936381

Keras : 손실 함수들

- MeanSquaredError: 정답 레이블과 예측값 사이의 평균 제곱 오차를 계산한다.
- BinaryCrossentropy: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을 계산한다(예를 들어서 강아지, 강아지 아님).
- CategoricalCrossentropy: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을 계산한다(예를 들어서 강아지, 고양이, 호랑이).
정답 레이블은 원핫 인코딩으로 제공되어야 한다.
- SparseCategoricalCrossentropy: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을 계산한다 (예를 들어서 강아지, 고양이, 호랑이).
정답 레이블은 정수로 제공되어야 한다.

범주형 데이터 처리

- 입력 데이터 중에는 “male”, “female”과 같이 카테고리를 가지는 데이터들이 아주 많다. -> 숫자로 바꾸어야 한다.



간단한 경우

```
for ix in train.index:  
    if train.loc[ix, 'Sex']=="male":  
        train.loc[ix, 'Sex']=1  
    else:  
        train.loc[ix, 'Sex']=0
```

일반적인 범주형 데이터 변환 방법

- 범주형 변수를 인코딩하는 3가지 방법
 - 정수 인코딩(Integer Encoding): 각 레이블이 정수로 매핑되는 경우이다.
 - 원-핫 인코딩(One Hot Encoding): 각 레이블이 이진 벡터에 매핑되는 경우이다.
 - 임베딩(Embedding): 범주의 분산된 표현이 학습되는 경우이다.



자연어 처리에서 주로 사용

원-핫 인코딩

- 원-핫 인코딩은 단 하나의 값만 1이고 나머지는 모두 0인 인코딩을 의미한다. -> 아주 많이 사용된다.

Country	Age	Salary
Korea	38	7200
Japan	27	4800
China	30	3100



Korea	Japan	China	Age	Salary
1	0	0	38	7200
0	1	0	27	4800
0	0	1	30	3100

원-핫 인코딩

```
import numpy as np
X = np.array([[ 'Korea', 44, 7200], [ 'Japan', 27, 4800], [ 'China', 30, 6100]])

from sklearn.preprocessing import OneHotEncoder
onehotencoder = OneHotEncoder()

# 원하는 열을 뽑아서 2차원 배열로 만들어서 전달하여야 한다.
XX = onehotencoder.fit_transform(X[:,0]).reshape(-1,1)).toarray()
print(XX)

X = np.delete(X, [0], axis=1)          # 0번째 열 삭제
X = np.concatenate((XX, X), axis = 1) # X와 XX를 붙인다.
print(X)
```

원-핫 인코딩(케라스 사용)

- 원-핫 인코딩은 케라스의 `to_categorical()`을 호출해서도 만들 수 있다.-> 아주 많이 사용된다.

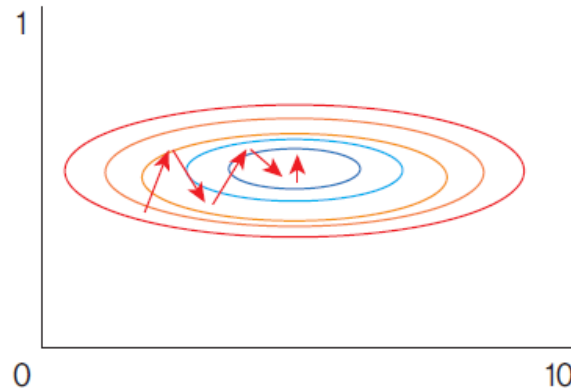
```
class_vector = [2, 6, 6, 1]

from tensorflow.keras.utils import to_categorical
output = to_categorical(class_vector, num_classes = 7, dtype = "int32")
print(output)
```

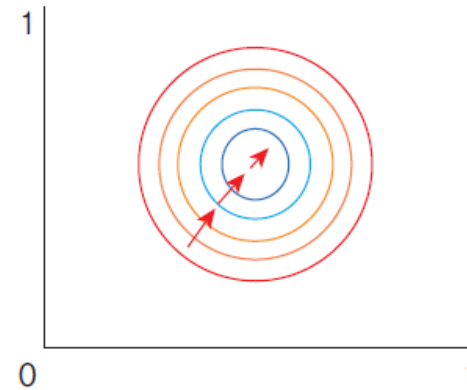
```
[[0 0 1 0 0 0 0]
 [0 0 0 0 0 0 1]
 [0 0 0 0 0 0 1]
 [0 1 0 0 0 0 0]]
```

데이터 정규화

- 신경망은 일련의 선형 조합과 비선형 활성화 함수를 통해 이러한 차이가 나는 입력을 결합하여 학습하게 되므로 각 입력과 관련된 매개 변수도 서로 다른 범위를 가지면서 학습된다.



특징값의 범위가
다른 경우

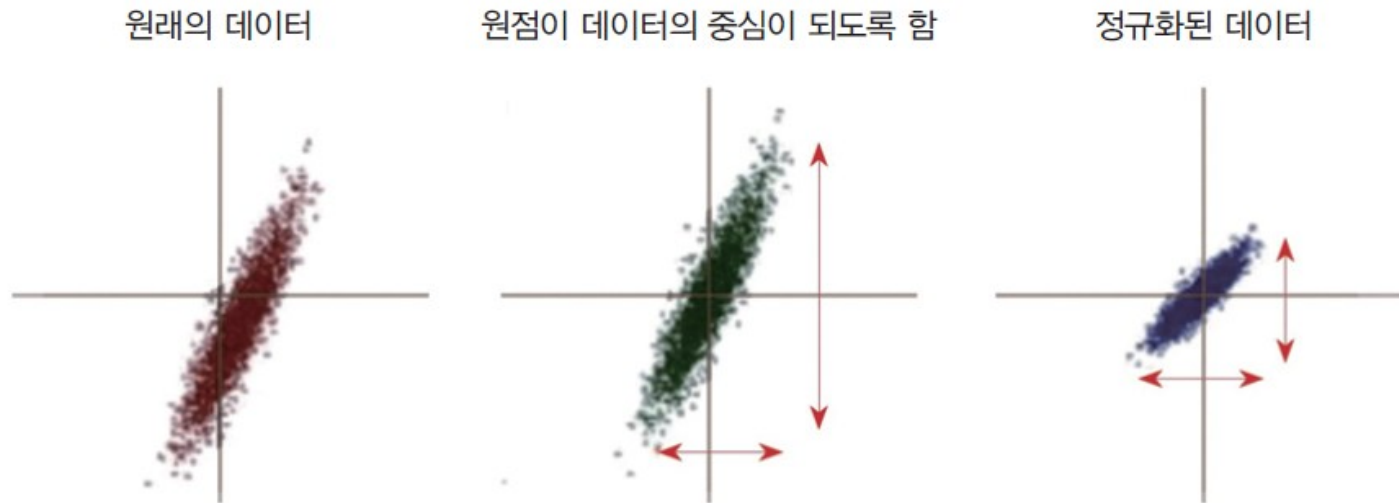


특징값이 $[0, 1]$ 에서
움직이는 경우

데이터 정규화의 영향

데이터 정규화

- 부동 소수점 숫자 정밀도와 관련된 문제를 피하기 위해, 입력 값이 대략 -1.0에서 1.0 범위에 있도록 하는 것이 좋다



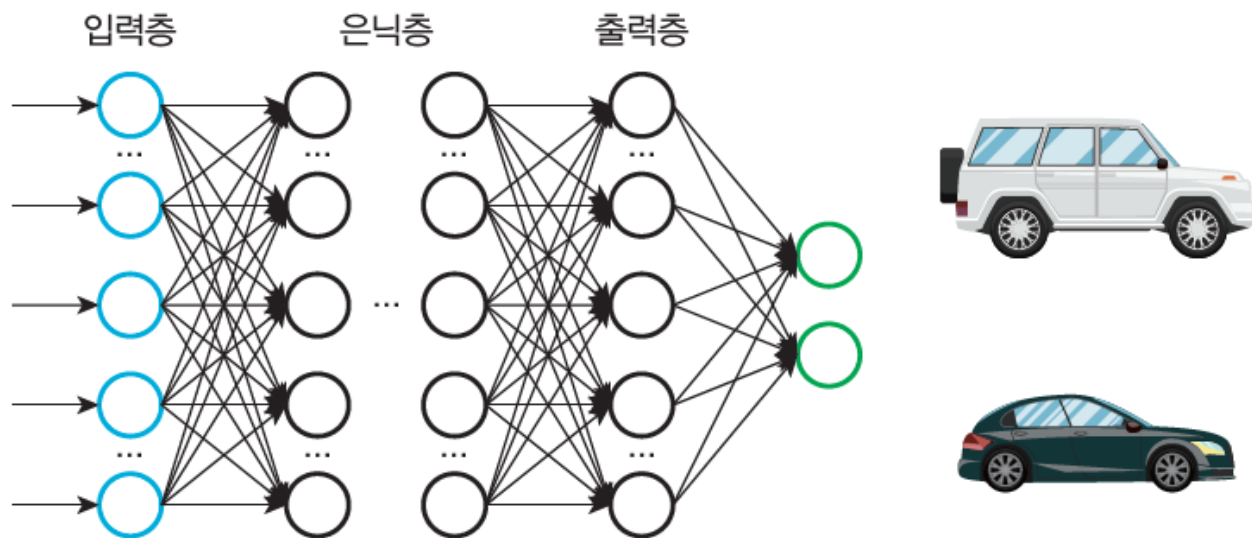
$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

케라스의 입력 데이터

- 넘파이 배열:
- TensorFlow Dataset 객체: 크기가 커서, 메모리에 한 번에 적재될 수 없는 경우에 디스크 또는 분산 파일 시스템에서 스트리밍될 수 있다.
- 파이썬 제너레이터: 예를 들어서 `keras.utils.Sequence` 클래스는 하드 디스크에 위치한 파일을 읽어서 순차적으로 케라스 모델로 공급할 수 있다.

예제

- 사람의 나이, 성별, 연간 수입을 기준으로, 선호하는 자동차의 타입(세단 아니면 SUV)을 예측할 신경망을 만들고 싶다고 가정하자



훈련 데이터

범주형 데이터

[0]	30	male	3800	SUV
[1]	36	female	4200	SEDAN
[2]	52	male	4000	SUV
[3]	42	female	4400	SEDAN

정규화 필요

[0]	-1.23	-1.0	-1.34	(1.0 0.0)
[1]	-0.49	1.0	0.45	(0.0 1.0)
[2]	1.48	-1.0	-0.45	(1.0 0.0)
[3]	0.25	1.0	1.34	(0.0 1.0)

sklearn의 데이터 정규화 방법

$$X_std = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))$$
$$X_scaled = X_std * (max - min) + min$$

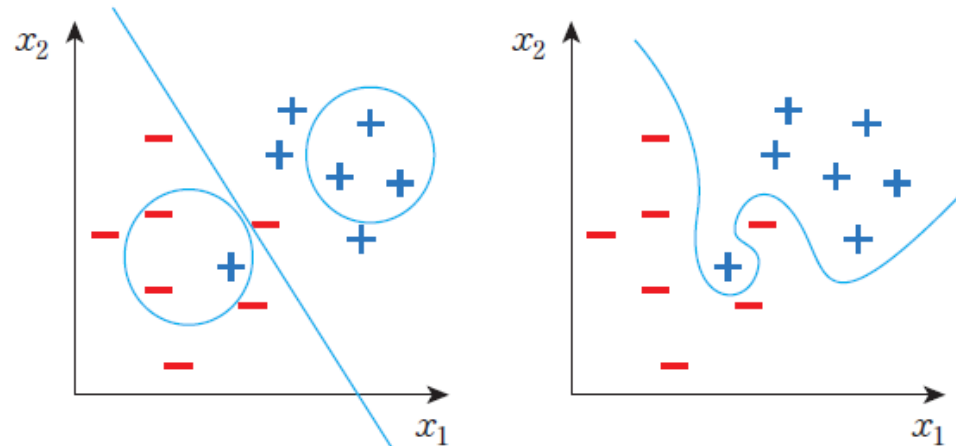
```
from sklearn.preprocessing import MinMaxScaler
data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]

scaler = MinMaxScaler()
scaler.fit(data) # 최대값과 최소값을 알아낸다.
print(scaler.transform(data)) # 데이터를 변환한다.
```

```
[[0.  0.  ]
 [0.25 0.25]
 [0.5  0.5  ]
 [1.   1.   ]]
```


딥러닝에서 과잉 적합과 과소 적합

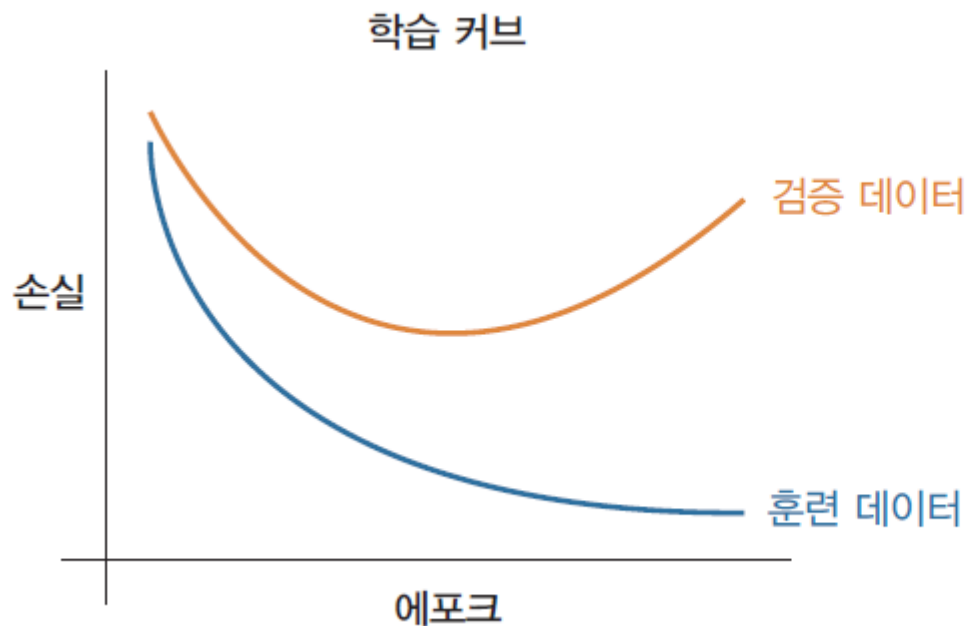
- 과잉 적합(over fitting)은 지나치게 훈련 데이터에 특화돼 실제 적용 시 좋지 못한 결과가 나오는 것을 말한다.



일반화에 실패!

과잉 적합의 예

딥러닝에서 과잉 적합을 아는 방법



훈련 데이터의 손실값은 계속 감소하지만 검증 데이터의 손실값은 오히려 증가하고 있네요!

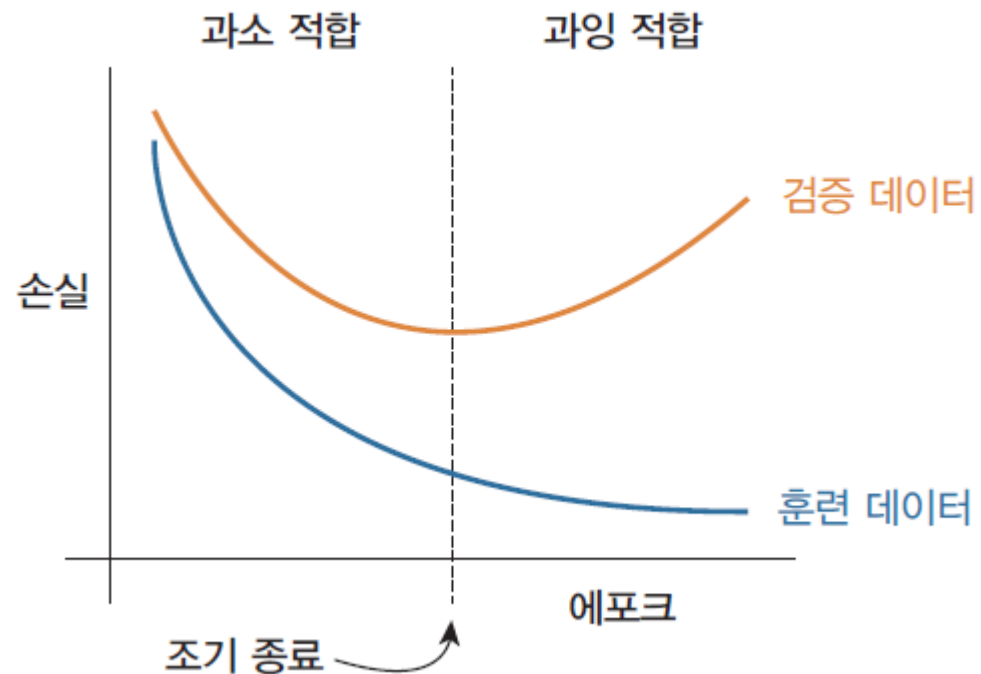


딥러닝에서 과잉 적합 방지 전략

- 조기 종료: 검증 손실이 증가하면 훈련을 조기에 종료한다.
- 가중치 규제 방법: 가중치의 절대값을 제한한다.
- 데이터 증강 방법: 데이터를 많이 만든다.
- 드롭아웃 방법: 몇 개의 뉴런을 쉬게 한다.

조기종료

- 검증 손실이 더 이상 감소하지 않는 것처럼 보일 때마다 훈련을 중단할 수 있다. 이런 식으로 훈련을 중단하는 것을 조기 종료 (Early Stopping)라고 한다.



가중치 초기화 문제

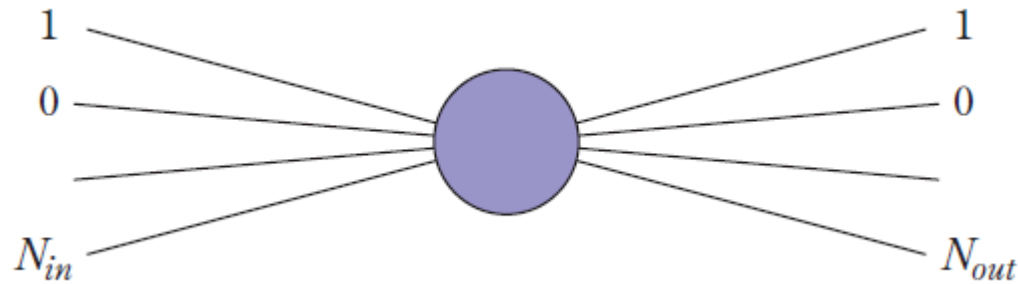
- 가중치의 초기값도 성능에 많은 영향을 끼친다. 신경망의 초기에는 연구가 부족하여서 초기값을 0으로 주는 경우도 많았다. -> 오차의 역전파가 되지 않는다.

$$\delta_j = \begin{cases} (out_j - t_j)f'(net_j) & j \text{가 출력층 노드이면} \\ (\sum_k w_{jk} \delta_k)f'(net_j) & j \text{가 은닉층 노드이면} \end{cases}$$

가중치가 0이라면 델타가 전달되지 않는다.

난수 가중치 초기값

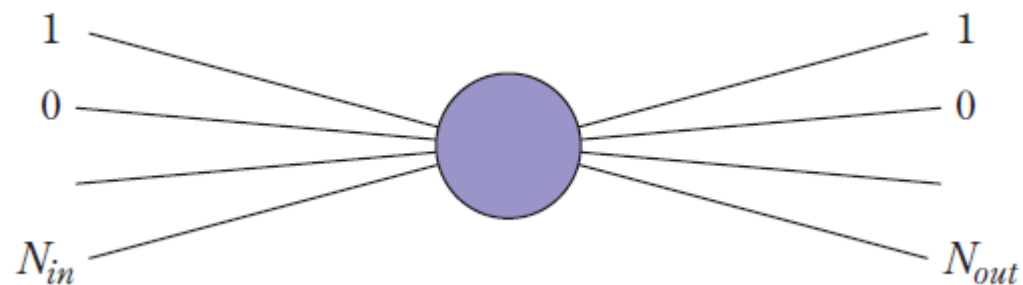
- 이상과 같은 이유로 가중치의 초기값은 난수로 결정되어야 한다.
- 또 반대로 너무 큰 가중치는 그래디언트 폭발(exploding gradients)을 일으킨다고 한다. 이 경우에는 학습이 수렴하지 않고 발산하게 된다
- Xavier의 방법



$$\text{var}(w_i) = \frac{1}{N_{in}}$$

난수 가중치 초기값

•He의 방법



$$var(w_i) = \frac{2}{(N_{in} + N_{out})}$$

가중치 규제

- 가중치의 값이 너무 크면, 판단 경계선이 복잡해지고 과잉 적합이 일어난다는 사실을 발견하였다

$$Loss = Cost + \lambda \sum |W|$$

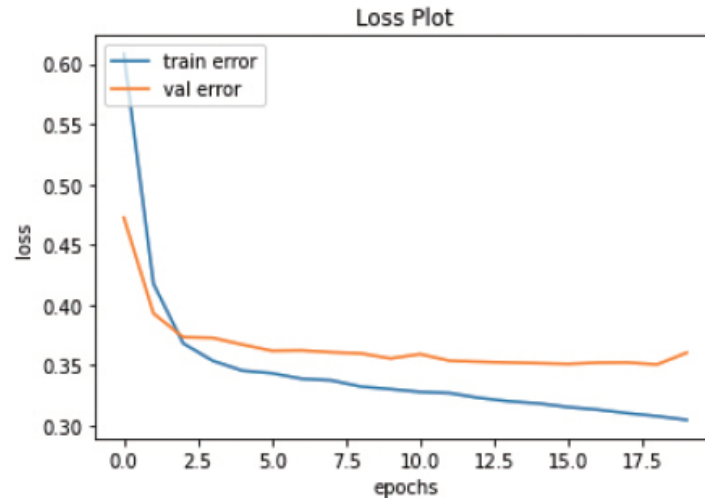
L1 규제

$$Loss = Cost + \lambda \sum W^2$$

L2 규제

과잉 적합 (규제 적용)

```
# 신경망 모델 구축
model = tf.keras.Sequential()
model.add(tf.keras.layers.Dense(16, kernel_regularizer=tf.keras.regularizers.l2(0.001),
activation='relu', input_shape=(1_000,)))
model.add(tf.keras.layers.Dense(16, kernel_regularizer=tf.keras.regularizers.l2(0.001),
activation='relu'))
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
```

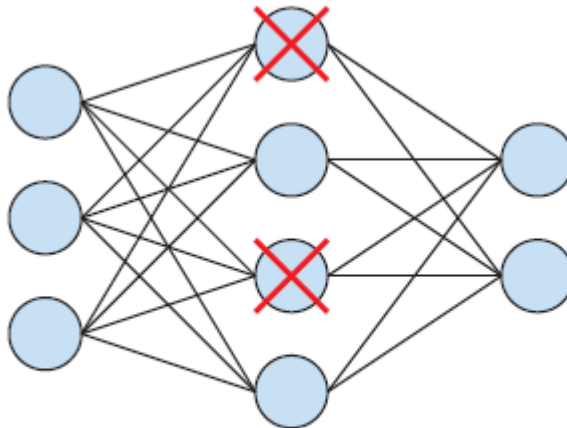


아직도 과잉 적합이지만 규제를
적용한 모델이 덜 과잉 적합됨을
알 수 있습니다.



드롭 아웃

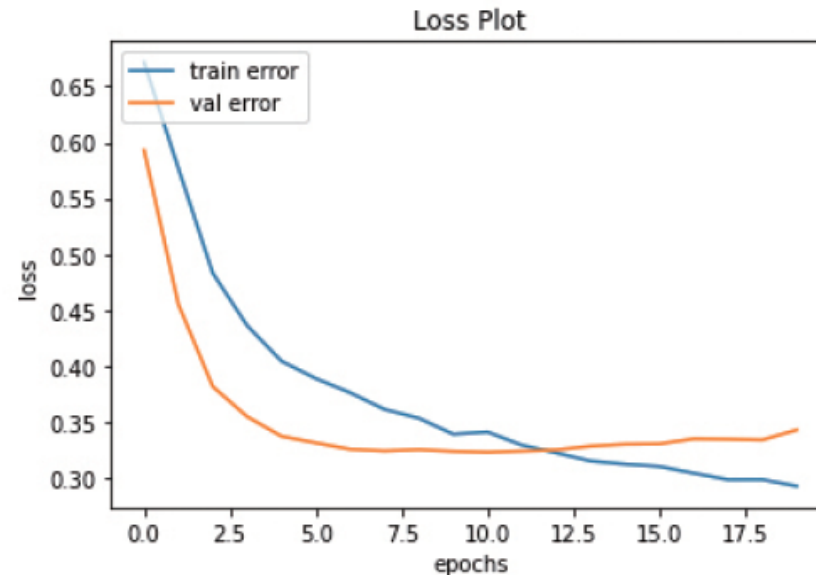
- 드롭아웃은 몇 개의 노드들을 학습 과정에서 랜덤하게 제외하는 것이다.



드롭아웃

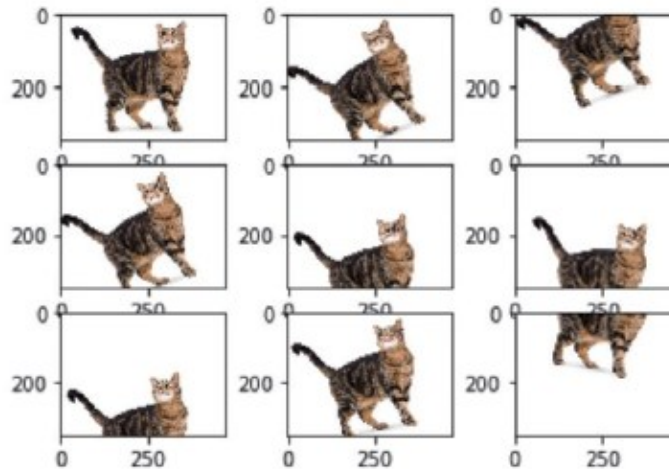
Dropout()

```
# 신경망 모델 구축
model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(1_000,)))
model.add(tf.keras.layers.Dense(16, activation='relu'))
model.add(tf.keras.layers.Dropout(0.5))
model.add(tf.keras.layers.Dense(16, activation='relu'))
model.add(tf.keras.layers.Dropout(0.5))
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
```



데이터 증강 방법

- 데이터 증강(data augmentation) 방법은 소량의 훈련 데이터에서 많은 훈련 데이터를 뽑아내는 방법이다.



데이터 증강 방법

예제: MNIST 필기체 숫자 인식

- MNIST 숫자를 MLP로 인식해본 경험이 있다. 동일한 데이터 세트에 대하여 이번에는 심층 신경망을 사용해보자. 얼마나 정확도가 증가할까?

```
import keras
mnist = keras.datasets.mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()
(X_train, X_test) = (X_train / 255.0, X_test / 255.0)

model = keras.Sequential()
model.add(keras.layers.Input(shape=(28, 28)))
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(512, activation='relu'))
model.add(keras.layers.Dropout(0.2))
model.add(keras.layers.Dense(10, activation='softmax'))#Sparse Categorical Crossentropy
```

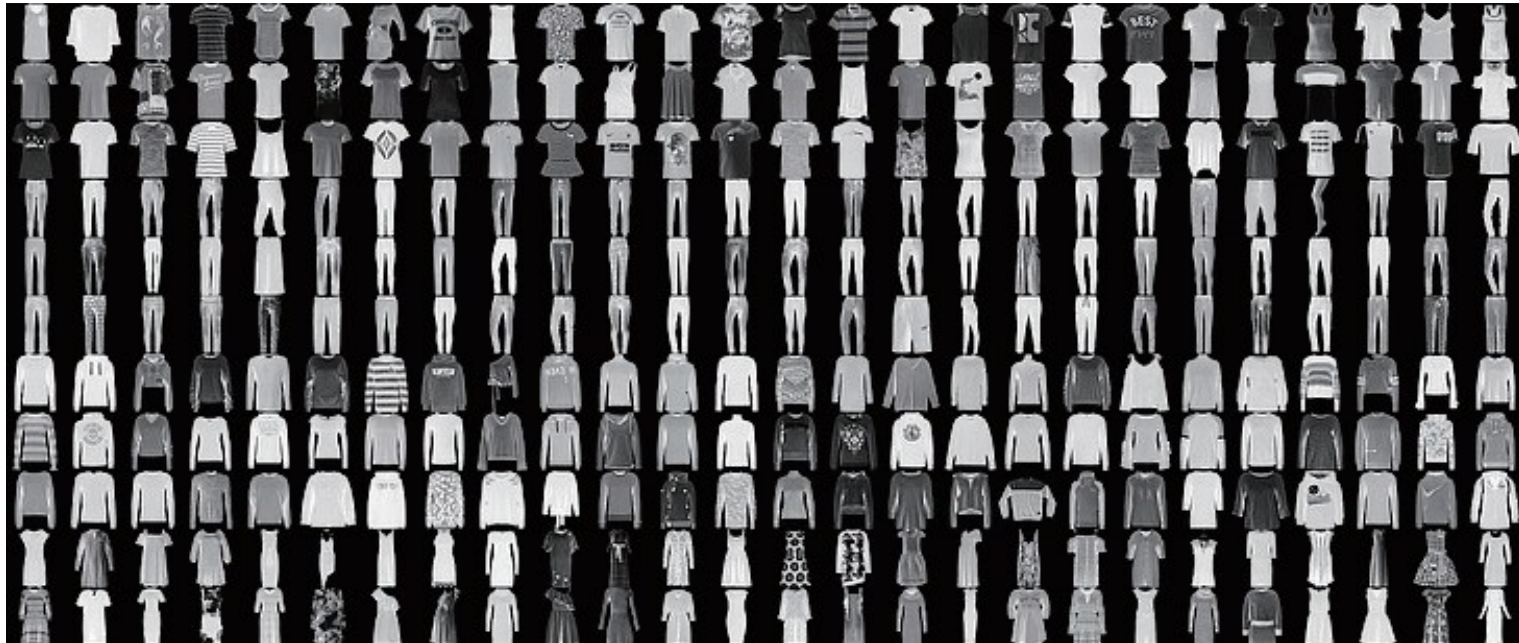
예제: MNIST 필기체 숫자 인식

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
              metrics=['accuracy']) # 0 ~ 9 Sparse Categorical Crossentropy  
  
model.fit(X_train, y_train, epochs=5)  
model.evaluate(X_test, y_test)
```

```
Epoch 1/5  
60000/60000 [=====] - 7s 116us/sample - loss: 0.2205 - acc: 0.9348  
Epoch 2/5  
60000/60000 [=====] - 7s 110us/sample - loss: 0.0969 - acc: 0.9700  
Epoch 3/5  
60000/60000 [=====] - 7s 109us/sample - loss: 0.0678 - acc: 0.9785  
Epoch 4/5  
60000/60000 [=====] - 6s 108us/sample - loss: 0.0529 - acc: 0.9834  
Epoch 5/5  
60000/60000 [=====] - 7s 108us/sample - loss: 0.0428 - acc: 0.9859  
10000/10000 [=====] - 0s 43us/sample - loss: 0.0645 - acc: 0.9795
```

예제: 패션 아이템 분류

- 이번 절에서는 텐서플로우 튜토리얼에 나오는 패션 아이템을 심층 신경망으로 분류하는 코드를 살펴보자. (https://www.tensorflow.org/tutorials/keras/basic_classification)



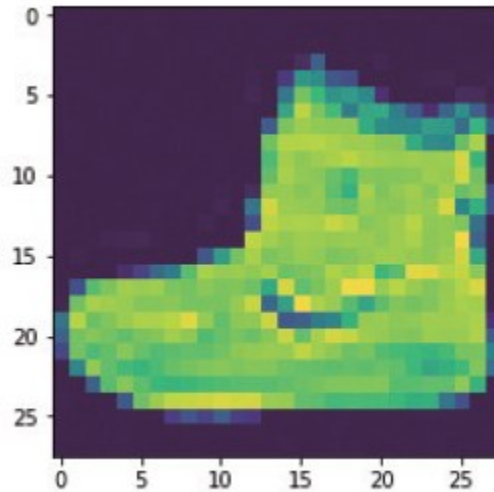
데이터 세트

- 이미지는 28x28 크기이고 픽셀 값은 0과 255 사이의 값이다. 레이블(label)은 0에서 9까지의 정수로서 패션 아이템의 범주를 나타낸다.

레이블	범주
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

하나의 데이터

- `plt.imshow(train_images[0])`



예제: 패션 아이템 분류

```
import tensorflow as tf
import keras
import numpy as np
import matplotlib.pyplot as plt

from keras.src import datasets, layers, models

fashion_mnist = keras.src.datasets.fashion_mnist

(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

plt.imshow(train_images[0])

train_images = train_images / 255.0

test_images = test_images / 255.0
```

예제: 패션 아이템 분류

```
model = keras.Sequential()  
Model.add(keras.layers.Input(shape=(28*28,))  
model.add(keras.layers.Flatten())  
model.add(keras.layers.Dense(128, activation='relu'))  
model.add(keras.layers.Dense(10, activation='softmax'))  
  
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])  
  
model.fit(train_images, train_labels, epochs=5)  
  
(test_loss, test_acc) = model.evaluate(test_images, test_labels)  
  
print('정확도:', test_acc)
```

```
10000/10000 [=====] - 0s 32us/sample - loss: 0.3560 -  
acc: 0.8701  
정확도: 0.8701
```

예제: 타이타닉 생존자 예측하기



생존자를 예측해봅시다. 어떤 부류의 사람들의 생존률이 높았을까요?
우리는 어떤 속성을 이용하여 이것을 예측할 수 있을까요?



Second



Third



Crew

라이브러리 임포트하기

<https://www.kaggle.com/c/titanic/data>

학습 데이터 : train.csv

테스트 데이터 : test.csv

pandas 를 이용해서 데이터를 읽어온다.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
import tensorflow as tf
```

학습데이터 다운로드

데이터 세트를 읽어들인다.

```
train = pd.read_csv("train.csv", sep=',')
```

```
test = pd.read_csv("test.csv", sep=',')
```

필요없는 컬럼을 삭제한다.

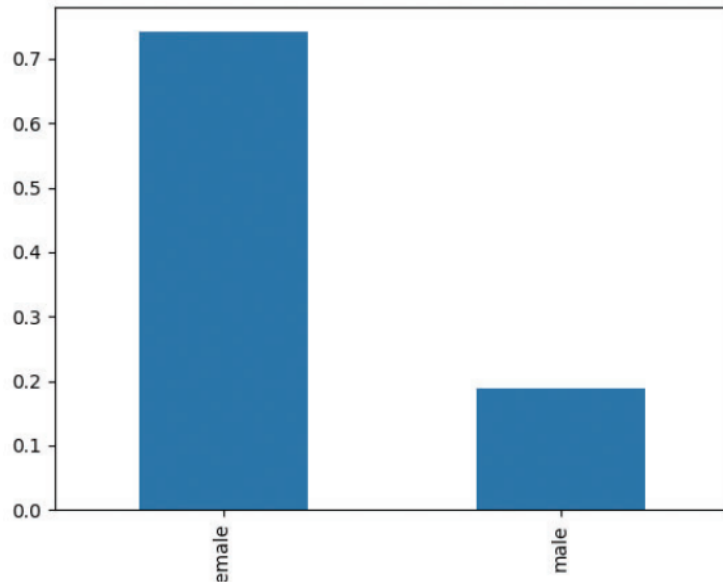
```
train.drop(['SibSp', 'Parch', 'Ticket', 'Embarked', 'Name', \
            'Cabin', 'PassengerId', 'Fare', 'Age'], inplace=True, axis=1)
```

결손치가 있는 데이터 행은 삭제한다.

```
train.dropna(inplace=True)
```

시각화

```
>>> df = train.groupby('Sex').mean()["Survived"]  
>>> df.plot(kind='bar')  
>>> plt.show()
```



여성의 생존률이 높은
것을 알 수 있네요!



데이터 정제하기

```
train.drop(['SibSp', 'Parch', 'Ticket', 'Embarked', 'Name', 'Cabin', 'PassengerId', 'Fare', 'Age'], inplace=True, axis=1)
```

```
>>> train.head()
```

	Survived	Pclass	Sex
0	0	3	male
1	1	1	female
2	1	3	female
3	1	1	female
4	0	3	male

범주형 데이터를 숫자값으로 변환

```
# 기호를 수치로 변환한다.  
for ix in train.index:  
    if train.loc[ix, 'Sex']=="male":  
        train.loc[ix, 'Sex']=1  
    else:  
        train.loc[ix, 'Sex']=0
```

```
# 2차원 배열을 1차원 배열로 평탄화한다.  
target = np.ravel(train.Survived)
```

```
# 생존여부를 학습 데이터에서 삭제한다.  
train.drop(['Survived'], inplace=True, axis=1)  
train = train.astype(float) # float형태로 형변환 해야 함.
```

컴퓨터는 숫자만 처리할 수 있다.
딥러닝은 0부터 1 사이의 실수만 처리 가능

케라스 모델 생성

케라스 모델을 생성한다.

```
model = keras.Sequential()
```

```
model.add(keras.layers.Input((2,)))
```

```
model.add(keras.layers.Dense(16, activation='relu'))
```

```
model.add(keras.layers.Dense(8, activation='relu'))
```

```
model.add(keras.layers.Dense(1, activation='sigmoid'))
```

케라스 모델을 컴파일한다.

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

케라스 모델을 학습시킨다.

```
model.fit(train, target, epochs=30, batch_size=1, verbose=1)
```

실행결과

```
...  
Epoch 29/30  
891/891 [=====] - 1s 753us/sample - loss: 0.4591 - acc:  
0.7677  
Epoch 30/30  
891/891 [=====] - 1s 753us/sample - loss: 0.4547 - acc:  
0.7789
```

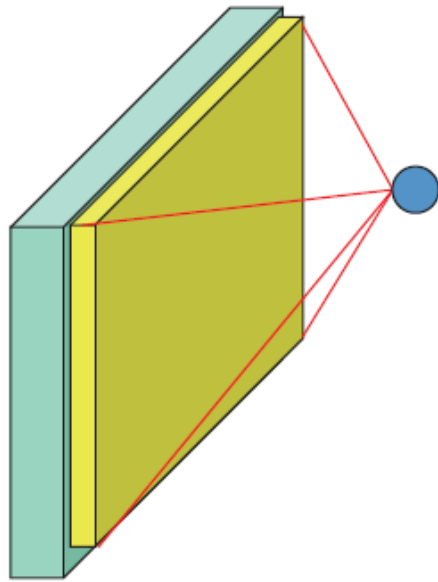
약 78% 정확도

CNN (컨벌루션 신경망)

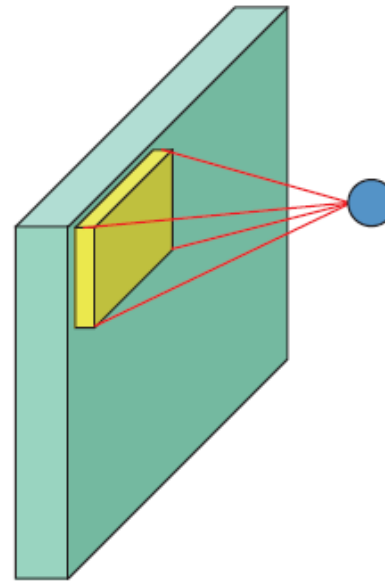


컨볼루션 신경망(CNN)

- 컨볼루션(Convolution Neural Network: CNN) 신경망에서는 하위 레이어의 노드들과 상위 레이어의 노드들이 부분적으로만 연결되어 있다.



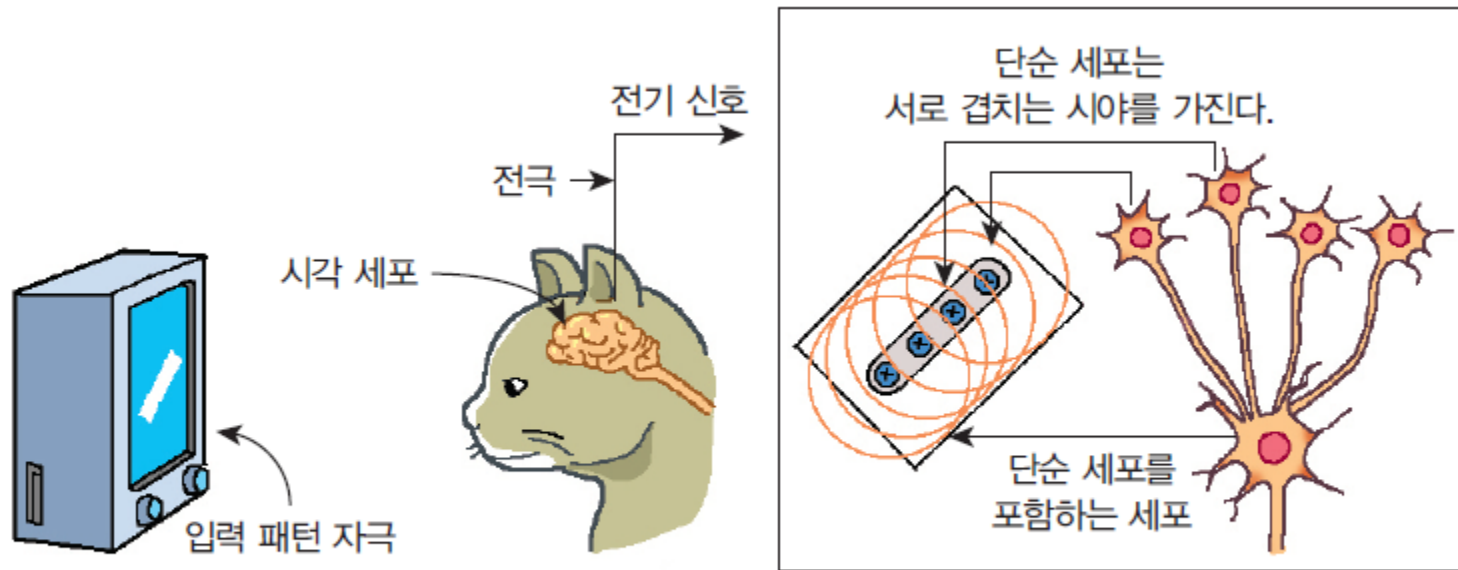
(a) 완전연결 신경망



(b) 컨볼루션 신경망

컨볼루션 신경망(CNN)

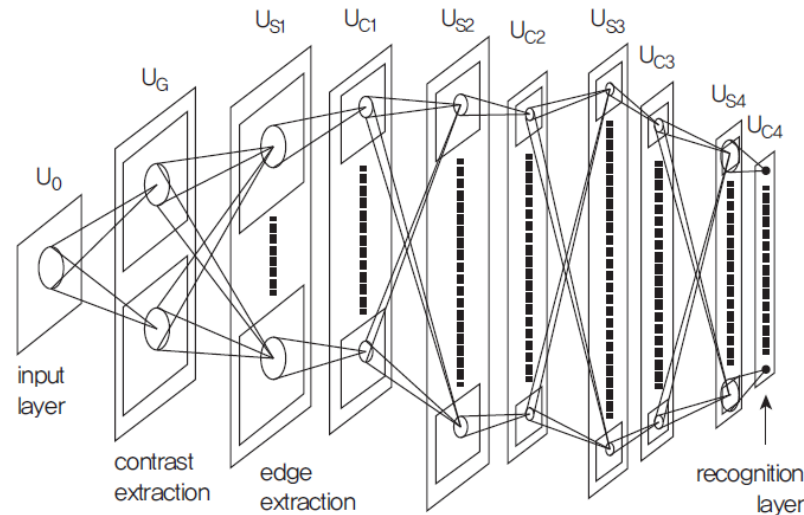
- 컨볼루션 신경망은 Hubel과 Wiesel이 발견한 고양이의 시각 세포에서부터 출발한다.



시각 피질 뉴런의 구조(출처: Hubel's book, Eye, Brain, and Vision)

네오코그니트론

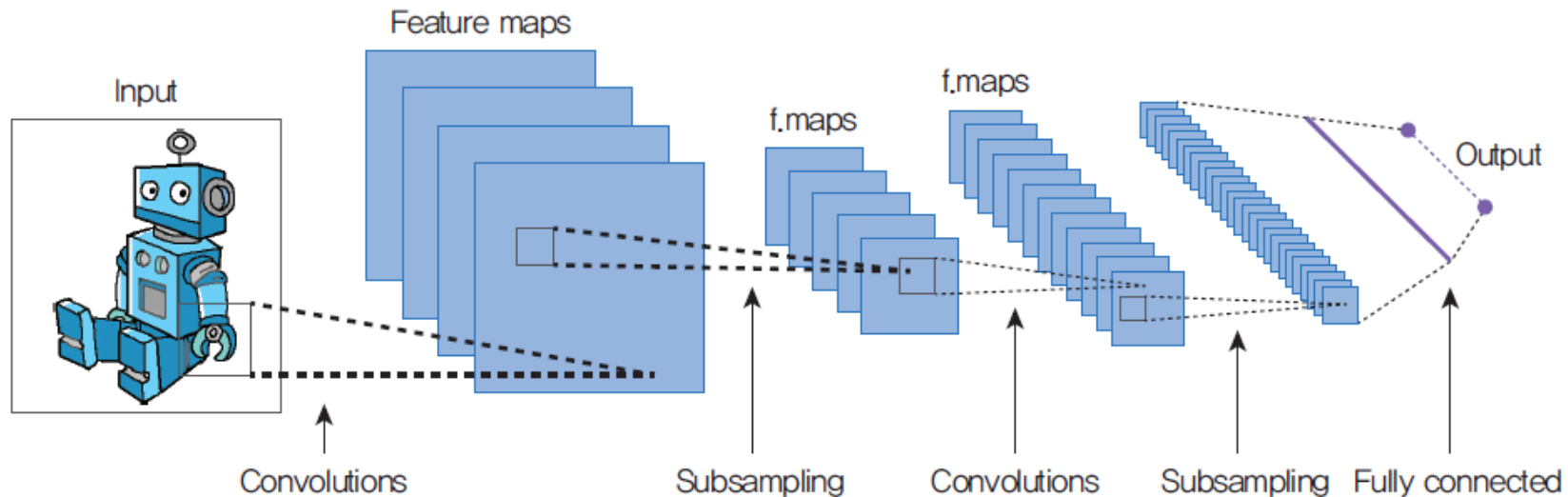
- “네오코그니트론(Neocognitron)”은 1980년 후쿠시마(Kunihiko Fukushima)에 의해 소개된 신경망 구조이다. 후쿠시마도 위에서 언급한 허벨과 위젤의 작업에서 영감을 받았다.
- 후쿠시마 는 허벨과 위젤이 발견한 단순 세포와 복합 세포를 패턴 인식에 사용하기 위해, 이 두 가지 유형의 세포로 이루어진 계층적인 모델을 제안했다. 또 네오코그니트론에서는 컨볼루션 신경망의 두 가지 기본 유형 레이어인, 컨볼루션 레이어와 서브 샘플링 레이어를 도입했다



네오코그니트론(출처: 후쿠시마의 논문)

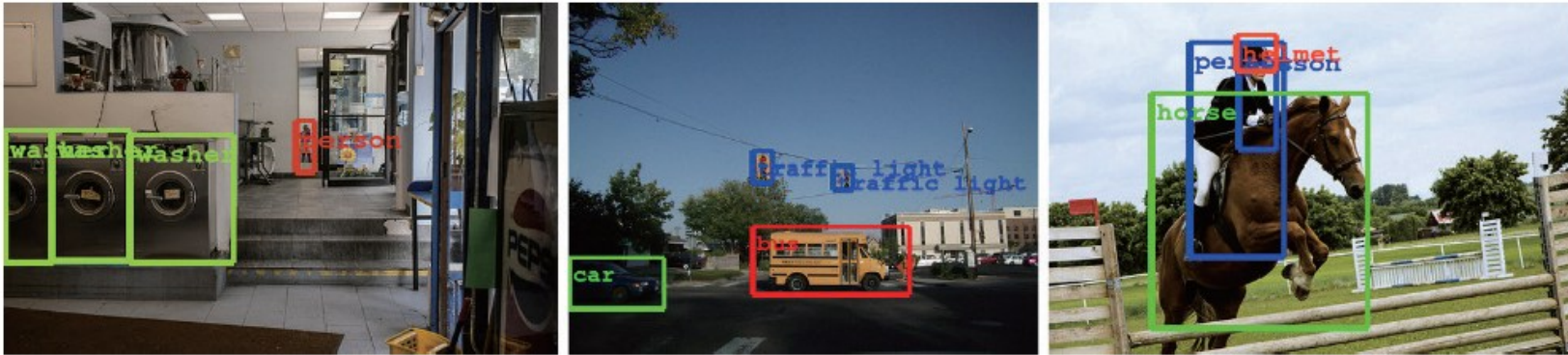
컨벌루션 신경망의 중요성

- 컨벌루션 신경망은 모든 신경망 구조 중에서 가장 강력한 성능을 보여주는 신경망 중의 하나이다.
- 컨벌루션 신경망은 2차원 형태의 입력을 처리하기 때문에, 이미지 처리에 특히 적합하다. 신경망의 각 레이어에서 일련의 필터가 이미지에 적용된다.



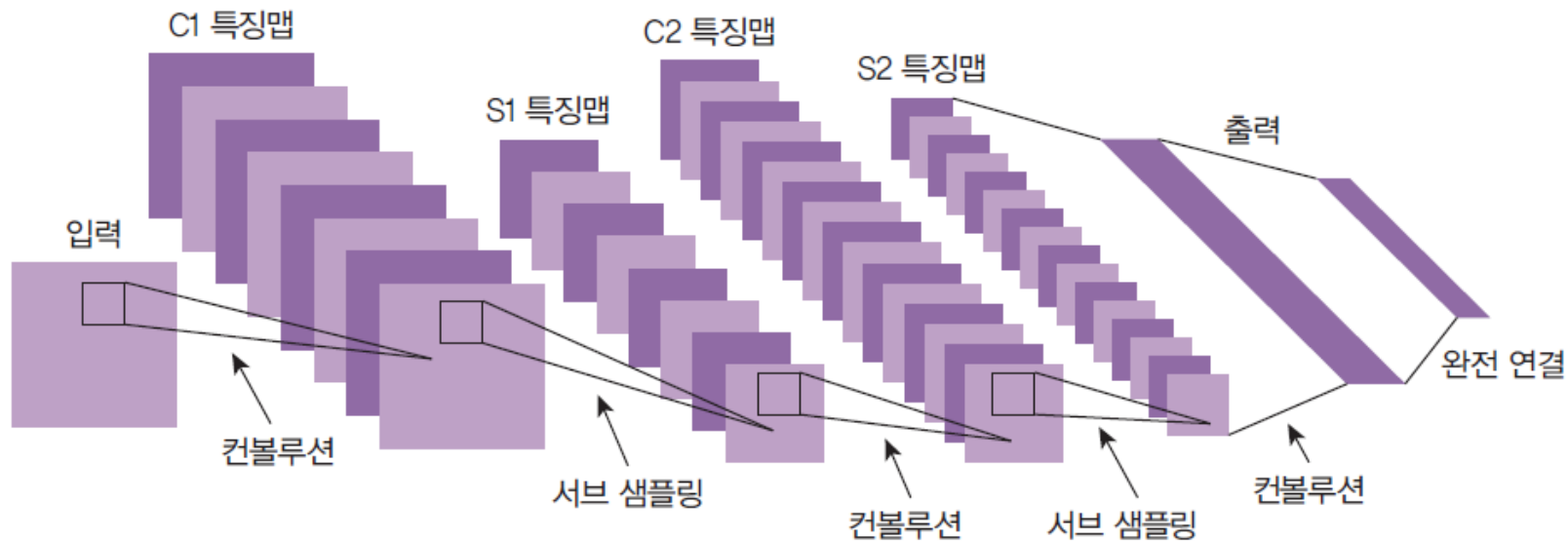
컨벌루션 신경망의 중요성

- 2012년도 영상 인식 경진 대회 ILSVRC에서 우승하여, 딥러닝의 우수성을 전 세계에 알리고 인공지능 부활의 신호탄이 되었던 신경망 모델도 컨벌루션 신경망이었다



CNN의 일반적인 구조

- 컨볼루션 신경망도 여러 레이어를 연결하여 신경망을 구축한다

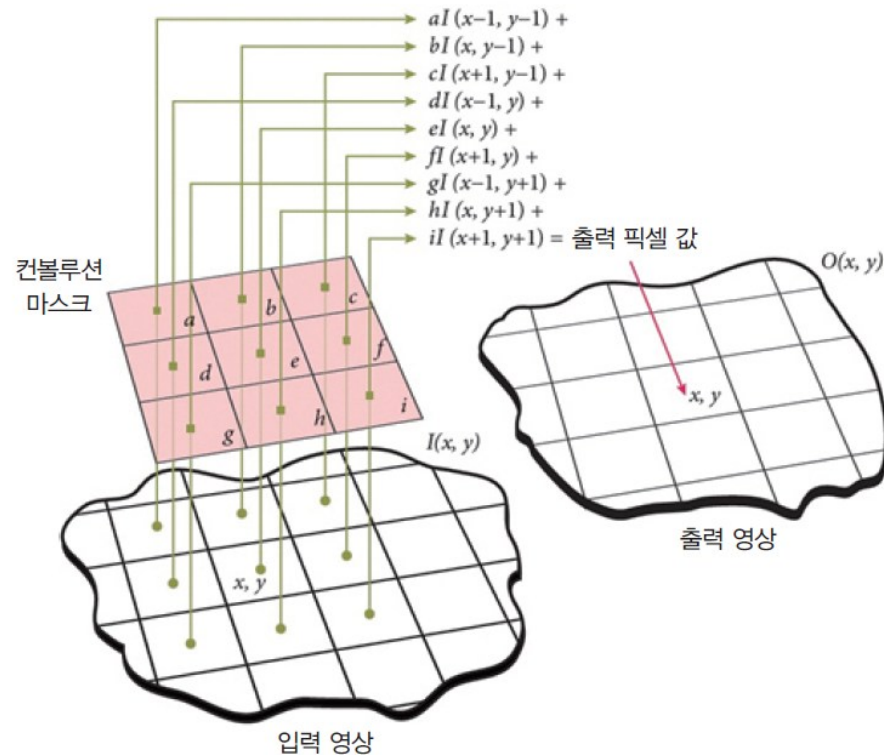


CNN의 일반적인 구조

- 입력층
- 입력층에서 컨벌루션 연산을 통하여 특징을 뽑아내는 특징맵 (feature map)이 존재
- 풀링(Pooling) 연산을 적용한다. 풀링 연산은 입력의 차원을 줄이는 연산이다.
- 컨벌루션 레이어와 풀링 레이어는 여러 번 되풀이 된다.
- 신경망의 맨 끝에는 완전히 연결된 구조의 전통적인 분류 신경망이 있어서 추출된 특징을 바탕으로 물체를 인식한다.

컨벌루션

- 컨벌루션은 주변 화소값들에 가중치를 곱해서 더한 후에 이것을 새로운 화소값으로 하는 연산이다.



영상 처리에서 컨벌루션 연산

컨벌루션 연산

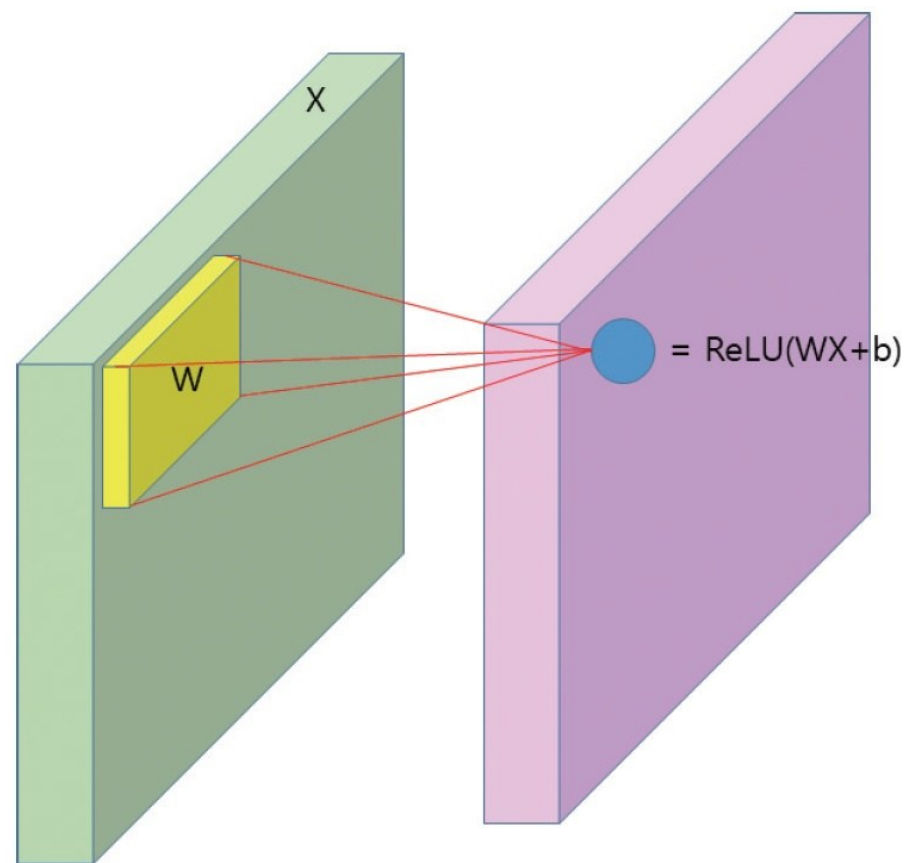
$$\begin{aligned} O(x, y) = & aI(x-1, y-1) + bI(x, y-1) + cI(x+1, y-1) \\ & + dI(x-1, y) + eI(x, y) + fI(x+1, y) \\ & + gI(x-1, y+1) + hI(x, y+1) + iI(x+1, y+1) \end{aligned}$$



$$O(x, y) = \sum_{k=-1}^{k=+1} \sum_{l=-1}^{l=+1} h(k, l) I(x+k, y+l)$$

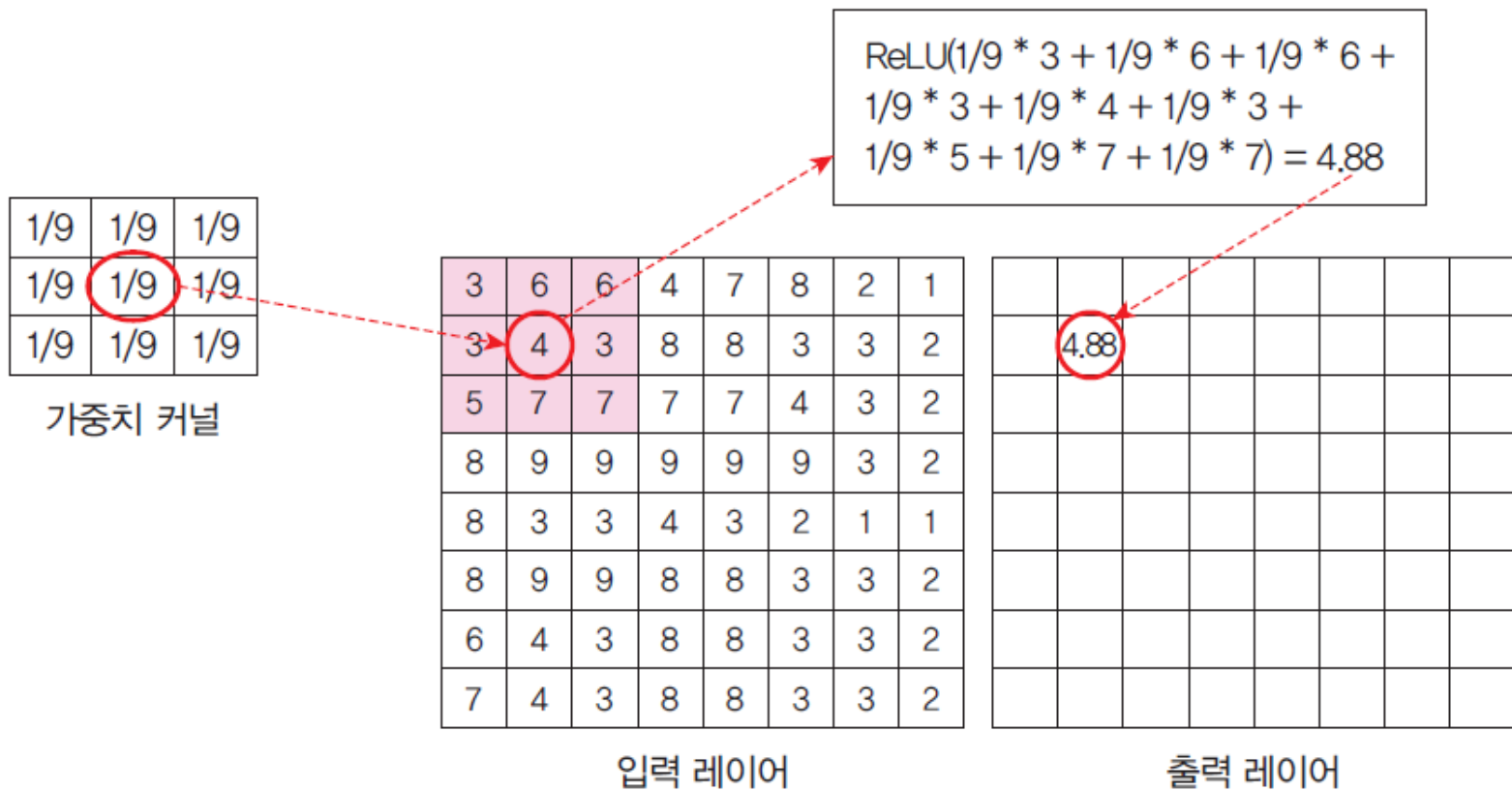
클래식한 관점에서 필터링이라고 하고, 이렇게 모인집합을 필터 बैं크라고 한다

컨볼루션

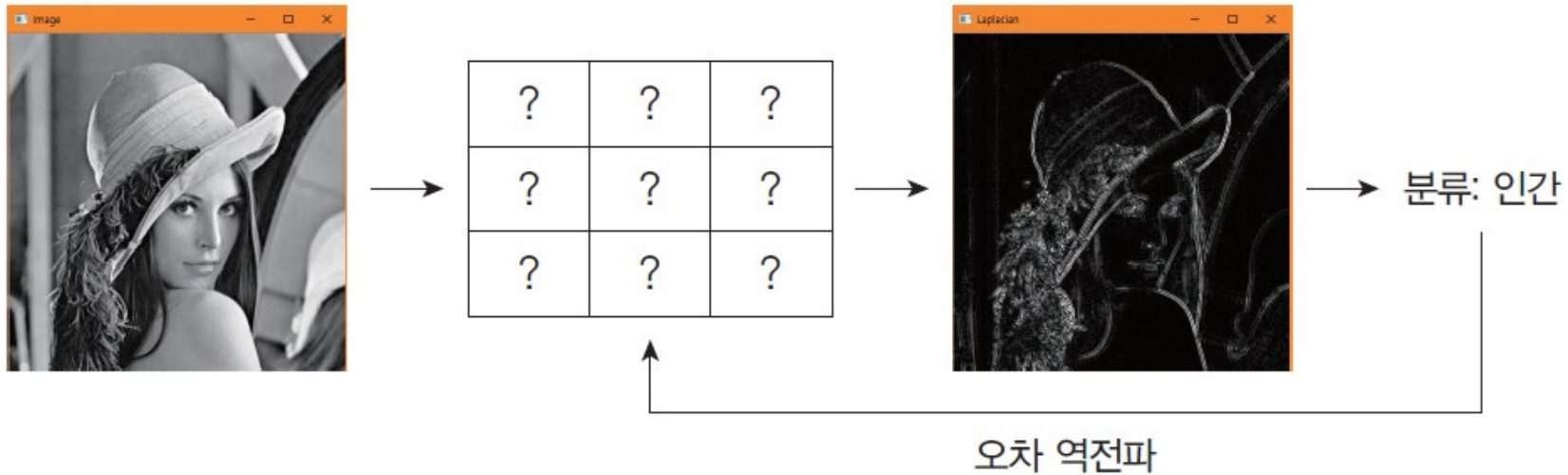


신경망에서의 컨볼루션 연산

컨벌루션의 구체적인 예



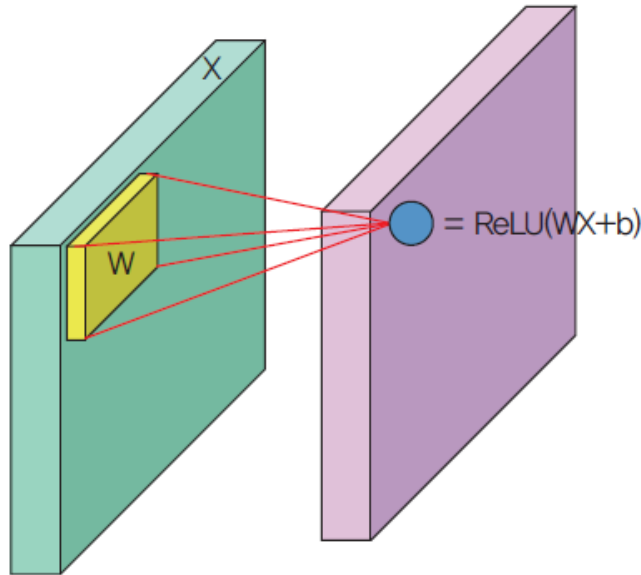
신경망에서는 필터의 가중치가 학습된다.



컨벌루션 신경망에서는 커널의 가중치들이 학습된다.

컨벌루션 신경망에서의 컨벌루션 연산

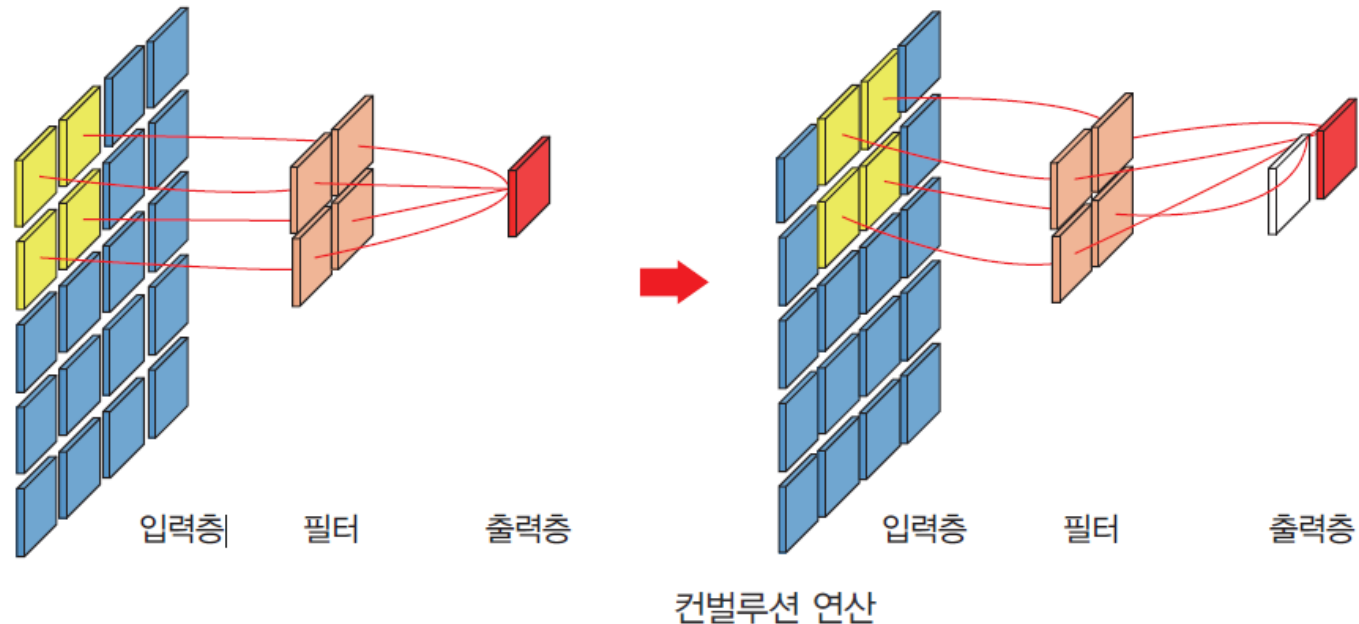
- 컨벌루션 신경망에서도 커널이 입력층의 각 화소를 중심으로 덮여 씌워진다. 앞 레이어의 값 X 는 각 커널 W 와 곱해져서 더해져서 $WX+b$ 가 된다.
- 이 계산값은 $\text{ReLU}()$ 와 같은 활성화 함수를 통과해서, 다음 레이어의 동일한 위치에 $\text{ReLU}(WX+b)$ 로 저장된다.



신경망에서의 컨벌루션 연산

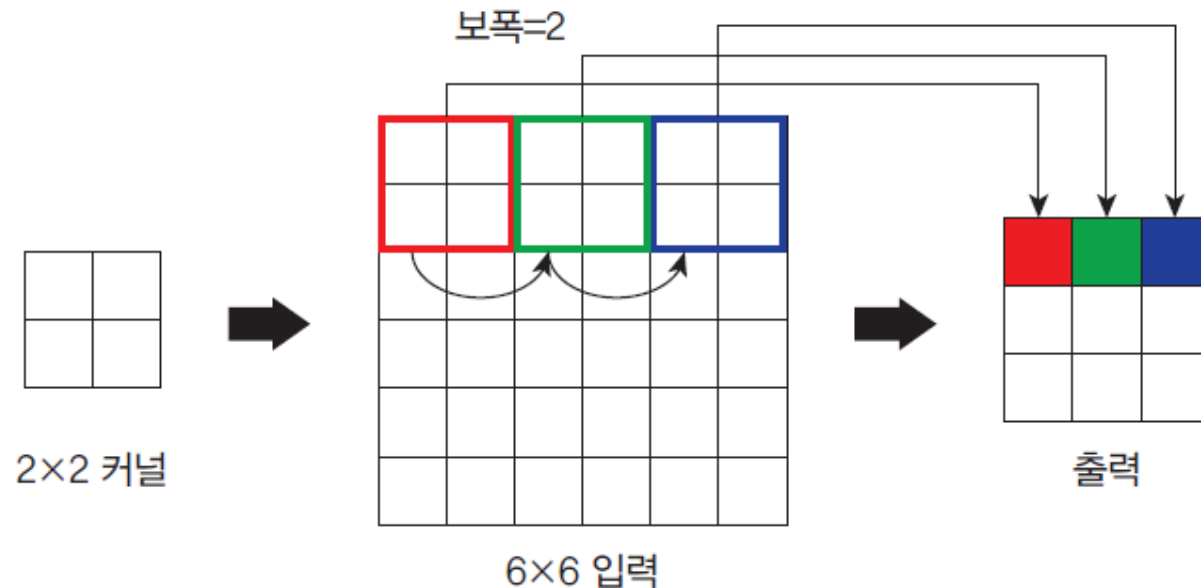
컨벌루션 레이어

- 여러 개의 필터를 이용할 수 있다.
- *필터의 값은 미리 정해진 것이 아니고 학습된다.*



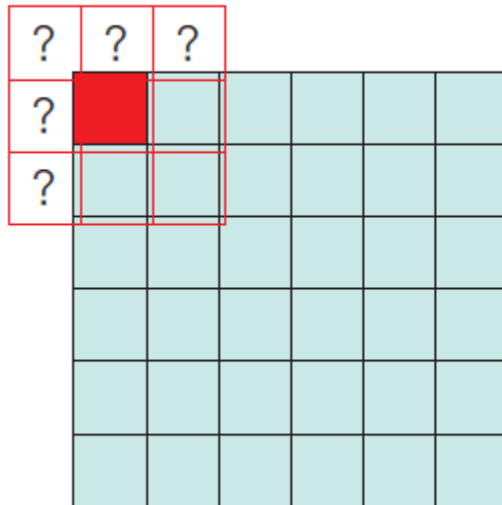
보폭

- 보폭(stride) 은 커널을 적용하는 거리이다. 보폭이 1이면 커널을 한 번에 1픽셀씩 이동하면서 커널을 적용하는 것이다.
- 보폭이 2라는 것은 하나씩 건너뛰면서 픽셀에 커널을 적용한다는 것을 의미한다.



패딩

- 패딩(padding)은 이미지의 가장자리를 처리하기 위한 기법이다.



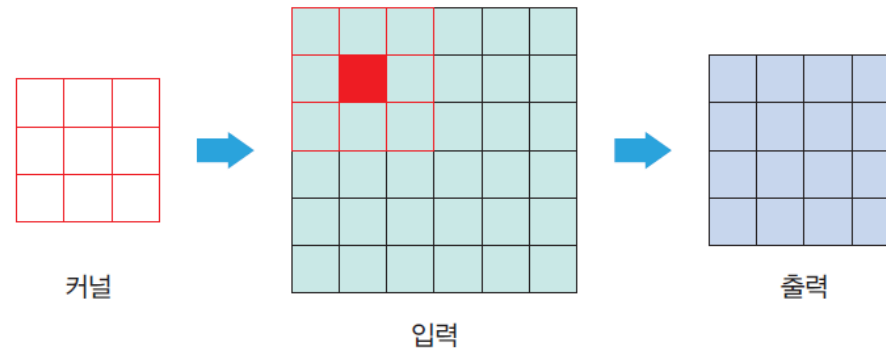
이미지의 가장 자리에 커널을
적용하려니, 커널 아래에 픽셀이
없네요. 어떻게 해야 할까요?



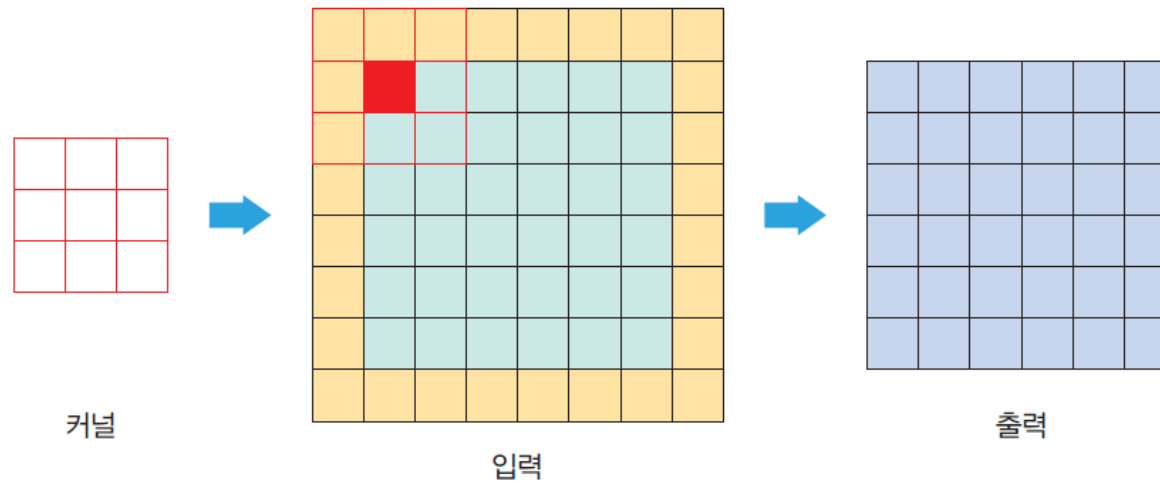
패딩이 필요한 이유

2가지 패딩 방법

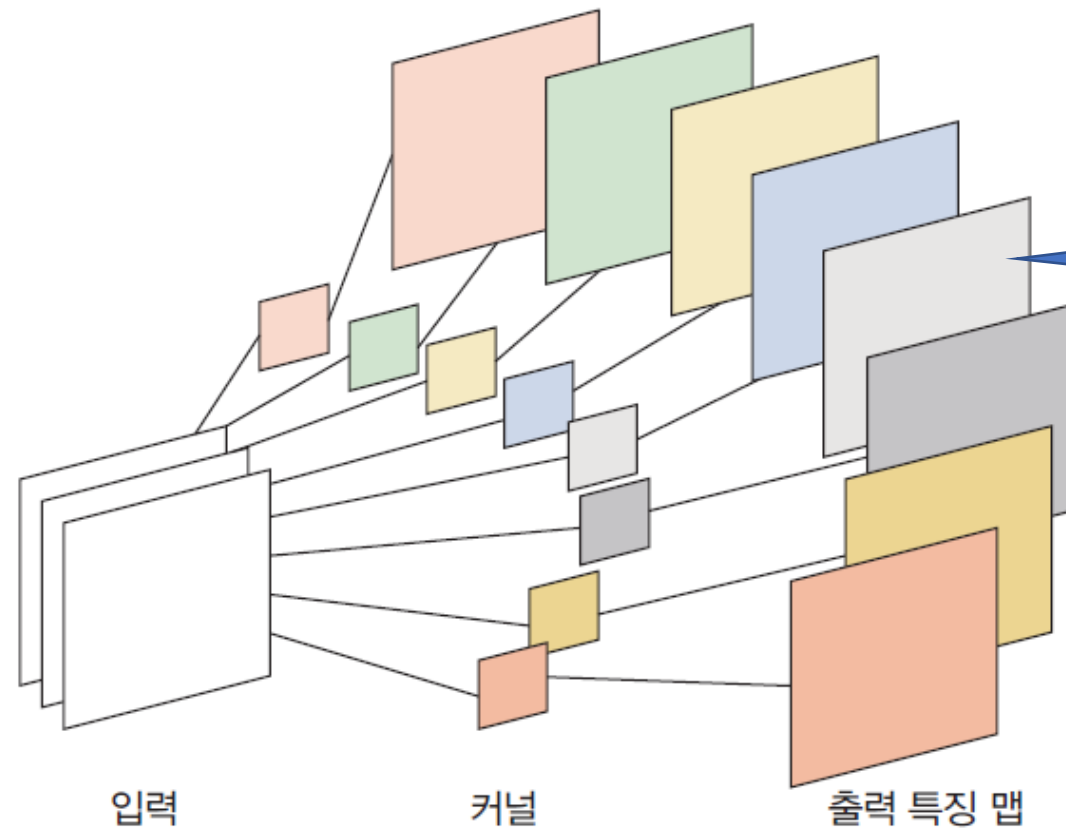
- Valid: 커널을 입력 이미지 안에서만 움직인다.



- Same : 입력 이미지의 주변을 특정값(예를 들면 0, 또는 이웃 픽셀값)으로 채우는 것



필터가 여러 개일 때의 컨벌루션 레이어

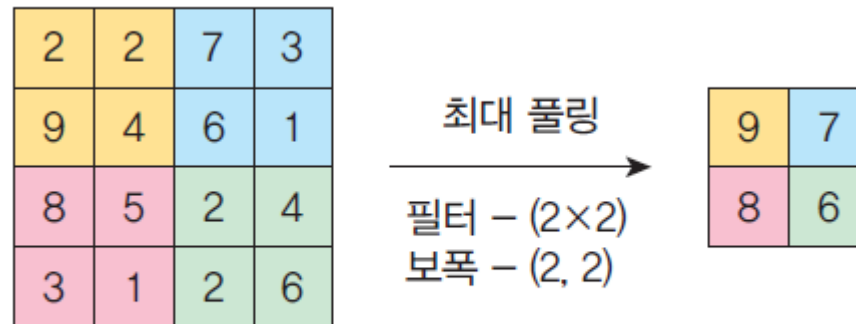


커널이 많아지면 자연스럽게 특징 맵의 개수도 많아진다.

필터가 여러 개일 때의 컨벌루션 레이어

풀링 또는 서브 샘플링

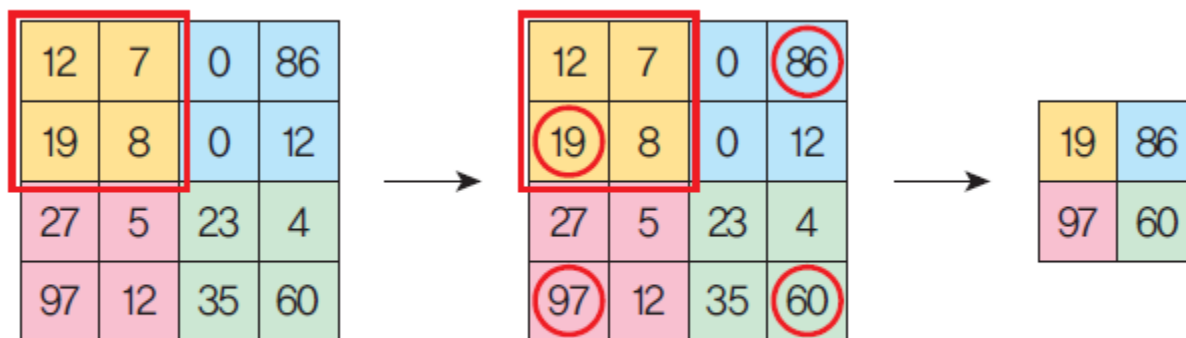
- 풀링(Pooling)이란 서브 샘플링이라고도 하는 것으로 입력 데이터의 크기를 줄이는 것이다.



풀링 레이어

최대 풀링

- 컨벌루션처럼 윈도우를 움직여서 윈도우 안에 있는 숫자 중에서 가장 큰 값만 출력하는 연산이다.

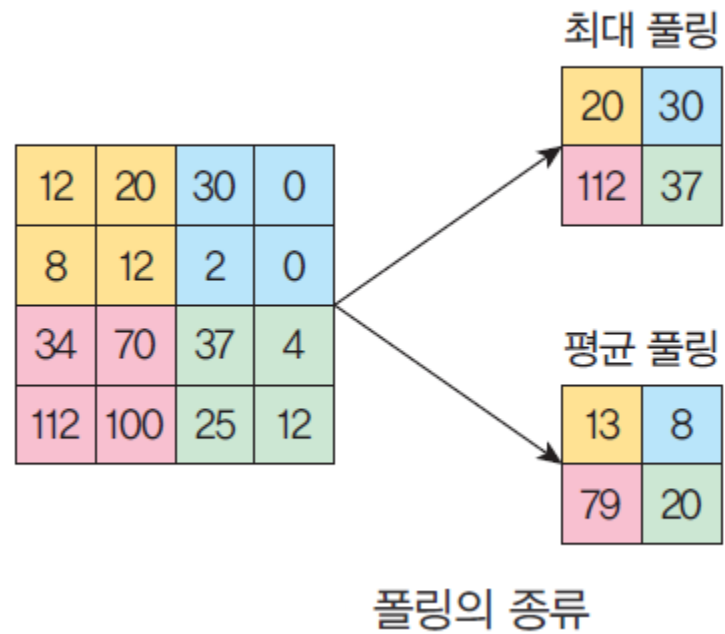


풀링 연산

풀링의 장점

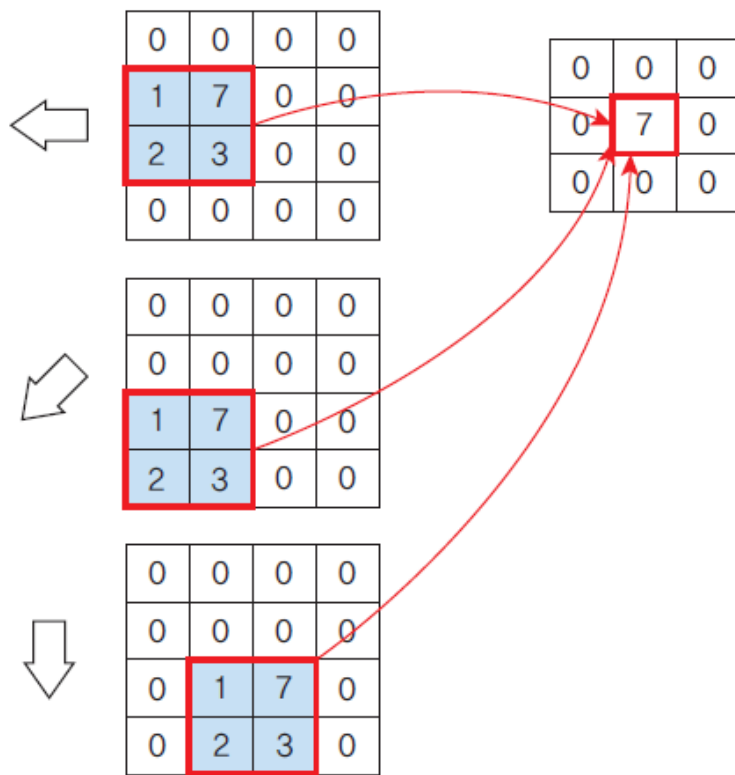
- 레이어의 크기가 작아지므로 계산이 빨라진다.
- 레이어의 크기가 작아진다는 것은 신경망의 매개변수가 작아진다는 것을 의미한다. 따라서 과적합이 나올 가능성이 줄어든다.
- 공간에서 물체의 이동이 있어도 결과는 변하지 않는다. 즉 물체의 공간이동에 대하여 둔감해지게 된다.

폴링의 종류

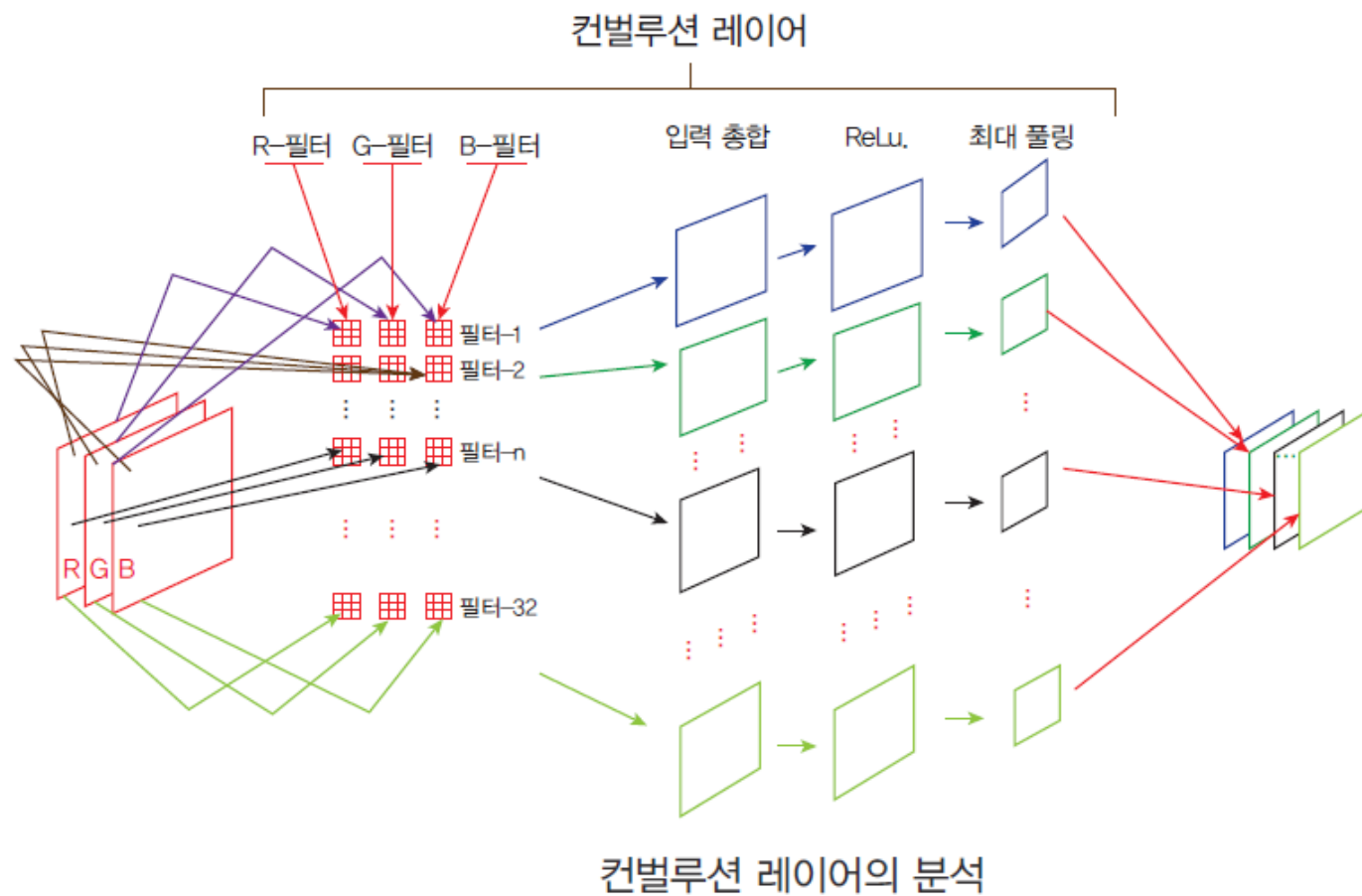


풀링의 장점

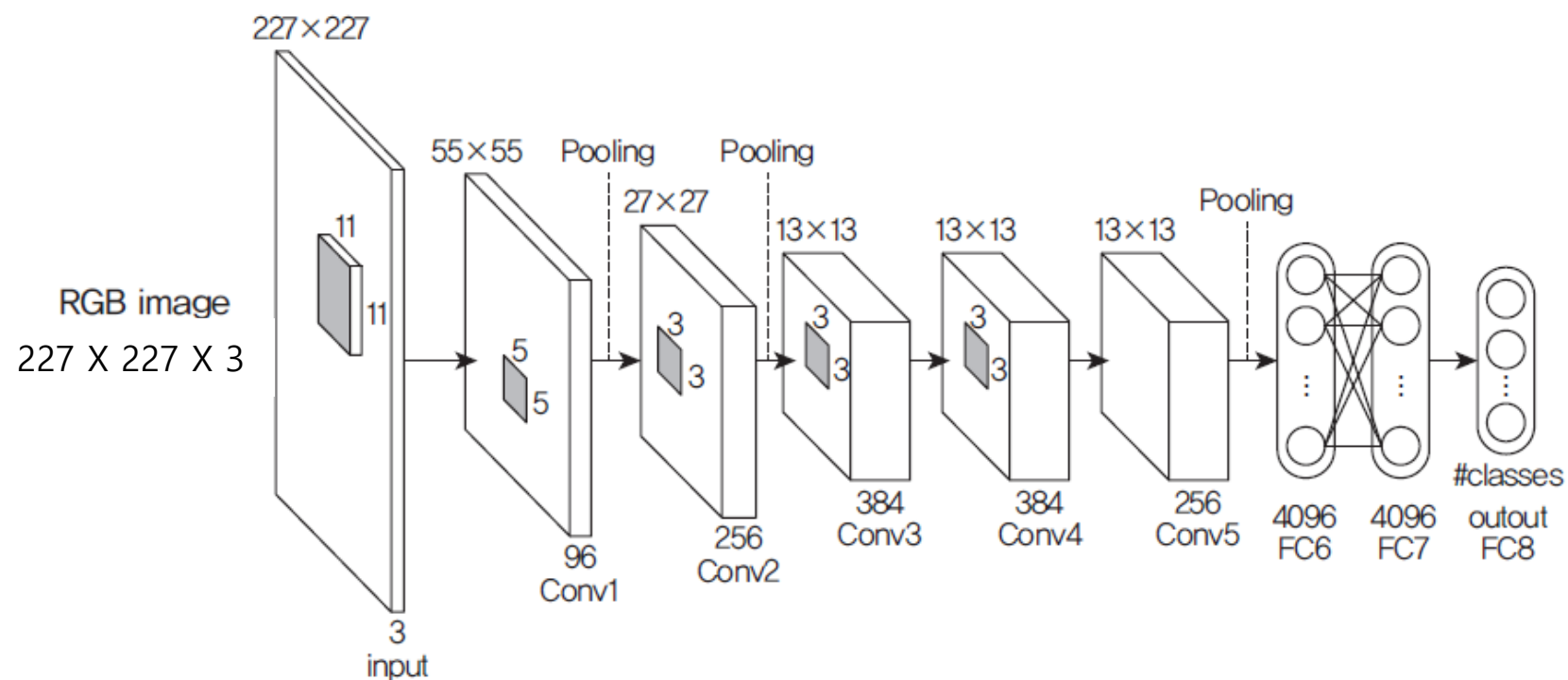
- 풀링 계층이 하는 작업 중에서 가장 중요한 것은 물체의 이동에 대하여 둔감하게 하는 것이다



컨벌루션 신경망의 해석



AlexNet



AlexNet

(1) 첫 번째 레이어 Conv1은 컨볼루션 레이어로 $11 \times 11 \times 3$ 커널을 96개를 사용하고 보폭은 4이고 패딩은 사용하지 않았다. 여기에 ReLu 활성화 함수를 적용하고 이어서, 3×3 겹치는 최대 풀링이 적용된다. 결과적으로 $27 \times 27 \times 96$ 크기의 특징맵이 나오게 된다.

(2) 두 번째 레이어 Conv2는 256개의 $5 \times 5 \times 48$ 크기의 커널을 사용하여 전 단계의 특징맵을 컨볼루션한다. 보폭은 1로, 패딩은 2로 설정한다. 따라서 $27 \times 27 \times 256$ 크기의 특징맵이 얻어진다. 이어서 3×3 최대 풀링을 보폭 2로 시행한다. 최종적으로 $13 \times 13 \times 256$ 특징맵을 얻는다.

(3) 세 번째 레이어, 네 번째 다섯 번째 레이어는 모두 유사하게 처리된다. 즉 보폭과 패딩이 모두 1로 설정된다. 커널의 개수만 달라진다.

케라스로 컨벌루션 신경망 구현하기

	클래스 이름	설명
컨벌루션 레이어	Conv1D, Conv2D, Conv3D, SeparableConv1D, SeparableConv2D, DepthwiseConv2D, Conv2DTranspose, Conv3DTranspose	컨벌루션 연산을 구현하는 레이어이다.
풀링 레이어	MaxPooling1D, MaxPooling2D AveragePooling1D, AveragePooling2D GlobalMaxPooling1D, GlobalMaxPooling2D GlobalAveragePooling1D, GlobalAveragePooling2D	몇 개의 값을 하나로 합치는 레이어이다.

Keras Convolution 사용하기

```
shape = (4, 28, 28, 3)
x = tf.random.normal(shape)
y = tf.keras.layers.Conv2D(2, 3, activation='relu', input_shape=shape[1:])(x)
print(y.shape)
```

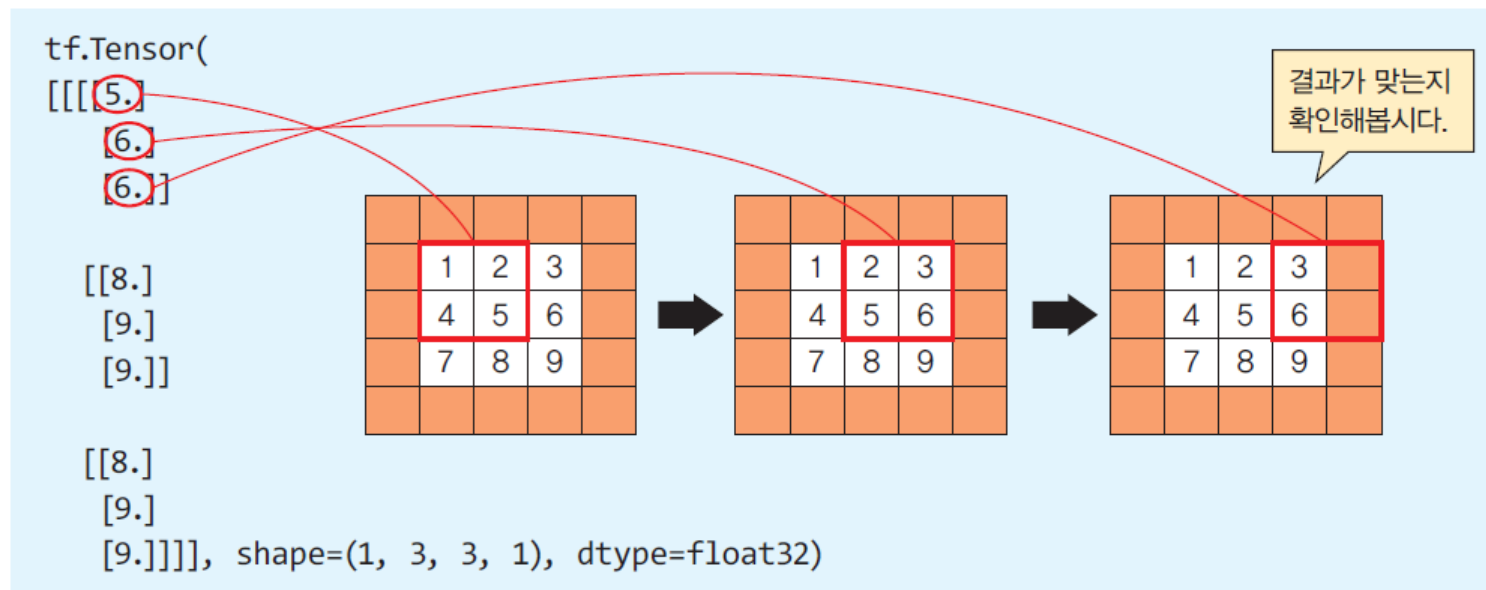
(4, 26, 26, 2)

컨벌루션 레이어

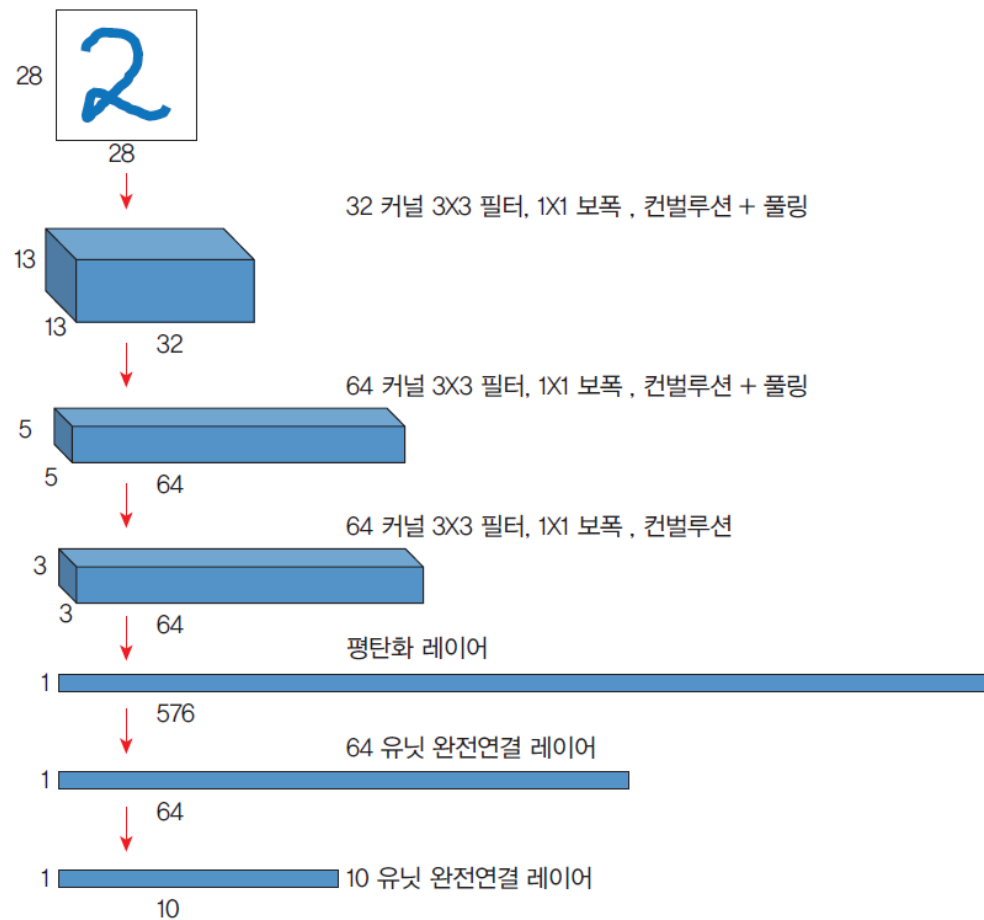
- `tf.keras.layers.MaxPooling2D(pool_size=(2, 2), strides=None, padding="valid")`
 - `pool_size`: 풀링 윈도우의 크기, 정수 또는 2개 정수의 튜플이다. (2, 2)라면 2x2 풀링 윈도우에서 최대값을 추출한다.
 - `strides`: 보폭, 각 풀링 단계에 대해 풀링 윈도우가 이동하는 거리를 지정한다.
 - `padding`: "valid"나 "same" 중의 하나이다. "valid"는 패딩이 없음을 의미한다. "same"은 출력이 입력과 동일한 높이 / 너비 치수를 갖도록 입력의 왼쪽 / 오른쪽 또는 위 / 아래에 균일하게 패딩한다.

MaxPooling2D

```
x = tf.constant([[1., 2., 3.], [4., 5., 6.], [7., 8., 9.]])
x = tf.reshape(x, [1, 3, 3, 1])
max_pool_2d = tf.keras.layers.MaxPooling2D(pool_size=(2, 2), strides=(1, 1), padding='valid')
print(max_pool_2d(x))
```



예제: MNIST 필기체 숫자 인식



예제: MNIST 필기체 숫자 인식

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()

train_images = train_images.reshape((60000, 28, 28, 1))
test_images = test_images.reshape((10000, 28, 28, 1))

# 픽셀 값을 0.0 ~ 1.0 사이로 정규화한다.

(train_images, test_images) = (train_images / 255.0, test_images / 255.0)
```

예제: MNIST 필기체 숫자 인식

```
model = models.Sequential()  
model.add(layers.Input((28,28,1)))  
model.add(layers.Conv2D(32, (3, 3), activation='relu'))  
model.add(layers.MaxPooling2D((2, 2)))  
model.add(layers.Conv2D(64, (3, 3), activation='relu'))  
model.add(layers.MaxPooling2D((2, 2)))  
model.add(layers.Conv2D(64, (3, 3), activation='relu'))  
  
model.add(layers.Flatten())  
model.add(layers.Dense(64, activation='relu'))  
model.add(layers.Dense(10, activation='softmax'))
```

예제: MNIST 필기체 숫자 인식

```
model.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
--------------	--------------	---------

conv2d_3 (Conv2D)	(None, 26, 26, 32)	320
-------------------	--------------------	-----

max_pooling2d_2 (MaxPooling2)	(None, 13, 13, 32)	0
-------------------------------	--------------------	---

conv2d_4 (Conv2D)	(None, 11, 11, 64)	18496
-------------------	--------------------	-------

max_pooling2d_3 (MaxPooling2)	(None, 5, 5, 64)	0
-------------------------------	------------------	---

conv2d_5 (Conv2D)	(None, 3, 3, 64)	36928
-------------------	------------------	-------

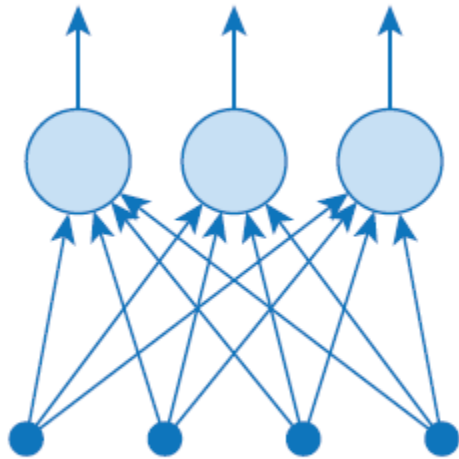
예제: MNIST 필기체 숫자 인식

```
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])  
  
model.fit(train_images, train_labels, epochs=5)
```

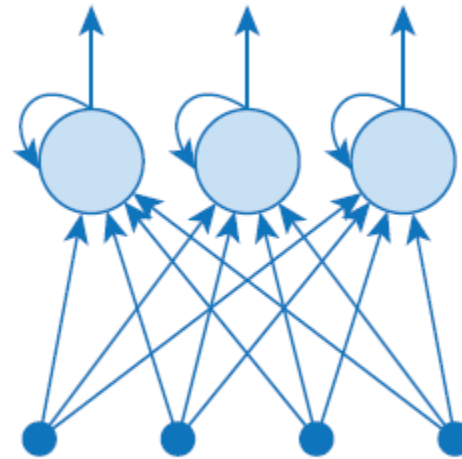
```
Epoch 1/5  
1875/1875 [=====] - 14s 7ms/step - loss: 0.1414 -  
accuracy: 0.9560  
...  
Epoch 5/5  
1875/1875 [=====] - 14s 7ms/step - loss: 0.0194 -  
accuracy: 0.9940
```

RNN의 구조

- 피드-포워드 신경망(feed-forward neural network) vs RNN

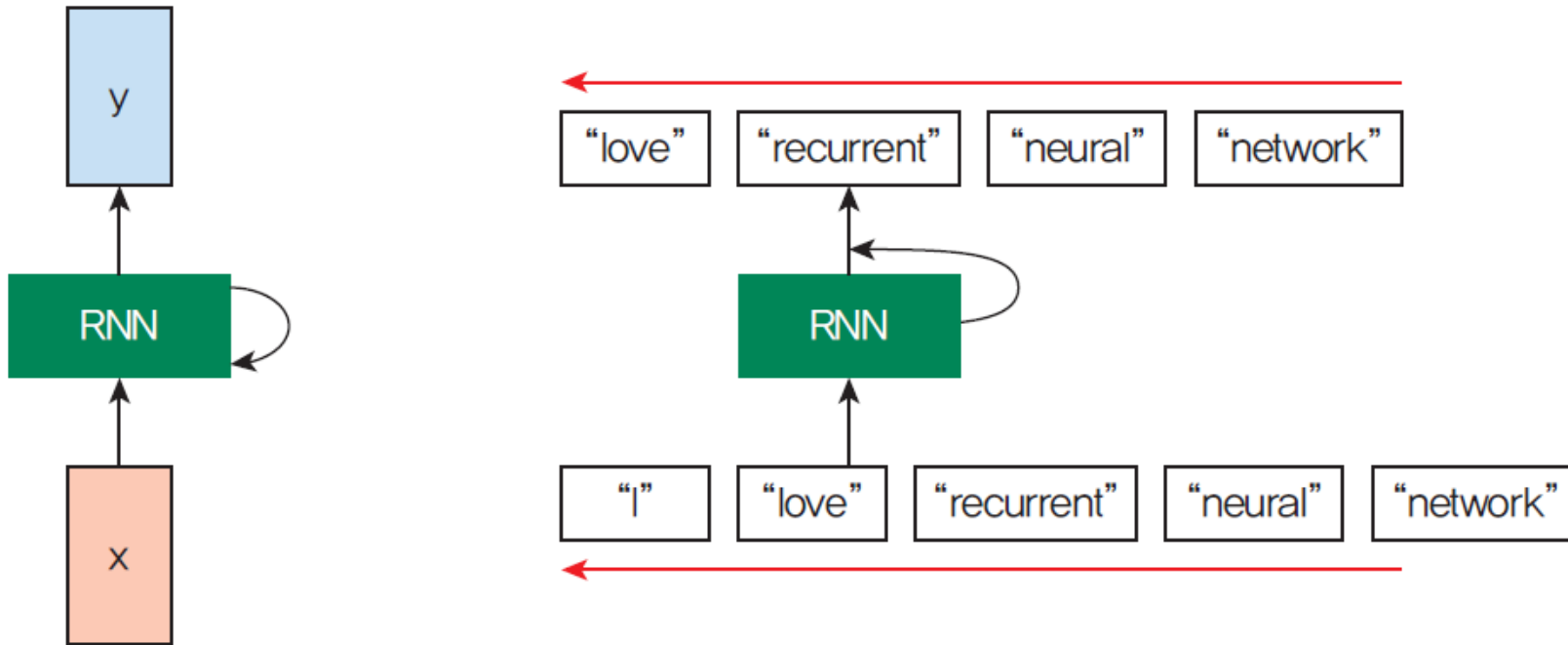


피드-포워드 신경망

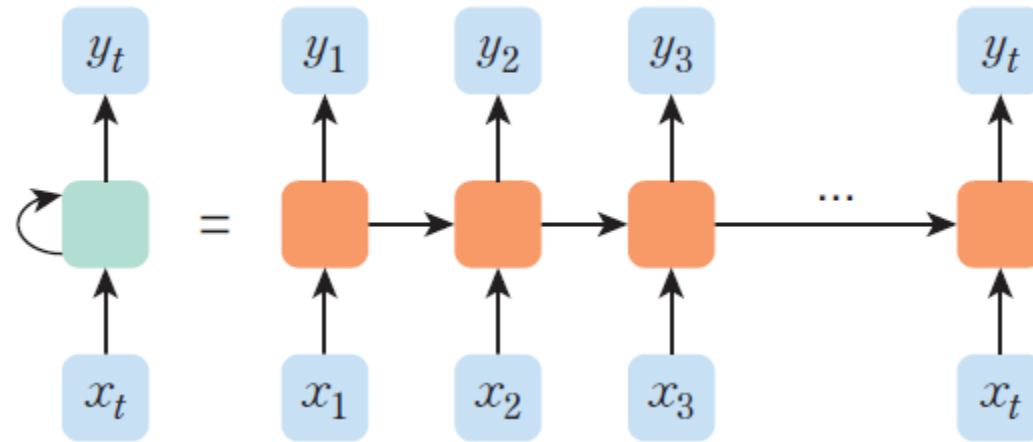


순환 신경망

다음 단어를 예측하는 RNN



순환 신경망을 펼쳐 놓은 그림

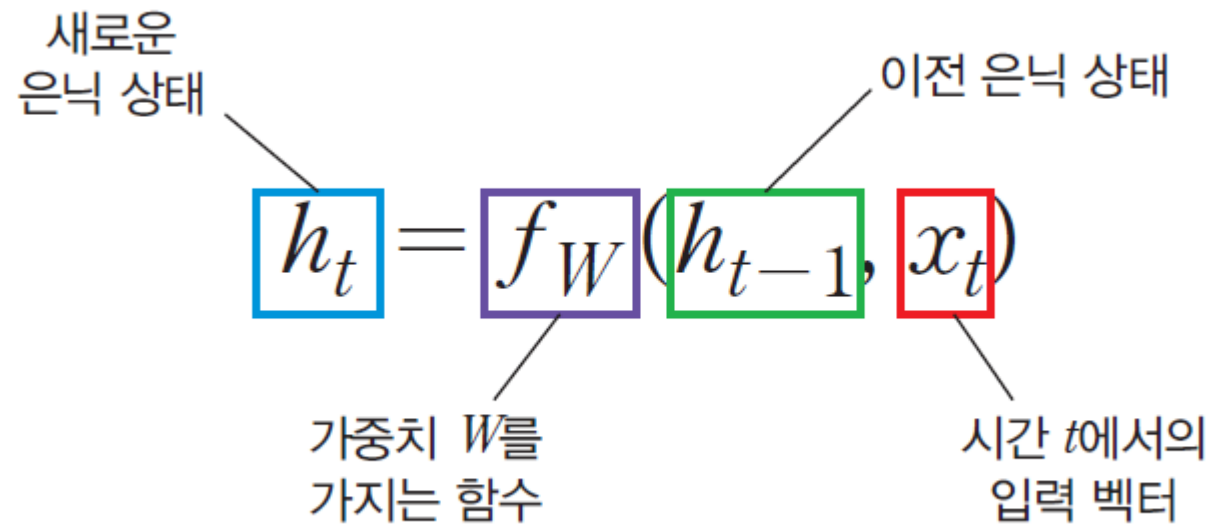


NOTE 은닉 상태, 입력 벡터, 출력 벡터

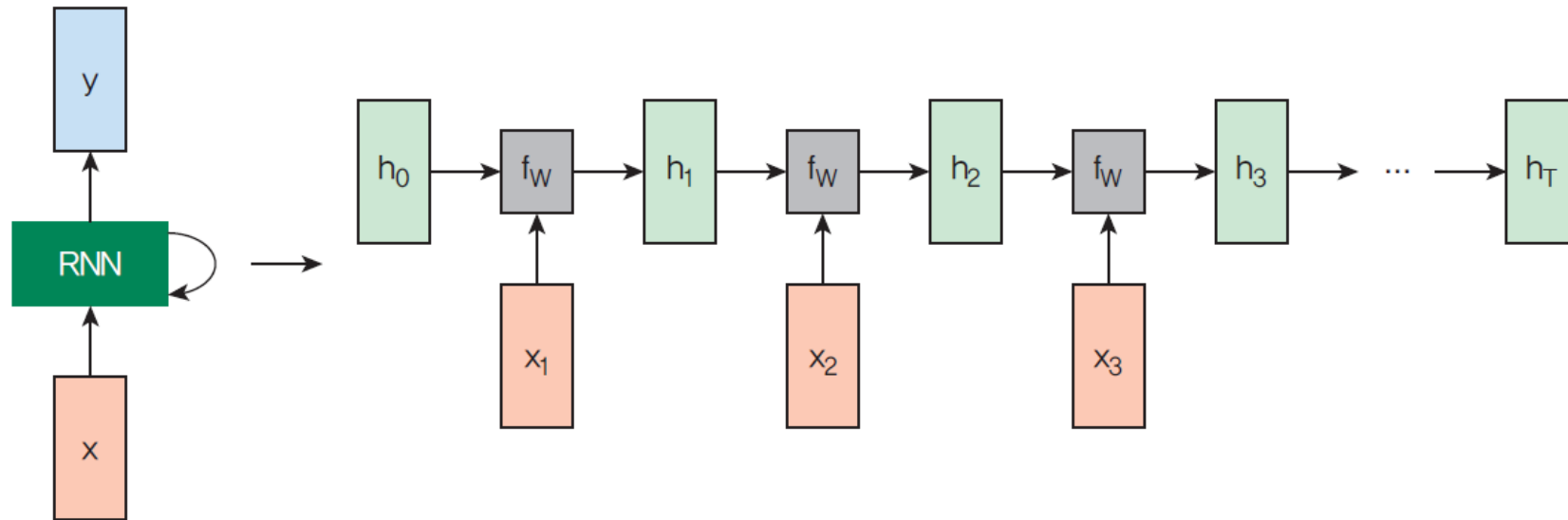
지금까지는 은닉층이란 용어를 사용했지만, 순환 신경망에서는 은닉 상태라는 용어를 주로 사용한다. 또 입력 층, 출력층보다는 입력 벡터, 출력 벡터라고 한다. 만약 주가 예측 신경망에서 지난 3일치의 데이터를 가지고, 다음 날짜의 주가를 예측한다면, 입력 벡터의 크기는 3일 것이다.

RNN의 동작

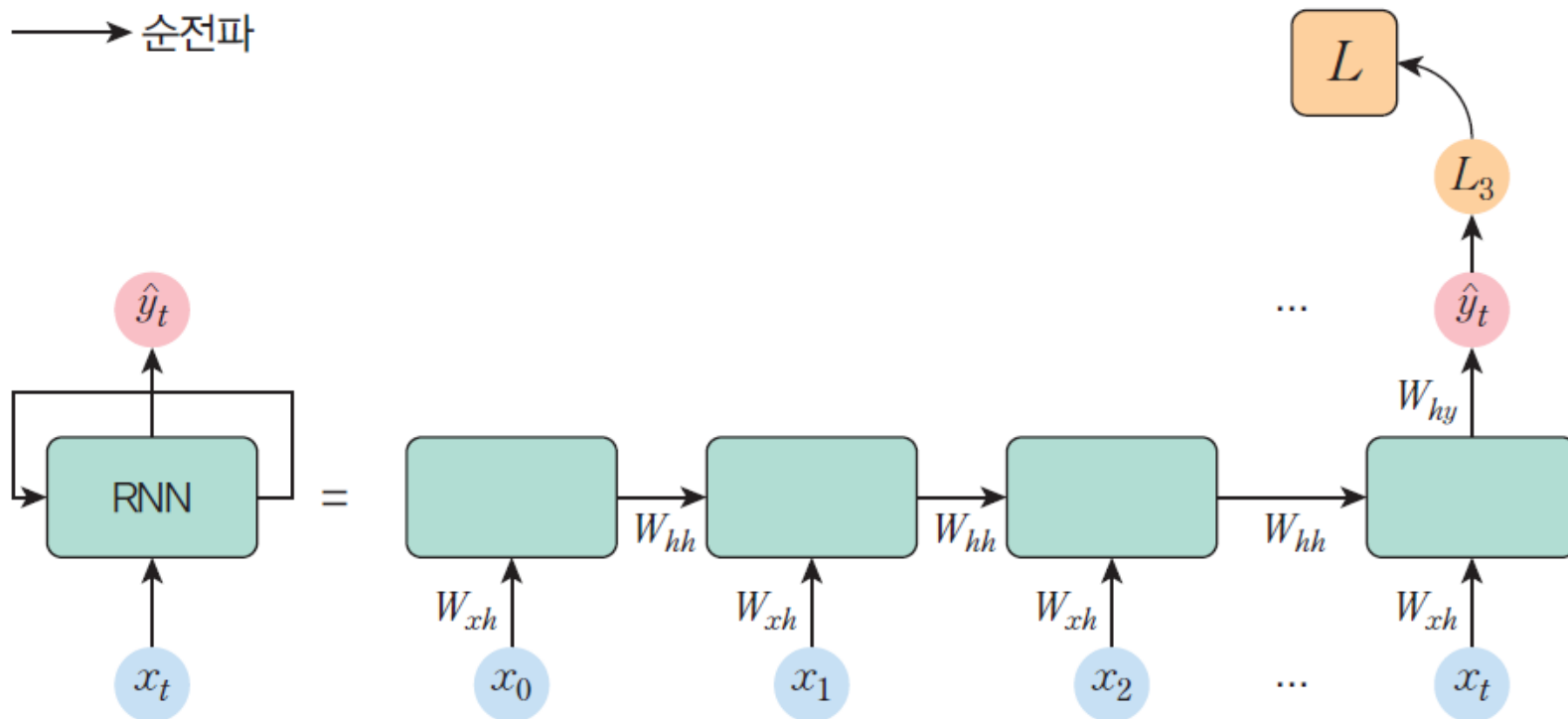
- 활성화 함수로는 tanh 함수를 주로 사용한다.



RNN의 순방향 패스

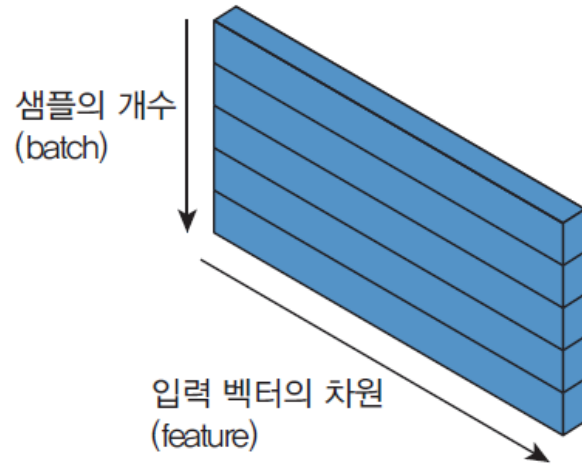


RNN의 순방향 패스

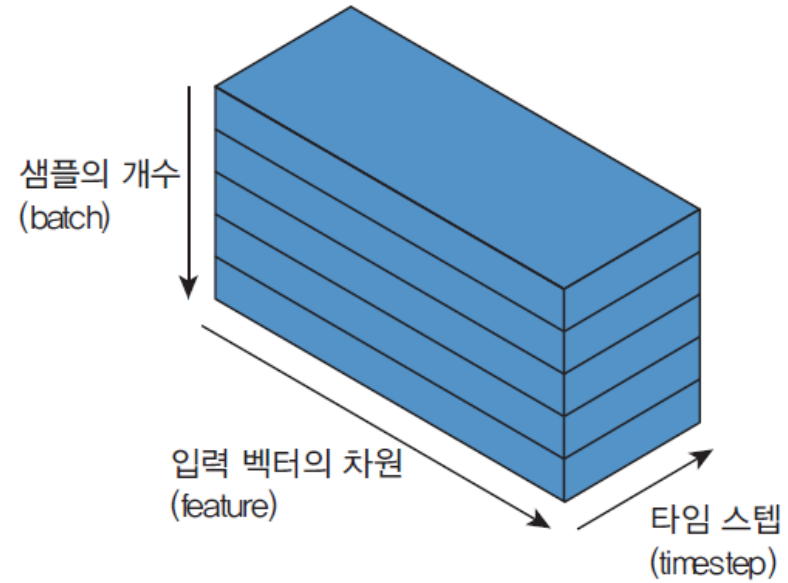


RNN에서의 입력 형상

피드-포워드 신경망



순환 신경망



메모리 셀의 각 시점에서의 모든 은닉 상태값을 반환받고 싶으면

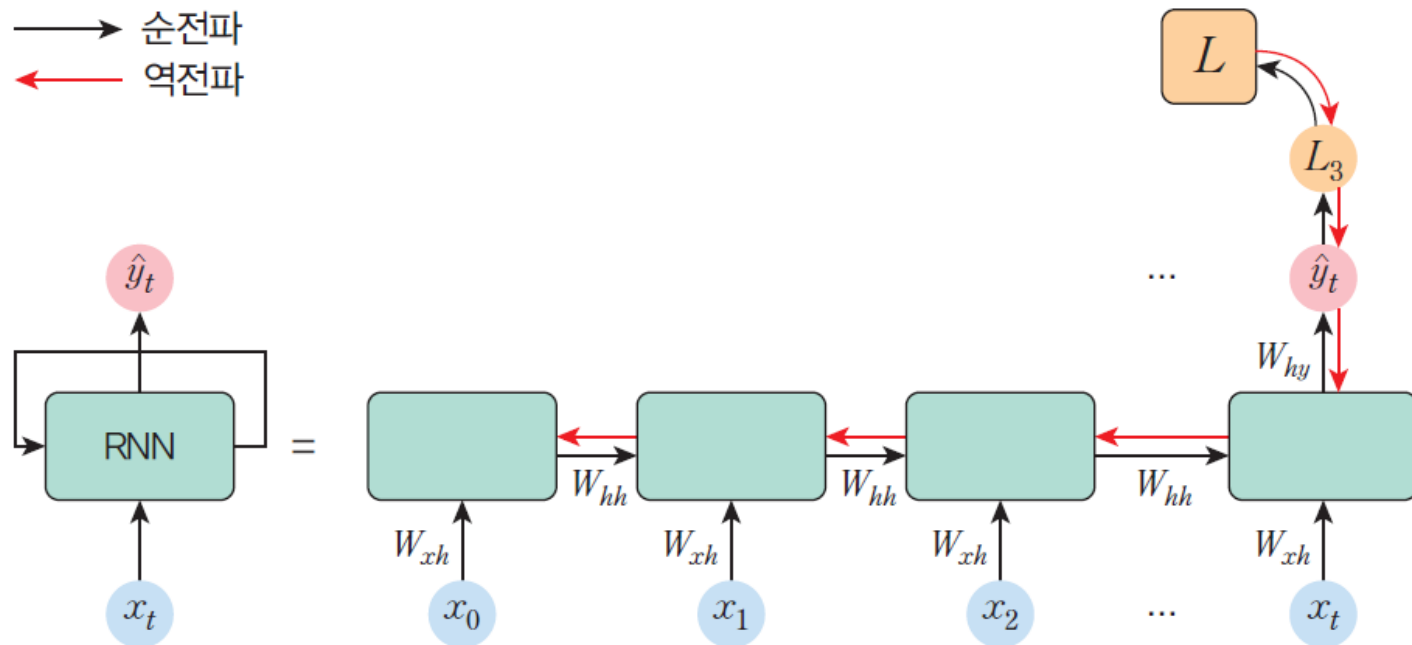
```
simple_rnn = tf.keras.layers.SimpleRNN(4, return_sequences=True, return_state=True)

whole_sequence_output, final_state = simple_rnn(inputs)

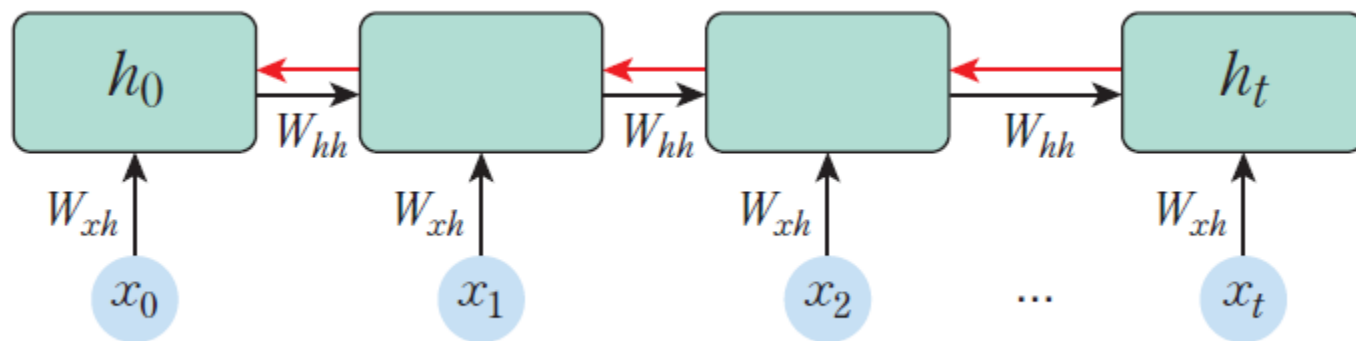
# whole_sequence_output의 형상은 `[32, 10, 4]`.
# final_state의 형상은 `[32, 4]`.
```

시간에 따른 역전파(BPTT)

- 순환 신경망에서는 계층을 통해 오류를 역전파하는 대신, 시간을 거슬러 올라가면서 그래디언트를 역전파한다.
- 이것을 BPTT(Backpropagation Through Time)라고 한다.

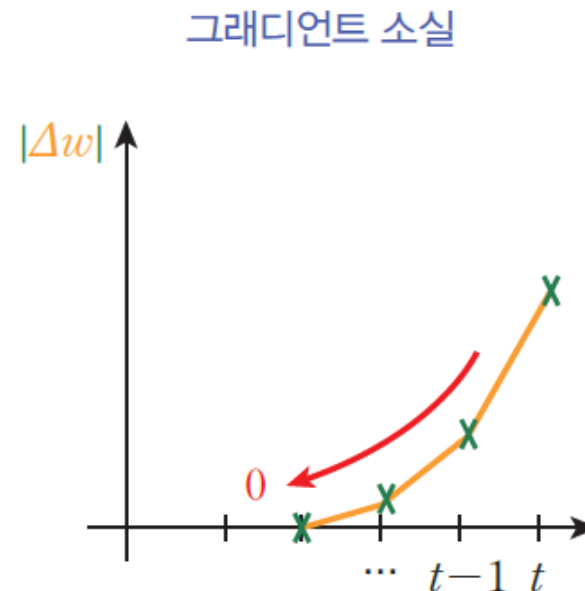
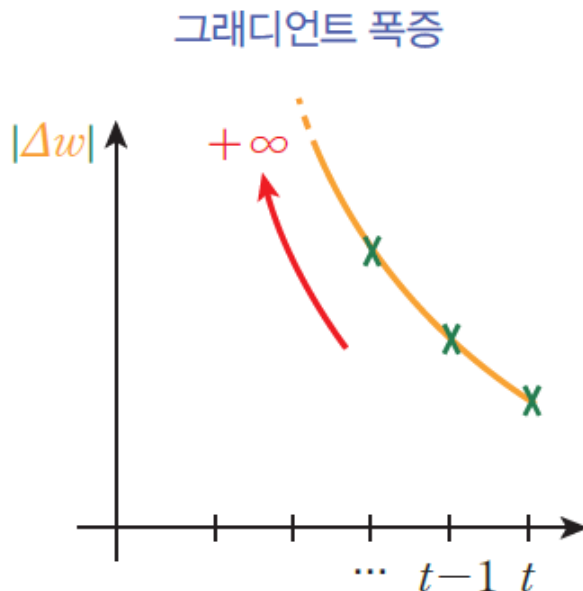


그래디언트 역전파 과정



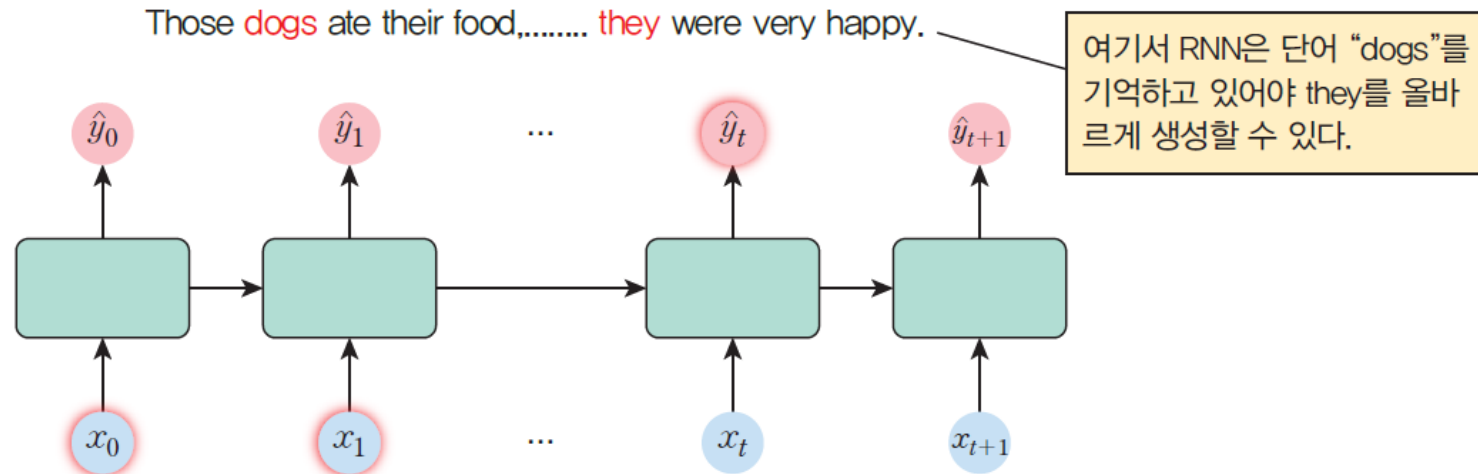
문제 발생

- 1보다 작은 값이 여러 번 곱해지는 경우 → 그래디언트가 점점 작아지다가 사라지게 된다.
- 1보다 큰 값이 여러 번 곱해지는 경우 → 그래디언트가 폭발적으로 증가한다.



그래디언트 소실 문제

- 그래디언트가 소실되면 학습이 진행되어도 먼 거리의 의존 관계는 파악하지 못하고 근거리의 의존 관계만을 중시하게 된다.

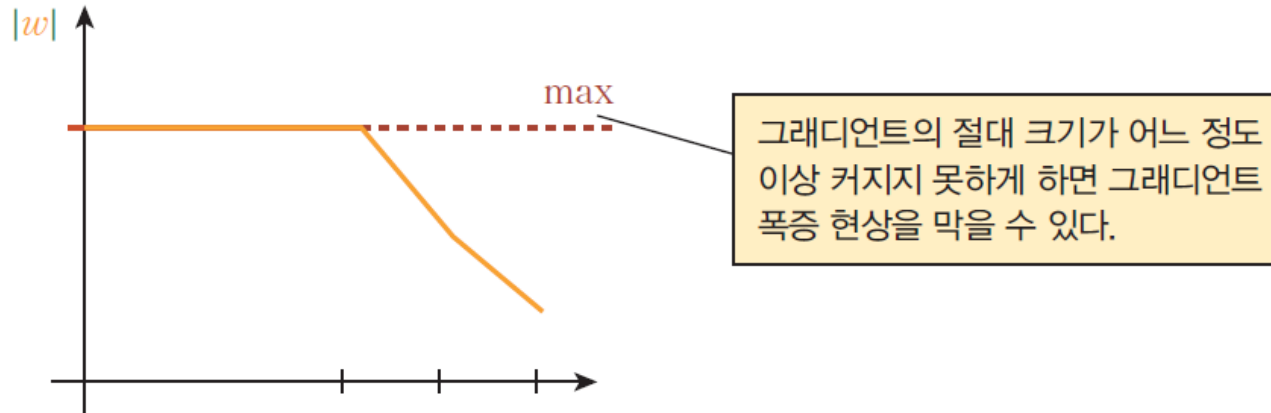


그래디언트 소실 방안

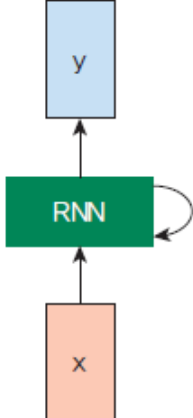
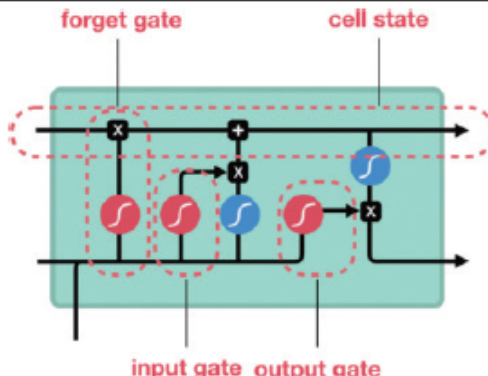
- 피드-포워드 신경망과 마찬가지로 활성화 함수를 ReLU로 바꾸면 도움이 된다.
- 또 다른 해결책으로는 가중치를 단위 행렬로 초기화하는 방법이 있다. 바이어스는 0으로 초기화하는 것도 도움이 된다.
- 또 하나의 해결책은 보다 복잡한 순환 유닛인 LSTM이나 GRU 같은 Gated Cell을 사용하는 것이다.
- 이들 게이트들은 어떤 정보를 지나가게 할 것인지를 제어할 수 있어서 장기적으로 기억할 수 있다.

그래디언트 폭증 방안

- 그래디언트 폭증은 그래디언트가 너무 커지는 것이다. 이때는 그래디언트를 일정 크기 이상 커지지 못하게 하는 방법이 많이 사용된다.

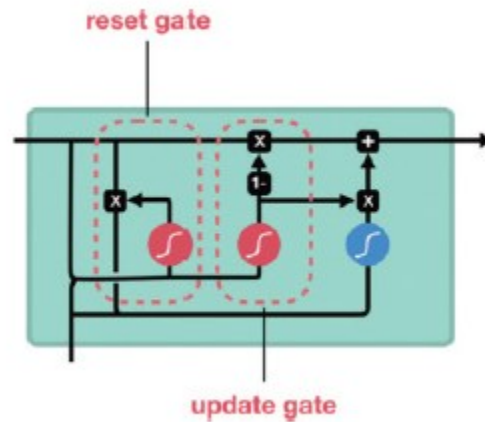


예제: Keras를 이용한 주가 예측

클래스	구조	설명
<code>keras.layers.SimpleRNN</code>		기본 순환 신경망이다. 앞 절에서 다룬 바 있다.
<code>keras.layers.LSTM</code>		1997년 Hochreiter & Schmidhuber 이 처음 제안한 순환 신경망 모델로서 앞 절에서 다룬 바 있다.

예제: Keras를 이용한 주가 예측

`keras.layers.GRU`



Cho et al. 등이 2014년도에서 처음 제안한 순환 신경망 모델이다. 이 책에서는 다루지 않았다.

주식데이터를 다운로드하고 그래프로 그려보자.

- 필요한 라이브러리
BeautifulSoup4,
finance-datareader를 설치한다

```
pip install BeautifulSoup4  
pip install finance-datareader
```


예제: 데이터를 다운로드하고 그래프로 그려보자.

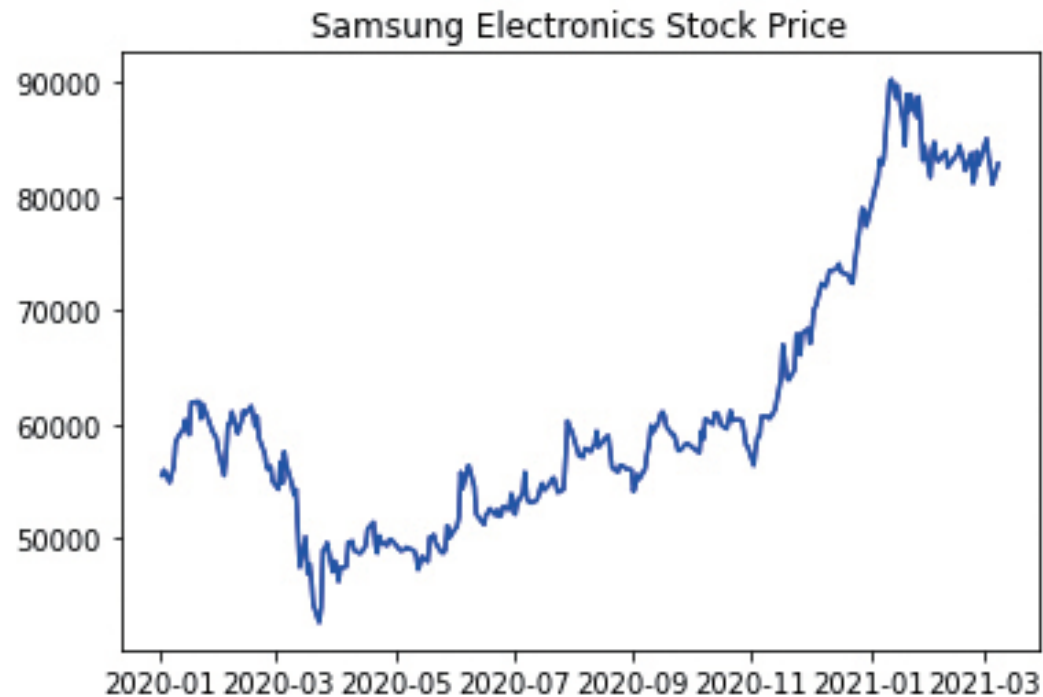
- 그래프를 그려보자.

```
# 라이브러리 포함
import FinanceDataReader as fdr
import numpy as np
import matplotlib.pyplot as plt

# 삼성전자 코드='005930', 2020년 데이터부터 다운로드
samsung = fdr.DataReader('005930', '2020')

# 시작가만 취한다.
seq_data = (samsung[['Open']]).to_numpy()
# 선형 그래프로 그린다.
plt.plot(seq_data, color='blue')
plt.title("Samsung Electronics Stock Price")
plt.xlabel("days")
plt.xlabel("")
plt.show()
```

예제: 데이터를 다운로드하고
그래프로 그려보자.



테스트용 주가 샘플 데이터 작성

- 데이터를 잘라서 샘플을 작성해보자

```
# 라이브러리 포함
import FinanceDataReader as fdr
import numpy as np
import matplotlib.pyplot as plt

# 삼성전자 코드='005930', 2020년 데이터부터 다운로드
samsung = fdr.DataReader('005930', '2020')

# 시작가만 취한다.
seq_data = (samsung[['Open']]).to_numpy()
# 선형 그래프로 그린다.
plt.plot(seq_data, color='blue')
plt.title("Samsung Electronics Stock Price")
plt.xlabel("days")
plt.xlabel("")
plt.show()
```

실행 결과

```
(284, 7, 1) (284, 1)  
[[55500]  
[56000]  
[54900]  
[55700]  
[56200]  
[58400]  
[58800]] [59600]
```

예제: Keras를 이용한 주가 예측

```
import FinanceDataReader as fdr
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

samsung = fdr.DataReader('005930', '2016')
print(samsung)

openValues = samsung[['Open']]

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range = (0, 1))
scaled = scaler.fit_transform(openValues)

TEST_SIZE = 200
train_data = scaled[:-TEST_SIZE]
test_data = scaled[-TEST_SIZE:]
```

예제: Keras를 이용한 주가 예측

```
def make_sample(data, window):  
    train = []  
    target = []  
    for i in range(len(data)-window):  
        train.append(data[i:i+window])  
        target.append(data[i+window])  
    return np.array(train), np.array(target)  
  
X_train, y_train = make_sample(train_data, 30)
```

예제: Keras를 이용한 주가 예측

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM

model = Sequential()

model.add(LSTM(16, input_shape=(X_train.shape[1], 1), activation='tanh', return_sequences=False))

model.add(Dense(1))

model.compile(optimizer = 'adam', loss = 'mean_squared_error')

model.fit(X_train, y_train, epochs = 100, batch_size = 16)
```

예제: Keras를 이용한 주가 예측

```
(X_test, y_test) = make_sample(test_data, 30)
pred = model.predict(X_test)

import matplotlib.pyplot as plt

plt.figure(figsize=(12, 9))
plt.plot(y_test, label='stock price')
plt.plot(pred, label='predicted stock price')
plt.legend()
plt.show()
```


실행 결과

